EECS 16B Designing Information Devices and Systems II

Note 0A: Overview

Welcome

As you've been told in 16A, the 16AB series is fundamentally about teaching you how to think like an engineer while making you stronger and more mathematically mature. The mathematical vehicle to do this is linear-algebra (and related concepts — especially in 16B the connections to calculus: differential equations most prominantly, but others as well) with the goal of you making these tools your own through your own hard work. This is done by showing you how you could have come with all the definitions yourself, as well as all the key proofs/derivations, and actually making you do many of these yourself — including deriving things not explicitly discussed in lecture/discussion, in ways that exercise the ways of thinking that are demonstrated for you in lecture and discussion. While mathematical curiosity is going to be fostered at points, the style is largely what is called "use-inspired[1]" where we show how each of the ideas emerge organically from engineering contexts, while being useful beyond the specific contexts that first motivated them.

The engineering contexts in 16AB have of course been carefully curated to allow both concepts *and* maturity to be built up step by step systematically. This is why 16A started with imaging, touched upon PageRank, engaged with touchscreens, and then went to the positioning module. This allowed you to develop the basic vocabulary of linear algebra, static linear circuit analysis, and inference by least squares and related ideas. Later in this note, we will go over the big picture application context for 16B: making cyborgs.

At times, 16AB use simple circuits to act as a bridge between the physical world and the much more generally applicable concepts that we want you to internalize. Circuits hit the "quadfecta" of being (1) extremely simple (so simple that we can proceed without any physics prerequisites while still engaging basic physical intuition) vehicles for understanding the interplay between modeling, analysis, and design; (2) amazingly well-approximated by linear component models as well as easy-to-understand nonlinearities like switches, together with interconnection using simple graphs; (3) easy/safe to engage with in a hands-on fashion in lab; while also being (4) incredibly useful building blocks in the real world.

We also keep coming back to the pattern of learning from data, another widely useful building block for real world problems. **By the end of 16B, you will actually have built a foundational understanding of both basic circuits and basic machine learning.**

1 Making cyborgs — the application thread

Throughout 16B, we will keep coming back to a big picture aspirational goal: making cyborgs. As engineers, we want to create a future in which machines can help human beings live better and fuller lives. One way to imagine doing so is cyborg technology. Imagine someone who has lost the use of a limb. We would like

[1]The term comes from a famous book by Donald Stokes: "Pasteur's Quadrant: Basic Science and Technological Innovation" that brought out how Engineering thinking is not encompassed by the naive dichotomy between "basic" and "applied" that exists in the popular view of the sciences. Instead, many key advances in the sciences (and mathematics) have actually been grounded in Engineering thinking where a broad set of goals creates the context for investigation and the systematization of knowledge. The popular example of "use-inspired" research is of Pasteur's foundational contributions to microbiology.

to be able to create an artificial limb that responds directly to their thoughts and helps them do what they want to do. In general, to have machines understand and respond to humans. This is futuristic technology (a civilization-scale endevour actually) that will take decades to fully realize, but that's exactly what we feel should motivate study in an introductory course like 16B. After all, those are the decades that you will be working!

How are we going to break this problem down into pieces that motivate aspects of the course? At a high level, there are three big tasks: extracting information from the brain and getting it into a computer, figuring out what the person wants, and then getting the physical machine to actually do what we want it to do.

1.1 The blocks that need to be understood

Each of these big tasks itself breaks down into sub tasks. But first, there is a higher level question that we need to tackle. How should we divide the work between what information-processing should be done using analog circuits and what should be done in a digital computer? To answer this, the course starts out by getting a basic understanding of why we can't expect digital computation to be infinitely fast. Once we understand that, we can start into doing those tasks with the following building blocks that process the information in stages:

1. After our physical device records electrical signals in the brain, we need to "filter" or clean the signal in the analog domain to remove as much interference as we can.
2. The filtered signal needs to be sampled as slowly as we can get away with and we need to be able to implicitly interpolate (inside the computer) using a global approximation of the continuous-time signal as needed.
3. The resulting sampled information needs to be summarized (dimensionality reduced) using simple and fast computations in a way that is specific to the application. We're going to need to use learning from data to get this compression right at design time.
4. The summarized information needs to be classified/interpreted so that we can infer what the person wants their cybernetic implants to do for them. Once again, we're going to need to use learning from data to figure out the right mapping to do this.
5. The current state of the mechanical implants, the inferred goal, and knowledge of the dynamics of the mechanical implants needs to be combined to plan a trajectory that achieves the goal. Here, learning is going to have to be used to learn a model for these dynamics that we can use to plan.
6. The current state, the planned trajectory, and knowledge of the dynamics needs to be used to do real-time responsive control to ensure that our real-world machine actually follows the planned trajectory despite having model inaccuracies and disturbances coming in from the world.

We're not going to cover those blocks in the order above, although we will be starting at the beginning. Instead, we will cover them in the order that best allows us to discover the required tools and ways of thinking as we go.

Obviously, it is fine if you don't understand what all this means right now. The point of the course is that you understand it by the end.

1.2 The toy version: our robot car

To give you a hands-on way of engaging with the above aspirational vision of making brain-controlled robotic body parts, we will have a toy version that lets you experience these ideas. Instead of capturing signals from electrodes in brains, we will use voice capture using a microphone. Instead of a robot limb, we will have a robot car that you will build. But within this toy context, you will engage with the blocks above — including the learning parts.

In fact, 16B is *the course* where you get to engage in a hands-on way with the entire modern machine-learning pipeline from building the data collection aparatus to collecting the data to learning from the data to actually deploying your learned model in the real world. And our main educational goal is that you actually understand every step of what is going on here, not as being able to call some magical libraries with the right incantations, but in a way that you understand fully what is going on and why it is working or not.

2 Behind the application thread: the key ideas

As you can see from the above, at a high level the distinction between 16A and 16B is that in 16A everything was largely static. In 16B, we will be engaging with dynamics a lot more. Mathematically, this means that we are really going to be leaning on not just the 16A prerequisite, but also calculus. In addition, there are ideas from our prerequisite 61A that we will be building upon, especially philosophically.

The ideas and concepts in the course are cumulative, and the story will keep building and deepening. At the same time, we will expect that your maturity will also keep developing alongside. While there are certainly going to be concepts and ideas that seem to be specific to each building block, we are actually going to keep circling back to a few core patterns of thought over and over again.

The main patterns are (in descending order of prevalence):

- Changing variables/coordinates — finding a coordinate system or way of looking at the problem that makes it as simple and transparent as possible.
- Learning from data.
- Iteration/Recursion/Induction — solving problems by taking one step at a time.
- Approximation — finding a way to divide the problem into the most important part, and the part that you ignore (possibly just for now).

Each of these is something that you have already been exposed to in 16A, 61A, and your calculus courses — but we're going to be greatly strengthening your understanding of their potential and versatility, especially as they combine together.

Again, it is fine if you don't understand how these work together just yet. The educational goal is that you understand by the end of the course since these are incredibly powerful ways of thinking that will serve you well across many different areas.

Along the way, we will also build a lot of mathematical and modeling tools. **Our core promise to you is that you will always know why we are doing whatever we are doing**, and even more importantly, we aspire to helping you understand how in principle, you could have discovered all these mathematical and modeling tools on your own. After all, we are laying the foundation for a long career in which you will constantly have to be learning and making discoveries for yourself.

3 How to succeed in 16B

The short answer for how to succeed is don't fall behind. Attend lecture, take your own notes, and participate in discussion. Read the official course notes as needed[2] and strive for actually understanding. Accept the fact that you might be confused when you first encounter any given HW problem — but try. Don't be afraid to try something that doesn't succeed at first. Overcoming what you don't understand and your own fears is a part of the learning process. Work with others and ask for help from course staff. This is a course in which truly understanding all the labs, homeworks, and discussions is the key to success on exams and beyond.

Acknowledgements

The reality is that these notes are the fruit of the combined effort of many people. In particular, credit for the overall vision of the Linear-Algebraic/Machine-Learning side of 16AB and how to present these ideas belongs largely to Prof. Gireeja Ranade. The overall aesthetic[3] of 16AB owes a great deal to both Gireeja Ranade and Elad Alon, the main instructors for the pilot Sp15 offering of 16A and main designers of the entire 16AB course sequence.

Lots of faculty have also contributed to and shaped the development of the technical material in 16B. These include Anant Sahai, Claire Tomlin, Murat Arcak, Miki Lustig, Babak Ayazifar, Michel Maharbiz, Kris Pister, and Jaijeet Roychowdhury. The first set of comprehensive notes for students was a reader put together by Murat Arcak that covered the material after the first module. In Fall '18, efforts led by Elad Alon initiated initial notes for the first module in 16 style. In Spring '19, the combined efforts of Anant Sahai, Kris Pister, and Jaijeet Roychowdhury led to some updated notes for the first module, and then in Fall '19, Anant Sahai led the effort to both prune down the course scope and get all the material standardized into a unified set of notes. For these unified 16B notes, the efforts of 16AB TAs Nikhil Shinde, Aditya Arun, and Rahul Arya have been invaluable in capturing the spirit of the course while getting the details right. It is also important to recognize all the feedback from students taking the course.

The notes exist in an ecosystem of resources together with the discussion worksheets and most importantly, the homework problems and their solutions. The labs and course project complement this written and computational material with hands-on experience with many of the course ideas.

Anyway, a major new and radically different freshman course sequence doesn't just happen — lots of people in the EECS department worked together for years to make all this a reality.

Contributors:

- Anant Sahai.

[2] Some people benefit from skimming course notes ahead of lecture and then reading them carefully after lecture. Others benefit from attending lecture without reading ahead, and then reading the appropriate sections of the official course notes carefully only after lecture in conjunction with their own notes. You should do what works for you, however 16B is definitely a course in which there is a big benefit to actively attending in-person lecture.

[3] Proper credit here also acknowledges the strong cultural influence of the faculty team that made EECS 70 and shaped it into a required course: Alistair Sinclair, Satish Rao, David Tse, Umesh Vazirani, Christos Papadimitriou, David Wagner, and Anant Sahai, along with pioneering TA's like Thomas Vidick, all helped make and polish the notes for 70. The 16AB+70 sequence all follow the aesthetic that mathematical ideas should have practical punchlines. 16AB further emphasizes the rooting of mathematical intuition in physical examples as well as virtual ones.

As far as linear algebra and differential equations go, the treatments in 16B in particular also owe a significant intellectual debt to William Kahan's notes and approach to dealing with this foundational material. For circuits, there is also a definite philosophical influence of Chua, Desoer, and Kuh's perspective. The unified view of computation and physical systems also owes a philosophical debt to Lee and Varaiya. All of the Berkeley faculty mentioned in this paragraph are now retired (and some are no longer among us), but their insights continue to reverberate here in this fresh treatment.

EECS 16B Designing Information Devices and Systems II

Note 0B: Circuit Equivalence

1 Equivalence

One aspect of circuit design that is distinctly different than most software engineering is that when we assemble a large circuit out of a component blocks, each of the blocks can potentially influence the behavior of the others. Does this mean that every addition or change to a circuit means that we need to completely re-analyze the entire system? No, because luckily, the ways they interact are limited in a very specific way that we will discuss. It turns out they actually interact through only 2 parameters, current I and voltage V. This leads to a new tool we will develop to help us when describing more complicated/complete circuit models; the concept of equivalence.

Equivalent circuits are used to simplify interactions between circuits. Let's take the simplest case where interactions are only through one pair of nodes. In that case, we just have two possible quantities: the voltage across the nodes and the current flowing through the connections. The relationship between this current and this voltage would then fully define the interactions between the circuits. This is where the idea of equivalence comes in. If we have a circuit that exhibits the same $I-V$ relationship from the standpoint of a pair of nodes, the other circuit (the one you are interacting with) can't tell the difference. The idea of equivalence is to be able to replace one (or both) of the interacting circuits with a simpler circuit that will give us the same overall behavior.

Before we move on, let's clarify what we mean by "equivalent": **Two circuits are equivalent if they have the same $I-V$ relationship.** (An example of an $I-V$ is that of a resistor, i.e., $V=IR$ or $I=\frac{V}{R}$). This is *exactly* what we mean by equivalence; be careful not to overextend this definition or apply others. For example, equivalence tells us nothing about the *power* in a circuit and one should be careful not to assume it does.

Now why is this possibly intuitively? Since voltage and current are governed by a linear relationship for all of the circuit elements we've learned about, and a line can be uniquely determined by exactly two points, we can capture the original circuit with a simplified circuit that has exactly two components: a voltage (or current) source and a resistor.

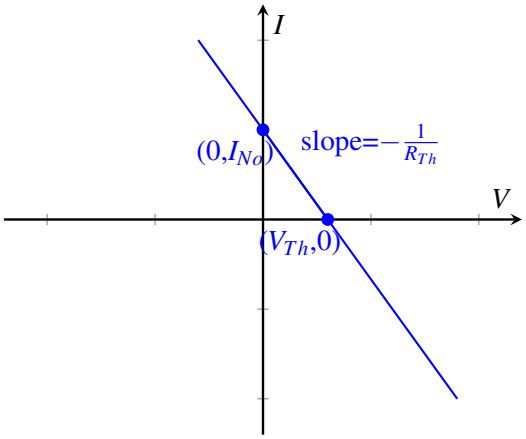
Definition 0.1 (equivalent circuit): **If we pick two terminals within a circuit, we say that another circuit is equivalent to the original circuit if it exhibits the same $I-V$ relationship at those two terminals.**

Note: From the standpoint of any other nodes in the circuit (i.e. any pairs of nodes), the circuit may or may not be equivalent. Furthermore, looking at the same circuit but examining a different pair of terminals may not produce equivalent $I-V$ relationship.

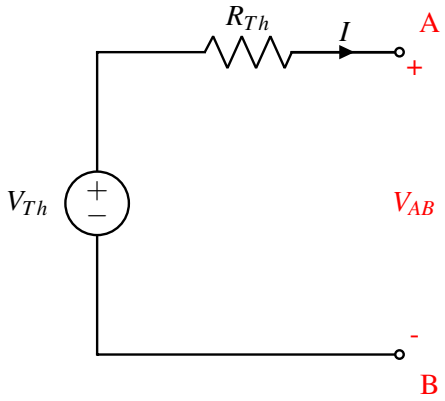
At a high level, what does it take (at a minimum) to construct a line? We can either use two points along the line, or one point and the slope of the line. Remember, the equivalent circuit of a circuit will have an identical IV curve, which is a line. In this class, we will construct these equivalent curves using a point and the slope. The two easiest points to collect along a line are the x-intercept (point with 0 current) and the y-intercept (point with 0 voltage).

There are two types of equivalent circuits we will construct: the *Thevenin* and the *Norton*. For the Thevenin equivelant we look at the intersection with the x-axis (zero current); for the Norton, we look at the intersection with the y-axis (zero voltage).

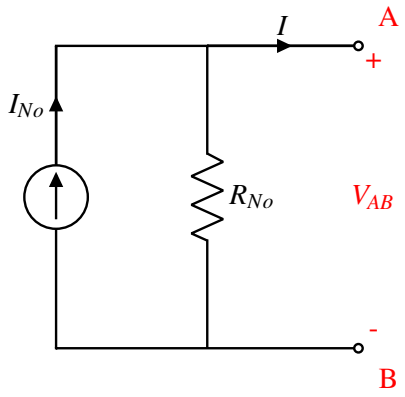
Next we figure out the slope of the line; remember, for an $I \times V$ curve, the slope is equal to the resistance.



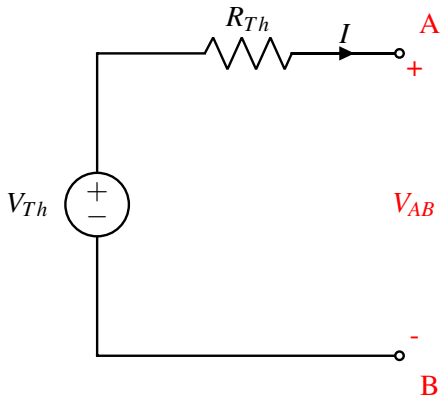
We call the first circuit below, containing a voltage source and a resistor the **Thevenin equivalent circuit**; we call the second circuit, containing a current source and a resistor, the **Norton equivalent circuit.** Once we simplify the original circuit to one of the above, we can easily figure out V_{out} no matter what resistor it is connected to on the right. In fact, we can convert **any** circuit into any one of these equivalent forms.



or



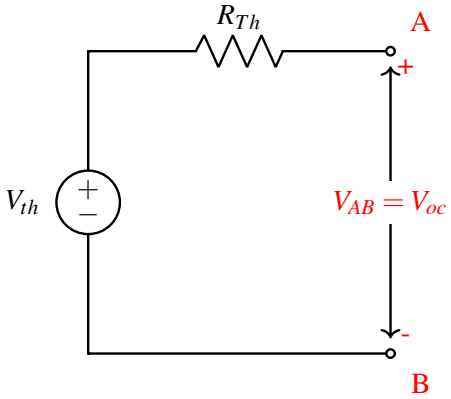
2 Thevenin Equivalent Circuit



Now how would you figure out V_{Th} and R_{Th} for the Thevenin equivalent circuit?

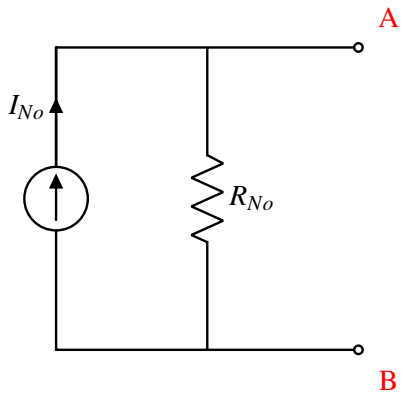
Concretely, the procedure to solve for the Thevenin equivalent is as follows:

Step 1, find V_{Th}: Connect an open circuit across the two output terminals and measure the voltage across them. This measured V_{OC} equals V_{Th}.



Step 2, find R_{Th}: Zero out any independent sources. Remember, this means voltage sources turn into a wire and current sources turn into an open circuit. Then apply either a test current into the terminal and measure the resultant voltage, or apply a test voltage and measure the resultant current. $R_{Th} = \frac{V_{test}}{I_{test}}$

3 Norton Equivalent Circuit



What about solving for the Norton equivalent circuit? First, note that R_{No} is equal to R_{Th}, since the slope of the IV curve is the same. Now, instead of looking at the V axis intercept, we find the intersection with the I-axis: At the intersection with the I-axis, the voltage drop between A and B is zero, which is equivalent to placing a wire between A and B (i.e. shorting A and B). We denote the current through the wire be I_{SC}.

To put it in terms of our standard procedure:

Step 1, find I_{No}: Connect a short circuit across the two output terminals and measure the current through it. This measured I_{SC} equals I_{No}.

Step 2, find R_{No}: Zero out any independent sources. Remember, this means voltage sources turn into a short circuit and current sources turn into an open circuit. Then apply either a test current into the terminal and measure the resultant voltage, or apply a test voltage and measure the resultant current. $R_{Th} = \frac{V_{test}}{I_{test}}$

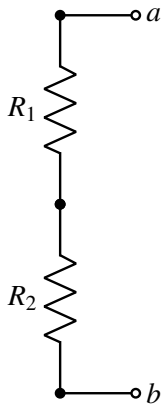
Note that the second step doesn't change because R_{No} is equal to R_{Th}!

4 Equivalence Examples

Here will we find the Thevenin equivalents for a set of simple circuits.

4.1 Series Resistors

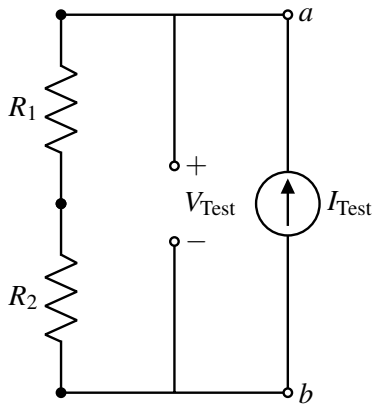
Consider the schematic:



Let's follow the procedure given above.

Step 1: Note that there is already an open circuit is already connected between terminals a and b. In this case there is no voltage or current source in the circuit. Therefore, the voltage at every node is the same, and therefore, $V_{ab,OC} = 0$. Remember, $V_{ab,OC} = V_{Th}$, so $V_{Th} = 0$.

Step 2: There is no source to zero out in this case. Since it will turn out to be the easier choice, we will apply a test current and measure the resulting voltage, as shown:



There is only one loop, and therefore all the currents in this circuit are the same.

$$V_{R1} = I_{\text{Test}} R1 \tag{1}$$

$$V_{R2} = I_{\text{Test}} R2 \tag{2}$$

$$V_{\text{Test}} = V_{R1} + V_{R2} = I_{\text{Test}} R_1 + I_{\text{Test}} R_2 \tag{3}$$

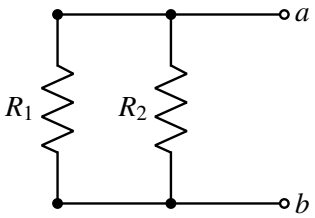
$$V_{\text{Test}} = (R_1 + R_2) I_{\text{Test}} \tag{4}$$

$$R_{Th} = \frac{V_{\text{Test}}}{I_{\text{Test}}} = R_1 + R_2 \tag{5}$$

We see that equivalent resistance of these two resistors is simply their sum. We call these resistors in **series**. Note that in order to be in series, the resistors have to have the exact same current through them.

4.2 Parallel Resistors

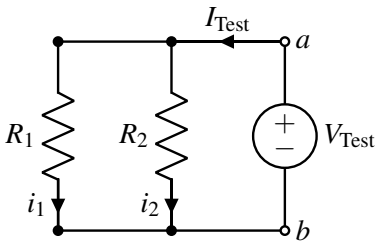
Another way to arrange a circuit with two resistors and no voltage source is as follows:



Let's again follow the procedure given above to find our equivalent circuit.

Step 1: Note that there is already an open circuit is already connected between terminals a and b. For the same reason as the prior example, $V_{ab,OC} = 0$ in this case. Therefore, $V_{Th} = 0$.

Step 2: There is no source to zero out in this case. Since it will turn out to be the easier choice, apply a test voltage and measure the resulting current, as shown:



To analyze this circuit, first we notice that the voltage drop over each resistor is equal to V_{Test}. This is because the voltage drop between node a and b is V_{Test}, and each resistor is connected to node a on one side and node b on the other.

First we use the I-V relationship of R_1.

$$V_{\text{Test}} = i_1 R_1 \tag{6}$$

$$i_1 = \frac{V_{\text{Test}}}{R_1} \tag{7}$$

Then we use the I-V relationship of R_2.

$$V_{\text{Test}} = i_2 R_2 \tag{8}$$

$$i_2 = \frac{V_{\text{Test}}}{R_2} \tag{9}$$

Finally, they are combined to calculate the equivalent resistance.

$$I_{\text{Test}} = i_1 + i_2 = \frac{V_{\text{Test}}}{R_1} + \frac{V_{\text{Test}}}{R_2} \tag{10}$$

$$\frac{I_{\text{Test}}}{V_{\text{Test}}} = \frac{1}{R_{Th}} = \frac{1}{R_1} + \frac{1}{R_2} \tag{11}$$

Rearranging this expression gives our final resistance:

$$R_{Th} = \frac{R_1 R_2}{R_1 + R_2} \tag{12}$$

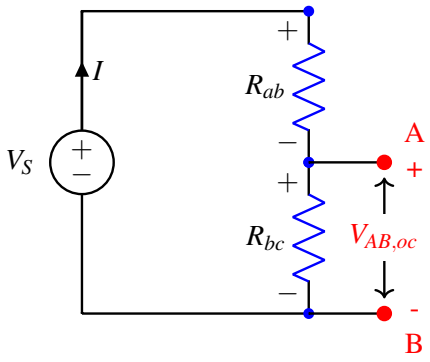
We call these resistors in **parallel**. Note that in order to be in parallel, the voltage across them has to be the same.

This mathematical relationship comes up often enough that it actually has a name: the "parallel operator", denoted $\|$. When we say $x \parallel y$, it means $\frac{xy}{x+y}$. Note that this is a mathematical operator and does not say anything about the actual configuration. In the case of resistors the parallel operator is used for parallel resistors, but for other components (like capacitors) this is *not* the case.

From these analyses, we now have a simple rule to tell if elements are in series or parallel. Series elements will have the exact same current through them due to KCL. Parallel elements will have the exact same voltage across them due to KVL.

4.3 Voltage Divider

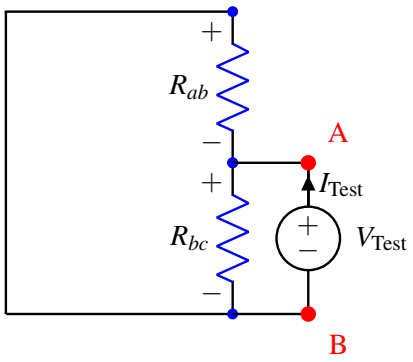
Now let's apply our analysis above to a voltage divider circuit shown below (which is very similar to the touchscreen). To figure out V_{th}, we solve for V_{oc} in the following circuit



Note that the same current flows through the two resistors. In addition, the voltage drop over the two resistors sums to V_s, so we can write $V_{Rab} = V_s - V_{AB,oc}$. Therefore, using Ohm's Law:

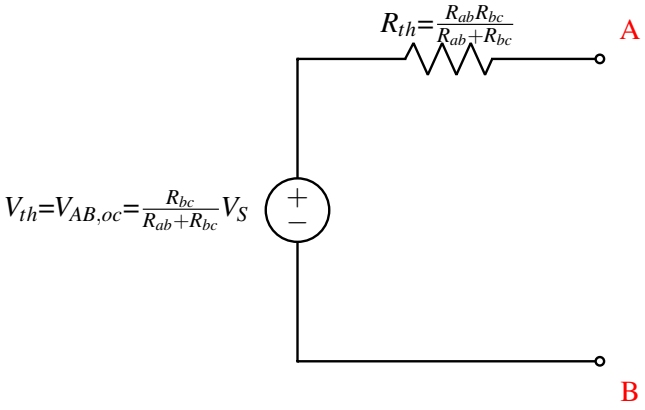
$$\begin{aligned} I_{Rab} &= I_{Rbc} \\ \frac{V_{Rab}}{R_{ab}} &= \frac{V_{AB,oc}}{R_{bc}} \\ \frac{V_s - V_{AB,oc}}{R_{ab}} &= \frac{V_{AB,oc}}{R_{bc}} \\ \frac{V_s}{R_{ab}} - \frac{V_{AB,oc}}{R_{ab}} &= \frac{V_{AB,oc}}{R_{bc}} \\ V_{AB,oc} &= \frac{R_{bc}}{R_{ab}+R_{bc}} V_s \end{aligned}$$

To figure out R_{th}, we zero out the independent source and apply a test voltage, measuring the resultant current.

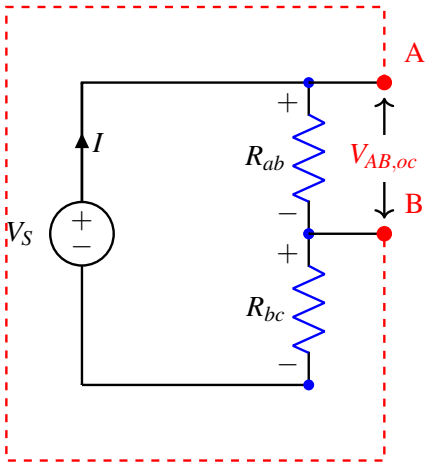


We can see that this is the same as the parallel resistor case we examined above: therefore, $R_{Th} = \frac{V_{\text{Test}}}{I_{\text{Test}}} = R_{ab} \parallel R_{bc}$.

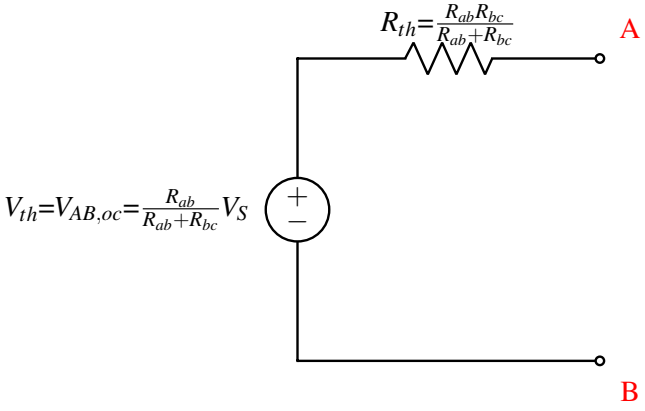
This gives us a resulting Thevenin equivalent circuit of:



What if we instead chose the upper two nodes (instead of the lower two nodes) as the two terminals (nodes A and B)? We can follow the same procedure to find an equivalent Thevenin circuit from the standpoint of these new nodes,



After following the same procedure we get the following equivalent circuit:



This is not the same result! In this case, the Thevenin voltages in the two circuits are different. This example shows how different pairs of nodes in the same circuit result in different equivalent circuits. In general, there is no guarantee that circuits will behave in the same way from the standpoint of different pairs of nodes.

5 Summary for Finding Equivalent Resistance

In general, there are three ways of finding the Thevenin/Norton equivalent resistance of a circuit. However, some of them only work in certain situations, so need to be used with caution.

(a) Zero out all independent sources and apply a V_{test} or I_{test} to calculate the resulting I_{test} or V_{test} respectively. $R_{eq} = \frac{V_{test}}{I_{test}}$.

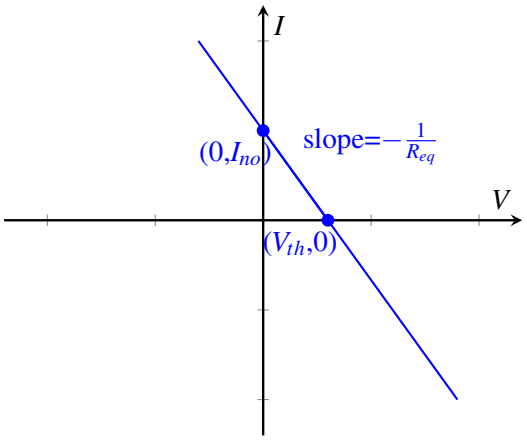
This is the method that we described in detail in the examples above, because **this method works for any circuit**. When in doubt, this method is the most reliable.

(b) Zero out all independent sources and reduce the entire remaining circuit into a single resistor using the series and parallel resistor formulas that were derived in Sections 4.1 and 4.2.

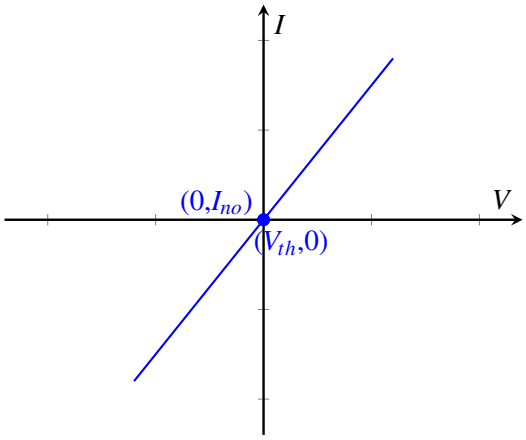
This method does not work if there are dependent sources. Remember that only independent sources are zeroed out, and there are no resistor formulas for dependent sources. In addition, some resistor configurations cannot be decomposed into combinations of parallel and series resistances.

(c) Calculate V_{th} and I_{no}, $R_{eq} = \frac{V_{th}}{I_{no}}$. This is an efficient method of finding R_{eq} if both the Thevenin and Norton equivalent circuits are being derived. Why does this work? Since the IV relationship is linear, we can calculate the slope (which is the reciprocal of resistance) from any two points. V_{th} and I_{no} are the points where the IV curve crosses the V and I axes, respectively (see the left-hand figure below).

However, this method does not work if V_{th} and I_{no} do not provide two unique points on the IV curve (see the right-hand figure below). Specifically, **this method only works if there is at least one independent source in the circuit.** When there are no independent sources, $V_{th} = I_{no} = 0$ which does not provide enough information to calculate R_{eq}.



$R_{eq} = \frac{V_{th}}{I_{no}}$



R_{eq} cannot be calculated from I_{no} and V_{th} alone

EECS 16B Designing Information Devices and Systems II
Note 1: Transistors, Differential Eqns.

1 Introduction to Transistors and RC circuits

Let us motivate this note by understanding the speed of digital computers. We can understand computers as a set of blocks that perform digital logic operations one after the another. You might be familiar with logic in the form of 'AND' and 'OR' operations in 'if' statements while programming. Logic gates are circuits that behave in a manner compatible with those logical operations. For this, high voltages are traditionally used to represent a logical 1 or TRUE and low voltages are traditionally used to represent a logical 0 or FALSE. These logical gate circuits are constructed out of physical devices called transistors. You will learn a lot more about digital circuits and how to construct logical gates out of transistors in 61C.

Here in 16B, we are going to focus on one of the most basic logical blocks: an inverter (or a 'NOT' gate). An inverter takes a boolean input and outputs the logical inverse: a 0 (low) maps to a 1 (high) and a 1 (high) maps to a 0 (low). The speed of computers is related to how quickly logical blocks can change their state and thus perform logic (for example, how quickly an inverter can output a 0 after its input changes to a 1). The underlying issues that limit the speed of an inverter are the same issues that impact all logical operations, so to understand what is going on, we will focus on inverters for simplicity.

One of the most basic circuits that we can use to analyze this change is an oscillator, a basic device that oscillates between zero and one. In fact such oscillators are quite common and are used as clocks in all the devices you regularly use! We can analyze the speed and behavior of such oscillators as a basic model to understand the more complex behavior of computers and their speed.

One way to create oscillators is by connecting together an odd prime number of inverters in a loop. Simply connecting an inverter in a loop will misbehave[1]. This type of oscillator is called a ring oscillator. By examining the signal in this oscillator after any inverter we can see that the signal must indeed oscillate between 0 and 1. We can create these inverters physically by using transistors as shown below:

[1] If we simply have one inverter connected in a loop, we will not have the switching behavior of the oscillator that we desire (depending on if the capacitor is appropriately sized). Since the circuit is fighting between high and low at the output it can stabilize at an intermediate value. In order to allow for oscillations, we need to chain more inverters in a loop. In fact, to prevent undesired behavior, we usually chain together a prime number of inverters. The reason why is related to properties of modulo arithmetic which you will learn in CS70 together with properties of signals studied in EE120.

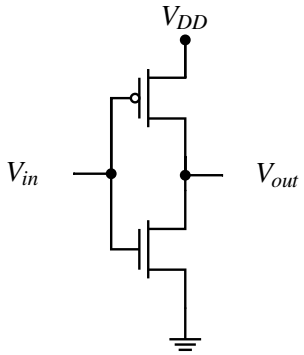


Figure 1: CMOS Inverter

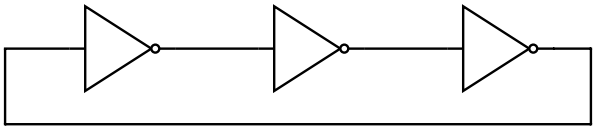


Figure 2: Ring oscillator with 3 inverters

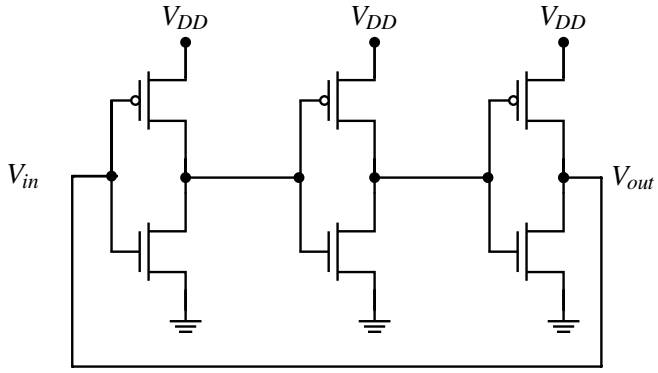


Figure 3: Ring oscillators with 3 CMOS inverters

At their most basic level, transistors can be modeled as switches that change state based on the voltage applied at their gates. There are three terminals on a transistor: a source terminal S, a drain terminal D, and a gate terminal G. As the names imply, the voltage on the Gate terminal determines whether the switch connecting the source and the drain is on (closed) or off (open). The slightly tricky part here comes from understanding "which voltage?" After all, you know from 16A that voltage is physically significant when measured across two points. For transistors, when we talk about the gate voltage, we are talking about the voltage at the gate relative to the voltage at the source. This is usually denoted $V_{GS} = V_G - V_S$.

There are two different kinds of transistors that we look at in what are called CMOS circuits: NMOS transistors and PMOS transistors. We'll get into more details soon, but basically, an NMOS transistor is on if the gate voltage V_{GS} is greater than some small positive threshold V_{tn}. This means that an NMOS transistor is on when the gate voltage is sufficiently higher than the source voltage. The threshold voltage tells us how much higher it needs to be. A PMOS transistor is on if the gate voltage V_{GS} is less than a small *negative* threshold $-V_{tp}$. This means that a PMOS transistor is on when the gate voltage is sufficiently lower than the source voltage. The threshold voltage tells us how much lower it needs to be. Note: because it is easy to get

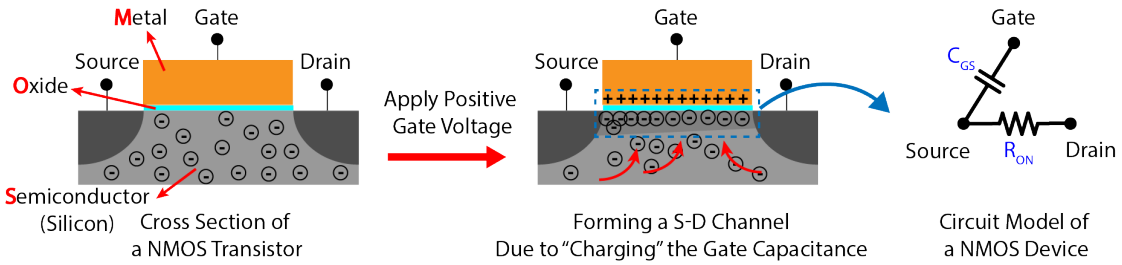


Figure 4: A toy view of a physical transistor to illustrate the "puddle" model of transistor operation. The "puddle" is the group of negative charges that have accumulated on the interface of the semiconductor and the oxide. https://en.wikipedia.org/wiki/Threshold_voltage has an interesting animation that literally shows the "puddle" growing as the gate voltage changes.

tripped up by sign errors when we have negative quantities in expressions where we cannot visibly see the signs, many people prefer to look at $V_{SG} = -V_{GS} = V_S - V_G$ when thinking about PMOS transistors.

At this point, you might be wondering "why do transistors physically behave in this manner?" A true answer to that question is out of scope for this course since it requires a knowledge of physics. However, there is a simple "charge puddle model" that gives you a heuristic sense for why transistors work the way that they do. Look at Figure 4. The idea of this model is that the gate of the transistor is like one terminal of a capacitor with the other part being in the silicon between the source and drain terminals. When the voltage on the capacitor is high enough in the right direction, a "puddle" of charge carriers forms in the silicon to balance out the charge being put on the gate. When the puddle is large enough (hence the finite threshold), it connects the source and the drain, allowing current to flow between them. The "source" is the terminal that can be viewed as where the relevant charge carriers spill from to form the puddle. For NMOS, these carriers are electrons having a negative charge. For PMOS, these carriers are what are called "holes" and they have a positive charge. Actually understanding this properly requires more physics; however, we tell you this just because it might help some of you remember the difference between PMOS and NMOS.

Using the most basic switch model of transistors, each inverter switches instantaneously. If each inverter switched instantaneously, then connecting them in a loop with an odd number of inverters would lead to inconsistent behavior! However, since we can implement this circuit in the real world there is clearly some aspect that we are missing. When such inconsistencies arise in our model, this can be a symptom of failing to properly understand real world behavior. In such cases, the usual approach is to approach the problem with a more detailed model. In this case, the oscillating behavior that we see is actually possible as there is a slight delay between the input and output of the inverters. You can think of these inverters in terms of relays or mechanical switches that are turned on and off by springs. The slight delay while the spring moves the switch from on to off and vice versa is what enables the oscillatory behavior that we see.

Since our simple switch model is not enough to understand this delayed behavior, we adopt a more detailed model for transistors. We model transistors as having some resistance (i.e. the "puddle" doesn't conduct perfectly) and some characteristic capacitance from their gates.[2] These models are illustrated in Figure 6 and Figure 7. It is important to note what might seem to be a peculiar convention: for NMOS transistors, the source S is on the bottom while for PMOS transistors, the source S is on the top. For our purposes, this is done to make the circuit for an inverter (as well as other logic gates done in CMOS style) more easy to draw on paper. Physically, it is related to the nature of the charge carriers in NMOS vs PMOS transistors.

[2]When dealing with these circuits in real-world integrated circuits, we also must deal with the capacitance of the wires.

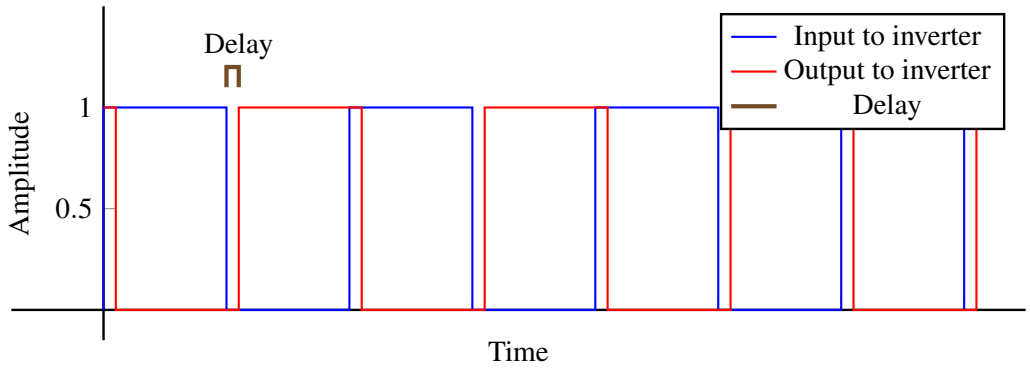


Figure 5: Delay in inverter output for simplified model.

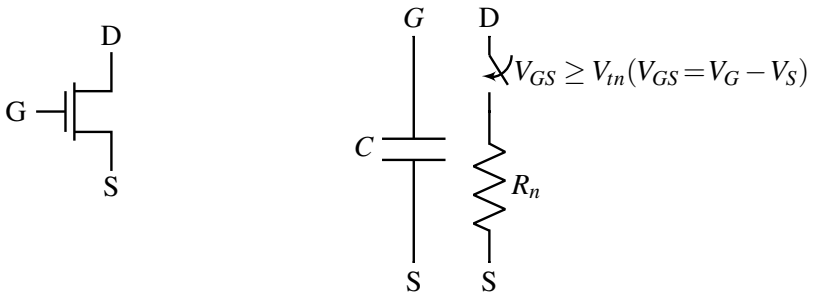


Figure 6: NMOS Transistor Resistor-switch model. The switch is on when V_{GS} is greater than the threshold voltage. Notice the convention: in NMOS Transistors, the Source terminal is at the bottom.

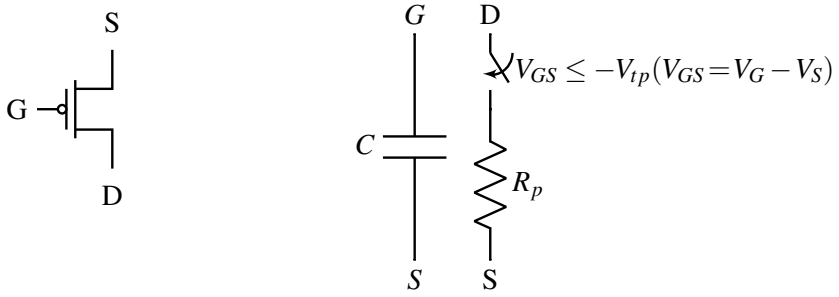


Figure 7: PMOS Transistor Resistor-switch model. The switch is on when V_{GS} is less than the negative threshold voltage. For reasons of human understanding and avoiding sign errors, we will often instead look to see if V_{SG} is greater than $|V_{tp}|$ instead. Notice the convention: in PMOS Transistors, the Source terminal is at the top.

This model for inverters can be used to redraw and analyze our oscillator made out of inverters from Figure 3

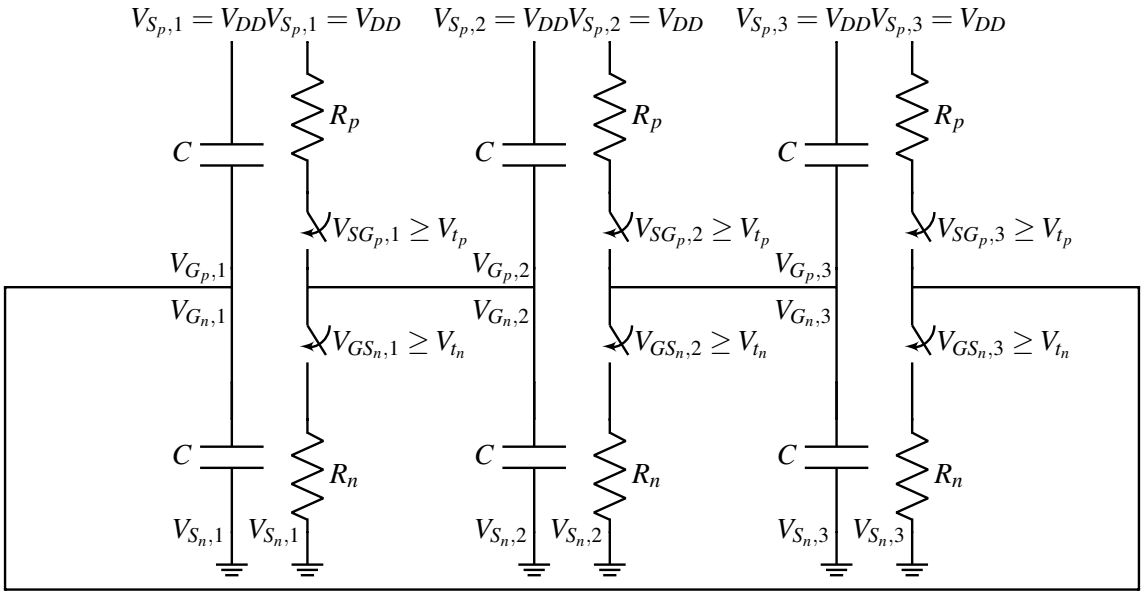


Figure 8: Ring oscillator with detailed transistor model: $V_{GS_n,i} = V_{G_n,i} - V_{S_n,i}$ and $V_{SG_p,i} = V_{S_p,i} - V_{G_p,i}$.

With this model we can see that each inverter drives some capacitance. This means that each inverter is pushing or draining charge from capacitors to cause the output to flip to a '1' or a '0'. To get an idea of how fast it takes for the inverter to change signals, let us examine the case of an inverter in the oscillator where the output started at 1 (high) switching to zero.

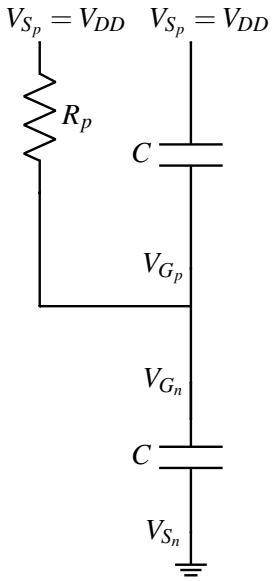


Figure 9: Inverter output at 1

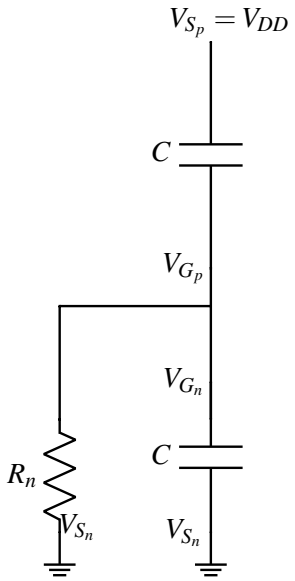


Figure 10: Inverter output at 0

If we condense the circuit down we see that switching the signal involves discharging the capacitor through a resistor. Similarly, if we look at an inverter going from outputting a 0 to outputting a 1, we get a capacitor charging through a resistor. We will show that this behavior is true later in the note. In the next section, we will analyze this with an intuitive approach.

2 Intuitive approach to RC circuits

Going with the example above, let us intuitively examine the voltage on a discharging capacitor over time.

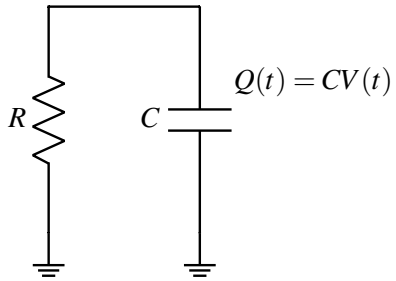


Figure 11: Capacitor discharging through circuit

Let us start out by writing the equations we know from EECS16A:

$$V = IR \tag{1}$$

$$Q = CV \tag{2}$$

Here since we are dealing with voltage and current changing over time we will use $V(t)$ and $I(t)$ in place of V and I to denote this time dependence.

We know that the voltage on the capacitor starts out high at some value we will call V_{DD}. This voltage $V(t)$ is the result of some charge $Q(t)$ on the capacitor. When we discharge the capacitor the charge leaves the capacitor as a current through the resistor and sinks into ground[3]. As dictated by Ohm's law the current through the resistor is $I(t) = \frac{V(t)}{R}$. If this voltage didn't change, then all the charge CV on the capacitor would drain away in exactly RC seconds. However as charge leaves the capacitor, the actualy $V(t)$ decreases and as a result the current $I(t)$ also decreases. This gives us the intuition that as the voltage drops, charge will leave the capacitor at a decreased rate due to the decrease in current. Thus the voltage drop will be a curve falling sharply first and then decreasing at some rate with the decrease in voltage. In fact, the slope on this discharge curve, at any point, will be such that the voltage would reach 0 in RC seconds (time) if we continue along that slope.

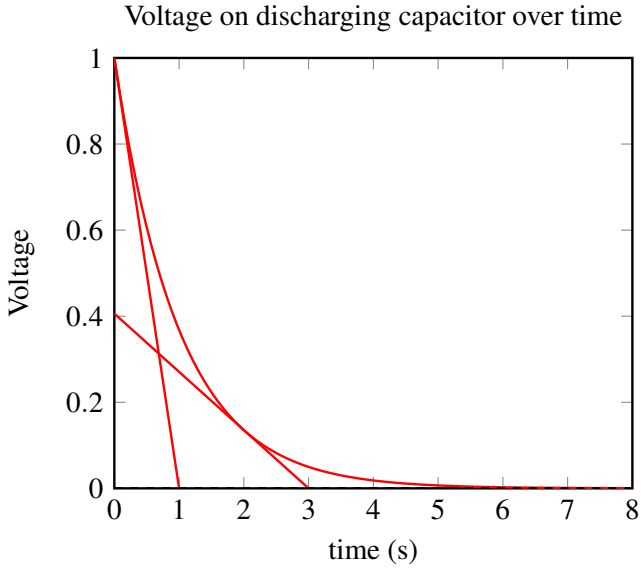


Figure 12: Voltage on capacitor discharging through resistor with RC constant = 1

Now the question arises, what exactly is this curve? Does it have a formula? In the following sections we will use a mathematical approach to solve for $V(t)$ exactly and validate our intuition.

3 Mathematical approach to RC circuits

To begin solving for $V(t)$ let us begin with the following approach that should be a familiar pattern from EECS16A:

- Establish variables for various parts of the circuit
- Utilize KCL (Kirchoff's current law) to establish current equations
- Establish remaining equations (branch equations) with the elements of the circuit

Let us start by establishing variables for the circuit:

[3] In reality ground is connected to the voltage source and completes a loop.

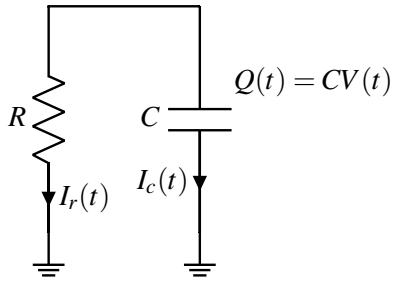


Figure 13: Capacitor discharging through circuit

Let $V(t)$ be the voltage across the capacitor, $Q(t)$ be the charge on the capacitor, $I_c(t)$ be the current flowing through the capacitor to ground, $I_r(t)$ be the current flowing through the resistor to ground, C be the capacitance of the capacitor and R be the resistance of the resistor.

To establish current equations we can focus on the node between the capacitor and resistor. Here we relate the currents to get the equation:

$$I_c(t) = -I_r(t)$$

For the remaining equations let us start by looking at the equation $Q(t) = CV(t)$. This charge, $Q(t)$, starts to leave the capacitor as current through the capacitor which in turn lowers the voltage on the capacitor. Since current is the change in charge over time we differentiate the formula for charge given above to get:

$$\frac{d}{dt}Q = C\frac{d}{dt}V(t) = I_c(t).$$

This gives us the following equations:

$$I_c(t) = C\frac{d}{dt}V(t) \tag{3}$$

$$\frac{V(t)}{R} = I_r(t) \tag{4}$$

$$I_c(t) = -I_r(t) \tag{5}$$

Simplifying the above equations by substitution we get:

$$\frac{d}{dt}V(t) = \frac{-V(t)}{RC}. \tag{6}$$

At first glance it seems like we have one equation and two unknowns: $\frac{d}{dt}V(t)$ and V. Normally we would not be able to make much progress with such a set up, however, these two unknowns are actually related by $\frac{d}{dt}$, the derivative operator. We can use this information to help arrive at a solution. An equation of this form, that relates the derivative of a function to something, is called a differential equation. We will learn how to solve it in the next section.

Now that we have derived an equation for a discharging capacitor we can show how our inverter output switching from 1 to 0 behaves like a discharging capacitor.

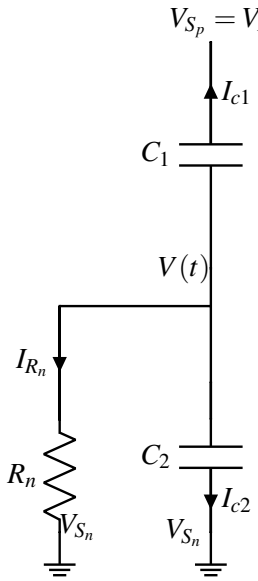


Figure 14: Inverter output at 0

In Figure 14 the inverter has just switched from outputting 1 to outputting 0. This means that the voltage $V(t)$ started at V_{DD} and decreases to 0 at steady state. We know the voltage across C_1 is $V(t) - V_{DD}$ and the voltage across C_2 is $V(t)$. Using this information we can set up a differential equation to solve for $V(t)$:

$$I_{c1} = C_1 \frac{d}{dt}(V(t) - V_{DD}) \tag{7}$$

$$I_{c2} = C_2 \frac{d}{dt} V(t) \tag{8}$$

$$I_{R_n} = \frac{V(t)}{R_n} \tag{9}$$

$$I_{c1} + I_{c2} + I_{R_n} = 0 \tag{10}$$

$$C_1 \frac{d}{dt}(V(t) - V_{DD}) + C_2 \frac{d}{dt} V(t) + \frac{V(t)}{R_n} = 0 \tag{11}$$

$$C_1 \frac{d}{dt}(V(t) - V_{DD}) + C_2 \frac{d}{dt} V(t) = -\frac{V(t)}{R_n} \tag{12}$$

$$C_1 \frac{d}{dt} V(t) + C_2 \frac{d}{dt} V(t) = -\frac{V(t)}{R_n} \tag{13}$$

$$(C_1 + C_2) \frac{d}{dt} V(t) = -\frac{V(t)}{R_n} \tag{14}$$

$$\frac{d}{dt} V(t) = -\frac{V(t)}{R_n (C_1 + C_2)} \tag{15}$$

This is exactly the same form of differential equation that we got for the discharging capacitor circuit with just a different value for capacitance! Thus we have shown that we can boil this inverter circuit down to a capacitor discharging through a resistor! (You can take a similar approach to show that an inverter that switches from 0 to 1 is akin to charging a capacitor through a resistor).

4 Differential equations

4.1 Simple scalar differential equations

Differential equations relate functions to their own derivatives. In this note, we will learn to solve simple first order scalar differential equations. To begin let us start with the simple example of:

$$\frac{d}{dt}x(t) = b$$

Here b is a particular given constant. If you recall from high school calculus to find x we can integrate both sides with respect to t:

$$\int \frac{d}{dt}x(t)dt = \int bdt \tag{16}$$

$$x(t) = bt + k_1 \tag{17}$$

where k_1 is any arbitrary constant.

We can check to see that this indeed satisfies $\frac{d}{dt}x(t) = b$ by calculating $\frac{d}{dt}x(t) = \frac{d}{dt}(bt + k_1) = \frac{d}{dt}(bt) + \frac{d}{dt}(k_1) = b + 0 = b$.

Recall from calculus that we resolved this ambiguous constant through definite integrals, where the bounds of integration were specified. When solving circuits or other problems involving differential equations, the definite integral approach is not always the most natural. This is because we instead often see this ambiguity-resolving information physically manifest as specific values for voltages or other state variables at a certain point in time. We will further discuss solving for the unknown constants in upcoming sections. Though the example here utilized a constant this method can be extended to differential equations of the form $\frac{d}{dt}x(t) = f(t)$ where $f(t)$ is any function that we can integrate.[4]

4.2 "Homogeneous" differential equations

Next, we work to extend this beyond $\frac{d}{dt}x(t) = f(t)$ to the more general first order differential equations of the form $\frac{d}{dt}x(t) = ax(t) + b$ where a and b are constants (In the context of circuits you will see that b will be the effect of voltage and current sources). We always want to start by thinking about the simplest case of the problem. Let us start by setting $b = 0$ with what are called homogeneous first order differential equations. The name isn't important, but this gives us $\frac{d}{dt}x(t) = ax(t)$ which feels simpler than the general form of such differential equations. In fact this equation is exactly like $\frac{d}{dt}V(t) = \frac{-V(t)}{RC}$, the differential equation for the voltage on a discharging capacitor, where $a = -\frac{1}{RC}$.

The format of this equation should remind you of eigenvalues you saw in EECS16A. There we saw $Ax = \lambda x$. This parallels $\frac{d}{dt}x(t) = ax(t)$, where the derivative (a linear operator), akin to the linear matrix operator, acts on some function $x(t)$ to give the same function $x(t)$ times some constant a.

Thus, intuition tells us that we should look for "eigenfunctions" of the derivative operator. In other words we need a function whose derivative equals itself times some constant. Recall that the function e^{at} satisfies

[4] The integration we refer to here is normal Riemann integration that you learned in high school; however, there are some cases, that are important in EECS and beyond the scope of this class, where you will also encounter Lebesgue integration. It is important also to realize that the only reason this "anti-derivative" approach to solving this kind of simple differential equation is valid is because you have proved the fundamental theorem of calculus in your basic calculus courses. Without that theorem, the use of integration here is nothing more than a heuristic.

this property perfectly! Furthermore, recall that when we solved for eigenvectors, we found an eigenspace where any linear combination of eigenvectors corresponding to a eigenvalue constituted a valid eigenvector. Similarly, here we see that, with the information given, any multiple of e^{at} satisfies the diffential equation. Hence, one solution will be in the form of $x(t) = k_2 e^{at}$ where k_2 can be any constant. Thus for our RC circuit, the solution should be something like $x(t) = k_2 e^{\frac{-t}{RC}}$.

So far, what we have done here is simply a form of glorified guessing. This is because while the eigenvalue/eigenvector idea above can be made rigorous (by appropriately and carefully defining the relevant infinite-dimensional vector spaces and linear operators on them), we haven't done so. We are just reasoning by analogy here. Anything obtained purely by analogy is just an educated guess. So we need to check to see if this guess is even a solution to the differential equation.

Let us validate by taking the derivative of $x(t)$:

$$\frac{d}{dt} k_2 e^{\frac{-t}{RC}} = \frac{-1}{RC} k_2 e^{\frac{-t}{RC}} \tag{18}$$

And it turns out that this worked!

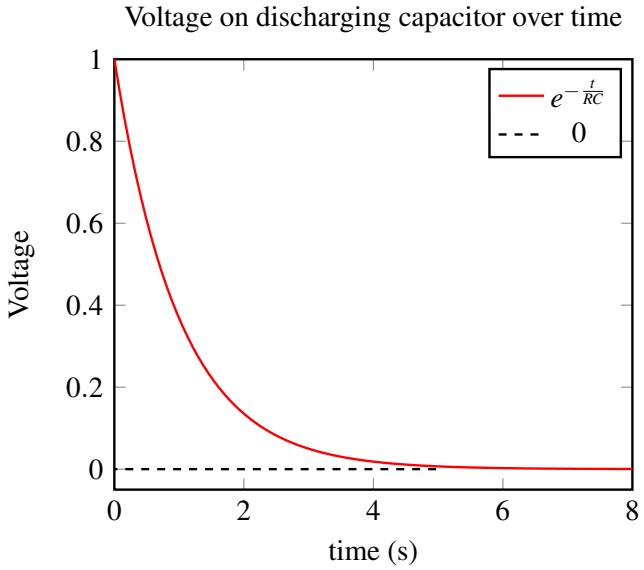


Figure 15

Uniqueness

Now that we have found a set of potential solutions, the other question that arises is whether there is a unique solution to the differential equation that we are solving. In both the cases above:

$$x = bt + k_1 \tag{19}$$

$$\frac{d}{dt} k_2 e^{at} = a k_2 e^{at} \tag{20}$$

we had an infinite number of solutions, as the constants k_1, k_2 in question could take any value and still satisfy the differential equation. We faced a similar issue when solving for voltages in a circuit in EECS16A. In EECS16A we always had one free variable and all the voltages that we solved were relative to that free variable. In order to properly solve for unique voltages in our circuit, we needed more information about

this free variable. This free variable was the ground that we were either given or arbitrarily placed in our circuit.

Thus, in order to restrict the set of solutions for these differential equations, we also need more information. Unlike the circuits in EECS16A, however, we cannot arbitrarily choose this free variable. Instead, it must be set to reflect the actual behavior of our circuit. Consequently, we can appropriately set this free variable using the actual value of the function at a specific time, which can be derived from the behavior of our circuit in the specific problem context.

Here, let $x(0) = c_0$. This information is called an "initial condition" and is usually specified as the value the function takes at time $t = 0$. Although the initial condition is often specified, other points and boundary conditions can be used as well; for example, we can also use steady state, i.e. $t \to 0\infty$, behavior to solve for the constants in question.[5]

Hence, our system is defined as follows:

$$\frac{d}{dt}x(t) = ax(t) \tag{21}$$

$$x(0) = c_0 \tag{22}$$

Revisiting the solution to the differential equation above with the initial condition we get:

$$x(t) = ke^{at} \tag{23}$$

$$x(0) = ke^0 \tag{24}$$

$$x(0) = k = c_0 \tag{25}$$

Thus, we get the solution $x(t) = c_0 e^{at}$. In the case of the discharging capacitor circuit, we set the initial condition $x(0) = V_{DD}$, where the capacitor was initially charged to V_{DD}. In this case solving for the constant multiplier gives us $k = V_{DD}$, yielding the solution $x(t) = V_{DD}e^{\frac{-t}{RC}}$. Finally, we have a specific solution to our problem.

4.3 Proof of uniqueness

Though we have one solution that satisfies the homogeneous differential equation and initial condition, we are not yet done. We need to ensure that we have not left out any solutions. To do so, we will prove that the solution we found is unique (Note: not all differential equations have unique solutions but we can prove the differential equation in this case does).

We start by assuming that there is another solution to the homogeneous differential equation. To argue for uniqueness we must show that the other solutions must be exactly the same as the solution that we have already found. Normally our first approach when proving equality is to manipulate both solutions to the same form, however, since we lack an expression for this hypothetical other solution, we must take another approach. We can show equality through one of two ways:

- The difference between the solutions is 0 everywhere the solution is defined.
- The ratio of the solutions is always 1.

[5] One question that may arise is whether it is valid to use times before $t = 0$ in order to compute the constant for the unique solution. Though we are free to use steady state behavior, the time in the problems we establish starts at time 0. Thus in the world of our problem there is no time before 0 and so it is invalid to consider times $t < 0$ to solve for unique solutions.

Showing either of the two statements will establish the equality between the two solutions, and hence prove the uniqueness of our original solution.

Proof:
We will prove equality with the second approach[6]. We have shown one solution: $x(t) = c_0 e^{at}$. Let us assume there is another solution to the homogeneous differential equation: $y(t)$ such that $\frac{d}{dt}y(t) = ay(t)$.

We want to show that the ratio $\frac{y(t)}{c_0 e^{at}} = 1$ everywhere. Here, the constant c_0 in the denominator can be a bit tricky to handle since it might be 0. So let us instead consider the ratio $\frac{y(t)}{e^{at}}$. Here, we are safe in knowing that the denominator is not zero.

Now we analyze the derivative of this ratio:

$$\frac{d}{dt}\frac{y(t)}{e^{at}} = \frac{d}{dt}y(t)e^{-at} = -ay(t)e^{-at} + e^{-at}\frac{d}{dt}y(t) = -aye^{-at} + aye^{-at} = 0.$$

The derivative of the ratio is always zero. This means that the ratio must be a constant. But which constant? For this, we can look at the initial conditions. From the initial conditions, we already know that $y(0) = c_0$ and $e^0 = 1$. So, the constant is just c_0. In other words, the ratio $\frac{y(t)}{e^{at}} = c_0$ always, and so $y(t) = c_0 e^{at}$ always as well.

It is important to remember that this uniqueness proof is actually crucial. This is what allows us to use guess-and-check to solve differential equations with any confidence in the answer. Since essentially all so-called-methods for solving differential equations are really just ways of guessing, uniqueness is vital to being able to make progress.

4.4 Nonhomogeneous differential equations

Now that we have learned to solve homogenous differential equations (those for which the all zero solution is a potential solution), let us learn to solve nonhomogeneous differential equations where "all zero" is not a possible solution. These are differential equations of the form shown below. The motivating example we use here is a charging capacitor in an RC circuit. This is akin to an inverter switching from '0' to '1'.

$$\frac{dx}{dt} = ax + b, \text{ where } b \neq 0 \tag{26}$$

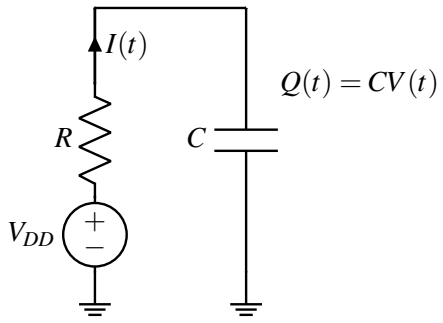


Figure 16: Capacitor charging through resistor circuit

[6]In general when faced with multiple approaches to solve a problem it is advantageous to try them all. As you attempt the problem, some approaches will inevitably appear easier. You can follow through with these approaches to arrive at a solution!

In general when solving unknown problems we want to try to reformulate them in terms of problems we know how to solve. So we formulate the above as a homogeneous differential equation that we know how to solve, and we want to do so in a way that doesn't make any assumptions.

In terms of circuits these homogeneous equations corresponded to our circuit having steady state values of 0. Here, however, the steady state value converges to some constant instead. To force this into a homogeneous differential equation we need to change variables so as to add an offset to shift the steady state value of our circuit to 0. We can accomplish this mathematically by substituting $\tilde{x}(t) = x(t) + \frac{b}{a}$ or $x(t) = \tilde{x}(t) - \frac{b}{a}$. This is simply a change of variables. We have represented our variable in a more convenient form without making any assumptions and changing the initial problem statement![7] With this substitution we get:

$$\frac{d}{dt}\tilde{x}(t) = a\tilde{x}(t) \tag{27}$$

This is exactly the homogeneous case we learned to solve before! Using the techniques described in the previous section we can find that $\tilde{x}(t) = ke^{at}$. We can now re-substitute to get $x(t) = ke^{at} - \frac{b}{a}$. As before we need an initial condition to solve for the constant k. Let the initial condition be $x(0) = c_0$. Plugging this in, we get:

$$c_0 = ke^0 - \frac{b}{a} \tag{28}$$

$$k = c_0 + \frac{b}{a} \tag{29}$$

$$x(t) = (c_0 + \frac{b}{a})e^{at} - \frac{b}{a} \tag{30}$$

To check this let us plug our solution back into the original differential equation in Equation (26). Since we start with $x(t) = (c_0 + \frac{b}{a})e^{at} - \frac{b}{a}$, we expect:

$$\text{LHS: } \frac{d}{dt}x(t) = \frac{d}{dt}\left(\left(c_0 + \frac{b}{a}\right)e^{at} - \frac{b}{a}\right) = a\left(c_0 + \frac{b}{a}\right)e^{at} - 0 = a\left(c_0 + \frac{b}{a}\right)e^{at} \tag{31}$$

$$\text{RHS: } a\left(\left(c_0 + \frac{b}{a}\right)e^{at} - \frac{b}{a}\right) + b = a\left(c_0 + \frac{b}{a}\right)e^{at} - b + b = a\left(c_0 + \frac{b}{a}\right)e^{at} \tag{32}$$

Hence, LHS = RHS, and $\frac{d}{dt}x(t) = ax(t) + b$ is solved by $x(t) = (c_0 + \frac{b}{a})e^{at} - \frac{b}{a}$. In fact it can be shown that it is uniquely [8] solved by $x(t) = (c_0 + \frac{b}{a})e^{at} - \frac{b}{a}$.

With these techniques, let us go back and approach the motivating example of a charging capacitor circuit. In the above circuit we know that $I = C\frac{d}{dt}V(t)$ and that $V_{DD} - V(t) = I(t)R$ by Kirchoff's law. Combining these equations together we get that $-RC\frac{d}{dt}V(t) = V(t) - V_{DD}$. Rewriting this as $\frac{d}{dt}V(t) = \frac{-V(t)}{RC} + \frac{V_{DD}}{RC}$ we get a nonhomogeneous differential equation as we discussed above where $a = -\frac{1}{RC}$ and $b = \frac{V_{DD}}{RC}$.

As in the generalized example, we perform a substitution: $V(t) = \tilde{V}(t) + V_{DD}$. Substituting this back in, we get:

$$\frac{d}{dt}(\tilde{V}(t) + V_{DD}) = \frac{-(\tilde{V}(t) + V_{DD})}{RC} + \frac{V_{DD}}{RC} \tag{33}$$

[7] For example when solving $x + 2 = 7$ We can represent x as $x = \tilde{x} + 2$ or $x = \tilde{x} + 5$ if that makes the problem more convenient. With $x = \tilde{x} + 5$ we get $(\tilde{x} + 5) + 2 = 7$ Which we can solve to get $\tilde{x} = 0$ and $x = 5$. The initial problem is unchanged as x was simply a variable that we chose to represent in a more convenient manner $\tilde{x}$.

[8] We already proved uniqueness in the homogeneous case. This is in fact sufficient to prove uniqueness in the nonhomogeneous case, as we can simply to do a change of variables to phrase our problem as a homogeneous differential equation and prove uniqueness for this reformatted problem.

Since V_{DD} is a constant, $\frac{d}{dt}V_{DD} = 0$. This gives us:

$$\frac{d}{dt}\tilde{V}(t) = -\frac{\tilde{V}(t)}{RC} \tag{34}$$

We can solve this homogeneous differential equation to get $\tilde{V}(t) = ke^{\frac{-t}{RC}}$. To solve for the value of k we resubstitute and use our initial condition: $V(0) = 0$.

$$V(t) = \tilde{V}(t) + V_{DD} \tag{35}$$
$$V(t) = ke^{\frac{-t}{RC}} + V_{DD} \tag{36}$$
$$0 = ke^{0} + V_{DD} \tag{37}$$
$$\therefore k = -V_{DD} \tag{38}$$

With this, we find that the solution to our differential equation is

$$V(t) = V_{DD}(1 - e^{\frac{-t}{RC}}) \tag{39}$$

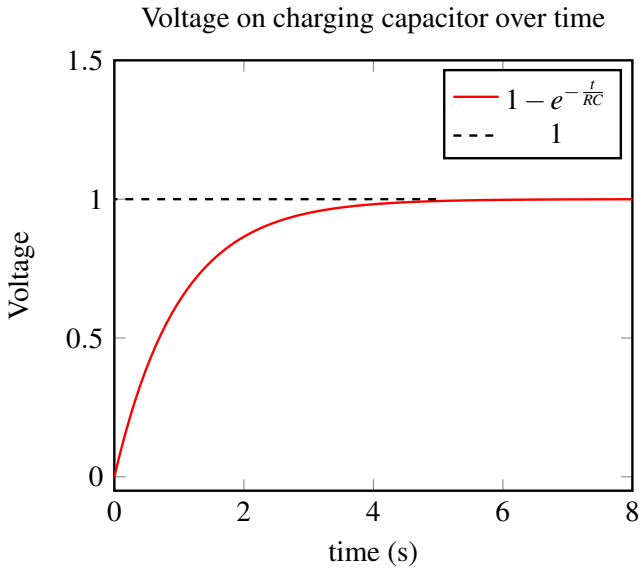


Figure 17

5 Nonlinear differential equations

In the above sections we only talked about linear differential equations where $\frac{d}{dt}x(t) = ax(t) + b$. However, in general you may encounter differential equations like $\frac{d}{dt}x(t) = x(t)^2$ and other such nonlinear functions of $x(t)$.

In general, there are various "techniques" that can be used to attempt to guess potential solutions for such equations. At the end of the day, all of these guesses need to be checked and the appropriate uniqueness theorems proved to make sure that we have got the single true solution. Only then can this solution used for any predictive purposes.

Without a uniqueness theorem, such solutions cannot be trusted for prediction. In the homework, you will see an example that illustrates how a seemingly innocuous differential equation can have non-unique solutions. In that homework, we will also share another heauristic technique that can be used to guess solutions to nonlinear differential equations — a technique known as "separation of variables." There are many such heuristic techniques out there, and different ones tend to work for different types of equations. You will encounter these techniques in later courses alongside the kinds of differential equations for which they tend to work.

Contributors:

- Neelesh Ramachandran.
- Nikhil Shinde.
- Anant Sahai.
- Aditya Arun.

EECS 16B Designing Information Devices and Systems II

Note 2: Transient Analysis and Inputs

1 Introduction

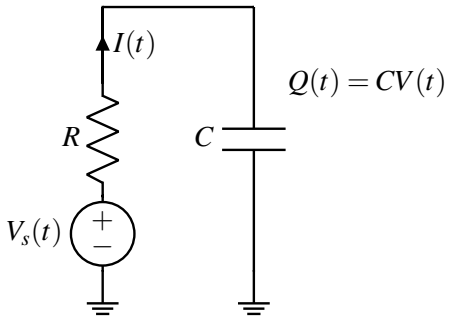


Figure 1: Capacitor charging through a circuit with a resistor. We can imagine that the voltage source is changing with time (that is, we express it as a function of time).

In the previous note we learned to solve for the transient voltage $V(t)$ on a capacitor charging up through a resistor. Recall that we solved the following differential equation:

$$\frac{d}{dt}V(t) = -\frac{V(t)}{RC} + \frac{V_{DD}}{RC}$$

to get the solution $\forall t \geq 0$:

$$V(t) = V_{DD}(1 - e^{-\frac{t}{RC}}). \tag{1}$$

In the previous note, we had viewed the "input" as coming from switches reconfiguring themselves. When switches changed from one state to another, what held steady across that instantaneous switch was the charge (and hence voltage) on the capacitors. The previous configuration's end state provided the initial state for the next configuration. Alternatively however, we can also think of the voltage that we used to charge up the capacitor as an input to our circuit and allow that voltage $V_s(t)$ to change with time. As far as the capacitor was concerned in the previous note, it might as well have faced a piecewise constant input that had:

$$V_s(t) = \begin{cases} 0 & t < 0 \\ V_{DD} & t \geq 0 \end{cases}$$

Though we have only shown this for positive inputs where $V_{DD} > 0$, we can also see that this applies to negative inputs such as $-V_{DD}$. Plugging in a negative input, we get the following differential equation:

$$\frac{d}{dt}V(t) = -\frac{V(t)}{RC} - \frac{V_{DD}}{RC} \tag{2}$$

Using the same kind of change of variables as we did in the previous note, we can solve the above equation to get for $t \geq 0$,

$$V(t) = V_{DD}(e^{-\frac{t}{RC}} - 1)$$

while $V(t) = 0$ for $t < 0$. This is the response of the circuit to inputs of the form:

$$V_s(t) = \begin{cases} 0 & t < 0 \\ -V_{DD} & t \geq 0 \end{cases}$$

Can we take what we know to build to understand more interesting inputs?

2 Time-varying Piecewise Constant Inputs: Two Illustrative Cases

Having analyzed these basic cases, we want to consider how to deal with inputs that change over time in a more interesting fashion. We have a strategy that we think should work — treat piecewise constant inputs in the same way that we dealt with circuits with switches changing configuration. Make the state (charge on the capacitor) be instantaneously constant across the configuration change, and just solve the differential equation with that initial condition. Let us start by considering the most basic changing input that we can think of: A voltage turning on to some value V_{DD} and then turning off.

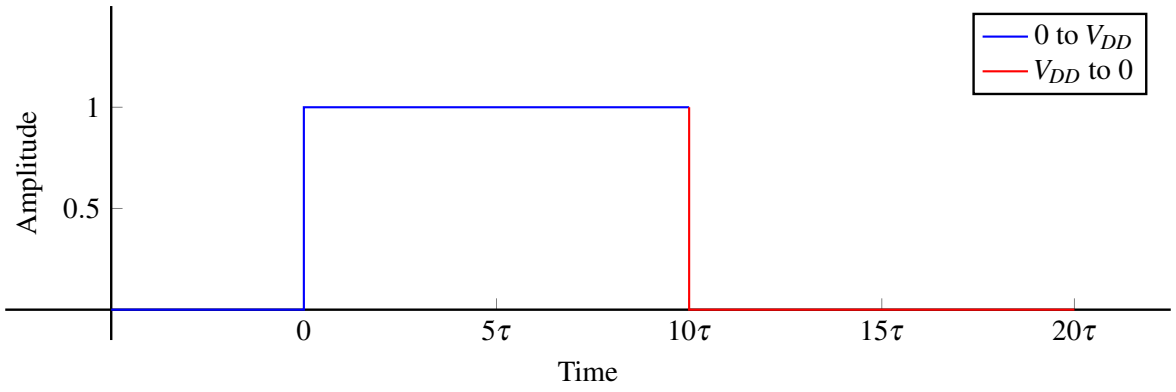


Figure 2: On and Off input: On for 10τ. Here $\tau = RC$ is the RC time constant for the circuit.

As always, when analyzing these more complex problems, we try to phrase them in terms of problems that we already know how to solve. We can look at this case as a combination of two piecewise constant cases: A constant zero input steady until some time T switching instantly to a steady constant 1 input until time $T + D$ (here D is some constant representing how long we hold at V_{DD}), falling back to zero again for the rest of time beyond $T + D$.

If $D \gg \tau$ then the circuit has the opportunity to settle to steady state. We treat the circuit in 2 different time intervals; the first with initial condition at 0 and the second with initial condition at V_{DD} (the value that the circuit settled to in the first interval from T to $T + D$).

Before we continue let us establish some notation. We use $V_i(t_{\text{int}})$ to denote the voltage on the capacitor during the i^{th} time interval that we are analyzing. Let t be absolute time starting at 0 and let t_{int} be the time from the beginning of the i^{th} interval until now. This latter time internal to the interval is useful conceptually.[1]

First Interval Analysis: Analyzing the circuit for time $t \in [0, 10\tau]$ with initial condition $V(0) = 0$ and constant input V_{DD} starting at time $t = 0$, we get the differential equation in Equation (1) (where the initial condition is $V(0) = 0$):

[1] This notation might be a bit confusing initially, but reading the casework and analysis below will help clarify.

$$\frac{d}{dt}V(t) = -\frac{V(t)}{RC} + \frac{V_{DD}}{RC}$$

Recall the solution to this type of differential equation is:

$$V(t) = ke^{-\frac{t}{RC}} + V_{DD}$$

Here k is some constant that we will solve for using initial conditions. Plugging in the initial condition, we get:

$$\begin{aligned} V(0) - V_{DD} &= k \\ k &= -V_{DD} \\ \implies V_1(t_{\text{int}}) = V(t) &= V_{DD}(1 - e^{-\frac{t}{RC}}) \qquad t \in [0, 10\tau] \end{aligned}$$

The solution to this differential equation is the same as the charging capacitor case! Since the input is held at V_{DD} until time 10τ, the circuit has time to settle to essentially steady state. We can see this by plugging in $t = 10\tau$:

$$\begin{aligned} V(10\tau) = V_1(10\tau) &= V_{DD}(1 - e^{-\frac{10\tau}{RC}}) \\ &= V_{DD}(1 - e^{-10}) \\ &\approx V_{DD}(1 - 0.00004539) \\ &\approx V_{DD} \end{aligned}$$

Thus we have shown that by time $t = 10\tau$ the capacitor has approximately reached the steady state voltage V_{DD}. We can now think about what happens for the next chunk of time ($t \in [10\tau, 20\tau]$).

Second Interval Analysis: We now have a new initial condition: $V(10\tau) = V_1(10\tau) = V_2(0) \approx V_{DD}$.

Using this initial condition information, the definition $t_{\text{int}} = t - 10\tau$, and the steps above, we can solve for $V_2(t_{\text{int}})$:

$$\frac{d}{dt}V(t) = -\frac{V(t)}{RC} + 0$$

Recall the solution to this type of differential equation is $V(t) = ke^{-\frac{t}{RC}}$. Plugging in the initial condition, we get $V_2(t_{\text{int}}) = V_{DD}(e^{-\frac{t_{\text{int}}}{RC}})$. And so $V(t) = V_{DD}(e^{-\frac{t-10\tau}{RC}})$ for $t \in [10\tau, 20\tau]$. Here, we also see that 10τ after the input switch, the voltage $V(t)$ again seems to reach steady state:

$$\begin{aligned} V(20\tau) = V_2(10\tau) &= V_{DD}(e^{-\frac{20\tau - 10\tau}{RC}}) \\ &= V_{DD}(e^{-10}) \\ &\approx V_{DD}(0.00004539) \\ &\approx 0. \end{aligned}$$

We can summarize the results we have just derived in Figure 3. This is one kind of behavior — when the transients are isolated from each other (because the time period is long and allows the circuit's response to the previous piecewise input to reach steady state). However there is also the case when the duration $D < \tau$ (or D is not too much greater than τ). In such a case our circuit does not have the opportunity to settle into steady state before the input changes. In such a case, we would need to calculate the exact voltage at the time our input changes to a 0 so that we could use an accurate initial condition for the second interval. Consider the case illustrated in Figure 4 where the input is only V_{DD} for a duration of one $\tau = RC$ time constant.

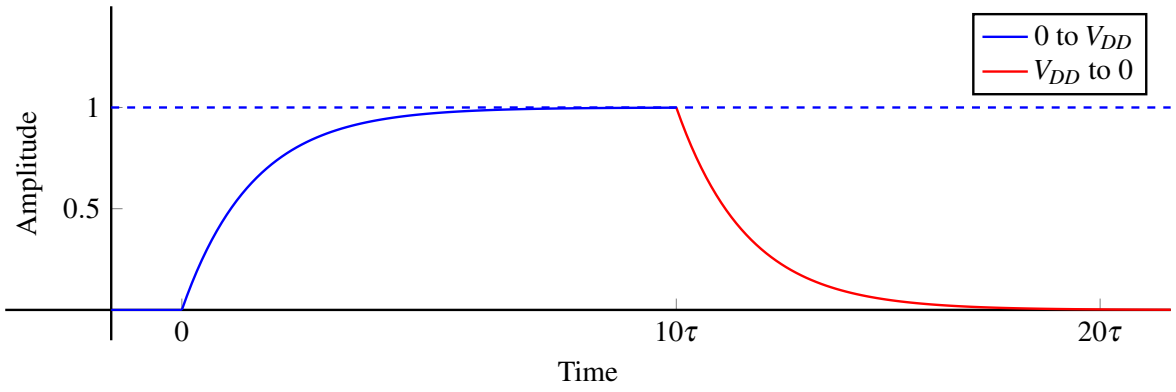


Figure 3: $V(t)$ for On and Off input: On for 10τ and then off

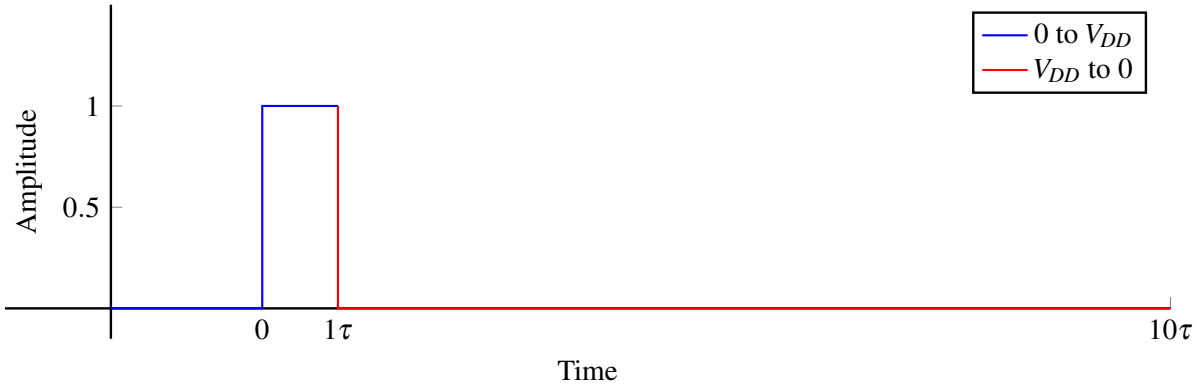


Figure 4: On and Off input: On for 1τ

Since the conditions for time $t \in [0, 1\tau]$ are the same as the case before we end up with the same equation for $V_1(t)$:

$$V_1(t_{\text{int}}) = V(t) = V_{DD}(1 - e^{-\frac{t}{RC}}) \quad t \in [0, 1\tau]$$

However, since the input V_{DD} is now only held for 1τ, the circuit does not get a chance to reach steady state before transitioning to the next stage when the input shifts from V_{DD} to 0.

$$\begin{aligned} V(1\tau) &= V_{DD}(1 - e^{-1}) \\ &\approx V_{DD}(1 - 0.36787) \\ &\not\approx V_{DD}. \end{aligned}$$

Thus, now we can no longer use V_{DD} as our initial condition. Instead, we have to now explicitly calculate our initial condition using the information we got from solving for $V_1(t_{\text{int}})$ in the first time interval. As defined above, let the function for the voltage in the second interval be $V_2(t)$ such that $V_2(t_{\text{int}}) = V(t)$ for $t \in [1\tau, 10\tau]$, where $t_{\text{int}} = t - 1\tau$. Having solved for $V_1(1\tau)$ we now have a new initial condition: $V(1\tau) = V_2(0) = V_{DD}(0.63212)$.

Solving the differential equation for the second interval and plugging in our initial condition, we get:

$$V_2(t_{\text{int}}) = V_{DD} \cdot 0.63212(e^{-\frac{t_{\text{int}}}{RC}})$$

in terms of time internal to that interval. In terms of absolute time:

$$V(t) = V_{DD} \cdot 0.63212(e^{-\frac{t-1\tau}{RC}}) \qquad t \in [1\tau, 10\tau]$$

This is illustrated in Figure 5.

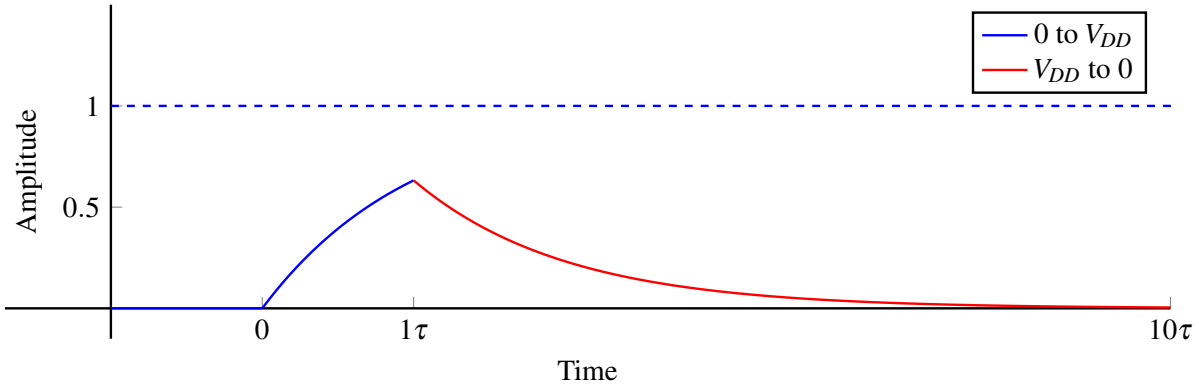


Figure 5: $V(t)$ for On and Off input: On for 1τ, Off afterwards.

2.1 More Examples and Cases

At this point, we can use what we know to analyze and understand many different examples.

2.2 Case 1: Input is at 0 and then V_{DD} Long Enough to Reach Steady State

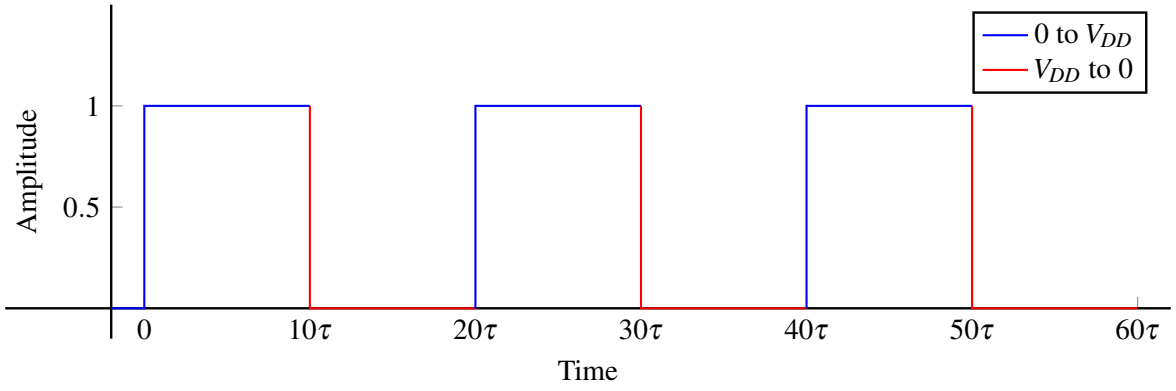


Figure 6: Case 1 Input: Input where both states reach steady state

The first case to consider is when our repeated time varying input is held at V_{DD} and then held at 0 long enough to reach steady state in both directions. This is illustrated in Figure 6. The output voltage is illustrated in Figure 7.

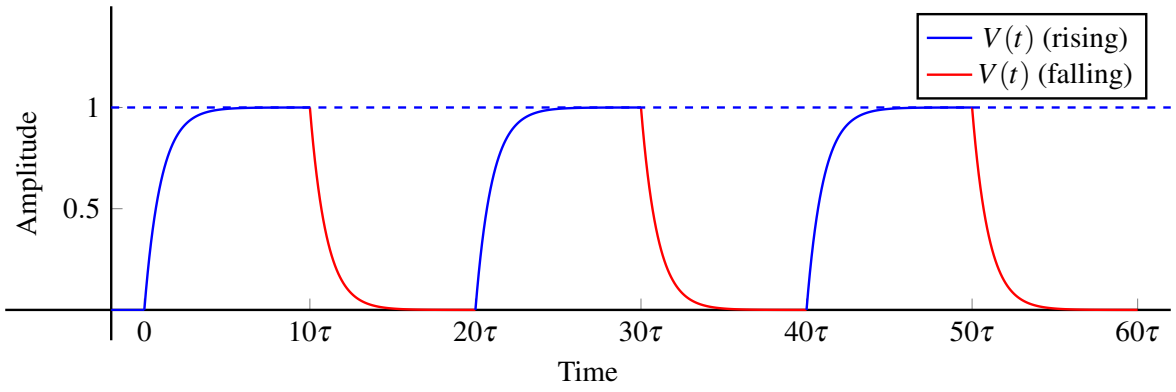


Figure 7: Case 1 Output: Transient voltage for repeated switching when both directions reach steady state.

2.3 Case 2: Input is at 0 Long Enough to Settle, and Does Not Settle at V_{DD}

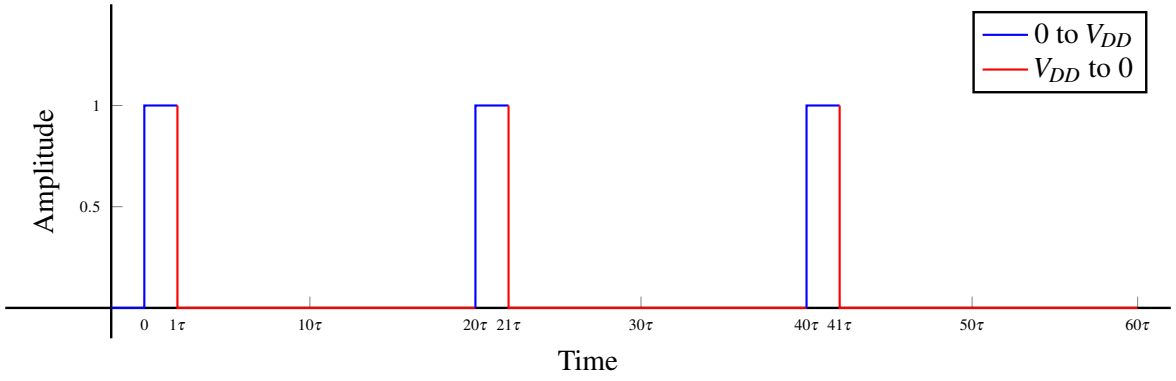


Figure 8: Case 2 Input: Input where the state settles only at 0.

The second case to consider is when our repeated time varying input is at 0 long enough to reach steady state but not at V_{DD} long enough to do so (or vice versa). This input is illustrated in Figure 8. The corresponding output voltage is illustrated in Figure 9.

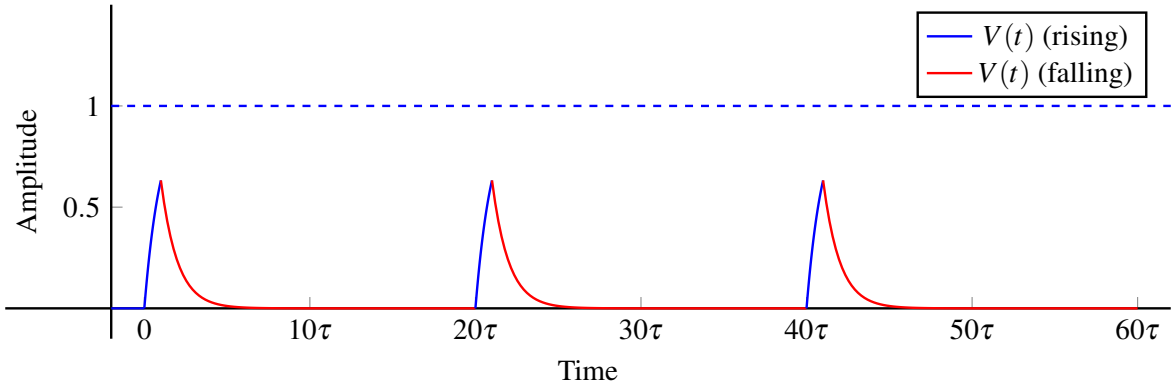


Figure 9: Case 2 Output: Output where only one state of the input settles

2.4 Case 3: The Input is Not Held Long Enough at 0 or V_{DD} Long Enough to Settle.

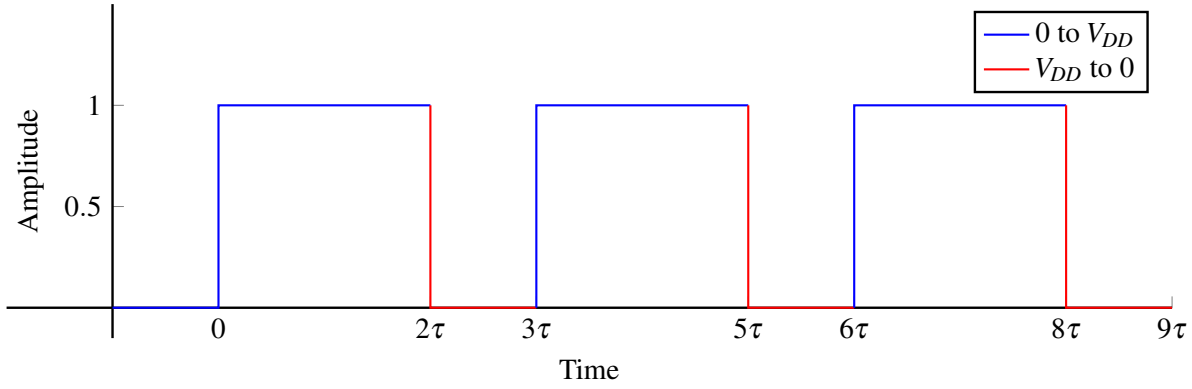


Figure 10: Case 3 Input: Input where both states of the input do not settle

The third case to consider is when our repeated time varying input is not at 0 or V_{DD} long enough to reach steady state for either extreme. This is illustrated in Figure 10. The output voltage is illustrated in Figure 11.

For this kind of case, we had no choice but to go interval by interval:

(a) Solve the differential equation to get a function for voltage changing with time.
(b) Solve for the initial condition using the previous interval's solution.
(c) Plug in the initial condition to the solution of the differential equation for the current interval.

Notice that in this case, the magnitude of the voltage on the capacitor seems to have a very slight upward trajectory (the top end of the rising edge seems to go higher and higher each time).

Can we figure out what this sawtooth shape will eventually start looking like? It will stay a sawtooth, and we know that each tooth will be 3τ long. But where will the top and bottom of the teeth be? This is an interesting exercise to think about.

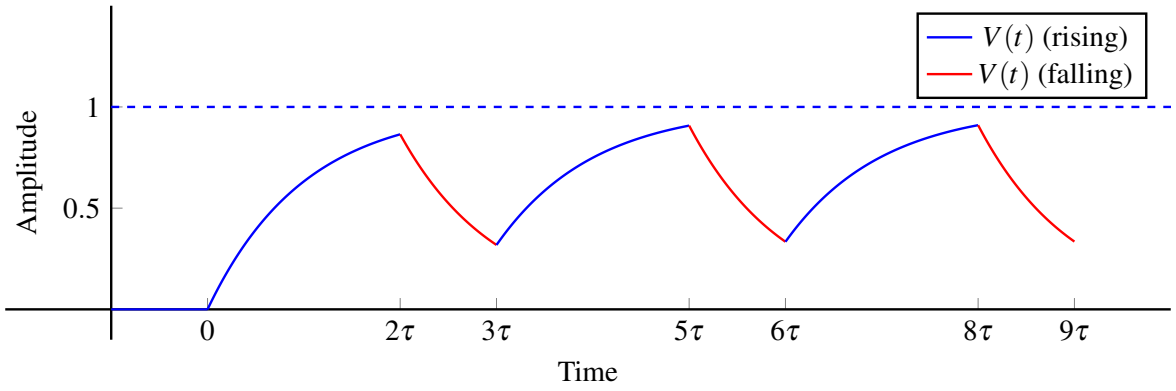


Figure 11: Case 3 Output: Transient voltage for repeated switch when both states of the input do not settle: Notice how the peak voltage goes gently up over time.

3 Building To General Inputs (Functions of Time), Not Necessarily Piecewise Constant

3.1 Guessing/Deriving a Solution for General Input Functions

Now that we know how to deal with repeated transients, we want to move towards analyzing any function of t. That is, we would like to be able to deal with a differential equation of the form:

$$\frac{d}{dt}V(t) = \lambda V(t) - \lambda u(t)$$

where $u(t)$ is any function of time. However, up until now, we have only dealt with piecewise constant inputs and repeated cases of these piecewise constants.

To analyze more complicated functions, we can start by approximating them as being piecewise constant over fixed interval widths Δ — which we know how to solve from what we have seen so far. That is, we can analyze these like repeated transients by finding new initial conditions and using those at every transition point.

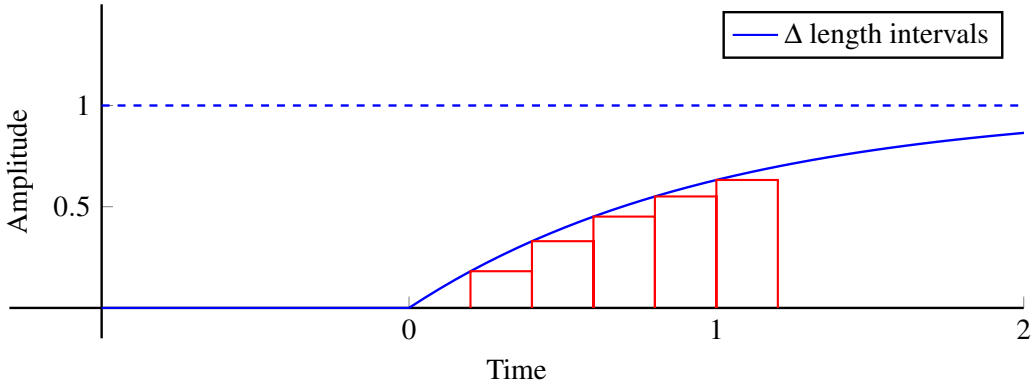


Figure 12: Our style of approximating a general function by something that is piecewise constant. This is akin to a Riemann sum.

Given some initial condition, let our approximated problem take the form of a differential equation with a piecewise constant input. Namely, for the i-th interval for $t \in (i\Delta, (i+1)\Delta]$:

$$\frac{d}{dt}V(t) = \lambda V(t) - \lambda u(i\Delta)$$

where $u(i\Delta)$ is a constant value (the value of the input function $u(t)$ at time $t = i\Delta$).

This parallels $\frac{d}{dt}V(t) = -\frac{V(t)}{RC} + \frac{V_{DD}}{RC}$ where $\lambda = -\frac{1}{RC}$, and where our input function is just the constant V_{DD} or 0 as we saw in the previous section.

Using what we know, we can solve the differential equation for this interval to get:

$$V(t_{\text{int}}) = ke^{\lambda t_{\text{int}}} + u(i\Delta).$$

where $t_{\text{int}} = t - i\Delta$ is the time internal to this interval, and the initial condition for this interval $v_i = k + u(i\Delta)$. Consequently:

$$V(t_{\text{int}}) = (v_i - u(i\Delta))e^{\lambda t_{\text{int}}} + u(i\Delta).$$

We can use the above formulation to solve for the transients over distinct intervals of width Δ. We can use this transient behavior to solve for the value of $V(t)$ at the end of the Δ long interval to get the initial condition for the next interval, and continue the process for the rest of the input function. Using this process, we can start to approximate the solutions to differential equations of the form:

$$\frac{d}{dt}V(t) = \lambda V(t) - \lambda u(t) \tag{3}$$

where $u(t)$ is some arbitrary input function. To proceed with this method, let us define some terms.

Let $V_i(t)$ be the solution of the differential equation for the i-th time interval. Let t be the absolute time starting time at 0 and let $t_{\text{int}} = t - i\Delta$ be the relative time that starts at 0 at the beginning of each interval (the i defining the i-th interval is implicit whenever we are using t_{int}). Let v_i be the initial condition for the i-th time interval and $u(i\Delta)$ (which is just a sample of our input function $u(t)$ at time $t = i\Delta$) be the constant input for the i-th time interval.

Consequently, $v_i = V_{i-1}(t_{\text{int}} = \Delta)$, and:

$$V(t) = \begin{cases} V_0(t_{\text{int}} = t) & t \in [0, \Delta] \\ V_1(t_{\text{int}} = t - \Delta) & t \in [\Delta, 2\Delta] \\ V_2(t_{\text{int}} = t - 2\Delta) & t \in [2\Delta, 3\Delta] \end{cases}$$

By the equations above, we have:

$$\begin{aligned} V_0(t_{\text{int}}) &= (v_0 - u(0))e^{\lambda t_{\text{int}}} + u(0) \\ V_1(t_{\text{int}}) &= (v_1 - u(\Delta))e^{\lambda t_{\text{int}}} + u(\Delta) \\ V_2(t_{\text{int}}) &= (v_2 - u(2\Delta))e^{\lambda t_{\text{int}}} + u(2\Delta) \end{aligned}$$

Since each interval is Δ long, the initial condition for $v_{i+1} = V_i(t_{\text{int}} = \Delta)$. As we try to evaluate $V(t)$ at a certain point, we have to repeat the process of finding the transient behavior, then using it to find the initial condition, and finally plugging in that initial condition to find the next transient behavior, over and over until we reach the time interval of interest. We can grind this out in a relatively mindless fashion:[2]

$$\begin{aligned} v_1 &= \left(v_0 - u(0)\right)e^{\lambda\Delta} + u(0) \\ V_1(t_{\text{int}}) &= \left(\left(\left(v_0 - u(0)\right)e^{\lambda\Delta} + u(0)\right) - u(\Delta)\right)e^{\lambda t_{\text{int}}} + u(\Delta) \\ v_2 = V_1(\Delta) &= \left(\left(\left(v_0 - u(0)\right)e^{\lambda\Delta} + u(0)\right) - u(\Delta)\right)e^{\lambda\Delta} + u(\Delta) \\ V_2(t_{\text{int}}) &= \left(\left(\left(\left(\left(v_0 - u(0)\right)e^{\lambda\Delta} + u(0)\right) - u(\Delta)\right)e^{\lambda\Delta} + u(\Delta)\right) - u(2\Delta)\right)e^{\lambda t_{\text{int}}} + u(2\Delta) \\ v_3 &= (((((v_0 - u(0))e^{\lambda\Delta} + u(0)) - u(\Delta))e^{\lambda\Delta} + u(\Delta)) - u(2\Delta))e^{\lambda\Delta} + u(2\Delta) \\ V_3(t_{\text{int}}) &= (((((((v_0 - u(0))e^{\lambda\Delta} + u(0)) - u(\Delta))e^{\lambda\Delta} + u(\Delta)) - u(2\Delta))e^{\lambda\Delta} + u(2\Delta)) - u(3\Delta))e^{\lambda t_{\text{int}}} + u(3\Delta) \\ v_4 &= (((((((v_0 - u(0))e^{\lambda\Delta} + u(0)) - u(\Delta))e^{\lambda\Delta} + u(\Delta)) - u(2\Delta))e^{\lambda\Delta} + u(2\Delta)) - u(3\Delta))e^{\lambda\Delta} + u(3\Delta) \\ V_4(t_{\text{int}}) &= ((((((((((v_0 - u(0))e^{\lambda\Delta} + u(0)) - u(\Delta))e^{\lambda\Delta} + u(\Delta)) - u(2\Delta))e^{\lambda\Delta} + u(2\Delta)) - u(3\Delta))e^{\lambda\Delta} \\ &\quad + u(3\Delta)) - u(4\Delta))e^{\lambda t_{\text{int}}} + u(4\Delta) \end{aligned}$$

[2] Note the way the inputs end up "coupling together" recursively, such that at some later time, the inputs applied until that time all contribute to the voltage in a predictable way.

We can then arrive at a final expression:

$$V_4(t_{\text{int}}) = v_0 e^{\lambda(4\Delta+t_{\text{int}})} + u(0)(e^{\lambda(3\Delta+t_{\text{int}})} - e^{\lambda(4\Delta+t_{\text{int}})}) + u(1\Delta)(e^{\lambda(2\Delta+t_{\text{int}})} - e^{\lambda(3\Delta+t_{\text{int}})}) + u(2\Delta)(e^{\lambda(\Delta+t_{\text{int}})} - e^{\lambda(2\Delta+t_{\text{int}})}) + u(3\Delta)(e^{\lambda(t_{\text{int}})} - e^{\lambda(\Delta+t_{\text{int}})}) + u(4\Delta)(1 - e^{\lambda(t_{\text{int}})})$$

But as we can see, chaining through the transient effects of all these constant inputs to get to some time t can be quite annoying[3]. Fortunately, there's a pattern to this that we can spot the pattern in the equations. Substituting $t_{\text{int}} = t - 4\Delta$ into the equation for the 4th interval we get:

$$V(t) = v_0 e^{\lambda(t)} + u(0)(e^{\lambda(t-\Delta)} - e^{\lambda(t)}) + u(1\Delta)(e^{\lambda(t-2\Delta)} - e^{\lambda(t-\Delta)}) + u(2\Delta)(e^{\lambda(t-3\Delta)} - e^{\lambda(t-2\Delta)}) + u(3\Delta)(e^{\lambda(t-4\Delta)} - e^{\lambda(t-3\Delta)}) + u(4\Delta)(1 - e^{\lambda(t-4\Delta)})$$

If we focus on the end of this interval $t = 5\Delta$, we can represent $1 = e^{\lambda(t-5\Delta)}$. With this substitution we can rewrite the above sum as:

$$V(t = 5\Delta) = v_0 e^{\lambda(t)} + u(0)(e^{\lambda(t-\Delta)} - e^{\lambda(t)}) + u(1\Delta)(e^{\lambda(t-2\Delta)} - e^{\lambda(t-\Delta)}) + u(2\Delta)(e^{\lambda(t-3\Delta)} - e^{\lambda(t-2\Delta)}) + u(3\Delta)(e^{\lambda(t-4\Delta)} - e^{\lambda(t-3\Delta)}) + u(4\Delta)(e^{\lambda(t-5\Delta)} - e^{\lambda(t-4\Delta)})$$

and capture the regularity using summation notation:

$$V(t = 5\Delta) = v_0 e^{\lambda(t)} + \sum_{i=0}^{4} u(i\Delta)\left(e^{\lambda(t-(i+1)\Delta)} - e^{\lambda(t-i\Delta)}\right)$$

Looking at the pattern for this sum of 4, we can extrapolate/guess this to be a sum of any $t = n\Delta$.

$$\begin{aligned} V(t = n\Delta) &= v_0 e^{\lambda(t)} + \sum_{i=0}^{n-1} u(i\Delta)\left(e^{\lambda(t-(i+1)\Delta)} - e^{\lambda(t-i\Delta)}\right) \\ &= v_0 e^{\lambda(t)} + \sum_{i=0}^{n-1} u(i\Delta) e^{\lambda(t-i\Delta)}(e^{-\lambda\Delta} - 1). \end{aligned}$$

When solving for $V(t = n\Delta)$ this way, we get an estimate of the voltage on the capacitor when the true input is not piecewise constant to begin with. But we can make this estimate better by making our Δ decrease and get infinitesimally small. Then, for any fixed actual time t, the corresponding n would go to ∞ as $\Delta \to 0$. Precisely, we can choose $\Delta = \frac{t}{n}$ and then take a limit:

$$\lim_{n\to\infty} V(t) = v_0 e^{\lambda t} + \lim_{n\to\infty} \sum_{i=0}^{n-1} u(i\Delta) e^{\lambda(t-i\Delta)}(e^{-\lambda\Delta} - 1)$$

This sum looks almost like a Reimann sum, except that it has $(e^{-\lambda\Delta} - 1)$ instead of something proportional to the small $\Delta = \frac{t}{n}$. To simplify this, let us recall the Taylor series approximation for e^x.

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

[3] Indeed, we stopped doing the nice parentheses in the middle because of how long the equation became; the logic from the first ones extends down directly but becomes cumbersome rapidly

Noticing that $\lambda\Delta$ is small, keeping the first two terms of the exponential's Taylor expansion, and plugging this into the above equation we get:

$$\begin{aligned}\lim_{n\to\infty} V(t) &\approx v_0 e^{\lambda t} + \lim_{n\to\infty}\sum_{i=0}^{n} u(i\Delta)e^{\lambda(t-i\Delta)}(1-\lambda\Delta-1)\\ &= v_0 e^{\lambda t} + \lim_{n\to\infty}\sum_{i=0}^{n} u(i\Delta)e^{\lambda(t-i\Delta)}(-\lambda\Delta)\\ &= v_0 e^{\lambda t} + \lim_{\Delta\to 0}(-\lambda)\sum_{i=0}^{\frac{t}{\Delta}} u(i\Delta)e^{\lambda(t-i\Delta)}\Delta\end{aligned}$$

The sum: $\sum_{i=0}^{\frac{t}{\Delta}} u(i\Delta)e^{\lambda(t-i\Delta)}\Delta$ reminds us of a Riemann sum from calculus. Using this knowledge we can turn the infinite summation into an integral:

$$\lim_{\Delta\to 0}\sum_{i=0}^{\frac{t}{\Delta}} u(i\Delta)e^{\lambda(t-i\Delta)}\Delta = \int_0^t u(\theta)e^{\lambda(t-\theta)}d\theta$$

This gives us the limiting solution:

$$V(t) = v_0 e^{\lambda t} - \lambda\int_0^t u(\theta)e^{\lambda(t-\theta)}d\theta$$

We made some approximations along the way, but intuitively, all of those approximations get more and more accurate as $\Delta \to 0$. So now have a generalized way for solving differential equations with any input that is a function of t! Note that in all our calculations, we did not make any assumptions about λ, or even the input being real. Thus our derivation is equally applicable to complex λ and complex inputs.

Also notice that we started off trying to solve the differential equation:

$$\frac{d}{dt}V(t) = \lambda V(t) - \lambda u(i\Delta)$$

This was simply to match the differential equation when solving for the voltage on the capacitor. We can use the same methods as above to derive a solution to the differential equation:

$$\frac{d}{dt}x(t) = \lambda x(t) + u(t)$$

and get

$$x(t) = x_0 e^{\lambda t} + \int_0^t e^{\lambda(t-\theta)}u(\theta)d\theta.$$

This, we will see discussed in detail in discussion.

3.2 Checking Our Solution

During the previous section's derivation, we might have seemed a little aggressive with approximations and limits. This is understandable. However, you have likely seen limits like the above in calculus, as well as approximations like the above in calculus. But the new concept is to see them both together; we need to check if our solution makes any sense and then understand if it is indeed correct.

3.2.1 Plug in a Known Function

In order to check our solution to the differential equation, the first thing to do is to plug in an input whose solution we already know and trust. Let us plug in a constant input that is 1 for time $t \geq 0$. Using our solution for $V(t)$ we get:

$$V(t) = v_0 e^{\lambda(t)} + (-\lambda)\int_0^t 1 e^{\lambda(t-\theta)} d\theta$$

where for our capacitor circuit. $\lambda = -\frac{1}{RC}$ and the initial condition $v_0 = 0$.

$$\begin{aligned}
V(t) &= v_0 e^{\lambda} + (-\lambda)\int_0^t 1 e^{\lambda(t-\theta)} d\theta \\
&= v_0 e^{\frac{-t}{RC}} + (\frac{1}{RC})\int_0^t 1 e^{-\frac{1}{RC}(t-\theta)} d\theta \\
&= (\frac{1}{RC})\int_0^t 1 e^{-\frac{1}{RC}(t-\theta)} d\theta \\
&= (\frac{1}{RC})(RC)\left[e^{-\frac{1}{RC}(t-\theta)}\right]_0^t \\
&= \left[e^{-\frac{1}{RC}(t-t)} - e^{-\frac{1}{RC}(t-0)}\right] \\
&= 1 - e^{-\frac{1}{RC}(t)}
\end{aligned}$$

This is exactly the equation for a charging capacitor: $V(t) = V_{DD}\left(1 - e^{\frac{-t}{RC}}\right)$ where $V_{DD} = 1$, and this is exactly what we expect with this constant input! So this makes sense. The solution also makes sense for a zero input.

3.2.2 Plug into the Original Differential Equation

We can further verify this by plugging the guessed solution $V(t) = v_0 e^{\lambda(t)} - \lambda \int_0^t u(\theta) e^{\lambda(t-\theta)} d\theta$ into the original differential equation:

$$\frac{d}{dt}V(t) = \lambda V(t) - \lambda u(t)$$

Doing so:

$$\frac{d}{dt}V(t) = \frac{d}{dt}\left[v_0 e^{\lambda t} + (-\lambda)\int_0^t u(\theta) e^{\lambda(t-\theta)} d\theta\right]$$

We can then use the fundamental theorem of calculus to compute the derivative[4]:

$$\begin{aligned}
\frac{d}{dt}V(t) &= \lambda v_0 e^{\lambda t} + (-\lambda)\left[1 e^{\lambda(t-t)} u(t) + \int_0^t u(\theta)\lambda e^{\lambda(t-\theta)} d\theta\right] \\
&= \lambda v_0 e^{\lambda t} + (-\lambda)\left[u(t) + \lambda\int_0^t u(\theta) e^{\lambda(t-\theta)} d\theta\right] \\
&= \lambda\left[v_0 e^{\lambda t} - \lambda\int_0^t u(\theta) e^{\lambda(t-\theta)} d\theta\right] - \lambda u(t).
\end{aligned}$$

[4]Recall that the fundamental theorem can be used to apply the derivative to the integral in a chain rule like fashion. We first take the derivative of the upper limit of the integral times the upper limit plugged into the inside of the integral. To this, we add the integral of the derivative of the inside of the integral. The latter term can be viewed as corresponding to bringing the derivative inside a summation. The first term corresponds to understanding that the number of terms essentially depends on t, and so the "last term" in the sum has to do with the derivative with respect to the upper limit of the integral. If you don't remember this, look up the Fundamental Theorem of Calculus in Leibniz form.

Notice that the expression within the square brackets is just $V(t) = v_0 e^{\lambda t} - \lambda \int_0^t u(\theta) e^{\lambda(t-\theta)} d\theta$ and so replacing this, we get $\frac{d}{dt}V(t) = \lambda V(t) - \lambda u(t)$. This means our guessed solution satisfies the original differential equation!

For the initial condition, $V(0) = v_0 e^{\lambda 0)} - \lambda \int_0^0 u(\theta) e^{\lambda(t-\theta)} d\theta = v_0 e^0 + 0 = v_0$, so that matches up as well.

Now that we have showed a solution to the differential equation, it is important to consider uniqueness. You will do this in your homework! The key trick is to consider the difference $z(t) = x(t) - y(t)$ of two candidate solutions $x(t)$ and $y(t)$. If you take the derivative $\frac{d}{dt}z(t)$, you will see that this must solve the differential equation $\frac{d}{dt}z(t) = \lambda z(t)$ with no input, together with the initial condition $z(0) = x(0) - y(0) = 0$. Since this differential equation has a unique solution $0e^{\lambda t} = 0$ for all $t \geq 0$, it must be the case that $z(t) = 0$ and hence $x(t) = y(t)$. So solutions must be unique. Because we have found one, we have found the only one!

3.3 Trying This Out

Using the above formula, let us try it out for some interesting inputs. Assuming we have the same differential equation: $\frac{d}{dt}V(t) = \lambda V(t) - \lambda u(t)$, let us find an expression for $V(t)$ when the input $u(t) = t^k e^{\lambda t}$ for $t \geq 0$ and some $k > -1$ with the initial condition $v_0 = 0$.

Plugging into the solution above, we get:

$$\begin{aligned} V(t) &= (-\lambda) \int_0^t \theta^k e^{\lambda\theta} e^{\lambda(t-\theta)} d\theta \\ &= (-\lambda) \int_0^t \theta^k e^{\lambda t} d\theta \\ &= (-\lambda) e^{\lambda t} \int_0^t \theta^k d\theta \\ &= (-\lambda) e^{\lambda t} t^{k+1} \frac{1}{k+1}. \end{aligned}$$

This turns out to be important later, but for now, it is just an interesting example.

Contributors:

- Neelesh Ramachandran.
- Nikhil Shinde.
- Anant Sahai.
- Aditya Arun.

EECS 16B Designing Information Devices and Systems II

Note j: Complex Numbers

1 What are Complex Numbers?

1.1 Introduction

Most students have some basic background in complex numbers ($\mathbb{C}$) from high school. The purpose of this note is to solidify this understanding. With that, let's begin with the most basic definition: $j = \sqrt{-1}$ (in most engineering discplines, we will use j, so as not confuse ourselves with current or the identity matrix I. Besides, numpy uses j as well). All complex numbers are of the form $z = a + jb$, where a is the **real part** and b is the **imaginary part**. This form is more commonly known as the **rectangular form**, and as we will see, addition in this form is very easy and akin to vector addition.

The **complex conjugate**[1] of z, represented by $\bar{z}$ (and sometimes by z^*), is defined as follows:

$$\bar{z} = \overline{(a+jb)} = a - jb \tag{1}$$

The **magnitude** of a complex number, z, is given by

$$|z| = \sqrt{a^2 + b^2} \tag{2}$$

and the **phase** is given by

$$\theta = \angle z = \text{atan2}(b, a) \tag{3}$$

Here, atan2(y,x) is a function[2] that returns the angle from the positive x-axis to the vector from the origin to the point (x,y).

1.2 Basic Operations

Let's say we have two complex numbers $z_1 = a_1 + jb_1$ and $z_2 = a_2 + jb_2$. As you may recall, addition is defined as follows:

$$z_1 + z_2 = (a_1 + a_2) + j(b_1 + b_2) \tag{4}$$

This should look very similar to the addition of vectors in 2D. Multiplication is a little more complicated, but it behaves very similarly to polynomial multiplication, except the indeterminate (i.e. the variable) is replaced by j, and we have $j^2 = -1$:

$$\begin{aligned} z_1 \times z_2 &= (a_1 + jb_1) \times (a_2 + jb_2) && (5)\\ &= a_1a_2 + jb_1a_2 + ja_1b_2 + j^2b_1b_2 && (6)\\ &= (a_1a_2 - b_1b_2) + j(a_1b_2 + a_2b_1) && (7) \end{aligned}$$

[1] $\bar{z}$ is also used in digital logic to represent 'NOT' operation

[2] See its relation to $\tan^{-1}\left(\frac{y}{x}\right)$ at https://en.wikipedia.org/wiki/Atan2.

Before we move on to division, let's see what the multiplicative inverse would look like:

$$\begin{aligned}\frac{1}{z} &= \frac{1}{a+jb} = \frac{1}{a+jb} \times \frac{a-jb}{a-jb} \\ &= \frac{a-jb}{a^2+b^2} = \frac{a}{a^2+b^2} - j\frac{b}{a^2+b^2}\end{aligned} \tag{8}$$

Above, we multiply both the numerator and denominator by $\bar{z}$. This allows us to make the denominator real, and we also observe $\bar{z} \times z = |z|^2$.

Following the same train of thought, let's define division as well:

$$\frac{z_1}{z_2} = \frac{a_1 + jb_1}{a_2 + jb_2} \tag{9}$$

$$= \frac{(a_1 + jb_1) \times (a_2 - jb_2)}{a_2^2 + b_2^2} \tag{10}$$

$$= \frac{(a_1 a_2 + b_1 b_2) - j(a_1 b_2 - a_2 b_1)}{a_2^2 + b_2^2} \tag{11}$$

In eq. (10), we subsititute the multiplicative inverse found in eq. (8), and we continue by carrying out the multiplication as defined in eq. (7).

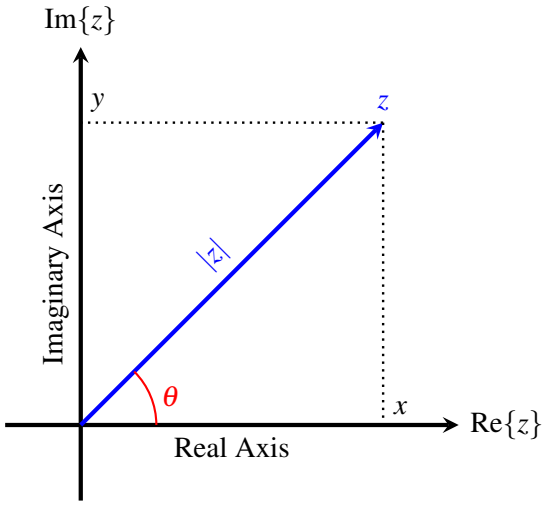


Figure 1: Complex number z depicted as a vector in the complex plane.

1.3 Complex Plane

The complex plane is an extension of the real number line; by adding another independent axis to represent the purely imaginary numbers (i.e. those with the real part equal to zero) we can represent all complex numbers. As shown in Figure 1, a complex number $z = x + jy$ has an intercept at x along the **real axis** and y along the **imaginary axis**. We can also see visually that the magnitude of z is its distance from the origin, and the phase is the angle from the positive real axis.

2 The Polar Form

2.1 Revisiting Multiplication

With our understanding of the complex plane, let's see how a complex number $z = 1 + j0$ changes as we multiply it with $z_1 = 1 + j$ repeatedly:

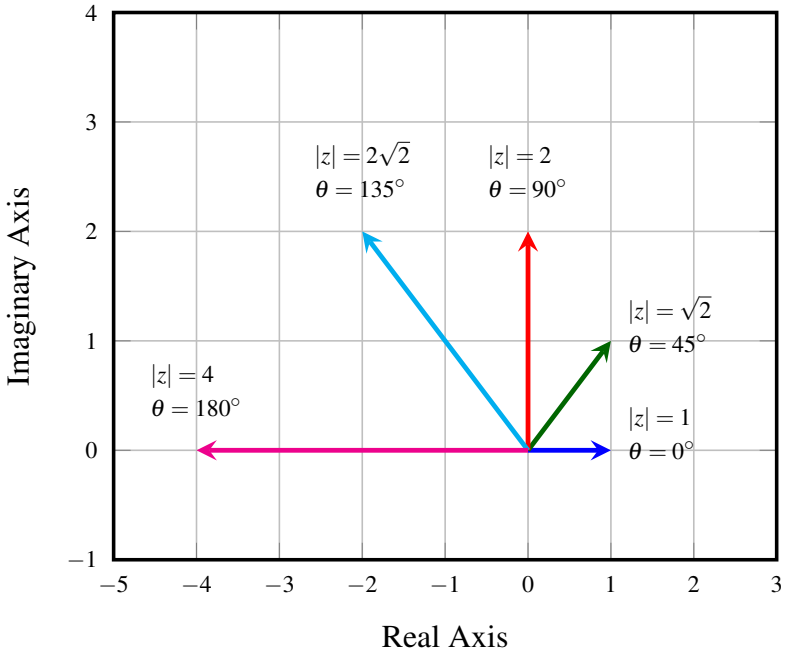


Figure 2: Multiplying z (in blue) by z_1 repeatedly

It looks like z seems to be rotating by 45°. Is this an arbitrary angle? Not really, as the angle of rotation seems to related to the phase $\angle z_1 = 45°$. So, can we think of some representation of complex numbers where this additive property of angles is natural?

How can we turn multiplication into some kind of addition? Well, we can represent the phase in an exponent. What if we used 'e' as the base for our exponent, and just set $z = e^{\angle z}$? Unfortunately, a pure real exponential will blow up as $\angle z$ increases, but this is not the behaviour we see with complex number angles. It is easy to see that any real number in the exponent couldn't possibly behave the right way — angles spin around while real exponents either go to zero or blow up. So if the term in the exponent can't be real, for now, let's take a leap of faith and hope that something imaginary works! i.e. Multiply the term in the exponent by j, i.e. $z = e^{j\angle z_2}$. We don't yet know how this will behave, but let's push forward.

The above motivation just captured the rotation property of multiplication by complex numbers. Futhermore, there is some scaling as well. More precisely, for the example of $1 + j$, it is scaling by $\sqrt{2}$. This cannot be a coincidence, so how can we express this scaling with our new form? We could multiply the magnitude of z_2 with the exponential, i.e. $z_2 = |z_2|e^{j\angle z_2}$.

Now, let's try to isolate the effect of rotation by multipling $z = 1 + j0$ by $z_2 = \frac{1}{\sqrt{2}} + j\frac{1}{\sqrt{2}}$ (note that $|z_2| = 1$) repeatedly. We get the following plot:

Clearly, we have a rotation by 45° as before, and we do not have any scaling. So, our intuition of multiplying the magnitude with the exponential agrees with this example as well. This is good news and increases our

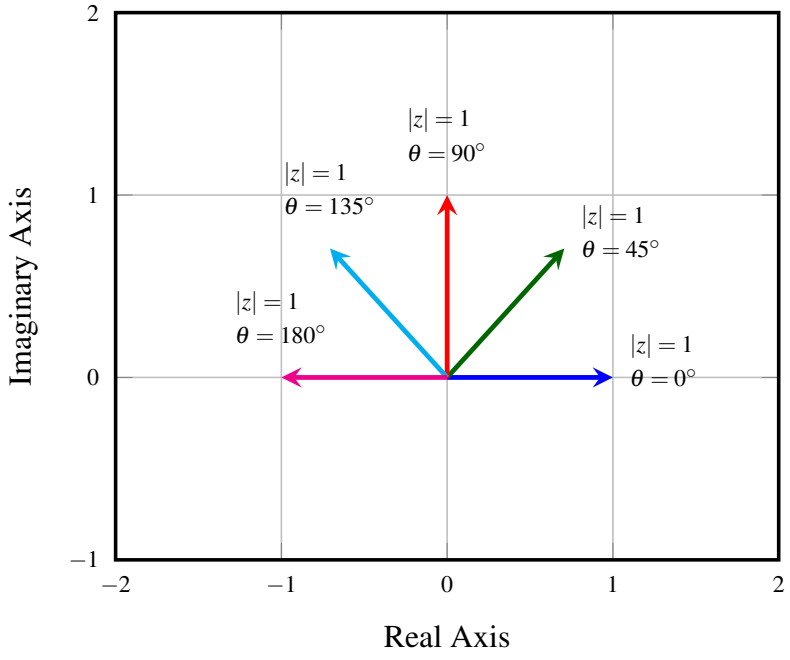


Figure 3: Multiplying z (in blue) by z_2 repeatedly

degree of comfort and confidence with our guess. But we really do need to justify it more thoroughly to be able to lean on it.

2.2 Developing the Polar Form and Euler's Equation

In the last section we conjectured that:

$$z = |z|e^{j\angle z} \tag{12}$$

But what does $e^{j\angle z}$ even mean? It is a natural first step, from our experiences, to use the exponential's definition in the form of its Taylor's expansion around 0 (or the Maclaurin's expansion of '$e^{(\cdot)}$'):

$$\begin{aligned}
z &= |z|e^{j\theta} && (13)\\
&= |z|\left(1 + j\theta + \frac{(j\theta)^2}{2!} + \frac{(j\theta)^3}{3!} + \cdots + \frac{(j\theta)^{2n}}{2n!} + \frac{(j\theta)^{2n+1}}{(2n+1)!} + \cdots\right) && (14)\\
&= |z|\left(1 + j\theta - \frac{\theta^2}{2!} - j\frac{\theta^3}{3!} + \cdots + (-1)^n\frac{\theta^{2n}}{2n!} + j(-1)^n\frac{\theta^{2n+1}}{(2n+1)!} + \cdots\right) && (15)\\
&= |z|\left[\left(1 - \frac{\theta^2}{2} + \frac{\theta^4}{4} - \cdots + (-1)^n\frac{\theta^{2n}}{2n!} + \cdots\right) + j\left(\theta - \frac{\theta^3}{3} + \cdots + (-1)^n\frac{\theta^{2n+1}}{(2n+1)!} + \cdots\right)\right] && (16)\\
&= |z|\left(\cos(\theta) + j\sin(\theta)\right). && (17)
\end{aligned}$$

Here, $\theta = \angle z$ and the angles are clearly measured in radians. At eq. (15), we used the fact that j^k has a regular periodically repeating pattern to it: $j^0 = +1$, $j^1 = +j$, $j^2 = -1$, $j^3 = -j$, $j^4 = +1$, and so it goes.

Basically we just need to look at the remainder[3] that we get after we divide k by 4.

If that remainder is 0 (i.e. k is 0 plus a multiple of 4 or, $k = 4\ell + 0$ for some integer ℓ), then $j^k = +1$.
If that remainder is 1 (i.e. k is 1 plus a multiple of 4 or, $k = 4\ell + 1$ for some integer ℓ), then $j^k = +j$.
If that remainder is 2 (i.e. k is 2 plus a multiple of 4 or, $k = 4\ell + 2$ for some integer ℓ), then $j^k = -1$.
If that remainder is 3 (i.e. k is 3 plus a multiple of 4 or, $k = 4\ell + 3$ for some integer ℓ), then $j^k = -j$.

Going from eq. (16) to eq. (17), we recognize and substitute for the Taylor expansions of sine and cosine around at 0. Let's analyze our result in eq. (17) with reference to the diagram of the complex plane in Figure 1:

Recalling basic definitions from trigonometry, we can see that $x = |z|\cos(\theta)$ and $y = |z|\sin(\theta)$, hence $z = x + jy = \cos(\theta) + j\sin(\theta)$, agreeing with our previous result. Hence, the form we guessed in the earlier section was correct and we have come back full circle, connecting with the rectangular form we discussed in Section 1.

Concept Check: Using Euler's equation:

$$e^{j\theta} = \cos(\theta) + j\sin(\theta) \tag{18}$$

write sine and cosine as sums of complex exponentials.

Solution:

$$e^{-j\theta} = \cos(-\theta) + j\sin(-\theta) = \cos(\theta) - j\sin(\theta) \tag{19}$$

Adding and Subtracting eq. (18) and eq. (19), we get:

$$2\cos(\theta) = e^{j\theta} + e^{-j\theta} \Rightarrow \cos(\theta) = \frac{e^{j\theta} + e^{-j\theta}}{2} \tag{20}$$

$$2j\sin(\theta) = e^{j\theta} - e^{-j\theta} \Rightarrow \sin(\theta) = \frac{e^{j\theta} - e^{-j\theta}}{2j} = \frac{-je^{j\theta} + je^{-j\theta}}{2} \tag{21}$$

This is the source of the famous identity $e^{j\pi} + 1 = 0$, connecting five very fundamental numbers together: 0 (the additive identity), 1 (the multiplicative identity), e (the base of the natural logarithm, defined because we want a function whose derivative was itself), j (the basic imaginary number $\sqrt{-1}$), and π (the area of a perfect circle with radius 1). Such remarkable beauty is what marks the subject of complex analysis[4] more generally.

2.3 Conclusion

The polar form of z is a very important representation as it greatly simplifies multiplication. The polar form of $z = a + jb$ is given as follows:

$$z = |z|e^{j\angle z}$$

[3] Such remainder operations are often called mod operations, and they play a major role in our follow-on course 70. But in 16B, you will begin to be prepared for thinking about such cyclic behavior because the complex numbers exhibit it very naturally.

[4] The historical tradition within Electrical Engineering has traditionally placed a lot of emphasis on complex analysis alongside real analysis and linear algebra. You will see some evidence of this in later courses like 120 and 128, and even in courses like 105, 140, and 142. However, the 16AB sequence is almost completely rooted in linear algebra because linear algebra is far simpler, and while sometimes lacking the delicate beauty of complex analysis, it can be more robust. At advanced levels, you need to know both. There is a reason why Complex Analysis 185 is considered an absolutely core course for Math majors and minors, along with Real Analysis 104, Linear Algebra 110, and Abstract Algebra 113. In the EECS lower division, 16AB hit basic linear algebra, 16B touches on some ideas from real analysis and only skirts vague hints of complex analysis, and 70 touches on some ideas from abstract algebra. We encourage our students to take the core upper division Math courses for a deeper understanding and appreciation for these foundational conceptual tools.

The conjugate of z is given as $\overline{z} = |z|e^{-j\angle z}$ since $\sin(-\theta) = -\sin(\theta)$ while $\cos(-\theta) = \cos(\theta)$ from trigonometry.

Multiplication in this form is given as:

$$z_1 \times z_2 = (|z_1|e^{j\angle z_1}) \times (|z_2|e^{\angle z_2}) = (|z_1| \cdot |z_2|)e^{j(\angle z_1 + \angle z_2)}$$

If we look carefully, we can realize that complex multiplication is nothing but a rotation operation, followed by scaling. We are essentially rotating z_1 by $\angle z_2$ in the counter-clockwise direction[5], and scaling it by a factor of $|z_2|$. Of course, this conclusion would have been difficult to make precise without the use of polar forms. We will solidfy this view of rotations further in the next section where we will model complex numbers as matrices to augment the vector intuition we have gained thus far.

2.4 Useful Identities

Complex Number Properties

Rectangular vs. polar forms: $z = x + jy = |z|e^{j\theta}$

where $|z| = \sqrt{z\overline{z}} = \sqrt{x^2+y^2}$, $\theta = \text{atan2}(y,x)$. We can also write $x = |z|\cos\theta$, $y = |z|\sin\theta$.

Euler's identity: $e^{j\theta} = \cos\theta + j\sin\theta$

$$\sin\theta = \frac{e^{j\theta} - e^{-j\theta}}{2j},\ \cos\theta = \frac{e^{j\theta} + e^{-j\theta}}{2}$$

Complex conjugate: $\overline{z} = x - jy = |z|e^{-j\theta}$

$$\overline{(z+w)} = \overline{z} + \overline{w},\ \overline{(z-w)} = \overline{z} - \overline{w}$$

$$\overline{(zw)} = \overline{z}\,\overline{w},\ \overline{\left(\frac{z}{w}\right)} = \frac{\overline{z}}{\overline{w}}$$

$\overline{z} = z \Leftrightarrow z$ is real

$\overline{z} = -z \Leftrightarrow z$ is purely complex, i.e. no real part

$$\overline{(z^n)} = (\overline{z})^n$$

Complex Algebra

Let $z_1 = x_1 + jy_1 = |z_1|e^{j\theta_1}$, $z_2 = x_2 + jy_2 = |z_2|e^{j\theta_2}$.

(Note: we adopt the easier representation between rectangular form and polar form.)

Addition: $z_1 + z_2 = (x_1 + x_2) + j(y_1 + y_2)$

Multiplication: $z_1 z_2 = |z_1||z_2|e^{j(\theta_1+\theta_2)}$

Division: $\frac{z_1}{z_2} = \frac{|z_1|}{|z_2|}e^{j(\theta_1-\theta_2)}$

Power: $z_1^n = |z_1|^n e^{jn\theta_1}$

$$z_1^{\frac{1}{2}} = \pm|z_1|^{\frac{1}{2}}e^{j\frac{\theta_1}{2}}$$

(Note: Just like square roots are not unique, other fractional powers of z_1 are not unique as well)

Useful Relations

$$-1 = j^2 = e^{j\pi} = e^{-j\pi}$$

$$j = e^{j\frac{\pi}{2}} = \sqrt{-1}$$

$$-j = -e^{j\frac{\pi}{2}} = e^{-j\frac{\pi}{2}}$$

$$\sqrt{j} = (e^{j\frac{\pi}{2}})^{\frac{1}{2}} = \pm e^{j\frac{\pi}{4}} = \frac{\pm(1+j)}{\sqrt{2}}$$

Concept Check: Verify the above identities for yourself if you have not done so in prior classes.

3 Complex Numbers Represented As Matrices (OPTIONAL, not in scope)

Viewing complex numbers as vectors definitely seems attractive and it does fit into our visualization of the complex plane, but it has a major flaw — vectors do not naturally multiply, but complex numbers do. In fact, multiplication is the *raison d'etre* for complex numbers. So, how do we get a better model? What both adds and multiplies? Enter matrices, and more specifically scaled rotation matrices.

[5] It is common to define the counter-clockwise direction as the positive direction while measuring angles in the Cartesian plane.

3.1 Matrix form of rotations

But first, what is a rotation matrix? To begin answering this question, we need to first understand what a rotation tranformation would look like. Rotating the vector $\vec{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ by angle θ in the counter clockwise direction would give us $\vec{\tilde{e}}_1 = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}$, and similarly for $\vec{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, we will get $\vec{\tilde{e}}_2 = \begin{bmatrix} -\sin(\theta) \\ \cos(\theta) \end{bmatrix}$. Hence, we can describe the rotation transform (by angle θ) as the following matrix:

$$\vec{\tilde{v}} = R_\theta \vec{v} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \vec{v} \tag{22}$$

An important thing to note is this rotation matrix has orthonormal columns[6]. Next, what would happen if we rotated a vector by θ_1 and then by θ_2? Well, it would be equivalent to rotating it by $\theta_1 + \theta_2$, hence we have:

$$R_{\theta_1} \times R_{\theta_2} = \begin{bmatrix} \cos(\theta_1) & -\sin(\theta_1) \\ \sin(\theta_1) & \cos(\theta_1) \end{bmatrix} \begin{bmatrix} \cos(\theta_2) & -\sin(\theta_2) \\ \sin(\theta_2) & \cos(\theta_2) \end{bmatrix} = \begin{bmatrix} \cos(\theta_1+\theta_2) & -\sin(\theta_1+\theta_2) \\ \sin(\theta_1+\theta_2) & \cos(\theta_1+\theta_2) \end{bmatrix} = R_{\theta_1+\theta_2} \tag{23}$$

Concept Check: Use basic trigonometry (in particular, the sum-angle formulas for sine and cosine that you probably derived in high school) to check the equality established in eq. (23). Futhermore, rotations in 2D are commutative[7] as well. Show that this is true by proving $R_{\theta_1} \times R_{\theta_2} = R_{\theta_2} \times R_{\theta_1}$

When we look back at rotation matrix in eq. (22), it bears some resemblance to the Euler form (eq. (17)) we discovered in the previous section. If we have a complex number $z = a + jb = \cos(\theta) + j\sin(\theta)$, where $|z| = 1$ (for simplicity, we will look at scaling a bit later) and $\angle z = \theta$, then we could define a matrix $Z_{(a,b)}$ as follows:

$$Z_{(a,b)} = \begin{bmatrix} a & -b \\ b & a \end{bmatrix} \tag{24}$$

Concept Check: Check that this matrix has orthogonal columns.

We can express the fundamentally two-dimensonal nature of such matrices by expressing them using a clear basis:

$$Z_{(a,b)} = aI + bJ \text{ where } I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \text{ and } J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}. \tag{25}$$

Notice that $J^2 = -I$ above, and so the matrix J acts like the counterpart of the basic imaginary number j.

What is a complex conjugate in this representation? What can we do to swap the b and $-b$ in the matrix above? Indeed we see that **transposing** the matrix corresponds to complex conjugation of the underlying complex number. It has no effect on a scaled identity matrix which would correspond to a purely real number. But $J^T = -J$.

[6] The columns in a matrix with orthonormal columns all have norm 1 and are mutually orthogonal to each other (i.e. their inner products with each other are zero). Such matrices are commonly referred to as **orthogonal** matrices in the mathematical literature.

[7] This commutative property for rotations only holds for 2D spaces, and not for 3D spaces. Take a second to think about this!

Next, let's look at the scaling. In this case, we have $z = a + jb$, with $|z| = \sqrt{a^2+b^2}$. To account for this in our matrix model, we can factor out $|z|$ as follows:

$$Z_{a,b} = \sqrt{a^2+b^2}\begin{bmatrix} \frac{a}{\sqrt{a^2+b^2}} & -\frac{b}{\sqrt{a^2+b^2}} \\ \frac{b}{\sqrt{a^2+b^2}} & \frac{a}{\sqrt{a^2+b^2}} \end{bmatrix} \tag{26}$$

Looking at the above form, the factor out in the front is responsible for the scaling. From the figure below, we can find $\theta = \text{atan2}(b,a)$ such that $\cos(\theta) = \frac{a}{\sqrt{a^2+b^2}}$ and $\sin(\theta) = \frac{b}{\sqrt{a^2+b^2}}$.

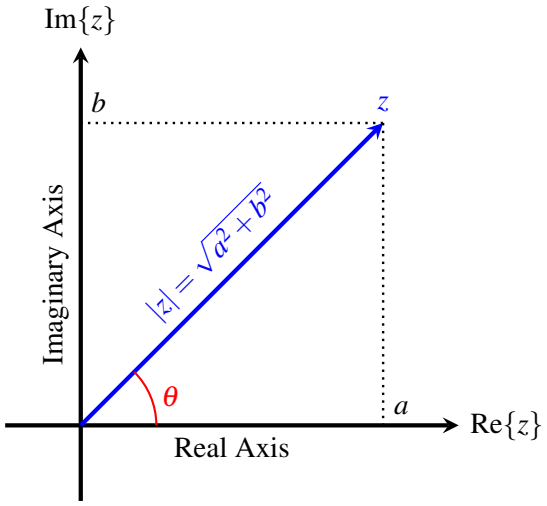


Figure 4: Complex number $z = a + jb$ represented as a vector in the complex plane.

Now, let's see if this model fits with everything that we know about complex arithmetic.

3.1.1 Addition:

For two complex numbers, $z_1 = a_1 + jb_1$ and $z_2 = a_2 + jb_2$, we have:

$$Z_{(a_1,b_1)} + Z_{(a_2,b_2)} = \begin{bmatrix} a_1+a_2 & -(b_1+b_2) \\ b_1+b_2 & a_1+a_2 \end{bmatrix} = Z_{(a_1+a_2,b_1+b_2)}$$

Hence, it satisfies our definition of addition.

3.1.2 Multiplication by real number:

Let $z = a + jb$, then $\lambda z = \lambda a + j\lambda b$, where λ is a real number. This can be easily extended to our matrix form as well:

$$\lambda Z_{(a,b)} = \lambda \begin{bmatrix} a & -b \\ b & a \end{bmatrix} = \begin{bmatrix} \lambda a & -\lambda b \\ \lambda b & \lambda a \end{bmatrix} = Z_{(\lambda a,\lambda b)}$$

3.1.3 Multiplication by a complex number:

Finally, and the reason we are pursuing this representation, multiplication by another complex number. Let $z_1 = a_1 + jb_1$ and $z_2 = a_2 + jb_2$, then we have $z_1 \times z_2 = (a_1a_2 - b_1b_2) + j(a_1b_2 + a_1b_1)$. Let's check if this

is the case with matrix multiplication:

$$Z_{(a_1,b_1)} \times Z_{(a_2,b_2)} = \begin{bmatrix} a_1 & -b_1 \\ b_1 & a_1 \end{bmatrix} \begin{bmatrix} a_2 & -b_2 \\ b_2 & a_2 \end{bmatrix} = \begin{bmatrix} a_1a_2 - b_1b_2 & -(a_1b_2 + a_2b_1) \\ a_1b_2 + a_2b_1 & a_1a_2 - b_1b_2 \end{bmatrix} = Z_{(a_1a_2 - b_1b_2, a_1b_2 + a_2b_1)}$$

Note that since our 2D rotations are communtative, so are the multiplications with complex numbers.

Multiplication by the complex conjugate also clearly gives an identity matrix with the magnitude squared along the diagonal.

It turns out, although we will not show this here, that even the natural generalization of exponentiation to matrices, called the matrix exponential, works with this matrix representation for complex numbers. This allows Euler's formula $e^{a+jb} = e^a e^{jb} = e^a(\cos b + j \sin b)$ to still apply for this model.

Contributors:

- Aditya Arun.
- Anant Sahai.
- Nikhil Shinde.
- Neelesh Ramachandran.

EECS 16B Designing Information Devices and Systems II

Note 3A: Vector Differential Equations

1 Matrix Form

1.1 Introduction

Last time we saw how to solve simple scalar first order linear differential equations.

Now, we are going to expand our tool set and learn how to tackle multivariable differential equations. (You will see in homeworks that these exact same ideas will also equip us to deal with higher-order differential equations where we have second and third derivatives involved.) Let's motivate the need to understand two dimensional systems with the following circuit.

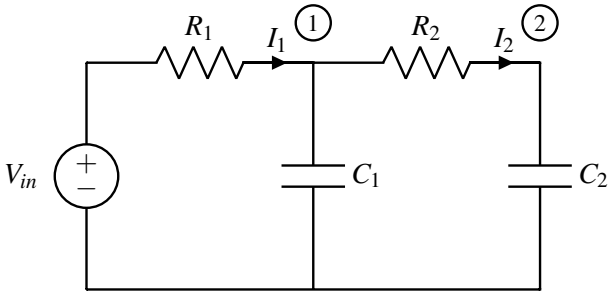


Figure 1: Two dimensional system

Let $C_1 = C_2 = 1\mu\text{F}$, $R_1 = \frac{1}{3}\text{M}\Omega$ and $R_2 = \frac{1}{2}\text{M}\Omega$.[1]

Concept Check: Before we begin, how would you solve for the above system if $R_2 = 0$?

Answer: If $R_2 = 0$, then the C_1 and C_2 are connected in parallel and can be combined into a single effective capacitance of $C_1 + C_2$. This is like what happened earlier when we had two gate capacitances in parallel.

As in Note 1, let us first consider the discharging case. In the above system, let $V_{\text{in}} = 1V$ for time $t < 0$, and $V_{\text{in}} = 0V$ for time $t \geq 0$. With this in mind, we have two steady state conditions:

(a) Initial condition $t = 0$: The voltage has been charging the capacitors for an infinite amount of time. Hence, both capacitors have voltage $V_{C_1} = V_{C_2} = 1\text{V}$ and current $I_1 = I_2 = 0\text{A}$.

(b) As $t \to \infty$: After the capacitors have been allowed to discharge for a long period of time, they carry no charges on their plates, hence $V_{C_1} = V_{C_2} = 0\text{V}$.

Next, let us solve for the transients, i.e. how does our system go from (a) to (b)? First we need to set up the circuit equations. We will define $V_1 = V_{C_1}$ and $V_2 = V_{C_2}$ as the voltages across our capacitors.

[1] The SI prefixes 'M' and 'μ' stand for "mega" and "micro," and correspond to the decimal multiples of 10^6 and 10^{-6} respectively.

$$V_2 = V_1 - I_2 R_2 \tag{1}$$

$$I_2 = C_2 \frac{d}{dt} V_2 \tag{2}$$

$$0 - V_1 = I_1 R_1 \Rightarrow I_1 = -\frac{V_1}{R_1} \tag{3}$$

$$I_1 = I_2 + C_1 \frac{d}{dt} V_1 \tag{4}$$

For eq. (1) and eq. (2), we look at the current through Node (2), using Ohm's law for the former and voltage-current relation for the capacitor in the latter. Similarly, we find eq. (3) and eq. (4) by looking at Node (1). Similar to what we had seen in a single capacitor system, we have effectively introduced two new variables - $\frac{d}{dt}V_1$ and $\frac{d}{dt}V_2$. Next, to solve for the transients, we need to first define our system variables. **The standard approach that we will always take is to make anything that gets differentiated into a state variable.** Hence, we will need two state variables, V_1 and V_2.

1.2 Systems of Differential Equations

We get a system of differential equations by isolating the derivative terms and solving for them in terms of their non-differentiated selves. You can view this as doing the downward pass of Gaussian Elimination using an ordering of the variables so that the $\frac{d}{dt}\cdot$ variables come second to last and their non-differentiated counterparts come last. Because there will be more unknowns than equations, Gaussian elimination will stop on the last $\frac{d}{dt}$-ed variable. Then, we can do a limited upward-pass (back-substitution pass) of Gaussian elimination to purge any dependence of one $\frac{d}{dt}$-ed variable on any other $\frac{d}{dt}$-ed variable. This gives rise to the system of differential equations in a systematic way — it is the lower block that remains after doing this back-substitution. In principle, every other circuit quantity of interest can be found once we know how the state variables and their derivatives behave.

$$\frac{d}{dt}V_1(t) = -\left(\frac{1}{R_1 C_1} + \frac{1}{R_2 C_1}\right) V_1(t) + \frac{V_2(t)}{R_2 C_1} \tag{5}$$

$$\frac{d}{dt}V_2(t) = \frac{V_1(t)}{R_2 C_2} - \frac{V_2(t)}{R_2 C_2} \tag{6}$$

The steps outlining how we obtained eq. (5) and eq. (6) are in section 4.

The fact that every linear circuit involving capacitors (and as we will see inductors as well) can eventually be expressed into this form is a close mathematical relative of the Norton/Thevenin equivalence that we saw earlier in the course (or in 16A). Both are effectively just consequences of Gaussian Elimination.

Concept Check: Take a second to work out setting up the above differential equations. It is important to try to reduce all your branch equations from the circuit analysis to as few variable as possible.

As seen in EE16A, when encountering a system of equations, we try to put it into vector/matrix form to try

to solve it. So, let's put eq. (5) and eq. (6) in the matrix differential form. We define $\vec{x}(t) = \begin{bmatrix} V_1(t) \\ V_2(t) \end{bmatrix}$, and:

$$\begin{bmatrix} \frac{dV_1}{dt} \\ \frac{dV_2}{dt} \end{bmatrix} = \begin{bmatrix} -\left(\frac{1}{R_1C_1} + \frac{1}{R_2C_1}\right)V_1 + \frac{V_2}{R_2C_1} \\ \frac{V_1}{R_2C_2} - \frac{V_2}{R_2C_2} \end{bmatrix} \tag{7}$$

$$= \begin{bmatrix} -\left(\frac{1}{R_1C_1} + \frac{1}{R_2C_1}\right) & \frac{1}{R_2C_1} \\ \frac{1}{R_2C_2} & -\frac{1}{R_2C_2} \end{bmatrix} \begin{bmatrix} V_1(t) \\ V_2(t) \end{bmatrix} \tag{8}$$

$$\frac{d}{dt}\vec{x}(t) = \begin{bmatrix} -5 & 2 \\ 2 & -2 \end{bmatrix} \vec{x}(t) \tag{9}$$

In eq. (9), we have substituted in for the component values defined above so that we get a matrix with concrete numbers in it.

Quick Aside: You may come across a dot over the variable of interest (e.g. $\dot{V}_1$) as short hand to represent the $\frac{d}{dt}$ operator. This is alternatively known as Newton's notation and is sometimes used for conciseness, especially in fields influenced by Physics notation. Similarly, we sometimes define $\ddot{V} \equiv \frac{d^2}{dt^2}V$ to make it faster to write second derivatives. However, we won't be using Newton's notation in this course.

In general, we want to set up our systems in the follwing generic form:

$$\frac{d}{dt}\vec{x}(t) = A\vec{x}(t) + \vec{b} \tag{10}$$

So, why do we choose this form? Because it most closely resembles the first order scalar differential equation we studied in the previous note, and our goal in this note is to massage this equation to convert it to a collection of first order differential equations. In context of our above example, $A = \begin{bmatrix} -5 & 2 \\ 2 & -2 \end{bmatrix}$ and $\vec{b} = \vec{0}$ because there is no external voltage or current being applied for $t \geq 0$ in the above example.

2 Diagonalization by means of an eigenbasis

2.1 Eigenvalues and Eigenvectors

Before we delve into solving our system of differential equations, let's review a core concept from EECS16A - eigenvalues and eigenvectors. Please note this will be an abridged version of the content covered in EECS16A. If you have further questions regarding any of these concepts, please review the notes from EECS16A.

What are **eigenvalues** and **eigenvectors**? An eigenvector is a vector that is only scaled by a value (the corresponding eigenvalue) when acted upon by a matrix, i.e.

$$A\vec{v}_\lambda = \lambda\vec{v}_\lambda \tag{11}$$

Here, $\vec{v}_\lambda \neq \vec{0}$ is an eigenvector of A which corresponds to the scalar λ eigenvalue. If we look at all the eigenvectors of the matrix A corresponding to a single λ, these together form a subspace known as the

λ-eigenspace. Each distinct eigenvalue is associated with its own nontrivial eigenspace[2]. We can see this subspace property by rearranging the above equation into:

$$(A - \lambda I_n)\vec{v} = \vec{0}. \tag{12}$$

Here, I_n is the $n \times n$ identity matrix. Since $A\vec{v}_\lambda = \lambda\vec{v}_\lambda$, we know that the matrix $A - \lambda I_n$ has a nullspace (namely the eigenspace corresponding to the eigenvalue λ of A). Consequently, this matrix $(A - \lambda I_n)$ must be non-invertible and thus $\det(A - \lambda I_n) = 0$. Remember, the determinant of a square matrix measures the oriented volume[3] of the unit hypercube as transformed by that matrix. Non-invertible matrices destroy information by squashing the space along at least one direction (the nullspace directions), and so result in something with volume zero.

This gives us the **characteristic equation**[4] and solving this equation will give us the eigenvalues λ. As a working example, let's compute the eigenvalues of $A = \begin{bmatrix} 1 & 2 \\ 4 & 3 \end{bmatrix}$:

$$\det(A - \lambda I_2) = \det\left(\begin{bmatrix} 1 & 2 \\ 4 & 3 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}\right) = \det\left(\begin{bmatrix} 1-\lambda & 2 \\ 4 & 3-\lambda \end{bmatrix}\right) \tag{13}$$

Taking the determinant and setting it to zero, we get the characteristic equation:

$$0 = (1-\lambda)(3-\lambda) - (4)(2) = \lambda^2 - 4\lambda - 5 = (\lambda - 5)(\lambda + 1) \tag{14}$$

Setting eq. (14) to 0 and solving the quadratic equation, we get the eigenvalues as $\lambda_1 = 5$ and $\lambda_2 = -1$. Each eigenvalue will have it's own eigenspace, and it will define the nullspace of the matrix $A - \lambda I_n$. Hence to find the eigenspace, we can just find the relevant nullspace[5]

$$(A - \lambda_1 I_2)\vec{v}_1 = (A - 5I_n)\vec{v}_1 = \begin{bmatrix} -4 & 2 \\ 4 & -2 \end{bmatrix}\begin{bmatrix} v_{11} \\ v_{12} \end{bmatrix} = \vec{0} \tag{15}$$

We see that both rows provide redundant information - $4v_{11} - 2v_{12} = 0$, and hence an eigenvector is $\vec{v}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$.

[2] In 16A, you also saw some interesting properties of these eigenspaces. For example, any collection of nonzero vectors drawn from eigenspaces corresponding to distinct eigenvalues is linearly independent. This is proved by contradiction—it is easy to see for two distinct eigenvalues. (A vector can't simultaneously be two different multiples of the same thing!) From there, the proof continues to three vectors. If one is expressible as a combination of the others, then those two must be expressible as a multiple of each other, which we have already ruled out. And so on. If you don't remember this proof, it is worth doing for yourself. Anyway, as a result, there cannot be more than n distinct eigenvalues for an $n \times n$ matrix since each one must be associated with its own eigenspace, and there can only be n linearly independent directions in an n-dimensional space.

[3] Recall how the determinant is computed by doing Gaussian elimination. Multiplication of a row by -1 incurs a factor of -1 because it is like looking through a mirror—a mirror flips orientation. Every time we swap two rows, we also incur a factor of -1—this is because swapping two rows is like rotating and then looking through a mirror. Rotating doesn't change volume, but the mirror flips the orientation. Adding a multiple of one row to another does nothing to the oriented volume because it is like shearing a cube—just as pushing over a deck of cards doesn't change the volume of the deck, shearing doesn't change volume either. Scaling a row by a positive number has that scaling effect on the volume—and so by reversing the scalings done in Gaussian Elimination (which gets us to the Identity), we can get the volume effect of the original matrix.

[4] The characteristic equation is often also referred to as the characteristic polynomial

[5] You know from EECS 16A how you can find nullspaces systematically using Gaussian Elimination. Simply run Gaussian elimination until it terminates—you will have at least one row of zeros. Then work your way backward from the end finding free variables (variables that you did not "pivot" on — i.e. variables that you never explicitly tried to eliminate from lower equations—these are variables that got eliminated on their own!) and then expressing others in terms of them. This will give you a basis for the nullspace.

Concept Check: Find an eigenvector $\vec{v}_2$ correspoding to λ_2.

Answer: A second eigenvector is $\vec{v}_2 = \alpha \begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Note that any $\alpha \neq 0$ would still be a valid eigenvector.

2.2 Repeated and Complex Eigenvalues

2.2.1 Repeated Eigenvalues

For a 2×2 matrix, it's possible that the two eigenvalues that you end up with have the same value, leading to a phenomenon called a **repeated eigenvalues**. This repeated eigenvalue can have one or two dimensional eigenspace (unlike a single, unrepeated eigenvalue, which will only have a one dimensional eigenspace).

For example, the following matrix has a repeated eigenvalue of λ.

$$A = \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}$$

The λ**-eigenspace** of this matrix is all of $\mathbb{R}^2$ since for any vector $\vec{v} \in \mathbb{R}^2$, $A\vec{v} = \lambda\vec{v}$.

We can also have examples like

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

that have a single eigenvalue $\lambda = 0$. (Easy to see by looking at the characteristic equation $\lambda^2 = 0$.) In this case, the relevant eigenspace is one-dimensional—only $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and its multiples are eigenvectors here. (If there were any other linearly independent eigenvectors corresponding to the 0 eigenvalue, then everything would have to be an eigenvector and that would mean the matrix mapped everything to the zero vector. But the matrix is clearly not the zero matrix, so that isn't what is going on.)

2.2.2 Complex Eigenvalues

It can also be the case that when we solve $\det(A - \lambda I_n) = 0$, there will be no real solutions to λ. Consider the rotation matrix (described in Note j):

$$R_\theta = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \tag{16}$$

Concept Check: When does R_θ have real eigenvalues? Why? **Answer:** For $\theta = 0^\circ$ or $\theta = 180^\circ$.

However, when θ does not correspond to 0° or 180° rotation, there are no real vectors that are scaled versions of themselves after the transformation. This results in complex eigenvalues. For example, let's look at 45° rotation:

$$R = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

$$\det(R - \lambda I) = \frac{1}{2}(1-\lambda)(1-\lambda) + \frac{1}{2}$$

Setting this determinant equal to zero and solving yields the complex eigenvalues, $\lambda = \frac{1}{\sqrt{2}}(1+j)$ and $\lambda = \frac{1}{\sqrt{2}}(1-j)$, which makes sense because there are no physical (real) eigenvectors for a rotation transformation in two dimensions.

2.3 Change of Basis and Diagonalization

Let's start with a vector $\vec{v} \in \mathbb{R}^n$. This vector represents a point in space. When you think about this vector written out using the coordinates, you are scaling the vectors in the standard basis (i.e. the columns of the identity matrix, I_n) by the components of $\vec{v}$ and then adding them up:

$$\vec{v} = I_n\vec{v} = v_1\begin{bmatrix}1\\0\\\vdots\\0\end{bmatrix} + v_2\begin{bmatrix}0\\1\\\vdots\\0\end{bmatrix} + \cdots + v_n\begin{bmatrix}0\\0\\\vdots\\1\end{bmatrix}$$

But, suppose that I think about this vector in terms of a different set of directions. We can define a new coordinate system using a basis, i.e. n linearly independent vectors $\vec{b}_1, \vec{b}_2, \ldots \vec{b}_n$. Now, let's say I have a vector $\vec{u}$ which I am representing as $\begin{bmatrix}u_1\\u_2\\\vdots\\u_n\end{bmatrix}$ measuring with respect to my coordinate system. How can I translate this to the coordinates you are familiar with? Well, instead of scaling the vectors of the standard basis, we are now scaling the vectors defining my new basis. Assuming that both of us were thinking of the same physical point in space, then the vector $\vec{v}$ in the standard basis is:

$$\vec{v} = u_1\vec{b}_1 + u_2\vec{b}_2 + \cdots + u_n\vec{b}_n \tag{17}$$

$$= \begin{bmatrix}| & | & & |\\ \vec{b}_1 & \vec{b}_2 & \cdots & \vec{b}_n\\ | & | & & |\end{bmatrix}\begin{bmatrix}u_1\\u_2\\\vdots\\u_n\end{bmatrix} \tag{18}$$

$$= B\vec{u} \tag{19}$$

Hence, to tranform a vector from my basis to your standard basis involves just a matrix multiplication. This can be expressed in a diagram given in Figure 2 using the up and down arrows.

Next, let's see what happens if we have a matrix A that transforms vectors. Consider the 2×2 matrix example, $A = \begin{bmatrix}1 & 2\\4 & 3\end{bmatrix}$. We can imagine this matrix is performing a linear tranformation, $\vec{y} = A\vec{x}$. The question is whether there is another basis within which this transformation is much simpler to understand.

What can we wish for? The transformation is clearly doing something nontrivial, and so it is not possible to find a basis in which the transformation is just the identity matrix. Nor is one going to be found where the matrix is just the zero matrix. So not all wishes can be fulfilled. What about wishing for a diagonal transformation? That would certainly be simpler to understand. But is that a reasonable wish? Let's see whether it is in this case.

Let's suppose we had our new basis B so that $\vec{x} = B\tilde{\vec{x}}$ and correspondingly, $\tilde{\vec{x}} = B^{-1}\vec{x}$. Similarly, $\vec{y} = B\tilde{\vec{y}}$ and

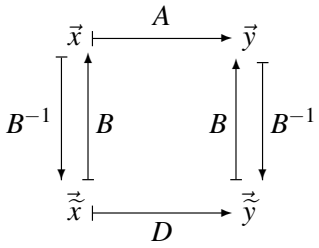


Figure 2: Change of Basis Mapping. The top row has everything in the standard basis. The bottom row is in B-basis. The matrix D is supposed to represent the same linear transformation as A, except that it does so for vectors expressed in B-basis. It turns out $D = B^{-1}AB$ since vectors are columns. We use tilde-ed coordinates for things in the lower basis.

correspondingly, $\vec{\tilde{y}} = B^{-1}\vec{y}$. Then:

$$A\vec{x} = AB\vec{\tilde{x}} \tag{20}$$
$$= A(\tilde{x}_1\vec{b}_1 + \tilde{x}_2\vec{b}_2) \tag{21}$$
$$= \tilde{x}_1 A\vec{b}_1 + \tilde{x}_2 A\vec{b}_2 \tag{22}$$

Now, if we chose our new basis to be the ones defined by the eigenvectors of A, then we can simplify:

$$= \tilde{x}_1\lambda_1\vec{b}_1 + \tilde{x}_2\lambda_2\vec{b}_2 \tag{23}$$
$$= \begin{bmatrix} | & | \\ \vec{b}_1 & \vec{b}_2 \\ | & | \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{x}_2 \end{bmatrix} \tag{24}$$
$$= BD\vec{\tilde{x}} \tag{25}$$
$$= BDB^{-1}\vec{x}, \tag{26}$$

where D is the diagonal matrix of eigenvalues and B is a matrix with the corresponding eigenvectors as its columns. Thus we have proved that $A = BDB^{-1}$. Furthermore, this also means that $D = B^{-1}AB$.

So, our wish can indeed be fulfilled—at least for this specific matrix. In general, the pattern we have used above will hold whenever we can find an eigenbasis—a full basis consisting of eigenvectors. Why? From the eigenvalue-eigenvector relationship we have $AB = [A\vec{b}_1, A\vec{b}_2, \cdots, A\vec{b}_n] = [\lambda_1\vec{b}_1, \lambda_2\vec{b}_2, \cdots, \lambda_n\vec{b}_n] = BD$. If B is full rank and thus invertible, we can left multiply both sides by B^{-1} to get:

$$B^{-1}AB = B^{-1}BD = D = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & \lambda_n \end{bmatrix}$$

But we know that there also exist matrices for which an eigenbasis does not exist. For those, this trick won't work.

3 How to Solve?

Now that we understand how to convert our systems of differential equations into vector/matrix form, let's understand how to solve them. The first thing we should try to do is to use the tools we developed for the scalar case. But, how can we do this, when our equations seem to be fundamentally dependent on two independent variables? Clearly, we wouldn't have this problem of 'coupling' if our A matrix were diagonal. So, let's try to change our basis to one where the A matrix does become diagonal.

Note that if our system of differential equations were 'uncoupled' to begin with, our A matrix would have been diagonal. Hence, we could proceed to solve the first order differential equations independently of each other, as seen in Note 1. Consider the following circuit:

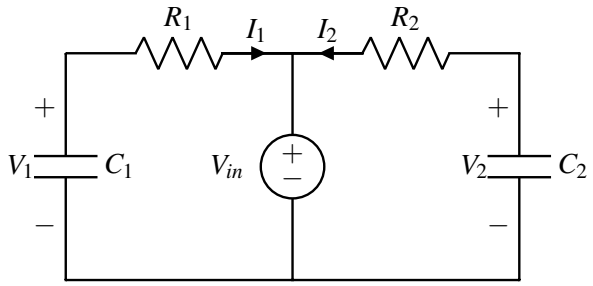


Figure 3: Diagonal System

Both the capacitors have been charged to V_{in} and at $t = 0$, we set $V_{in} = 0V$, and allow the capacitors to discharge. Hence our initial conditions are $V_1(0) = V_2(0) = V_{in}$. We get the following branch equations:

$$I_1 = -C_1 \frac{d}{dt} V_1 = \frac{V_1}{R_1} \tag{27}$$

$$I_2 = -C_2 \frac{d}{dt} V_2 = \frac{V_2}{R_2} \tag{28}$$

Hence from eq. (27) and eq. (28), we get the following uncoupled differential equation:

$$\frac{d}{dt}\begin{bmatrix} V_1(t) \\ V_2(t) \end{bmatrix} = \begin{bmatrix} -\frac{1}{R_1 C_1} & 0 \\ 0 & -\frac{1}{R_2 C_2} \end{bmatrix} \begin{bmatrix} V_1(t) \\ V_2(t) \end{bmatrix} \tag{29}$$

Concept Check: We can easily solve the above system of equations by separately solving for V_1 and V_2. Review Note 1 if you are unsure about how to solve for the voltages.

Coming back to our original system, we define $\vec{x}(t) = \begin{bmatrix} V_1(t) \\ V_2(t) \end{bmatrix}$. Then, in eq. (9), we set up the differential equations as follows:

$$\frac{d}{dt}\vec{x}(t) = \begin{bmatrix} -5 & 2 \\ 2 & -2 \end{bmatrix} \vec{x}(t) \tag{30}$$

As discussed, let's begin by finding a coordinate system in which the transformation represented by this matrix is simply diagonal.

First, we must find its eigenvalues, i.e. the roots of its characteristic equation:

$$\det(\lambda I - A) = \det\left(\begin{bmatrix} \lambda+5 & -2 \\ -2 & \lambda+2 \end{bmatrix}\right) \tag{31}$$

$$= (\lambda+5)(\lambda+2) - 4 \tag{32}$$

$$= \lambda^2 + 7\lambda + 6 = (\lambda+6)(\lambda+1) \tag{33}$$

Solving the above characteristic equation, we get the eigenvalues as $\lambda_1 = -6$ and $\lambda_2 = -1$ and hence we can find a set of eigenvectors as $\vec{v}_1 = \begin{bmatrix} -\frac{2}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} \end{bmatrix}$ and $\vec{v}_2 = \begin{bmatrix} \frac{1}{\sqrt{5}} \\ \frac{2}{\sqrt{5}} \end{bmatrix}$. Defining the new basis as $V = [\vec{v}_1, \vec{v}_2]$, we get the diagonal matrix $\Lambda = \begin{bmatrix} -6 & 0 \\ 0 & -1 \end{bmatrix}$.

Hence, we can write $A = V\Lambda V^{-1}$ and rewrite eq. (9) as follows:

$$\frac{d}{dt}\vec{x}(t) = V\Lambda V^{-1}\vec{x}(t) \tag{34}$$

$$= \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix} \begin{bmatrix} -6 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix}^{-1} \vec{x}(t) \tag{35}$$

Let's perform a change of basis by setting $\vec{\tilde{x}}(t) = \begin{bmatrix} \tilde{x}_1(t) \\ \tilde{x}_2(t) \end{bmatrix} = V^{-1}\vec{x}(t)$. By left multiplying V^{-1} on both sides of eq. (34), we get the following:

$$V^{-1}\frac{d}{dt}\vec{x}(t) = V^{-1}A\vec{x}(t) = V^{-1}V\Lambda V^{-1}\vec{x}(t) \tag{36}$$

$$\implies \frac{d}{dt}V^{-1}\vec{x}(t) = \Lambda\vec{\tilde{x}}(t) \tag{37}$$

$$\implies \frac{d}{dt}\vec{\tilde{x}} = \begin{bmatrix} -6 & 0 \\ 0 & -1 \end{bmatrix}\vec{\tilde{x}}(t) \tag{38}$$

Because V^{-1} is a constant matrix, we can pull it inside the derivative to go from eq. (36) to eq. (37). In eq. (38), we have successfully uncoupled our equations and we can proceed to solve them independenly as mentioned earlier:

$$\frac{d}{dt}\tilde{x}_1(t) = -6\tilde{x}_1 t) \tag{39}$$

$$\implies \tilde{x}_1 = k_1 e^{-6t} \tag{40}$$

$$\frac{d}{dt}\tilde{x}_2(t) = -\tilde{x}_2(t) \tag{41}$$

$$\implies \tilde{x}_2 = k_2 e^{-t} \tag{42}$$

Next, we need to solve for our constants k_1 and k_2. Recall our initial conditions, $V_1(0) = V_2(0) = 1V$. Hence,

$\widetilde{x}_1(0)$ and $\widetilde{x}_2(0)$ are given by[6]:

$$\vec{\widetilde{x}}(0) = V^{-1}\begin{bmatrix} V_1(0) \\ V_2(0) \end{bmatrix} \tag{43}$$

$$\Rightarrow \begin{bmatrix} \widetilde{x}_1(0) \\ \widetilde{x}_2(0) \end{bmatrix} = \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \tag{44}$$

$$= \begin{bmatrix} -\frac{1}{\sqrt{5}} \\ \frac{3}{\sqrt{5}} \end{bmatrix} \tag{45}$$

Hence, $k_1 = -\frac{1}{\sqrt{5}}$ and $k_2 = \frac{3}{\sqrt{5}}$. Now, we can transform back into our original basis as follows to find $V_1(t)$ and $V_2(t)$:

$$\vec{x} = V\vec{\widetilde{x}} \tag{46}$$

$$= \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix} \begin{bmatrix} -\frac{1}{\sqrt{5}}e^{-6t} \\ \frac{3}{\sqrt{5}}e^{-t} \end{bmatrix} \tag{47}$$

$$= \begin{bmatrix} \frac{2}{5}e^{-6t} + \frac{3}{5}e^{-t} \\ -\frac{1}{5}e^{-6t} + \frac{6}{5}e^{-t} \end{bmatrix} \tag{48}$$

For $t \geq 0$, we find that $V_1 = \frac{2}{5}e^{-6t} + \frac{3}{5}e^{-t}$ and $V_2 = -\frac{1}{5}e^{-6t} + \frac{6}{5}e^{-t}$. Figure 4 is a plot of our solutions:

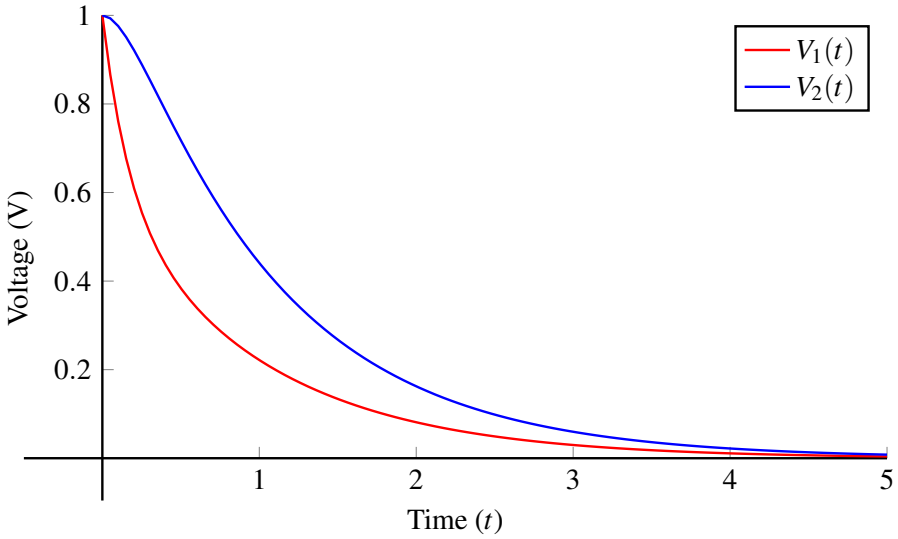
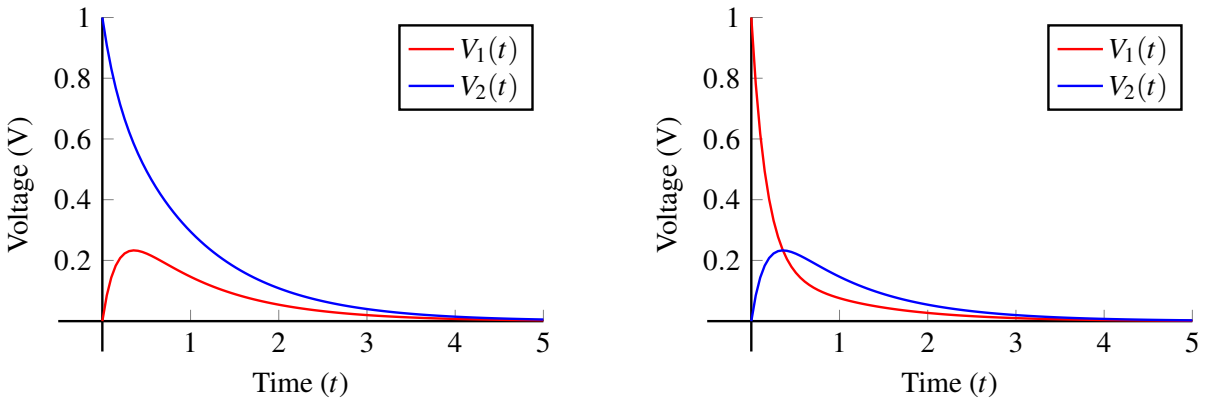


Figure 4: Initial Conditions: $V_1(0) = 1V$ and $V_2(0) = 1V$. Homogeneous Case Solution.

Using the same argument, we can see how the voltage will vary with different initial conditions (as shown in Figure 5):

[6]Note that in this case, $V^{-1} = V^\top = V$ so when we plug in V^{-1} below, it's the same as plugging in V.



(a) Initial Conditions: $V_1(0) = 0V$ and $V_2(0) = 1V$. Here, $V_1(t) = -\frac{2}{5}e^{-6t} + \frac{2}{5}e^{-t}$, and $V_2(t) = \frac{1}{5}e^{-6t} + \frac{4}{5}e^{-t}$.

(b) Initial Conditions: $V_1(0) = 1V$ and $V_2(0) = 0V$. Here, $V_1(t) = \frac{4}{5}e^{-6t} + \frac{1}{5}e^{-t}$, and $V_2(t) = -\frac{2}{5}e^{-6t} + \frac{2}{5}e^{-t}$.

Figure 5: Voltage transients for different initial conditions on the capacitors. The specific solutions are in the subcaptions for your reference.

Concept Check: Take a minute to qualitatively reason about the initial increase in voltages in Figure 5.

Now that we have a good understanding of the homogenous case, let's look at the voltage transients of charging our two capacitor system. In this case, we have two uncharged capacitors, i.e. $V_1(0) = V_2(0) = 0V$, and we apply a voltage $V_{in} = 1V$ for time $t > 0$. We get the following equations:

$$V_2 = V_1 - I_2 R_2 \tag{49}$$

$$I_2 = C_2 \frac{d}{dt} V_2 \tag{50}$$

$$V_{in} - V_1 = I_1 R_1 \Rightarrow I_1 = \frac{V_{in} - V_1}{R_1} \tag{51}$$

$$I_1 = I_2 + C_1 \frac{d}{dt} V_1 \tag{52}$$

Hence, our matrix differential equation is:

$$\frac{d}{dt}\vec{x}(t) = \frac{d}{dt}\begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = \begin{bmatrix} -\left(\frac{1}{R_1C_1} + \frac{1}{R_2C_1}\right) & \frac{1}{R_2C_1} \\ \frac{1}{R_2C_2} & -\frac{1}{R_2C_2} \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} + \begin{bmatrix} \frac{V_{in}}{R_1C_1} \\ 0 \end{bmatrix} \tag{53}$$

$$= \begin{bmatrix} -5 & 2 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} + \begin{bmatrix} 3 \\ 0 \end{bmatrix} \tag{54}$$

$$= A\vec{x}(t) + \vec{b}. \tag{55}$$

Now the system has a non-homogeneous term which we haven't seen before, but let's see if diagonalization

can help us simplify the problem. Letting $\vec{\tilde{x}}(t) = V^{-1}\vec{x}(t)$ and using $A = V\Lambda V^{-1}$, we plug in:

$$\frac{d}{dt}\vec{\tilde{x}}(t) = \frac{d}{dt}V^{-1}\vec{x}(t) = V^{-1}\left(\frac{d}{dt}\vec{x}(t)\right) \tag{56}$$
$$= V^{-1}\left(A\vec{x}(t) + b\right) \tag{57}$$
$$= V^{-1}A\vec{x}(t) + V^{-1}b \tag{58}$$
$$= V^{-1}AV\vec{\tilde{x}}(t) + V^{-1}b \tag{59}$$
$$= \Lambda\vec{\tilde{x}}(t) + V^{-1}b. \tag{60}$$

In this case, recall that from earlier in the notes

$$A = \begin{bmatrix} -5 & 2 \\ 2 & -2 \end{bmatrix} = \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix} \begin{bmatrix} -6 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix}^{-1} \tag{61}$$

so then

$$\frac{d}{dt}\vec{\tilde{x}}(t) = \Lambda\vec{\tilde{x}}(t) + V^{-1}b \tag{62}$$
$$= \begin{bmatrix} -6 & 0 \\ 0 & -1 \end{bmatrix}\vec{\tilde{x}}(t) + \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix}^{-1}\begin{bmatrix} 3 \\ 0 \end{bmatrix} \tag{63}$$
$$= \begin{bmatrix} -6 & 0 \\ 0 & -1 \end{bmatrix}\vec{\tilde{x}}(t) + \begin{bmatrix} -\frac{6}{\sqrt{5}} \\ \frac{3}{\sqrt{5}} \end{bmatrix}. \tag{64}$$

We can thus write the following uncoupled system of differential equations:

$$\frac{d}{dt}\tilde{x}_1(t) = -6\tilde{x}_1(t) - \frac{6}{\sqrt{5}} \tag{65}$$
$$\frac{d}{dt}\tilde{x}_2(t) = -\tilde{x}_2(t) + \frac{3}{\sqrt{5}} \tag{66}$$

We already know how to solve these non-homogeneous differential equations! (see Section 4.4 in Note 1):

$$\tilde{x}_1(t) = e^{-6t}\left(\tilde{x}_1(0) + \frac{1}{\sqrt{5}}\right) - \frac{1}{\sqrt{5}} \tag{67}$$
$$\tilde{x}_2(t) = e^{-t}\left(\tilde{x}_2(0) - \frac{3}{\sqrt{5}}\right) + \frac{3}{\sqrt{5}} \tag{68}$$

To get a full function for $\vec{\tilde{x}}(t)$, we need to find $\vec{\tilde{x}}(0)$. By the definition of $\vec{\tilde{x}}(t)$,

$$\vec{\tilde{x}}(t) = V^{-1}\vec{x}(t) \implies \vec{\tilde{x}}(0) = V^{-1}\vec{x}(0) = \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix}^{-1}\begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \tag{69}$$

Thus plugging these values in for $\vec{\tilde{x}}(t)$,

$$\tilde{x}_1(t) = \frac{1}{\sqrt{5}}e^{-6t} - \frac{1}{\sqrt{5}} \tag{70}$$
$$\tilde{x}_2(t) = -\frac{3}{\sqrt{5}}e^{-t} + \frac{3}{\sqrt{5}} \tag{71}$$

or, putting it together,

$$\vec{\tilde{x}}(t) = \begin{bmatrix} \frac{1}{\sqrt{5}}e^{-6t} - \frac{1}{\sqrt{5}} \\ -\frac{3}{\sqrt{5}}e^{-t} + \frac{3}{\sqrt{5}} \end{bmatrix} \tag{72}$$

Remember, we require $\vec{x}(t)$, not $\vec{\tilde{x}}(t)$. By definition of $\vec{\tilde{x}}(t)$,

$$\vec{x}(t) = V\vec{\tilde{x}}(t) \tag{73}$$

$$= \begin{bmatrix} -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{5}}e^{-6t} - \frac{1}{\sqrt{5}} \\ -\frac{3}{\sqrt{5}}e^{-t} + \frac{3}{\sqrt{5}} \end{bmatrix} \tag{74}$$

$$= \begin{bmatrix} -\frac{2}{5}e^{-6t} - \frac{3}{5}e^{-t} + 1 \\ \frac{1}{5}e^{-6t} - \frac{6}{5}e^{-t} + 1 \end{bmatrix}. \tag{75}$$

Finally, we have $V_1(t) = 1 - \frac{2}{5}e^{-6t} - \frac{3}{5}e^{-t}$ and $V_2(t) = 1 + \frac{1}{5}e^{-6t} - \frac{6}{5}e^{-t}$.

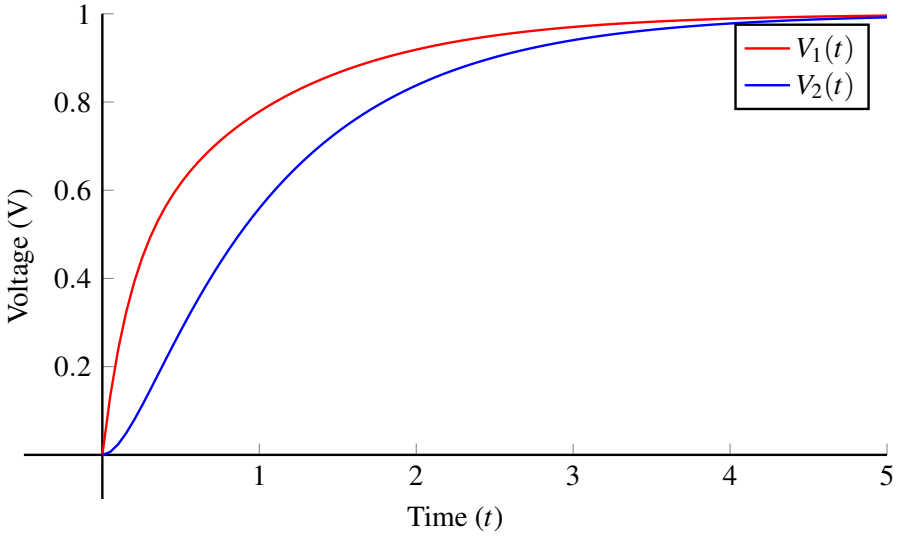


Figure 6: Initial Conditions: $V_1(0) = 0V$ and $V_2(0) = 0V$. Nonhomogeneous Case Solution.

4 Appendix: Algebra for Formation of Matrix Equation

We start with the following equations:

$$V_2 = V_1 - I_2R_2 \tag{76}$$

$$I_2 = C_2\frac{d}{dt}V_2 \tag{77}$$

$$0 - V_1 = I_1R_1 \Rightarrow I_1 = -\frac{V_1}{R_1} \tag{78}$$

$$I_1 = I_2 + C_1\frac{d}{dt}V_1 \tag{79}$$

Our goal is to rearrange this into a form where the derivatives of our state quantities (voltages in this case) are expressed in terms of the voltages themselves and the input voltage.[7] Keeping this in mind, we start with

[7] There are multiple ways to arrive at eq. (5) and eq. (6); this is just one valid sequence of steps.

the simpler equation to derive:

$$\text{eq. (77)} \implies \frac{d}{dt}V_2(t) = \frac{I_2}{C_2} \tag{80}$$

$$\text{eq. (76)} \implies I_2 = \frac{V_1(t)}{R_2} - \frac{V_2(t)}{R_2} \tag{81}$$

$$\frac{d}{dt}V_2(t) = \frac{V_1(t)}{R_2C_2} - \frac{V_2(t)}{R_2C_2} \tag{82}$$

And now:

$$\text{eq. (79)} \implies \frac{d}{dt}V_1(t) = \frac{I_1}{C_1} - \frac{I_2}{C_1} \tag{83}$$

$$\text{eq. (81), eq. (78)} \implies \frac{d}{dt}V_1(t) = \frac{I_1}{C_1} - \frac{\frac{V_1(t)}{R_2} - \frac{V_2(t)}{R_2}}{C_1} \tag{84}$$

$$= \frac{-\frac{V_1(t)}{R_1}}{C_1} - \frac{\frac{V_1(t)}{R_2} - \frac{V_2(t)}{R_2}}{C_1} \tag{85}$$

Simplifying:

$$\frac{d}{dt}V_1(t) = -\frac{V_1(t)}{R_1C_1} - \frac{V_1(t)}{R_2C_1} + \frac{V_2(t)}{R_2C_1} \tag{86}$$

$$\frac{d}{dt}V_1(t) = \left(-\frac{1}{R_1C_1} - \frac{1}{R_2C_1}\right)V_1(t) + \frac{1}{R_2C_1}V_2(t) \tag{87}$$

And these two equations (eq. (87), eq. (82)) exactly match eq. (5), eq. (6).

Contributors:

- Aditya Arun.
- Anant Sahai.
- Nikhil Shinde.
- Kareem Ahmad.
- Jennifer Shih.
- Neelesh Ramachandran.

EECS 16B Designing Information Devices and Systems II

Note 3B: Inductors and RLC Circuits

1 Inductors

Let's introduce a new passive component, an inductor. This new component will help us design more interesting circuits and introduce oscillations within our circuits.

1.1 Basics

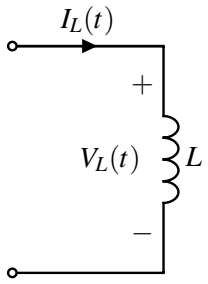


Figure 1: Example Inductor Circuit

The voltage across the inductor is related to its current as follows:

$$V_L(t) = L\frac{dI_L(t)}{dt}. \tag{1}$$

where L is the *inductance* of the inductor. The SI unit of inductance is Henry (H). Looking at eq. (1) we can observe that an inductor behaves as a capacitor with the roles of current and voltage reversed.

Concept Check: The current across the inductor cannot change instantaneously. Why?

Solution: If our current changes instantaneously, then $\frac{d}{dt}I_L \to \infty$, and from eq. (1) the voltage across the inductor $V_L \to \infty$, which is not possible. Hence, our current cannot change instantaneously.

In steady state, when the current flowing through an inductor is constant, there is no voltage drop across the inductor. This makes sense, since an inductor is essentially a spool of wire wrapped around a conductor. Similarly, if the current through the inductor is changing, there will be a voltage drop across the inductor. The energy stored in the inductor turns out to be $E_L = \frac{1}{2}LI^2$, but we won't be using this very much in EECS16B. We are only mentioning it here because it helps us interpret what is happening later.

1.2 Physics behind Inductors

(not in scope for EECS 16B, just for information)

Inductors store energy in a magnetic field. In the same way that a capacitor separates charge (Q) and this leads to an electric field ($\vec{E}$), anytime current flows down a conductor, it creates a magnetic field ($\vec{B}$). Likewise, the magnetic field can store energy. Their behavior can be described using **Faraday's Law of Induction**.

The magnitude of magnetic field created by a straight wire is pretty small, so we usually use other geometries if we want to create a useful inductances. A **solenoid** is a good example, where we wind a wire around a conductor like a copper rod:

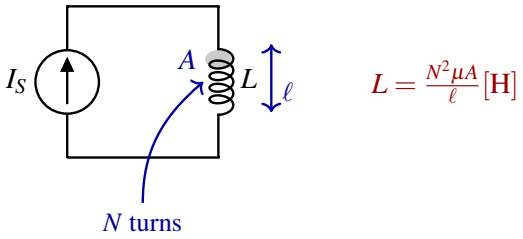


Figure 2: The Inductance of a Solenoid: a wire coiled around something.

Note that the inductance (L) depends on **geometry** and a material property called **magnetic permeability** (μ) of the solenoid core material. In the case of the solenoid in 2, the inductance depends on the number of turns (N), the length of the solenoid (l) and the area (A) of the loops. Inductors are useful in many applications such as wireless communications, chargers, DC-DC converters, key card locks, transformers in the power grid, etc. But in many high speed applications, their presence might be undesirable as they create delays in the time response of the circuit.

1.3 Equivalence Relations

Now that we have the basics, let's derive the equivalence relations for series and parallel combinations of inductors. We will find that these are similar to those of resistors. Why? Because the law governing an inductor $V_L = L\frac{d}{dt}I_L$ involves a proportionality constant L that multiplies a current-like quantity to give a voltage. In a resistor, the resistance R multiplies current to give a voltage.

1.3.1 Series Equivalence

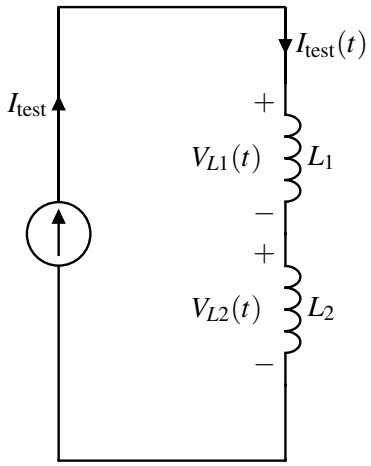


Figure 3: Series Inductor Circuit

Let's apply a $\frac{dI_{\text{test}}}{dt}$ through the two inductors, then

$$V_{L1}(t) + V_{L2}(t) = V_L(t)$$

where, $V_L(t)$ is the voltage across the two inductors. From VI relationship for inductors, we get

$$L_1 \frac{dI_{\text{test}}}{dt} + L_2 \frac{dI_{\text{test}}}{dt} = V_L(t) \tag{2}$$

$$(L_1 + L_2) \frac{dI_{\text{test}}}{dt} = V_L(t) \tag{3}$$

$$L_{\text{eq}} \frac{dI_{\text{test}}}{dt} = V_L(t) \tag{4}$$

where, $L_{\text{eq}} = L_1 + L_2$.

1.3.2 Parallel Equivalence

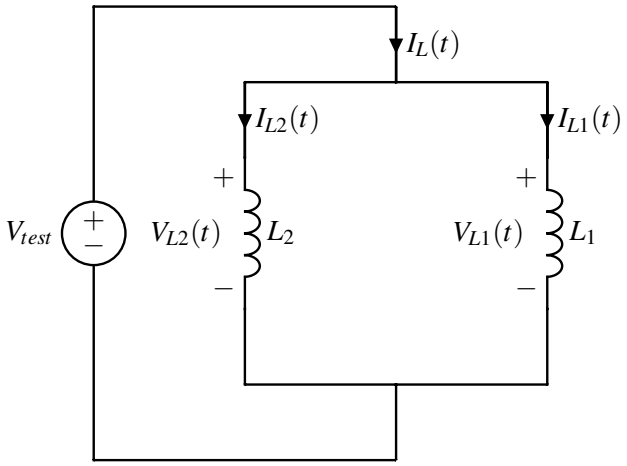


Figure 4: Parallel Inductor Circuit

We apply at V_{test} across the parallel combination. We have

$$V_{L1}(t) = V_{L2}(t) = V_{\text{test}}(t)$$

$$L_1 \frac{dI_{L1}}{dt} = L_2 \frac{dI_{L2}}{dt} = L_{\text{eq}} \frac{dI_L}{dt}$$

and from KCL, we have

$$I_L(t) = I_{L1}(t) + I_{L2}(t)$$

Differentiating with respect to time, and substituting from the above equality,

$$\frac{dI_L}{dt} = \frac{dI_{L1}}{dt} + \frac{dI_{L2}}{dt} \tag{5}$$

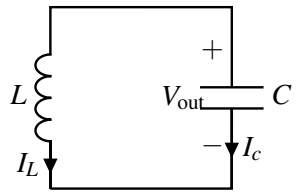
$$\frac{dI_L}{dt} = \frac{L_{\text{eq}}}{L_1} \frac{dI_L}{dt} + \frac{L_{\text{eq}}}{L_2} \frac{dI_L}{dt} \tag{6}$$

$$\frac{1}{L_{\text{eq}}} = \frac{1}{L_1} + \frac{1}{L_2} \tag{7}$$

2 LC Tank

In our two capacitor circuit example, we found that our eigenvalues were real. But, we could also encounter a system whose eigenvalues are complex. In this section, we will explore a circuit, commonly known as an LC tank, whose matrix will have purely imaginary eigenvalues.

In the following circuit, we have an inductor $L = 10\text{nH}$ and capacitor $C = 10\text{pF}$ in parallel. Let $I_L(0) = 50\text{mA}$ and $V_{\text{out}}(0) = 0\text{V}$:



Since the inductor and capacitor are in parallel:

$$V_L = V_C = V_{\text{out}}$$

KCL gives:

$$\begin{aligned} I_L = -I_c &= -C\frac{dV_{\text{out}}}{dt} \\ \frac{dV_{\text{out}}}{dt} &= -\frac{1}{C}I_L \\ V_L = V_{\text{out}} &= L\frac{dI_L}{dt} \\ \frac{dI_L}{dt} &= \frac{1}{L}V_{\text{out}} \end{aligned}$$

Putting it into matrix form, as before:

$$\begin{bmatrix} \frac{d}{dt}V_{\text{out}} \\ \frac{d}{dt}I_L \end{bmatrix} = \begin{bmatrix} 0 & -\frac{1}{C} \\ \frac{1}{L} & 0 \end{bmatrix} \begin{bmatrix} V_{\text{out}} \\ I_L \end{bmatrix} \tag{8}$$

Finding the eigenvalues:

$$\det\left(\begin{bmatrix} -\lambda & -\frac{1}{C} \\ \frac{1}{L} & -\lambda \end{bmatrix}\right) = \lambda^2 + \frac{1}{LC} = 0 \tag{9}$$

$$\implies \lambda_{1,2} = 0 \pm j\frac{1}{\sqrt{LC}} \tag{10}$$

Next, we can find the eigenvectors of the above matrix as $v_1 = \begin{bmatrix} j\sqrt{\frac{L}{C}} \\ 1 \end{bmatrix}$ and $v_2 = \begin{bmatrix} -j\sqrt{\frac{L}{C}} \\ 1 \end{bmatrix}$. We can use these vectors to transform our coordinates to one where the matrix becomes diagonal. More concretely,

$$\begin{bmatrix} V_{\text{out}} \\ I_L \end{bmatrix} = \begin{bmatrix} | & | \\ v_1 & v_2 \\ | & | \end{bmatrix} \begin{bmatrix} \widetilde{V}_{\text{out}} \\ \widetilde{I}_L \end{bmatrix}$$

As discussed before, once in this new coordinates, our system becomes uncoupled, and we can solve for V_{out} and I_L as follows:

$$\begin{aligned}
\begin{bmatrix} \frac{d}{dt}\widetilde{V}_{\text{out}} \\ \frac{d}{dt}\tilde{I}_L \end{bmatrix} &= \begin{bmatrix} j\frac{1}{\sqrt{LC}} & 0 \\ 0 & -j\frac{1}{\sqrt{LC}} \end{bmatrix} \begin{bmatrix} \widetilde{V}_{\text{out}} \\ \tilde{I}_L \end{bmatrix} \\
\implies \frac{d}{dt}\widetilde{V}_{\text{out}} &= j\frac{1}{\sqrt{LC}}\widetilde{V}_{\text{out}} \\
\frac{d}{dt}\tilde{I}_L &= -j\frac{1}{\sqrt{LC}}\tilde{I}_L \\
\therefore \widetilde{V}_{\text{out}} &= \widetilde{k_1}e^{\frac{j}{\sqrt{LC}}t} \\
\tilde{I}_L &= \widetilde{k_2}e^{-\frac{j}{\sqrt{LC}}t}
\end{aligned}$$

Next, we need to find initial conditions in this new coordinate system. Substituting the given values,

$$\begin{aligned}
\begin{bmatrix} \widetilde{V}_{\text{out}}(0) \\ \tilde{I}_L(0) \end{bmatrix} &= \begin{bmatrix} j\sqrt{\frac{L}{C}} & -j\sqrt{\frac{L}{C}} \\ 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} V_{\text{out}}(0) \\ I_L(0) \end{bmatrix} \\
&= \frac{1}{j20\sqrt{10}} \begin{bmatrix} 1 & j10\sqrt{10} \\ -1 & j10\sqrt{10} \end{bmatrix} \begin{bmatrix} 0 \\ 0.05 \end{bmatrix} \\
&= \begin{bmatrix} 2.5\times10^{-2} \\ 2.5\times10^{-2} \end{bmatrix}
\end{aligned}$$

Hence, $\widetilde{k_1} = 2.5\times10^{-2}$ and $\widetilde{k_2} = 2.5\times10^{-2}$. Next, we can tranform back to our original coordinate system:

$$\begin{aligned}
\begin{bmatrix} V_{\text{out}} \\ I_L \end{bmatrix} &= \begin{bmatrix} j10\sqrt{10} & -j10\sqrt{10} \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2.5\times10^{-2}e^{j\sqrt{10}\times10^9 t} \\ 2.5\times10^{-2}e^{-j\sqrt{10}\times10^9 t} \end{bmatrix} \\
&= \begin{bmatrix} j0.25\sqrt{10}e^{j\sqrt{10}\times10^9 t} - j0.25\sqrt{10}e^{-j\sqrt{10}\times10^9 t} \\ 2.5\times10^{-2}e^{j\sqrt{10}\times10^9 t} + 2.5\times10^{-2}e^{-j\sqrt{10}\times10^9 t} \end{bmatrix}
\end{aligned}$$

Concept Check: Write the above sum of exponentials as sine and cosine. *Hint: Use the Euler form of sine and cosine we encountered in the complex number note.*

Based on the intuition we have gained above, let's guess a solution with pure sines and cosines, as follows:

$$V_{\text{out}}(t) = c_1\cos\left(\frac{1}{\sqrt{LC}}t\right) + c_2\sin\left(\frac{1}{\sqrt{LC}}t\right) \tag{11}$$

Next, plugging in initial conditions to solve for the constants:

$$\begin{aligned}
V_{\text{out}}(0) = 0 &= c_1 \\
I_c(0) = -I_L(0) &= -50\times10^{-3} \\
\frac{dV_{\text{out}}(0)}{dt} = \frac{1}{C}I_c(0) = \frac{-50\times10^{-3}}{10^{-11}} &= \frac{c_2}{\sqrt{10^{-8}\times10^{-11}}} \\
c_1 &= 0 \\
\implies c_2 = -\frac{5}{\sqrt{10}} &= -0.5\sqrt{10} \\
\implies V_{\text{out}}(t) &= -0.5\sqrt{10}\sin\left(\sqrt{10}\times10^9 t\right)
\end{aligned}$$

Notice that the amplitude of V_{out} is constant.[1]

Concept Check: Follow the same steps above to find the current, $I_L(t)$. *Hint: The current will also be of the form in eq.* (11)*, but with different constants.*

Solution:

$$I_L(t) = 50 \times 10^{-3} \cos\left(\sqrt{10} \times 10^9 t\right)$$

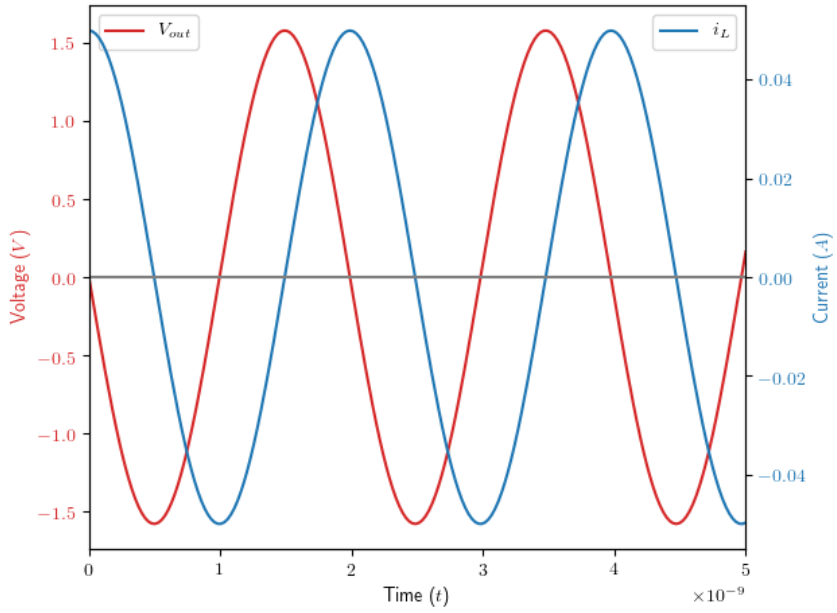


Figure 5: Voltage and Current response of LC Tank

Figure 5 plots the above solutions for the capacitor voltage and inductor current. This system is also called an oscillator because the circuit produces a repetitive voltage waveform under the right initial conditions.

From the above plots, we can see that the current and voltage are 90° out of phase, i.e. when the current is at its maximum or minimum, the voltage is at 0V, and vice versa. What does this mean for the energy stored in these components? We know that, energy in the capacitor, $E_C = \frac{1}{2}CV^2 = 1.25 \times 10^{-11} \sin^2\left(\sqrt{10} \times 10^9 t\right)$ and energy in the inductor, $E_L = 1.25 \times 10^{-11} \cos^2\left(\sqrt{10} \times 10^9 t\right)$. Figure 6 plots the these energies. As you can see, the total energy seems to be sloshing back and forth between the inductor and capacitor.

[1] And, in case the algebra is confusing, the $\frac{c_2}{\sqrt{10^{-8} \times 10^{-11}}}$ part comes from evaluating the derivative of the output voltage at time $t = 0$. That is, $\frac{d}{dt} V_{\text{out}}(t) = c_2 \cos\left(\frac{1}{\sqrt{LC}} t\right)$, and we know the value for this at $t = 0$. Plugging in L, C, gives this equation.

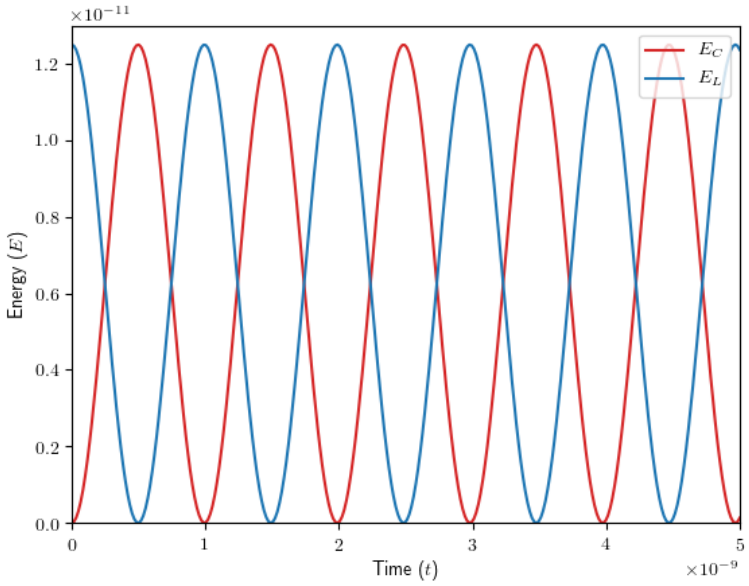
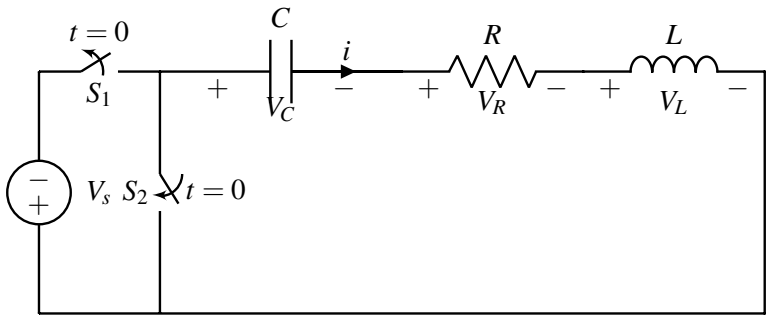


Figure 6: Energy stored in Inductor and Capacitor. Notice the sum is constant.

3 RLC Circuits and Higher Order Differential Equations

The LC tank we studied in the previous section was a very ideal case where we assumed there was no resistor in the system. But this is rarely the case, and we will need to understand how adding this third component will modify our differential equations.

To motivate our discussions, consider the following circuit, with component values $V_s = 4\text{V}$, $C = 2\text{fF}$, $R = 60\text{k}\Omega$, and $L = 1\mu\text{H}$. Before $t = 0$, switch S_1 is on while S_2 is off. At $t = 0$, both switches flip state (S_1 turns off and S_2 turns on):



This is something that you will work out for yourself in the homework. The key is simply to follow the steps marking anything with a derivative on it as a state variable, writing out the differential equations, and solving the system.

Contributors:

- Neelesh Ramachandran.

- Kristofer Pister.
- Utkarsh Singhal.
- Aditya Arun.
- Kyle Tanghe.
- Anant Sahai.
- Kareem Ahmad.
- Nikhil Shinde.

EECS 16B Designing Information Devices and Systems II
Note 4: Phasors

1 Overview

Frequency analysis focuses on analyzing the steady-state behavior of circuits with sinusoidal voltage and current sources — sometimes called AC circuit analysis. This note discusses techniques to simplify the analysis of general RLC circuits that have such inputs.

It's natural to wonder: what's so special about sinusoids? For one, sinusoidal sources are a very common category of inputs, making this form of analysis very useful. Also, there's a technique known as the Fourier Transform, that can express certain input signals as weighted sums of sinusoidal functions, and we can apply superposition.[1]

The most important reason, however, is that analyzing sinusoidal functions is *easy*! Whereas analyzing arbitrary input signals (like in transient analysis) requires us to solve a set of differential equations, it turns out that we can use a procedure very similar to the seven-step procedure from EECS16A in order to solve AC circuits with only sinusoidal sources.

An Important Remark on Notation: For clarity in this note, we will denote time-domain signals with lower-case letters, and phasors (section 4 onwards) with capital letters. Vectors of scalars (as featured in section 3) are denoted using the ˇ character. On occasion, we might have a term that seems to be a "vector of phasors" (such as a vector scaling $e^{j\omega t}$.) Here, we will still use ˇ notation.

2 Scalar Linear First-Order Differential Equations

We've already seen that general linear circuits with sources, resistors, capacitors, and inductors can be thought of as a system of linear, first-order, differential equations with sinusoidal input. By developing techniques to determine the steady state of such systems in general, we can hope to apply them to the special case of circuit analysis. Let's step back from circuits for a little bit and revisit the fundamentals.

First, we'll look at the scalar case, for simplicity. Consider the differential equation

$$\frac{d}{dt}x(t) = \lambda x(t) + u(t),$$

where the input $u(t)$ is of the form

$$u(t) = ke^{st}$$

where $s \neq \lambda$.

We've previously seen (in HW 3) how to solve this equation:

$$x(t) = \left(x_0 - \frac{k}{s-\lambda}\right)e^{\lambda t} + \frac{k}{s-\lambda}e^{st},$$

[1] This topic is one focus of classes like EE120 and beyond.

where $x(0) = x_0$ is the initial value of $x(t)$.

Interestingly, this is *almost* a scalar multiple of $u(t)$ - if only we could ignore the initial term involving $e^{\lambda t}$, then $x(t)$ would linearly depend on $u(t)$. Can we ignore this term? We can only do so by arguing that it goes to zero over time. Then, our "steady state" solution for $x(t)$ would involve only the e^{st} term, which seems to make our lives a lot easier.

When might that happen? Specifically, when does $e^{\lambda t} \to 0$ as $t \to \infty$? If λ were real, then the term decays to zero if and only if $\lambda < 0$. But what about for complex λ? We can try writing a complex λ in the form $\lambda = \lambda_r + j\lambda_i$, to try and reduce the problem to the real case. That is:

$$\begin{aligned} e^{\lambda t} &= e^{(\lambda_r + j\lambda_i)t} \\ &= e^{\lambda_r t} e^{j\lambda_i t} \end{aligned}$$

The $e^{\lambda_r t}$ is exactly the real case we just saw above. But what about the $e^{j\lambda_i t}$ term? Well, the only thing we can really do here is apply Euler's formula. Expanding our expression, we find that

$$e^{\lambda t} = e^{\lambda_r t}(\cos(\lambda_i t) + j\sin(\lambda_i t))$$

This expression seems promising! The first term in the product is a real exponential, which we know decays to zero exactly when $\text{Re}[\lambda] = \lambda_r < 0$. The second term is a sum of two sinusoids with unit amplitudes. Since the amplitude of each sinusoid is constant, their sum will not decay to zero or grow to infinity over time. Thus, the asymptotic behavior of the overall expression is governed solely by the first term ($e^{\lambda t}$ will decay to zero exactly when e^{λ_r} does). From earlier, we can see that this happens when $\lambda_r < 0$.

Looking back at our solution for $x(t)$, we now have a way to determine when the $e^{\lambda t}$ decays, and the condition can be applied for both real and complex λ.

3 Systems of Linear First-Order Differential Equations

Can we apply similar techniques to what we've just seen to a system of differential equations? Specifically, consider the system

$$\frac{d}{dt}\vec{x}(t) = A\vec{x}(t) + \vec{u}(t),$$

where A is a fixed, real, matrix. As before, we will consider only control inputs of a special form, where each component is of the form ke^{st} for some constant k. Said differently, we will only examine cases where $\vec{u}(t)$ can be expressed in the form

$$\vec{u}(t) = \vec{u}e^{st},$$

where $\vec{u}$ does not depend on t, and s is *not* an eigenvalue of the matrix A.

Inspired by our previous observations, let's make the guess that our solution $x(t)$ can be written as

$$\vec{x}(t) = \vec{x}e^{st}$$

where $\vec{\tilde{x}}$ does not depend on t. Substituting into our differential equation, we find that

$$\begin{aligned}\frac{d}{dt}\left(\vec{\tilde{x}}e^{st}\right) &= A\vec{\tilde{x}}e^{st} + \vec{\tilde{u}}e^{st} \\ s\vec{\tilde{x}}e^{st} &= A\vec{\tilde{x}}e^{st} + \vec{\tilde{u}}e^{st} \\ (sI-A)\vec{\tilde{x}}e^{st} &= \vec{\tilde{u}}e^{st}.\end{aligned}$$

Since the above equality must hold true for all t, we can equate the coefficients of e^{st} to obtain

$$(sI-A)\vec{\tilde{x}} = \vec{\tilde{u}}.$$

Now, to express $\vec{\tilde{x}}$ in terms of $\vec{\tilde{u}}$, we are tempted to multiply by the inverse of $sI-A$ on both sides. Recall that our working assumption for this analysis is that s is not an eigenvalue of A. Before taking the inverse, can we use our assumption to show this operation is valid? To proceed with proof by contradiction, imagine that $sI-A$ is not invertible (that is, $sI-A$ has a nonempty null space, containing some vector $\vec{y}$). We could then write:

$$\begin{aligned}&(sI-A)\vec{y} = \vec{0} \\ \implies &A\vec{y} = s\vec{y}\end{aligned}$$

so s would be an eigenvalue of A with corresponding eigenvector $\vec{y}$. But our assumption is that s is not an eigenvalue of A, so our "proof-by-contradiction" assumption that $sI-A$ is not invertible must have been wrong ($sI-A$ must be invertible).

Thus, we can rearrange our equation for $\vec{\tilde{x}}$ above, to express

$$\vec{\tilde{x}} = (sI-A)^{-1}\vec{\tilde{u}}.$$

It is straightforward to substitute this back into the expression for $x(t)$ and verify that it does indeed correspond to a valid solution for our original system of differential equations.

This is great! Starting with a system of differential equations with an input of a particular form, we can now use the above identity to construct a solution for $\vec{x}(t)$ without calculus!

But is this solution the one we will reach in the steady state? Assume, for simplicity, that A has a full set of eigenvectors $\vec{v}_1, \ldots, \vec{v}_n$ with corresponding eigenvalues $\lambda_1, \ldots, \lambda_n$. Then we know that we can diagonalize A to be

$$\begin{aligned}A &= \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix}^{-1} \\ &= V\Lambda V^{-1},\end{aligned}$$

where V and Λ are the eigenvector and eigenvalue matrices in the above diagonalization.

Thus, we can rewrite our differential equation for $x(t)$ as:

$$\frac{d}{dt}\vec{x}(t) = V\Lambda V^{-1}\vec{x}(t) + \vec{u}(t)$$
$$\frac{d}{dt}\left(V^{-1}\vec{x}(t)\right) = \Lambda(V^{-1}\vec{x}(t)) + V^{-1}\vec{u}(t).$$

As we have seen, this diagonalized (uncoupled) system of differential equations can be rewritten as a set of scalar differential equations of the form

$$\frac{d}{dt}\left(V^{-1}\vec{x}(t)\right)_i = \lambda_i\left(V^{-1}\vec{x}(t)\right)_i + \left(V^{-1}\vec{u}(t)\right)_i,$$

where the subscripted i represents the ith component of the associated vector, and λ_i is the ith eigenvalue of A.

Since $\left(V^{-1}\vec{u}(t)\right)_i$ is a multiple of e^{st} and $s \neq \lambda_i$, we know from our scalar results that the solution to $\left(V^{-1}\vec{x}(t)\right)_i$ can be expressed as a linear combination of $e^{\lambda t}$ and e^{st}, where the $e^{\lambda t}$ decays to zero over time if and only if $\text{Re}[\lambda_i] < 0$, yielding a steady state solution involving only a scalar multiple of e^{st}. Let this solution be:

$$\left(V^{-1}\vec{x}(t)\right)_i = \vec{\tilde{x}}_i e^{st}.$$

Thus, we can stack these solutions for i and pre-multiply by V to obtain

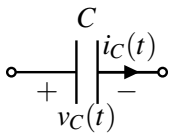
$$V^{-1}\vec{x}(t) = \vec{\tilde{x}}e^{st}$$
$$\vec{x}(t) = (V\vec{\tilde{x}})e^{st}.$$

Now, recall that our candidate solution $\vec{x}(t) = \vec{\tilde{x}}e^{st}$ was constructed to be the unique solution to our system that was a scalar multiple of e^{st}. Thus, our candidate solution is exactly the steady state solution to the system, which we will converge to exactly when the real components of all the eigenvalues λ_i of our state matrix A are less than zero.

4 Circuits with Exponential Inputs

For a large circuit involving resistors, capacitors, and inductors, if we tried to solve the circuit using differential equations, every capacitor and inductor adds an extra derivative to the system. Now let's take a look at what happens if our circuit was driven by inputs of the exponential function e^{st} for some constant s.

Consider a particular capacitor C within the circuit, with node voltages $v_+(t)$ and $v_-(t)$ at its two terminals (defining a capacitor voltage $v_c(t)$) and a current $i_C(t)$ flowing through it.:



At steady state, we know from our understanding of the differential equation story above that:

$$v_C(t) = \widetilde{V}_C e^{st} \qquad i_C(t) = \widetilde{I}e^{st}$$

for some scalars $\widetilde{V}_C$ and $\widetilde{I}_C$.

By the known differential equation for a capacitor, we have that

$$\begin{aligned} i_C(t) &= C\frac{d}{dt}v_C(t) \\ \widetilde{I}_C e^{st} &= C\frac{d}{dt}(\widetilde{V}_C e^{st}) \\ \widetilde{I}_C e^{st} &= Cs(\widetilde{V}_C)e^{st} \\ \implies \widetilde{I}_C &= Cs(\widetilde{V}_C). \end{aligned}$$

Critically, this equation resembles that of Ohm's Law! It is a purely linear equation without any time-dependence.

Similar equations can be obtained (this is a useful exercise to do) for an inductor $\widetilde{V}_L = \widetilde{I}_L \cdot sL$. and a resistor $\widetilde{V}_R = \widetilde{I}_R \cdot R$. Rewriting the capacitor relationship to be in the same form, we see $\widetilde{V}_C = \widetilde{I}_C \cdot (\frac{1}{Cs})$. This suggests that we can view capacitors, inductors, and resistors as all being similar. In effect, capacitors and inductors just have s-dependent resistances. These are called s-impedances. The term impedance is a generalization of the concept of resistance to allow for different resistances at different s-values (we will soon see what governs the value of s in a circuit context.). The impedance is defined as the voltage-current ratio (similar to resistance):

$$\widetilde{Z} = \frac{\widetilde{V}}{\widetilde{I}}$$

A capacitor has an s-impedance of $\frac{1}{Cs}$. An inductor has an s-impedance of Ls. And a resistor's s-impedance is just the same as its resistance R.

This reveals an approach for circuit analysis with exponential inputs, as long as all the inputs have the same s. We can replace all the independent voltage and current sources with constant voltages and currents corresponding to only the coefficients of e^{st}. We replace all capacitors and inductors with their corresponding impedances, and then analyze the entire circuit as though it only had resistances it it. This can be solved using the circuit analysis techniques studied in 16A, and we can interpret the results to get the steady-state solution for the system.

5 Sinusoids and Phasors

Unfortunately, there's one big issue with all the work we've done so far - specifically, the restrictions we imposed on our input $\vec{u}(t)$. We stated that $\vec{u}(t)$ should be expressed as

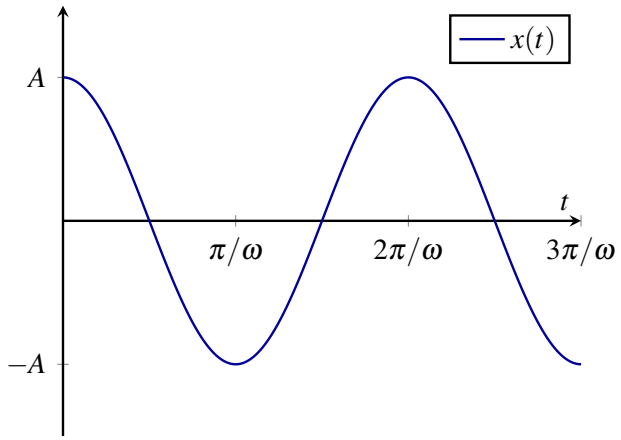
$$\vec{u}(t) = \vec{\tilde{u}}e^{st}$$

for some s. What kinds of s are probably useful? If $\mathrm{Re}(s) < 0$, then we know that the input approaches zero over time, so the steady state behavior of our system is probably not very interesting. Similarly, if $\mathrm{Re}(s) > 0$, then our input will grow to infinity over time, so our state will blow up! This only leaves the case $\mathrm{Re}(s) = 0$ as neither blowing up or decaying away.

So then what can our input look like? If $\mathrm{Re}(s) = 0$, then s must be purely imaginary. So our input will be a linear function of e^{st}, where s is a real multiple of j. From Euler's formula, we know that term has some sort of periodic, sinusoidal behavior.

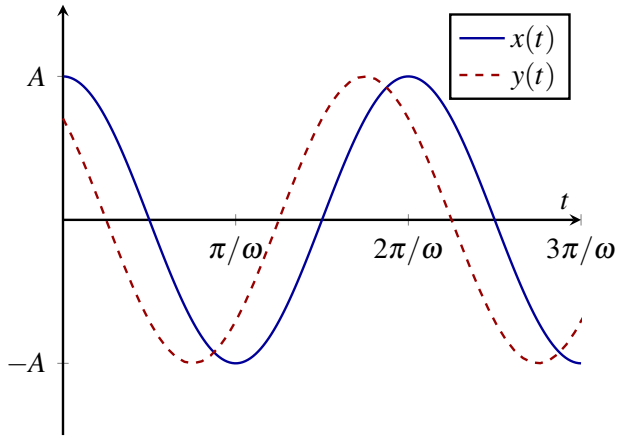
Consider the function $x(t) = A\cos\omega t$, where $x(t)$ can be thought of as representing an input in our circuit,

like an alternating current or voltage.



There are a couple properties of $x(t)$ that are apparent from the figure: we call the maximum value of $x(t)$ above the mean (in this case, the x-axis) the *amplitude* (A), and the spacing between repetitions of the function the *period* ($T = 2\pi/\omega$).

However, there's one other important property of sinusoids: their *phase*. Consider a similar function $y(t) = A\cos(\omega t + \phi)$.



Here, ϕ represents the *phase shift* of $y(t)$ with respect to $x(t)$. As can be seen, a positive phase shift moves the function to the left by that amount. In particular, notice that the sine and cosine functions are really the same sinusoid! It's just that there is a $\pi/2$ radian phase shift between them.

Now that we know a little about sinusoids, let's see how we can express a sinusoidal voltage input $v(t) = V_0\cos(\omega t + \phi)$ in terms of exponential functions. To do this, we will use complex numbers. We can combine Euler's formula with the properties of complex conjugates to determine that

$$e^{j\theta} + e^{-j\theta} = (\cos(\theta) + j\sin(\theta)) + (\cos(\theta) - j\sin(\theta)) = 2\cos(\theta).$$

In other words, starting with two complex exponentials, we have pulled out a purely real sinusoid! From the

above definition, we have that

$$\begin{aligned} \cos(\theta) &= \frac{1}{2}e^{j\theta} + \frac{1}{2}e^{-j\theta} \\ \implies \quad \cos(\omega t + \phi) &= \frac{1}{2}e^{j\omega t + j\phi} + \frac{1}{2}e^{-j\omega t - j\phi} \\ &= \frac{e^{j\phi}}{2}e^{j\omega t} + \frac{e^{-j\phi}}{2}e^{-j\omega t} \\ \implies \quad v(t) &= V_0\cos(\omega t + \phi) \\ &= \frac{V_0 e^{j\phi}}{2}e^{j\omega t} + \frac{V_0 e^{-j\phi}}{2}e^{-j\omega t}. \end{aligned}$$

Therefore, we can express an arbitrary sinusoid $v(t)$ as a linear combination of two exponential functions! Notice that the coefficients of the two exponential functions are complex conjugates of one another. Thus, we can rewrite the above as:[2]

$$v(t) = \frac{V_0 e^{j\phi}}{2}e^{j\omega t} + \frac{\overline{V_0 e^{j\phi}}}{2}e^{-j\omega t}.$$

Thus, the coefficient of the $e^{j\omega t}$ can be used to represent the entire sinusoid $v(t)$ (assuming the frequency ω is known). We call this coefficient the *phasor* representing $v(t)$, and denote it as

$$\widetilde{V} = \frac{V_0 e^{j\phi}}{2}.$$

Now, we know how to find the steady states of systems of differential equations with sinusoidal inputs! First, use the above transformation to write the input as a linear combination of exponential functions e^{st}. Then, for each exponential function, solve the equation $\vec{\tilde{x}} = (sI - A)^{-1}\vec{\tilde{u}}$ to determine the steady state solution $\vec{x}(t) = \vec{\tilde{x}}e^{st}$.[3] Finally, take the superposition of all these steady states, to obtain the steady state corresponding to the entire original input. We see a small symbolic example of this below.

This approach works great! But there's one further optimization we can add to simplify calculations. Let's consider the case when we are working with real, sinusoidal inputs of a fixed frequency ω. Then we know that our input can be represented as a linear combination of inputs of the forms $e^{j\omega t}$ and $e^{-j\omega t}$.

But, from our above construction, we know that this can't be just *any* linear combination! Specifically, the coefficients of $e^{j\omega t}$ and $e^{-j\omega t}$ must be complex conjugates of one another! Thus, we can write our input as

$$\vec{u}(t) = \vec{\tilde{u}}e^{j\omega t} + \overline{\vec{\tilde{u}}}e^{-j\omega t}.$$

Now, let our system be

$$\frac{d}{dt}\vec{x}(t) = A\vec{x}(t) + \vec{u}(t).$$

When $\vec{u} = \vec{\tilde{u}}e^{j\omega t}$, we know that the steady state $\vec{\tilde{x}}_1 e^{j\omega t}$ for $\vec{x}(t)$ is such that

$$(j\omega I - A)\vec{\tilde{x}}_1 = \vec{\tilde{u}}.$$

[2] Recall that $e^{-j\theta} = \overline{e^{j\theta}}$.

[3] Going forward, we will consistently use capital letters for phasors corresponding to lower-case time-domain sinusoidal quantities. Here, $\vec{\tilde{x}}$ and $\vec{\tilde{u}}$ are vectors of scalars, not phasors.

Similarly, when $\vec{u} = \vec{\tilde{u}} e^{-j\omega t}$, we know that the steady state $\vec{\tilde{x}}_2 e^{-j\omega t}$ for $\vec{x}(t)$ is such that

$$(-j\omega I - A)\vec{\tilde{x}}_2 = \vec{\tilde{u}}.$$

Then, we take the superposition of these two solutions to find the overall steady state solution:

$$\vec{x}(t) = \vec{\tilde{x}}_1 e^{j\omega t} + \vec{\tilde{x}}_2 e^{-j\omega t}.$$

This solution works, but it requires us to solve two linear equations to find both $\vec{\tilde{x}}_1$ and $\vec{\tilde{x}}_2$. These quantities appear to be fairly similar; is the relationship between them predictable? That is, can we solve for $\vec{\tilde{x}}_2$ in terms of $\vec{\tilde{x}}_1$?

The key observation to make is that (since A is a real matrix and $\overline{A} = A$):

$$\overline{j\omega I - A} = -j\omega I - A,$$

Thus, starting from the known solution for $\vec{\tilde{x}}_1$, we can take complex conjugates to obtain

$$\begin{aligned} & (j\omega I - A)\vec{\tilde{x}}_1 = \vec{\tilde{u}} \\ \implies \quad & \overline{(j\omega I - A)}\overline{\vec{\tilde{x}}_1} = \overline{\vec{\tilde{u}}} \\ \implies \quad & (-j\omega I - A)\overline{\vec{\tilde{x}}_1} = \overline{\vec{\tilde{u}}}. \end{aligned}$$

The above equation is exactly the equation that $\vec{\tilde{x}}_2$ has to satisfy. Matching terms from our earlier expression, we see that

$$\vec{\tilde{x}}_2 = \overline{\vec{\tilde{x}}_1},$$

so we can substitute and write our final solution for $x(t)$ as

$$\vec{x}(t) = \vec{\tilde{x}}_1 e^{j\omega t} + \overline{\vec{\tilde{x}}_1} e^{-j\omega t}.$$

So only one round of Gaussian elimination (or linear equation system solving generally) is needed, not two!

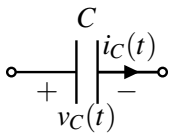
6 Circuit Quantities: Phasors

In principle, at this point we already know what to do when given a circuit with sinusoidal inputs all at the same frequency. But it can be helpful to revisit and solidify the derivations.

Let's apply the technique we've just developed to study the steady-state behavior of capacitors and inductors when supplied with a sinusoidal voltage or current signal. One key insight will help us proceed; with a phasor, we are representing a sinusoid by its amplitude and phase, but not the frequency. In fact, the frequency is tied to time as part of the ωt term, and in the section above, we noted that the phasor represents the *entire sinusoid*. We found this convenient mathematically, and could do this only because linear circuits (resistors, capacitors, and inductors) will never alter the frequency of a sinusoid. Why? This property comes from the fact that our generalized exponential input term, e^{st}, is an eigenfunction of differentiation (which appears in the definitions of inductor voltages and capacitor currents). Any sinusoid will be expressed in terms of $e^{j\omega t}$ and $e^{-j\omega t}$; let's take a closer look at an example to understand this.

6.1 Impedance of a Capacitor

We examine a capacitor provided with the sinusoidal voltage $v_C(t) = V_0 \cos(\omega t + \phi)$, as shown:



Note that we aren't assuming anything about the origin of the $v(t)$ – it could come from a voltage supply directly, or from some other complicated circuit. From before, we know that $v(t)$ has a phasor representation. Since:

$$v_C(t) = \frac{V_0 e^{j\phi}}{2} e^{j\omega t} + \frac{V_0 e^{-j\phi}}{2} e^{-j\omega t},$$

it can be represented by the phasor

$$\widetilde{V}_C = \frac{V_0 e^{j\phi}}{2}.$$

Now, by the capacitor equation, we know that:

$$\begin{aligned}
i_C(t) &= C\frac{d}{dt} v_C(t) \\
&= C\frac{d}{dt}\left(\frac{V_0 e^{j\phi}}{2} e^{j\omega t} + \frac{V_0 e^{-j\phi}}{2} e^{-j\omega t}\right) \\
&= C\frac{d}{dt}\left(\widetilde{V}_C e^{j\omega t} + \overline{\widetilde{V}}_C e^{-j\omega t}\right) \\
&= C\left(\widetilde{V}_C (j\omega) e^{j\omega t} + \overline{\widetilde{V}}_C (-j\omega) e^{-j\omega t}\right) \\
&= (j\omega C)\widetilde{V}_C e^{j\omega t} + (-j\omega C)\overline{\widetilde{V}}_C e^{-j\omega t}
\end{aligned}$$

Noting that we find phasors by taking the coefficient of the $e^{j\omega t}$ term (and recognizing that the coefficient of the $e^{-j\omega t}$ term is guaranteed to be the complex conjugate.). So we can represent the current as the phasor

$$\widetilde{I}_C = (j\omega C)\widetilde{V}_C.$$

In other words, having already shown that all steady state circuit quantities will be sinusoids with frequency ω in response to the input voltage, we can relate the phasors of the voltage across and the current through a capacitor by a ratio that depends only on the frequency and the capacitance.

This is exactly the same as the s-impedance story we told earlier. When dealing with sinusoidal inputs at frequency ω, we use $s = +j\omega$ and just call the s-impedance, the *impedance*. The $+j\omega$ is understood from context.

As before, this can be thought of as the "resistance" of a capacitor, since it relates the phasor representations of an element's voltage and current by a constant ratio. For a capacitor, the impedance is

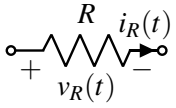
$$Z_C = \frac{\widetilde{V}_C}{\widetilde{I}_C} = \frac{1}{j\omega C}.$$

Interestingly, the impedance for the capacitor is imaginary.

We will now quickly perform a similar analysis for inductors and resistors.

6.2 Impedance of a Resistor

Imagine some resistor R labeled as follows:



Let $v_R(t)$ be represented by some phasor $\widetilde{V_R}$. Thus, by Ohm's Law,

$$\begin{aligned} v_R(t) &= \widetilde{V}_R e^{j\omega t} + \overline{\widetilde{V}}_R e^{-j\omega t} \\ \implies \quad i_R(t) &= \frac{1}{R} v(t) \\ &= \frac{\widetilde{V}_R}{R} e^{j\omega t} + \overline{\frac{\widetilde{V}_R}{R}} e^{-j\omega t}, \end{aligned}$$

so we may represent the output current with the phasor

$$\widetilde{I}_R = \frac{\widetilde{V}_R}{R},$$

so the impedance is

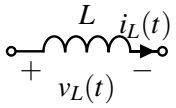
$$Z_R = \frac{\widetilde{V}_R}{\widetilde{I}_R} = R.$$

From this, we see that the impedance behaves very much like the resistance does, except that it generalizes to other circuit components.

6.3 Impedance of an Inductor

From our previous consideration of complex numbers, we have seen that any sinusoidal function can be represented by a phasor. Since we know that our steady states will all be sinusoids with the same frequency ω, we can start with a sinusoidal current and work in the opposite direction to calculate the impedance of an inductor, as follows.

Consider an inductor with voltage and current across it as follows:



Let the current $i_L(t)$ be represented by some phasor $\widetilde{I}_L$. Thus, by the equation of an inductor,

$$\begin{aligned} & i_L(t) = \widetilde{I}_L e^{j\omega t} + \overline{\widetilde{I}}_L e^{-j\omega t} \\ \implies \quad & v_L(t) = L\frac{di_L(t)}{dt} \\ & = (j\omega L)\widetilde{I}_L e^{j\omega t} - (j\omega L)\overline{\widetilde{I}}_L e^{-j\omega t} \\ & = (j\omega L)\widetilde{I}_L e^{j\omega t} + \overline{(j\omega L)\widetilde{I}_L} e^{-j\omega t}, \end{aligned}$$

so the voltage can be represented by the phasor

$$\widetilde{V}_L = j\omega L\widetilde{I}_L.$$

Thus, the impedance of an inductor is

$$Z_L = \frac{\widetilde{V}_L}{\widetilde{I}_L} = j\omega L.$$

6.4 A Remark on Conjugation

When we have an expression like $v(t) = \frac{V_0 e^{j\phi}}{2} e^{j\omega t} + \frac{V_0 e^{-j\phi}}{2} e^{-j\omega t}$, we have consistently been defining the phasor as $\widetilde{V} = \frac{V_0 e^{j\phi}}{2}$, the coefficient of the $e^{j\omega t}$ term. But what about the conjugate phasor $\overline{\widetilde{V}} = \frac{V_0 e^{-j\phi}}{2}$ associated with the complementary exponential, $e^{-j\omega t}$? Why does it not appear in our time-to-phasor-domain transformation?

Well, since the two phasor terms form a complex-conjugate pair, changing one automatically impacts the other. This balance happens in such a way that the ultimate time-domain signal stays fully real-valued. In a sense, having the conjugate phasor is necessary to "cancel out" the imaginary parts from the original phasor. Notice that $e^{j\omega t}$ has a real and imaginary component (think of it as circling around the unit circle in the complex plane.) Its imaginary part can only be cancelled by adding its complex conjugate (for any complex a, $a + \overline{a} = 2\text{Re}\{a\}$, which is fully real.)

7 Circuit Analysis

7.1 Summarizing the Connection Between Time- and Phasor-Domains

Given a general sinusoidal time-domain input signal $u(t) = A\cos(\omega t + \phi)$, we derived a way to conveniently represent $u(t)$ as a weighted sum of a complex exponential $e^{j\omega t}$ and its complex conjugate $e^{-j\omega t}$. This led us to naturally define the phasor, which is a term $\widetilde{U} = \frac{Ae^{j\phi}}{2}$. We showed how the phasor captures all the critical information about $u(t)$, but without the time-dependent terms. This transformation $u(t) \to \widetilde{U}$ is often called a *Phasor-Transform*.

$$\text{Phasor Transform: } u(t) = A\cos(\omega t + \phi) \to u(t) = \frac{Ae^{j\phi}}{2} e^{j\omega t} + \frac{Ae^{-j\phi}}{2} e^{-j\omega t} \implies \widetilde{U} = \frac{Ae^{j\phi}}{2}$$

Using $\widetilde{U}$, we have seen how to analyze the behavior of R, L, C circuit elements to find their voltages and currents. That's great, but we ultimately want to know what the output voltage or current is as a function of time. This motivates the *Inverse Phasor Transform*. The inverse transformation takes some $\widetilde{W} = Be^{j\psi}$ that we've solved for, and converts it into $w(t) = B\cos(\omega t + \psi)$. Notice that the frequency term stayed the

same throughout! This was one of our critical assumptions, related to how the RLC circuits we study only contain elements which preserve the frequency of the sinusoids they act on. They only impact the amplitude and phase.

$$\text{Inverse Phasor Transform:} \quad \widetilde{W} = \frac{Be^{j\psi}}{2} \rightarrow w(t) = B\cos(\omega t + \psi)$$

7.2 KCL with Phasors

In previous sections, we have essentially obtained "equivalents" to Ohm's Law for inductors and capacitors, using the impedance to relate their voltage and current phasors.

We will now try to show that a sum of sinusoidal functions is zero if and only if the sum of the phasors of each of those functions equals zero as well, to obtain a sort of "phasor-version" of KCL. Consider the sinusoids represented by the phasors here:

$$\widetilde{I}_1, \widetilde{I}_2, \ldots, \widetilde{I}_n.$$

Let $i_k(t)$ be the sinusoid represented by the phasor $\widetilde{I}_k$. Observe that:

$$\begin{aligned}
& & \widetilde{I}_1 + \widetilde{I}_2 + \ldots + \widetilde{I}_n &= 0 \\
\iff & & (\widetilde{I}_1 + \widetilde{I}_2 + \ldots + \widetilde{I}_n)e^{j\omega t} &= 0 \\
\iff & & (\widetilde{I}_1 + \widetilde{I}_2 + \ldots + \widetilde{I}_n)e^{j\omega t} + \overline{(\widetilde{I}_1 + \widetilde{I}_2 + \ldots + \widetilde{I}_n)}e^{-j\omega t} &= 0 \\
\iff & & \sum_{k=1}^{n} \widetilde{I}_k e^{j\omega t} + \overline{\widetilde{I}_k} e^{-j\omega t} &= 0 \\
\iff & & i_1(t) + i_2(t) + \ldots + i_n(t) &= 0,
\end{aligned}$$

so we have proved that a sum of sinusoids is zero if and only if the sum of their corresponding phasors is zero as well. This result can be thought of as a generalization of KCL to phasors.

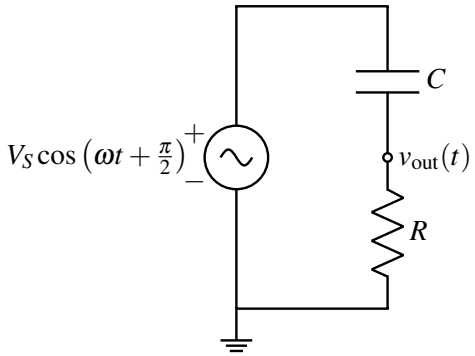
Putting everything together, we have now successfully generalized all of our techniques of DC analysis to frequency analysis.

7.3 Problem-Solving Process

We can outline the key steps to analyze and solve a general circuit using phasors:

(a) Confirm that the circuit can actually be usefully analyzed using phasors. *This requires the voltages and currents to be sinusoidal!*
(b) Convert all $v(t), i(t)$ information to the phasor-domain $(\widetilde{V}, \widetilde{I})$.
(c) Solve for the voltages and currents using EECS16A techniques (KCL, KVL, NVA, etc.)
(d) Convert all phasor results back to the time-domain.

We can now consider some basic circuits, to exercise this technique, and verify that it works correctly. Consider a voltage divider, where we introduce a capacitor in place of one of the resistors, as follows:



We are interested in knowing how the voltage $v_{\text{out}}(t)$ varies over time in response to the input supply, $u(t) = V_S \cos\left(\omega t + \frac{\pi}{2}\right)$. Recall that we proved the voltage divider equation in the context of DC circuit analysis. However, that proof carries over to the phasor domain in a straightforward manner. Thus, the phasor $\widetilde{V}_{\text{o}}$ representing the voltage $v_{\text{out}}(t)$ can be represented in terms of the phasor $\widetilde{U}$ representing the supply voltage as follows:

$$\widetilde{V}_{\text{o}} = \frac{Z_R}{Z_C + Z_R}\widetilde{U},$$

where Z_C and Z_R are the impedances of the capacitor and resistor, respectively. Note also that, since the supply is at frequency ω, all other voltages and currents in the system will also be at the same frequency ω. The circuit elements do not change this frequency. Thus, using our results from earlier, we know that:

$$Z_C = \frac{1}{j\omega C} \qquad Z_R = R$$

Note also that $\widetilde{U} = \frac{V_S e^{j\frac{\pi}{2}}}{2} = \frac{jV_S}{2}$, using our equation for $\cos\theta$ from earlier.

Substituting these values into our equation for $\widetilde{V}_{\text{o}}$, we find that

$$\widetilde{V}_{\text{o}} = \frac{R}{\frac{1}{j\omega C} + R}\frac{jV_S}{2} = \frac{\frac{jR}{2}}{\frac{1}{j\omega C} + R}V_S.$$

It'll be convenient to have a magnitude-phase representation (and to multiply both top and bottom by $j\omega C$ to rationalize the denominator):

$$\begin{aligned}
\widetilde{V}_{\text{o}} &= \frac{V_S \frac{jR}{2} j\omega C}{1 + j\omega RC} \\
&= \frac{-V_S\, \omega RC}{2\,(1 + j\omega RC)} \\
&= \frac{V_S\, \omega RC}{2\sqrt{1 + (\omega RC)^2}} e^{j\left(\pi - \text{atan2}(\omega RC, 1)\right)}
\end{aligned}$$

Now, we can use the Inverse Phasor Transform formula:

$$\begin{aligned} v_{\text{out}}(t) &= \widetilde{V}_{\text{o}} e^{j\omega t} + \overline{\widetilde{V}}_{\text{o}} e^{-j\omega t} \\ &= 2|\widetilde{V}_{\text{o}}| \cos\Big(\omega t + \measuredangle \widetilde{V}_{\text{o}}\Big) \\ &= \frac{V_S\, \omega RC}{\sqrt{1+(\omega RC)^2}} \cos\big(\omega t + \pi - \text{atan2}(\omega RC, 1)\big). \end{aligned}$$

This formula might look complicated, but it'll become significantly simpler when we have actual values of R, C, ω to plug in. An example of this will be at the start of the next Note!

A Warning

Be aware that in this course phasors are defined slightly differently from how it is often done elsewhere. Essentially, there is a factor of 2 difference.

In this course, we define the phasor representation $\widetilde{X}$ of a sinusoid $x(t)$ to be such that

$$x(t) = \widetilde{X} e^{j\omega t} + \overline{\widetilde{X}} e^{-j\omega t}.$$

However, elsewhere, the phasor representation may be defined such that

$$x(t) = \frac{1}{2}(\widetilde{X} e^{j\omega t} + \overline{\widetilde{X}} e^{-j\omega t}).$$

Our definition is more natural and aligns to what you will see in later courses when you learn about Laplace and Fourier transforms. This is because our definition arises from the mathematics, and the same spirit of definition works even when working with inputs of the form e^{st} where s is not a purely imaginary number.

But then why would anyone ever use the alternative, more common definition? Its main advantage is that the magnitude of the phasor equals the amplitude of the signal. For instance, if we have the signal $A\cos(\omega t + \phi)$, then the alternative definition yields the phasor $Ae^{j\phi}$, with magnitude A. In contrast, our definition yields the phasor $(A/2)e^{j\phi}$. The former definition is convenient when conducting physical observations - when using an oscilloscope, one can easily see[4] the amplitude A of a signal, not the half-amplitude $A/2$.

Furthermore, it turns out that there are some slight calculation advantages (i.e. it makes some formulas simpler) to the more common definition when working with power systems and power electronics, which you may see if you take the relevant upper-division EE courses. However, for the purposes of the scope of this course, our definition is simpler and easier to understand, so we will stick with it throughout.

Of course, if the mathematics is done correctly, there is no real difference between the two definitions, in that both describe the same physical behaviors. It is just easier to do the mathematics correctly with the definiton we use here.

Contributors:

- Neelesh Ramachandran.

[4] Actually in practice, if there is a DC component to the circuit — i.e. there are some inputs that are constants too — then the easiest thing to see is the peak-to-peak swing of the voltage which corresponds to twice the amplitude. So even the more common definition often forces the person using it to have to divide by two.

- Rahul Arya.
- Anant Sahai.
- Jaijeet Roychowdhury.

EECS 16B Designing Information Devices and Systems II

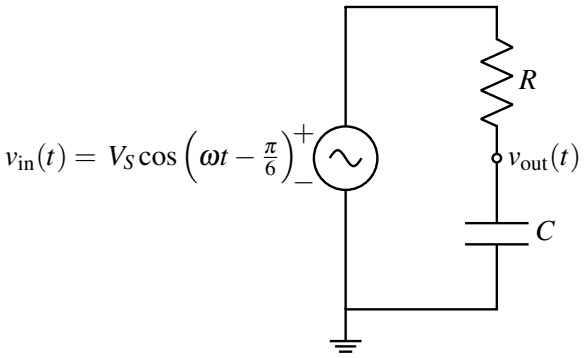
Note 5: Circuit Filters

Overview

In the previous note, we developed the method of Phasor Domain Analysis to analyze the steady state behavior of circuits with sinusoidal voltage and current sources. In this note, we will consider some applications of these circuits, and begin to explore techniques for designing these circuits to fit a set of requirements. While this note may seem very long, much of the space is consumed by figures, and each example provided comes with a complete set of steps.

1 Circuit Example and Transfer Functions

In the previous note, we analyzed a circuit that resembled a voltage divider, but replaced the first (top) resistor with a capacitor. Below, we consider a similar circuit where the roles of the capacitor and resistor are swapped. Our notation has changed, but only very slightly ($\widetilde{U} \to \widetilde{V}_{\text{in}}$, $\widetilde{V}_{\text{o}} \to \widetilde{V}_{\text{out}}$). We will also assign specific values to the input voltage and circuit elements to get a feel for a "complete" solution.



Let's say that $V_S = 5\text{V}$, $\omega = \sqrt{3} \cdot 10^5 \frac{\text{rad}}{\text{s}}$, $R = 1\text{k}\Omega$, and $C = 10\text{nF}$. We can calculate that the input voltage phasor $\widetilde{V}_{\text{in}}$ corresponding to $v_{\text{in}}(t) = V_S \cos\left(\omega t - \frac{\pi}{6}\right)$ is:

$$\widetilde{V}_{\text{in}} = \frac{V_S e^{-j\frac{\pi}{6}}}{2} = \frac{5e^{-j\frac{\pi}{6}}}{2}$$

Applying the equation for a voltage divider, we find the voltage phasor at $v_{\text{out}}(t)$ to be:

$$\widetilde{V}_{\text{out}} = \frac{\frac{1}{j\omega C}}{R + \frac{1}{j\omega C}} \widetilde{V}_{\text{in}}.$$

Keeping our results symbolic just a little while, we will look at the ratio of the output phasor to the input phasor. This term, $\frac{\widetilde{V}_{\text{out}}}{\widetilde{V}_{\text{in}}}$, is called the circuit's *transfer function* (denoted $H(\omega)$). It relates the output of the

circuit to the input while capturing the frequency dependence.

$$H(\omega) = \frac{\widetilde{V}_{\text{out}}}{\widetilde{V}_{\text{in}}} = \frac{\frac{1}{j\omega C}}{R + \frac{1}{j\omega C}}.$$

More broadly, transfer functions map an angular frequency ω to a ratio of two phasors - an input and an output - and are used to characterize the behavior of a system across a range of frequencies.[1]

Observe that as transfer functions are the ratio of two phasors, they are complex-valued in general. Thus, we can write

$$H(\omega) = |H(\omega)| e^{j\angle H(\omega)},$$

where $|H(\omega)|$ is the magnitude of the transfer function, and $\angle H(\omega)$ is the phase of the transfer function. In this example, we can rearrange $H(\omega)$ to become:

$$\begin{aligned} H(\omega) &= \frac{\frac{1}{j\omega C}}{R + \frac{1}{j\omega C}} = \frac{1}{1 + j\omega RC} \\ &= \frac{1}{1 + j\sqrt{3}} \end{aligned}$$

Now, we can compute the information we need about $H(\omega)$, symbolically and numerically. First, the magnitude[2]:

$$|H(\omega)| = \frac{1}{\sqrt{1 + (\omega RC)^2}} = \frac{1}{2}$$

Now, the phase[3]:

$$\begin{aligned} \angle H(\omega) = \angle(1) - \angle(1 + j\omega RC) &= -\operatorname{atan2}(\omega RC, 1) \\ &= -\operatorname{atan2}\left(\sqrt{3}, 1\right) \\ &= -\frac{\pi}{3} \end{aligned}$$

Now, we can compute the voltage phasor $\widetilde{V}_{\text{out}}$ using $H(\omega)$, $\widetilde{V}_{\text{in}}$ as follows:

$$\begin{aligned} \widetilde{V}_{\text{out}} &= H(\omega)\widetilde{V}_{\text{in}} \\ &= |H(\omega)| e^{j\angle H(\omega)} \widetilde{V}_{\text{in}} \\ &= \frac{1}{2} e^{-j\frac{\pi}{3}} \frac{5e^{-j\frac{\pi}{6}}}{2} \\ &= \frac{5}{4} e^{-j\frac{\pi}{2}} \end{aligned}$$

[1] Here, our transfer function relates two voltage phasors, but they can also relate two current phasors, or a voltage phasor with a current phasor, in a similar manner.

[2] Recall that $|\frac{z_1}{z_2}| = \frac{|z_1|}{|z_2|}$, where z_1, z_2 are both complex numbers $(a + jb)$ with magnitudes as $|z| = \sqrt{a^2 + b^2}$

[3] $\angle\left(\frac{z_1}{z_2}\right) = \angle(z_1) - \angle(z_2)$, where $\angle(z) = \operatorname{atan2}(b, a)$

Now that we have $\widetilde{V}_{\text{out}}$, we can solve for $v_{\text{out}}(t)$ using the Inverse Phasor Transform[4]:

$$\begin{aligned} v_{\text{out}}(t) &= \widetilde{V}_{\text{out}} e^{j\omega t} + \overline{\widetilde{V}_{\text{out}}} e^{-j\omega t} \\ &= \frac{5}{4} e^{-j\frac{\pi}{2}} e^{j\omega t} + \frac{5}{4} e^{j\frac{\pi}{2}} e^{-j\omega t} \\ &= \frac{5}{4}\left(e^{j\left(-\frac{\pi}{2}+\omega t\right)} + e^{j\left(\frac{\pi}{2}-\omega t\right)}\right) \\ &= \frac{5}{4}\left(e^{-j\left(\frac{\pi}{2}-\omega t\right)} + e^{j\left(\frac{\pi}{2}-\omega t\right)}\right) \\ &= \frac{5}{2}\cos\left(\sqrt{3}\cdot 10^5 t - \frac{\pi}{2}\right), \end{aligned}$$

We see here that the transfer function will scale the input signal by a factor of $H(\omega)$, and shift its phase by an angle $\measuredangle H(\omega)$. Equipped with the notion of a transfer function that scales and delays an input sinusoid, we can follow the following circuit analysis procedure:

- Express the sinusoidal input (current or voltage) as a phasor.
- Solve for the output phasor(s), which can be voltages or currents, as functions of the input phasor, and use the output to determine the transfer function.
- Observe the magnitude and phase shift of the transfer function at the specific input frequency to determine the behavior of the circuit.

Note that we have previously solved this same example involving the output of an RC circuit with a cosine input, purely in the time domain. That process involved far more calculation and painful trigonometry to simplify the end result. The usage of phasors simplifies our calculations greatly.

2 Filters

One very common use for AC circuits is as a *filter*, which blocks certain categories of frequencies. The key idea behind filters is that of *superposition*. In EECS16A, we saw that in circuits with independent sources, the circuit could be analzyzed with only 1 independent source one at a time (with the rest zeroed out).[5] Then, the final signal was the sum of the intermediate responses from each source.

Here, rather than looking at the superposition of independent sources, we will look at the superposition of input signals with different frequencies. Specifically, let's represent our generalized input signal as follows:

$$v_{\text{in}}(t) = \sum_i \left(\widetilde{U}_i e^{j\omega_i t} + \overline{\widetilde{U}_i} e^{-j\omega_i t}\right)$$

where each ω_i is some frequency with an associated phasor $\widetilde{U}_i$. Applying this input to a circuit with transfer function $H(\omega)$, the output voltage will be:

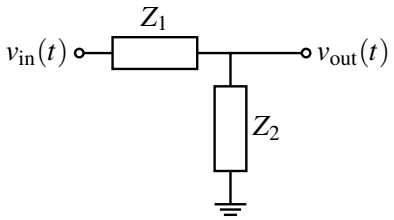
$$v_{\text{out}}(t) = \sum_i \left(\left(H(\omega_i)\widetilde{U}_i\right) e^{j\omega_i t} + \left(\overline{H}(\omega_i)\overline{\widetilde{U}_i}\right) e^{-j\omega_i t}\right)$$

That is, we can treat the superposition of signals of two different frequencies as if the two signals were separate. Each frequency sinusoid is affected by the circuit as governed by the transfer function evaluated at that frequency.

[4] If you can arrive the final answer by inspection, that's great! The steps are here for completeness.

[5] If there are dependent sources, this doesn't hold!

Now, we will consider various filter configurations, and study their behavior. For now, we will look at *L-filters*, which are filters consisting of two elements placed in series, as shown:

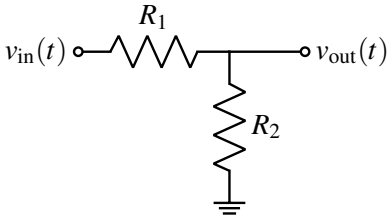


Recall from the impedance divider equation that the transfer function of this circuit is:

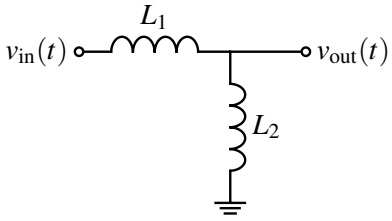
$$H(\omega) = \frac{Z_2}{Z_1 + Z_2}.$$

We will now consider all the possible forms of impedances that we can substitute into this equation. There are nine possible configurations, and we divide them into 3 main categories: Frequency-Independent, Low-Pass, High-Pass, and LC Tank.

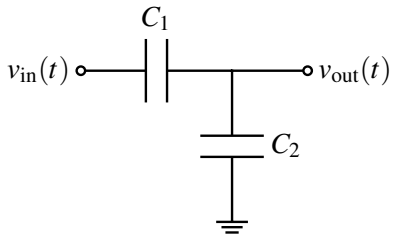
2.1 Frequency-Independent (RR, CC, LL)



(a) This forms a simple resistive voltage divider.



(b) A frequency-independent voltage divider using inductors.



(c) A frequency-independent voltage divider using capacitors.

Figure 1: Frequency-Independent $L-$filters.

R-R: Both components are resistors. We can compute:

$$H(\omega) = \frac{R_2}{R_1 + R_2}$$

Notice there's no frequency dependence here, which makes sense since this circuit only contains resistors.

L-L: Both components are inductors. Substituting the impedance of an inductor into our equation for the transfer function, we obtain:

$$H(\omega) = \frac{j\omega L_2}{j\omega L_1 + j\omega L_2} = \frac{L_2}{L_1 + L_2}$$

So,this circuit exhibits behavior similar to a resistive voltage divider (once again, independently of frequency). Notice, however, that unlike resistors, inductors do not dissipate energy.[6].

[6] A natural question to ask would be: what happens to the energy if we supply a DC voltage ($\omega = 0$) to this circuit? Surely there will be some current flowing between the two potentials, which would cause energy to be dissipated, but where can this energy go? The resolution to this issue relies on the observation that our expression for $H(\omega)$ resolves to $0/0$ for $\omega = 0$, and since our simplifications assumed that $\omega \neq 0$ (allowing us to divide by $j\omega$), the pure voltage-divider formula doesn't quite hold. In the case of $\omega = 0$, we can no longer model the inductors as being ideal, and their internal resistance will dissipate energy

C-C: Both components are capacitors. Substituting in the impedance of a capacitor:

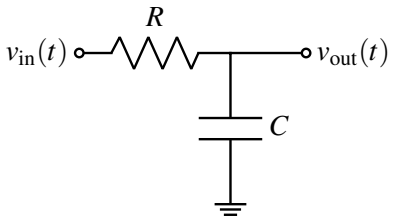
$$H(\omega) = \frac{\frac{1}{j\omega C_2}}{\frac{1}{j\omega C_1} + \frac{1}{j\omega C_2}} = \frac{C_1}{C_1 + C_2}.$$

Again, we get something very similar to a voltage divider. Here, however, when $\omega = 0$ no current is permitted to flow, since the capacitors act as open circuits. A charge-sharing argument would nevertheless demonstrate that $v_{\text{out}}(t)$ will remain as we have claimed.

Looking at these circuits' transfer functions, we see why we call them frequency-independent. Because the elements' interaction with frequency is of the same nature, it is cancelled in the transfer function expression. So we end up with transfer functions that don't contain ω terms.

2.2 Low-Pass (RC, LR)

R-C: We have a circuit with a resistor and capacitor, and we take the voltage across the capacitor.



We can compute the transfer function (the same as we calculated at the start of this note):

$$H(\omega) = \frac{\frac{1}{j\omega C}}{R + \frac{1}{j\omega C}} = \frac{1}{1 + j\omega RC}$$

Now we see some more interesting behavior, since the transfer function depends on frequency. But how can we see that this is a low-pass filter? As a quick intuition check, if $\omega = 0, H(\omega) = 1$ and if $\omega \to \infty$, then $H(\omega) \to 0$. This makes sense, but what happens in between 0 and ∞? Let's take a look at a couple of values[7] around a special frequency value, $\omega_c = \frac{1}{RC}$:

ω	$H(\omega)$	$\lvert H(\omega)\rvert$	$\angle H(\omega)$
$0.1\omega_c$	$\frac{1}{1 + j\,0.1}$	0.995	$-6°$
ω_c	$\frac{1}{1 + j}$	0.71	$-45°$
$10\omega_c$	$\frac{1}{1 + j\,10}$	0.1	$-84°$

We see here that $\omega_c = \frac{1}{RC}$ is a very important frequency to look at since this is around the frequency where the behavior of the filter starts to qualitatively change.[8] In fact, this is so important that we call this the **cutoff** frequency. Signals far below this frequency are allowed fully through, while signals above this frequency

[7] We will soon plot such transfer functions over a continuum of frequencies.

[8] Note how this is the reciprocal of the time constant from back when we studied inverter transitions. We'll explore the significance of τ in a later section.

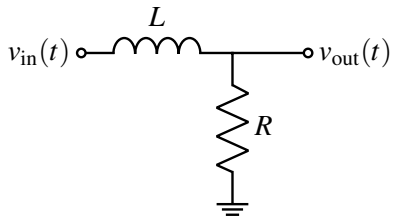
are largely blocked. So, this RC circuit constitutes a low-pass filter. We can write the low-pass RC transfer function in terms of ω_c: $H(\omega) = \frac{1}{1+\frac{j\omega}{\omega_c}}$.

We define the cutoff frequency ω_c,[9] as the point where:

$$|H(\omega_c)| = \frac{\max_\omega |H(\omega)|}{\sqrt{2}}$$

Here, $\max_\omega H(\omega)$ is the maximum magnitude of $H(\omega)$ over the full frequency range. For passive circuits containing passive components like R, L, C elements, this will usually be 1.

L-R: We have a circuit with a resistor and inductor, and we take the voltage across the resistor.



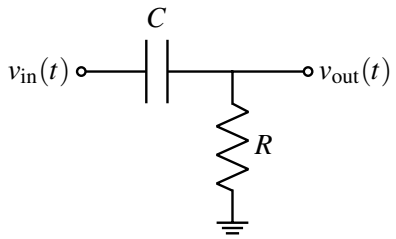
The above has the transfer function

$$H(\omega) = \frac{R}{R + j\omega L} = \frac{1}{1 + j\omega\frac{L}{R}} = \frac{1}{1 + {}^{j\omega}/_{\omega_c}}$$

with $\omega_c = \frac{R}{L}$. Here, $\tau = \frac{L}{R}$, as we saw in the transient analysis for an RL circuit (dis04B).

2.3 High-Pass (CR, RL)

C-R: We have a circuit with a resistor and capacitor, and we take the voltage across the resistor.



We compute:

$$H(\omega) = \frac{R}{R + \frac{1}{j\omega C}} = \frac{j\omega RC}{1 + j\omega RC}$$

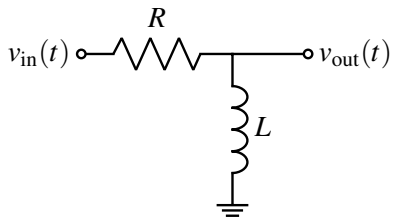
The qualitative behavior we see is different from the low-pass filters above; let's generate a table as we did before ($\omega_c = \frac{1}{RC}$):

[9] ω_c goes by many names, including cuttof frequency, corner frequency, critical frequency, and -3 dB frequency. If you see any of these, they all mean ω_c.

ω	$H(\omega)$	$\lvert H(\omega)\rvert$	$\angle H(\omega)$
$0.1\omega_c$	$\frac{j\,0.1}{1+j\,0.1}$	0.1	84°
ω_c	$\frac{j}{1+j}$	0.71	45°
$10\omega_c$	$\frac{j\,10}{1+j\,10}$	0.995	6°

Here, instead of passing low frequencies and blocking high frequencies, this circuit does the opposite!

R-L: We have a circuit with a resistor and inductor, and we take the voltage across the inductor.

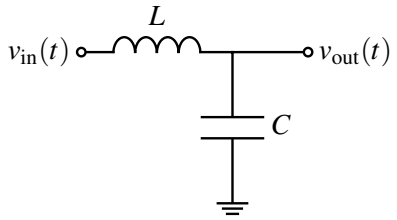


We compute ($\omega_c = \frac{R}{L}$):

$$H(\omega) = \frac{j\omega L}{R + j\omega L} = \frac{j\omega\frac{L}{R}}{1 + j\omega\frac{L}{R}} = \frac{\frac{j\omega}{\omega_c}}{1 + \frac{j\omega}{\omega_c}}$$

2.4 LC Tank (LC, CL)

We introduced LC circuits like the one below in Note 3B, and observed how energy in such a circuit alternates between being stored in the inductor's magnetic field and the capacitor's electric field.



We can compute its transfer function in the standard way and arrive at the expression below, but we will study them more in the context of resonance in RLC circuits.

$$H(\omega) = \frac{\frac{1}{j\omega C}}{\frac{1}{j\omega C} + j\omega L} = \frac{1}{1 - \omega^2 LC}$$

2.5 Summary of Filters

Well, that was a lot of filters! But looking closely at their transfer functions, we primarily see two distinct forms:

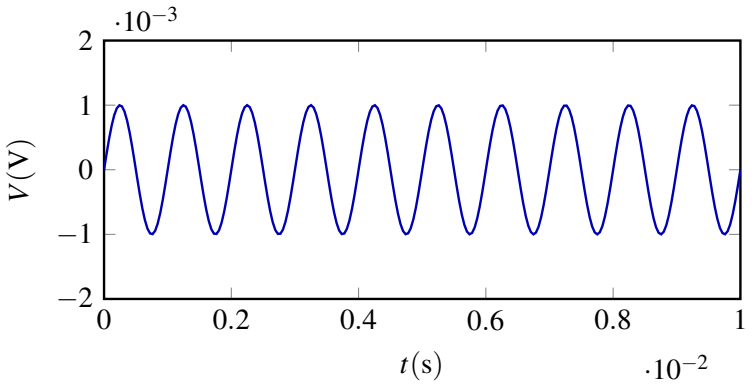
$$H_{LP}(\omega) = \frac{1}{1 + {}^{j\omega}/_{\omega_c}} \qquad H_{HP}(\omega) = \frac{{}^{j\omega}/_{\omega_c}}{1 + {}^{j\omega}/_{\omega_c}}$$

The first of these two transfer functions describes a *low-pass filter*, since for values of $\omega \to 0$, $H(\omega) \to 1$, while for values of $\omega \to \infty$, $H(\omega) \to 0$. In essence, low frequencies can pass through, while high frequencies cannot. In a symmetric manner, the second of these transfer functions describes a *high-pass filter*, which has the opposite behavior.

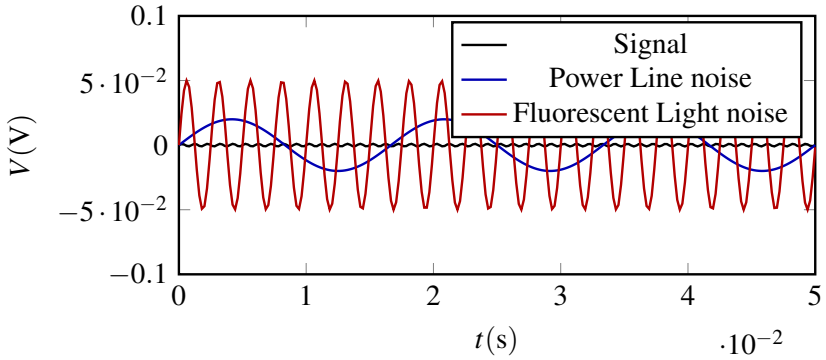
3 Design

Now that we have a better understanding of low- and high-pass filters, it's time to look at a toy example to understand why filters are important in practical applications. We also will recognize how useful it will be to have more powerful visualization tools than the tables in the previous section, and we will cover such tools in Note 6 on Bode Plots.

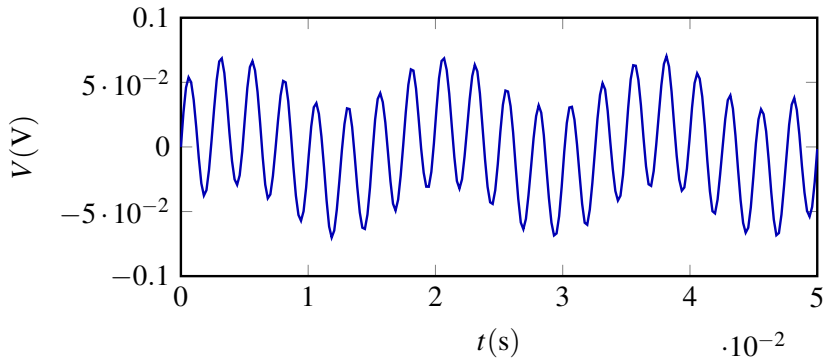
Imagine that we have a signal with amplitude 1mV and frequency approximately 1kHz, as shown:



Now, we introduce some 60Hz noise (a common contamination from power lines) at a stronger 20mV, as well as some 100kHz noise (such as from a malfunctioning fluorescent light ballast) at 50mV. These 3 waveforms (1 desired signal, and 2 undesired sources of noise) are shown below:



Adding these noise sources to our signal, we obtain the result:



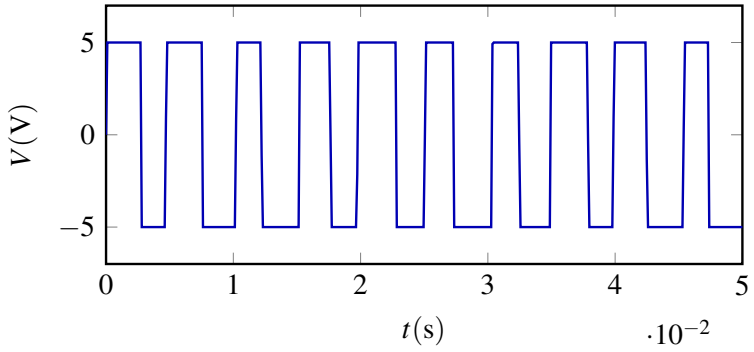
At this point, our original signal is not even visible. But, there's still hope! Someone might have told you about the magic of the Fourier transform[10], which can break up a signal into its constituent sinusoids. All we need to do is record this signal precisely in a digital format, send it to a computer, and let a software library figure the rest out.

Unfortunately, we are likely encoding this signal using something akin to a 4-bit ADC, much like the one from lab. If our ADC has a maximum voltage of 3.3 V, then the least significant bit indicates a precision of:

$$\frac{3.3\text{V}}{2^4} \approx 0.2\text{V}.$$

But our signal only has an amplitude of 1mV! What can we do? One natural option is to amplify our signal, such as by using an op-amp circuit, until the amplitude of our signal is on the same order as our measurement ability. Suppose we amplify our signal by a factor of $\frac{3.3\text{V}}{1\text{mV}} = 3300$. Unfortunately, when we do this, our noise signals will shoot up, reaching amplitudes of $50\text{mV} \cdot 3300 = 165\text{V}$!

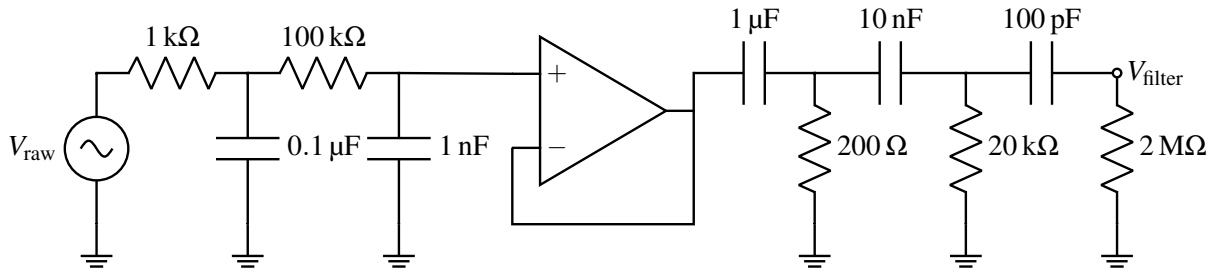
Of course, our op-amps can only produce voltages between their rails (perhaps around $\pm 5\,\text{V}$). So our amplified signal will rail much of the time. Plotting the end result of this amplification, we obtain[11]:



Notice that our signal is almost always railing, meaning that information has been irretrievably lost before even arriving at a computer. Therefore, analog processing is necessary to remove the noise. We want to remove both the high and low frequency noise while maintaining the amplitude of our signal frequency. We could then amplify the signal, without worrying about our op-amp railing. To do so, consider the following circuit:

[10] We will actually deal with this at the end of 16B.

[11] The diagram is actually fairly inaccurate due to limitations of the plotting software - it should really look more like a solid block of color.



The final amplification of V_{signal} is omitted for reasons of space, but can be constructed using an non-inverting amplifier.

Recall that our analysis of $L-$filters relied heavily on the voltage divider equation. That equation, in turn, assumed that all the current flowing into the top impedance flowed out of the bottom one. But here, some current flows out of the top impedance of each filter and into the next one in series (that is, later filtering stages load the previous ones). There are two ways to handle this issue. This first is to use a unity gain buffer, as done here between the low-pass and high-pass filter chains, but is not done between all pairs of adjacent individual filters.

Another option (that we selected here) is to try to make that current draw (the *load* of a filter) as low as possible, by increasing the impedance of subsequent filters. On the low-pass side, notice that the resistance of the resistor jumps by a factor of 100 from the first filter to the second, as does the impedance of the capacitors.[12] On the high-pass side, we again increase the impedance of each filter by a factor of 100 as well (for both the resistor and capacitor). Thus, each consecutive filter imposes only a very small load on the previous filters, and we approximate the filters as being independent.

Of course, it may not be clear why chaining multiple low-pass or high-pass filters is necessary. Why not just use a single filter of each type? The logic is that we want to built **higher-order** filters, described more (and with plots) in the next Note. Notice that the transfer function $H(\omega)$ of two filters chained together is the product of their two transfer functions $H_1(\omega)$ and $H_2(\omega)$.[13] Consider the case of two low-pass filters with time constant $\tau = RC$ driven by a frequency significantly greater than ω_p. Making linear approximations:

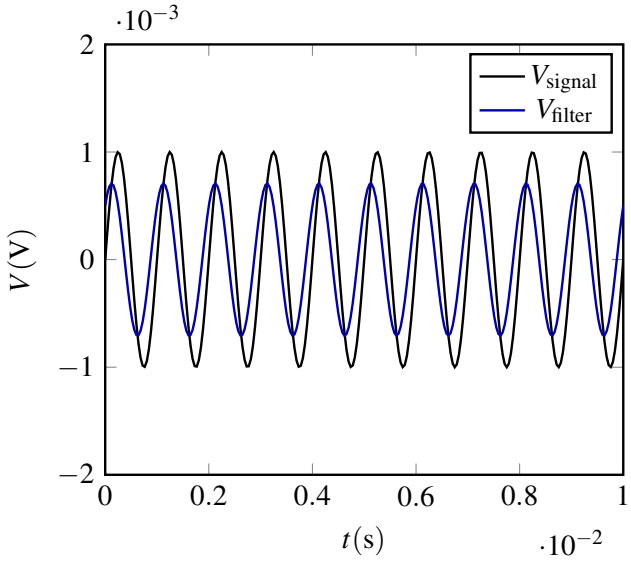
$$|H_1(\omega)| \approx \frac{1}{\omega RC}$$
$$|H_2(\omega)| \approx \frac{1}{\omega RC}$$
$$|H(\omega)| \approx \frac{1}{(\omega RC)^2}.$$

Notice that $|H(\omega)|$ drops with ω^2, rather than ω. If we were to plot it on a log-log plot, its slope would be -2, whereas $H_1(\omega)$ and $H_2(\omega)$ would only have a slope of -1. The magnitude of this slope is known as a filter's *roll-off*. Chaining filters enables a greater roll-off, so it attenuates undesired-frequency signals much more.

Looking at the ultimate end-result V_{filter} versus V_{signal}, we observe:

[12] A capacitor's impedance is inversely proportional to its capacitance, so reducing the capacitance actually increases the impedance.

[13] Assuming that the second filter draws very little current from the first, as is true here.



Notice that we have successfully used our low- and high-pass filters to reject undesired frequencies! However, we have attenuated our signal a bit, and its phase has also shifted. We may have to adjust our amplification factor a bit to compensate for this attenuation, but we can now actually measure our signal with just a 4-bit DAC! This issue of noise sources of different frequencies contaminating the desired signal is ubiquitous in circuit design, so it's quite nice that we can develop a solution to this problem using our knowledge about filters!

Contributors:

- Neelesh Ramachandran.
- Rahul Arya.
- Anant Sahai.
- Jaijeet Roychowdhury.
- Taejin Hwang.

EECS 16B Designing Information Devices and Systems II

Note 6: Bode Plots

Overview

Having analyzed our first order filters and gone through a design example in the previous Note to show why filter-design is important, we will now plot their transfer functions $H(\omega)$ (or frequency responses) using Bode Plots. In the previous Note, we generated tables of $|H(\omega)|$, $\angle H(\omega)$ at certain key values of ω_c, and while this gave some intuition, it didn't really show what happens at intermediate frequencies. There is immense value in visualizing transfer functions across a wide range of frequencies.

Throughout this section, we will use numerical approximations, which will not only prove useful in plotting a filter's frequency response but also will help us better understand its behavior.

1 Bode Plots

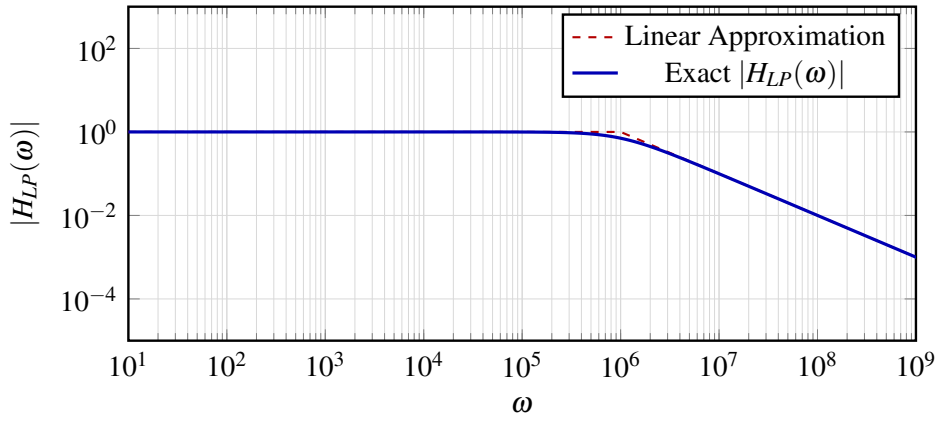
When we make Bode plots, we plot the frequency and magnitude on a logarithmic scale, and the angle in either degrees or radians. We use the logarithmic scale because it allows us to break up complex transfer functions into its constituent components. Let's start by generating Bode Plots for low-pass and high-pass first order filters, which will build our intuition. We will soon see how the analysis of more complex transfer functions can be broken down into parts.

1.1 Low-pass Filter

Recall our generalized model of a low-pass filter (perhaps RC or LR):

$$H_{LP}(\omega) = \frac{1}{1 + j\omega/\omega_c}$$

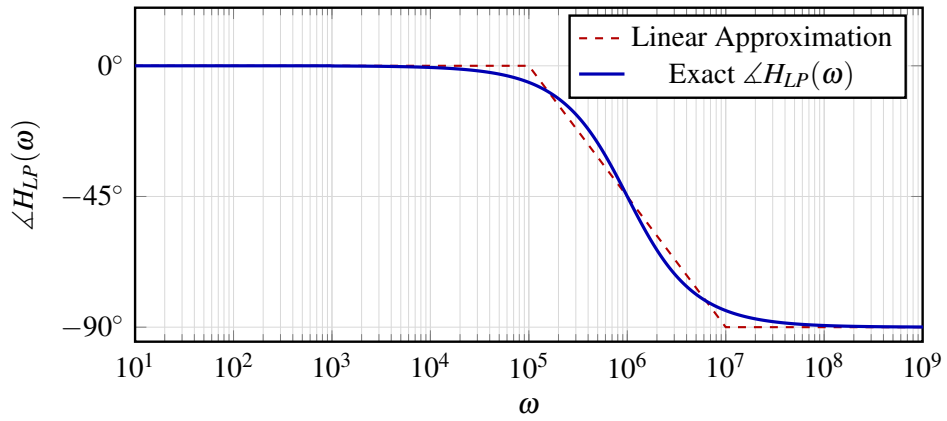
We plot the magnitude of the frequency response assuming $\omega_c = 10^6$. Note how $|H_{LP}(\omega)|$ is very close to



1 for $\omega < \omega_c$ and $|H_{LP}(\omega)|$ starts dropping off with slope -1 after ω_c. We can look at 3 distinct regions on the plot:

- $\omega \ll \omega_c$: $\implies j\omega/\omega_c \approx 0$. So, $H_{LP}(\omega) \approx 1$ and $|H_{LP}(\omega)| \approx 1$.
- $\omega = \omega_c$: $\implies H(\omega) = \frac{1}{1+j}$. So, $|H_{LP}(\omega)| = \frac{1}{\sqrt{2}}$.
- $\omega \gg \omega_c$: $\implies \omega/\omega_c \gg 1$. Therefore $H_{LP}(\omega) \approx -j\frac{\omega_c}{\omega}$. So, $|H_{LP}(\omega)| \approx \frac{\omega_c}{\omega}$. On a log scale, this means that $\log|H_{LP}(\omega)| \approx \log\omega_c - \log\omega$ explaining behavior of dropping off with slope -1.[1]

Now let's plot the phase of $H_{LP}(\omega)$: $\angle H_{LP}(\omega)$ is very close to 0 for $\omega < 0.1\omega_c$ and $\angle H_{LP}(\omega)$ is approxi-



mately $-\frac{\pi}{2}$ for $\omega > 10\omega_c$.

- $\omega \ll 0.1\omega_c \implies j\omega/\omega_c \approx 0$. So, $H_{LP}(\omega) \approx 1$ and $\angle H_{LP}(\omega) \approx 0$.
- $\omega = 0.1\omega_c \implies H_{LP}(0.1\omega_c) = \frac{1}{1+j\,0.1}$ and $\angle H_{LP}(\omega) \approx -6°$.
- $\omega = \omega_c \implies H_{LP}(\omega_c) = \frac{1}{1+j}$ and $\angle H_{LP}(\omega) = -45°$.
- $\omega = 10\omega_c \implies H_{LP}(10\omega_c) = \frac{1}{1+j\,10}$ and $\angle H_{LP}(\omega) \approx -84°$.
- $\omega \gg 10\omega_c \implies \omega/\omega_c \gg 10$. So, $H_{LP}(\omega) \approx -j \cdot 0$ [2] and $\angle H_{LP}(\omega) \approx -90°$.

We can now better understand the values of the magnitude and phase at $0.1\omega_c$, ω_c, $10\omega_c$ (as seen in the tables of Note 5).

1.2 High-pass Filter

We can similarly analyze our generalized high-pass filter model (CR, RL):

$$H_{HP}(\omega) = \frac{j\omega/\omega_c}{1 + j\omega/\omega_c}$$

we plot the magnitude of the frequency response, again assuming $\omega_c = 10^6$.

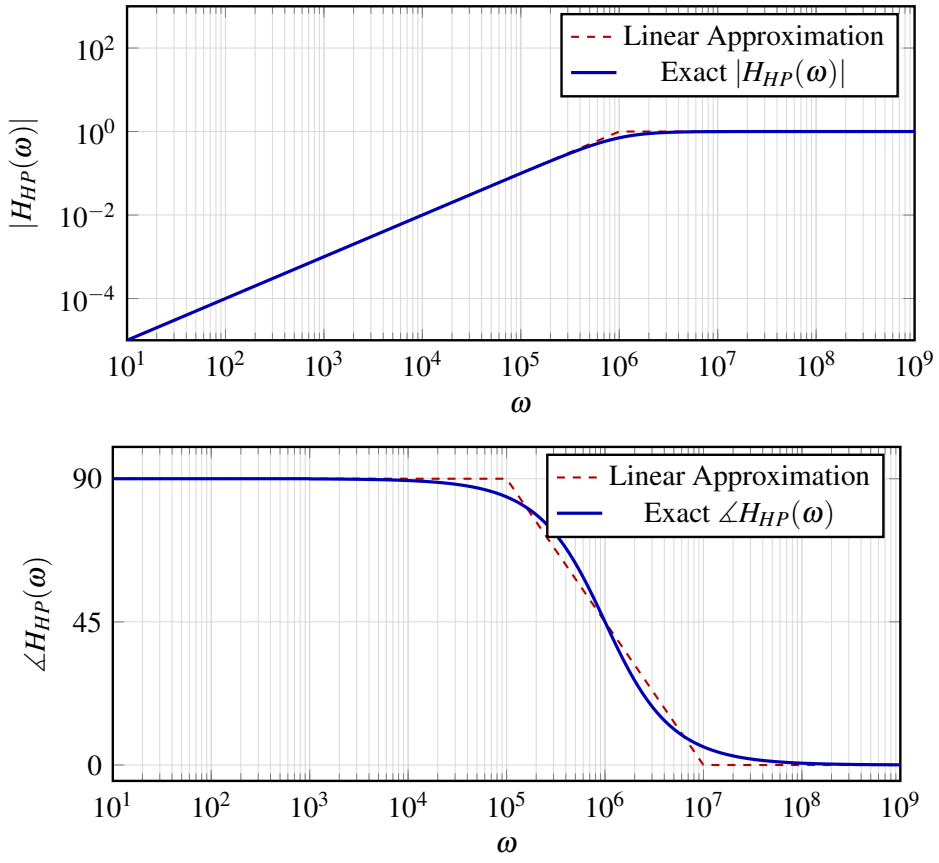
Here, $|H_{HP}(\omega)|$ rises with slope 1 for $\omega < \omega_c$ and $|H_{HP}(\omega)| \approx 1$ after ω_c. We analyze the plot:

- $\omega \ll \omega_c$, then $\omega/\omega_c \ll 1$. Therefore $H_{HP}(\omega) \approx j\frac{\omega}{\omega_c}$ which implies $|H_{HP}(\omega)| \approx \frac{\omega}{\omega_c}$. On a log scale, this means that $\log|H_{HP}(\omega)| \approx \log\omega - \log\omega_c$, which explainsthe rising slope of 1.
- $\omega = \omega_c$, then $H(\omega) = \frac{j}{1+j}$ meaning $|H_{HP}(\omega)| = \frac{1}{\sqrt{2}}$
- $\omega \gg \omega_c$, then $\omega/\omega_c \gg 1$. Therefore $H_{HP}(\omega) \approx 1$ which implies $|H_{HP}(\omega)| \approx 1$.

Now let's plot the phase of the transfer function $H_{HP}(\omega)$.

[1] Recall that the line $y = mx + b$ has slope m. In this case $y = \log|H_{LP}(\omega)|$ and $x = \log|\omega|$.

[2] The magnitude will always be greater than 0, meaning its phase will still be very close to $-90°$



$\measuredangle H_{HP}(\omega)$ is very close to $\frac{\pi}{2}$ for $\omega < 0.1\omega_c$ and $\measuredangle H_{HP}(\omega)$ is approximately 0 for $\omega > 10\omega_c$.

- $\omega \ll 0.1\omega_c \implies j\omega/\omega_c \approx 0$. So, $H_{HP}(\omega) \approx 0$ (but slightly positive) and so $\measuredangle H_{HP}(\omega) \approx 90°$.
- $\omega = 0.1\omega_c \implies H_{HP}(0.1\omega_c) = \frac{j\ 0.1}{1+j\ 0.1}$ and $\measuredangle H_{HP}(\omega) \approx 84°$.
- $\omega = \omega_c \implies H_{HP}(\omega_c) = \frac{j}{1+j}$ and $\measuredangle H_{HP}(\omega) = 45°$.
- $\omega = 10\omega_c \implies H_{HP}(10\omega_c) = \frac{j\ 10}{1+j\ 10}$ and $\measuredangle H_{HP}(\omega) \approx 6°$.
- $\omega \gg 10\omega_c \implies \omega/\omega_c \gg 10$. So, $H_{HP}(\omega) \approx 1$ and $\measuredangle H_{HP}(\omega) \approx 0°$.

2 Second Order Filters

We will now consider more complex systems and, in doing so, see the value in appropriate visualizations for transfer function behavior.

2.1 Band-Pass Filters

With the knowledge of low-pass filters that block out higher frequencies and high-pass filters that block out lower frequencies, how could we build a filter that lets a specific range of frequncies through? One idea could be to take the output of the low-pass filter and treat it as an input to the high-pass filter.

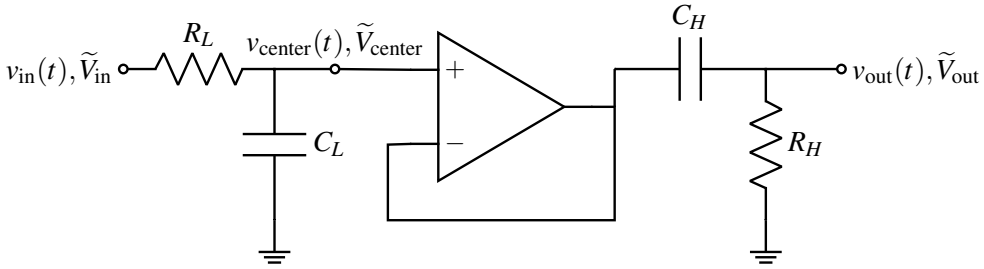


Figure 1: A Buffered Band-Pass filter, composed of low-pass and high-pass filter components.

Notice the op-amp serving as a unity gain buffer between the two filters. It is introduced to prevent the second circuit from loading the first.

The input voltage phasor is $\widetilde{V}_{\text{in}}$ at some frequency ω. Suppose the two filters have cutoff frequencies ω_{LP} and ω_{HP}, respectively. Thus, we can write that:

$$\widetilde{V}_{\text{center}} = H_{LP}(\omega)\widetilde{V}_{\text{in}}$$

Since $v_{\text{center}}(t)$ is the second filter's input, the output voltage phasor $\widetilde{V}_{\text{out}}$ is:

$$\widetilde{V}_{\text{out}} = H_{HP}(\omega)\widetilde{V}_{\text{center}} = H_{HP}(\omega)H_{LP}(\omega)\widetilde{V}_{\text{in}}$$

Thus, the net transfer function $H_{BP}(\omega)$ is:

$$H_{BP}(\omega) = H_{LP}(\omega)H_{HP}(\omega)$$

More generally, placing filters in series produces a circuit whose transfer function is the product of the individual transfer functions. From the properties of complex numbers and logarithms, we see that $\log|z_1 z_2| = \log|z_1| + \log|z_2|$. So, when plotting $|H_{BP}(\omega)|$ on a log-log plot, this corresponds to a graphical addition of the individual transfer functions. Note that we are still taking the product of the actual transfer functions, but it resembles a geomtrical sum: $|H_{LP}(\omega)| + |H_{HP}(\omega)|$. We see this idea on display in the examples at the end of this note. Similarly, for $\angle H_{BP}(\omega)$, we can again plot the sum, $\angle H_{LP}(\omega) + \angle H_{HP}(\omega)$.[3]

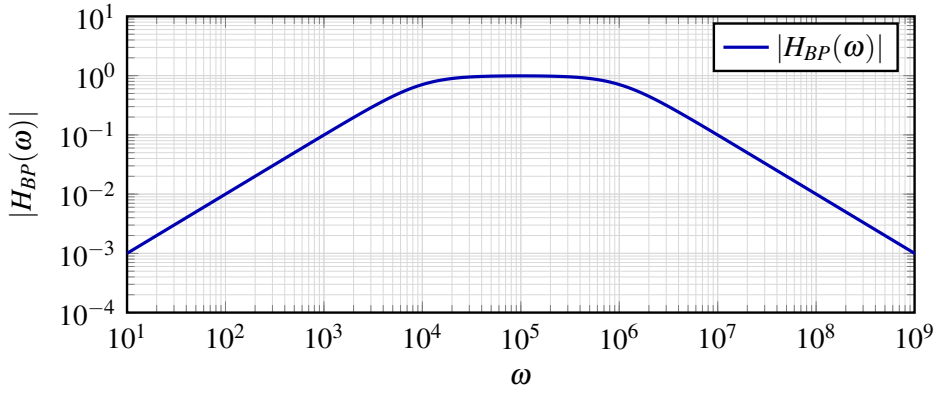
We can compute $H_{BP}(\omega)$ symbolically as:

$$H_{BP}(\omega) = H_{LP}(\omega)H_{HP}(\omega) = \frac{1}{1 + j\omega R_L C_L} \cdot \frac{j\omega R_H C_H}{1 + j\omega R_H C_H}$$

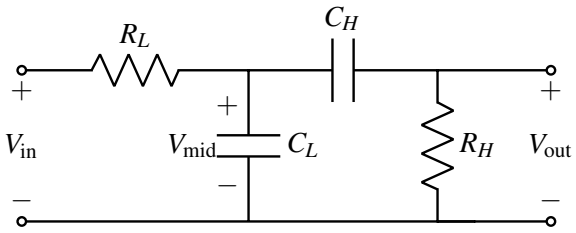
To find the cutoff frequencies of this filter, we can look at the points at which $H_{BP}(\omega_c) = \frac{1}{\sqrt{2}}$. But based on our approximations from before and from the cuttof frequencies $\omega_{LP} = \frac{1}{R_L C_L}$ and $\omega_{HP} = \frac{1}{R_H C_H}$, we can approximate $|H(\omega_{LP})| \approx \frac{1}{\sqrt{2}} \cdot 1$ and $|H(\omega_{HP})| \approx 1 \cdot \frac{1}{\sqrt{2}}$. This approximation holds best when the cuttof frequencies are spaced apart.

We've shown a convenient result! The cutoffs for the band-pass filter are identical to the individual cutoffs for the low and high-pass filters. We now plot $H_{BP}(\omega)$ with $\omega_{LP} = 10^6$ and $\omega_{HP} = 10^4$ to demonstrate the band-pass behavior.

[3] We must be careful, however, to note that in most of our plots, the x-axis does *not* correspond to 0, so we can't simply "stack" the two plots.



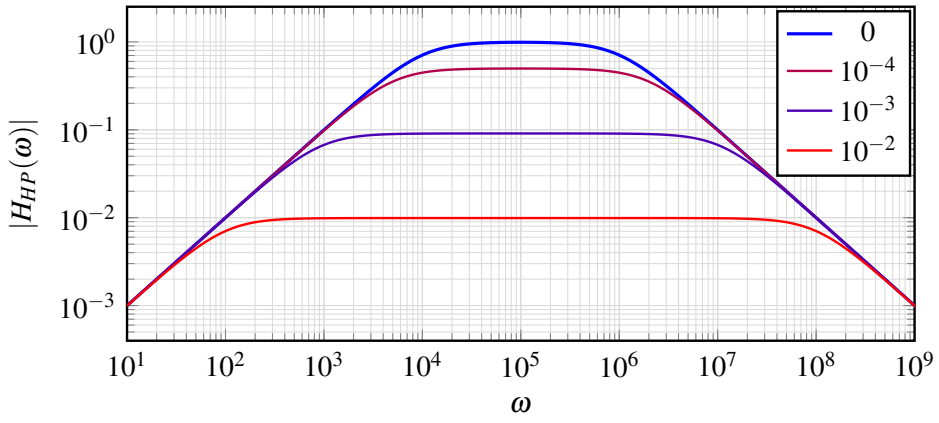
Our band-pass filter uses an op-amp, but what if we cascade our two filters, causing a loading effect?



We leave the derivation as an exercise (feel free to post your work on Piazza!), but computing the transfer function yields:

$$H(\omega) = \frac{j\omega R_H C_H}{(1 + j\omega R_L C_L)(1 + j\omega R_H C_H) + j\omega R_L C_H}$$

The loading effect introduces a term of $j\omega R_L C_H$ in the denominator. The relative size of this term determines the impact on the circuit. We plot some examples of the band-pass filter with identical low and high cutoff frequencies but different $R_L C_H$ values to show this loading effect.



Note how the maximum value of $H(\omega)$ decreases as $R_L C_H$ increases. In addition, the cutoff frequencies move further and further apart from the original $\omega_{LP} = \frac{1}{R_L C_L}$ and $\omega_{HP} = \frac{1}{R_H C_H}$.

2.2 Low-Pass Filters

From our analysis of low-pass filters, we saw that $|H(\omega)|$ dropped off by a factor of 10 for each factor-of-10 increase in frequency after ω_c. This is a desirable effect, but ideally, we would like to build a filter that drops off at a quicker rate after ω_c. Therefore, let's try cascading two low-pass filters of identical cutoff with a buffer in between. The diagram is exactly like

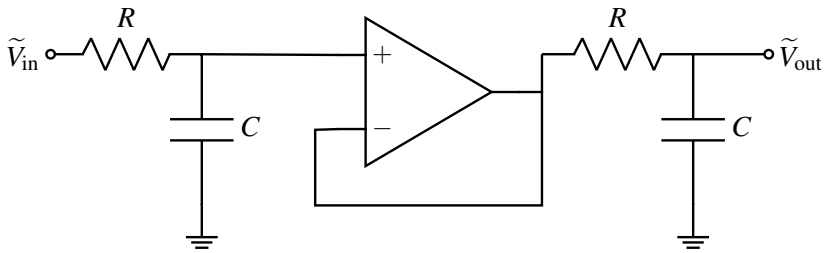


Figure 2: A Buffered Second Order Low-Pass filter. To achieve fast roll-off after ω_c, we must use the same R, C values in both filters.

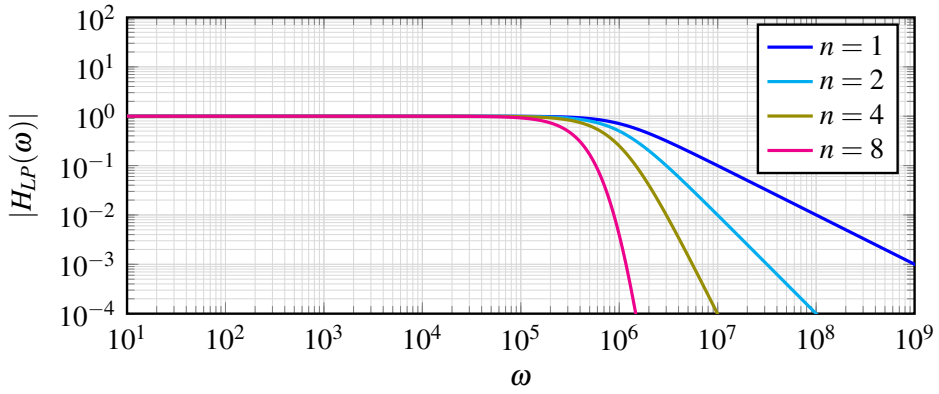
With similar analysis as for the band-pass filter:

$$H_{LP}(\omega) = \frac{1}{(1+j\omega RC)^2}$$

We can see that it does indeed drop off at a quicker rate (with slope 2 after the cutoff ω_c.). In fact, let's just generalize the system to be an $n-$th order low-pass filter (having n such RC stages), and plot $|H_{LP}(\omega)|$ for each! For an n^{th} order filter, we see a dropoff of slope n after the cutoff. We eventually approach an ideal low-pass filter[4] in which

$$H(\omega) = \begin{cases} 1 & \omega < \omega_c \\ 0 & \omega \geq \omega_c \end{cases} \tag{1}$$

We will explore this effect in more detail in the next section.



[4]This is where our hand-approximations and reality diverge a bit. If we use straight-lines, the transition will happen exactly at ω_c but what we see in the plot is a steady shift to the left as n increases. This is the exact result, and the difference arises from the fact that our approximation error (a fractor of $\frac{1}{\sqrt{2}}$) magnifies with each additional filter stage. The Bode approximation is unable to capture this behavior. If we wanted to build something closer to the ideal low-pass filter, we need to shift $\frac{1}{RC}$ to be slightly greater than ω_c.

3 Higher Order Bode Plots

We will now see how to plot a given transfer function by using straight-line approximations, and we will notice that a specific form of transfer function will make the process of plotting more systematic and simple.

3.1 Rational Transfer Functions

When we write the transfer function of an arbitrary circuit, it always takes the following form, called a "rational transfer function." We also like to factor the numerator and denominator, so that they become easier to work with and plot:

$$H(\omega) = K \cdot \frac{N(\omega)}{D(\omega)} = K \frac{(j\omega)^{N_{z0}} \left(1 + j\frac{\omega}{\omega_{z1}}\right)\left(1 + j\frac{\omega}{\omega_{z2}}\right)\cdots\left(1 + j\frac{\omega}{\omega_{zn}}\right)}{(j\omega)^{N_{p0}} \left(1 + j\frac{\omega}{\omega_{p1}}\right)\left(1 + j\frac{\omega}{\omega_{p2}}\right)\cdots\left(1 + j\frac{\omega}{\omega_{pm}}\right)} \tag{2}$$

To summarize the components, each transfer function is the product of constant gain K, one or more "origin-poles" ($(j\omega)$ in a denominator) or "origin zeros" ($(j\omega)$ in a numerator), and one or more poles ($\left(1 + j\frac{\omega}{\omega_{p_i}}\right)$ in the denominator) or zeros ($\left(1 + j\frac{\omega}{\omega_{z_i}}\right)$ in the numerator).

Here, we define the constants ω_z as "zeros" and ω_p as "poles."" The zeros are the roots of $N(\omega)$ while poles are the roots of $D(\omega)$. [5]

3.2 Composing Bode Plots

For two transfer functions $H_1(\omega)$ and $H_2(\omega)$, if $H(\omega) = H_1(\omega) \cdot H_2(\omega)$,

$$\log|H(\omega)| = \log|H_1(\omega) \cdot H_2(\omega)| = \log|H_1(\omega)| + \log|H_2(\omega)| \tag{3}$$

$$\angle H(\omega) = \angle(H_1(\omega) \cdot H_2(\omega)) = \angle H_1(\omega) + \angle H_2(\omega) \tag{4}$$

As a consequence, when plotting $|H(\omega)|$ on a log-log plot, we can simply plot $|H_1(\omega)|$ and $|H_2(\omega)|$ and add them up. This implies that we will be able to add the slopes of each pole and zero to provide a complete plot. In the next section we provide a further analysis on the meaning of zeros and poles and the idea of adding slopes.

3.3 Decibels (Optional)

We define the decibel as the following:

$$20\log_{10}(|H(\omega)|) = |H(\omega)| \text{ [dB]}$$

The origin of the decibel comes from looking at the ratio of the output and input power of the system.

$$|H(\omega)| \text{ [dB]} = 10\log\left|\frac{P_{\text{out}}}{P_{\text{in}}}\right| = 10\log\left|\frac{V_{\text{out}}}{V_{\text{in}}}\right|^2 = 20\log\left|\frac{V_{\text{out}}}{V_{\text{in}}}\right|$$

This means that 20 dB per decade is equivalent to one order of magnitude. We won't be using dB when plotting, but understanding the conversion to dB will help when reading other resources on Bode plots.

[5] Technically if $s = j\omega$, then the roots of $N(s)$ and $D(s)$ are $-\omega_z$ and $-\omega_p$. However, when plotting Bode plots, we refer to ω_z and ω_p as the zero and pole frequencies.

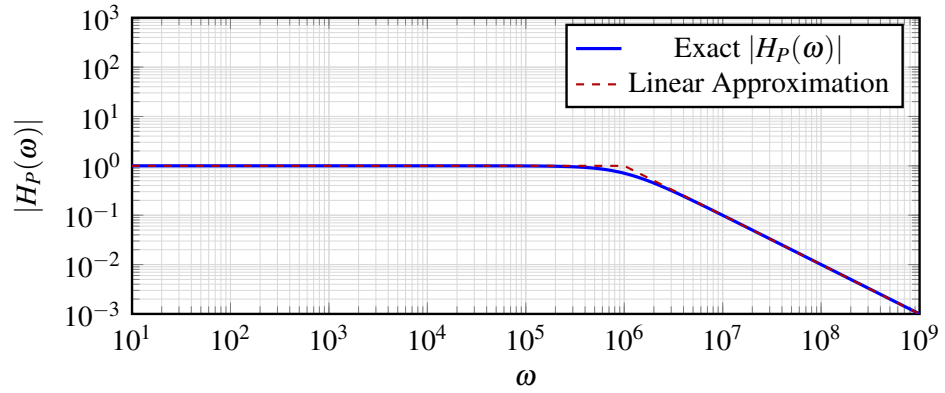
3.4 Poles, Zeros, and Constants

Single Pole, Single Zero

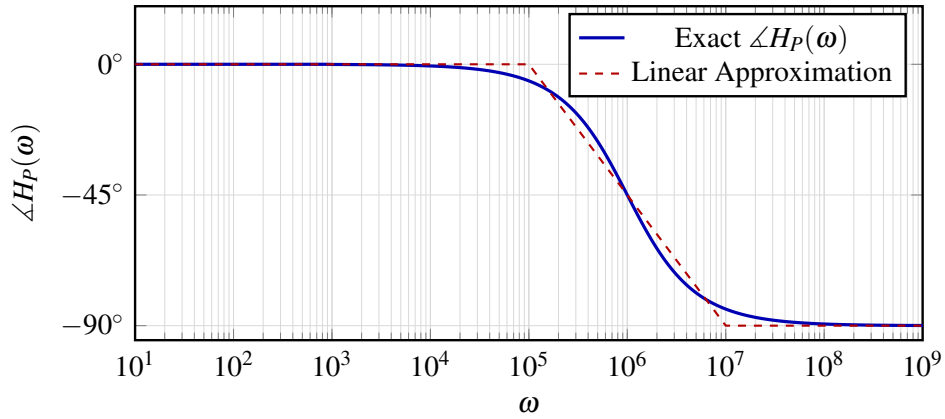
The notion of a **pole** and **zero** frequency is a generalization of the term cutoff frequency. Let's first look back at a plot of our RC low-pass filter:

$$H_P(\omega) = \frac{1}{1 + \frac{j\omega}{10^6}} \tag{5}$$

In what follows, pay special attention to the Linear Approximations! When drawing Bode plots, we claim that the plot drops off with a slope of -1 after a pole ω_p. This transfer function has a single pole at $\omega_p = 10^6$. The magnitude plot has a familiar shape (resembling a low-pass filter's transfer function!).



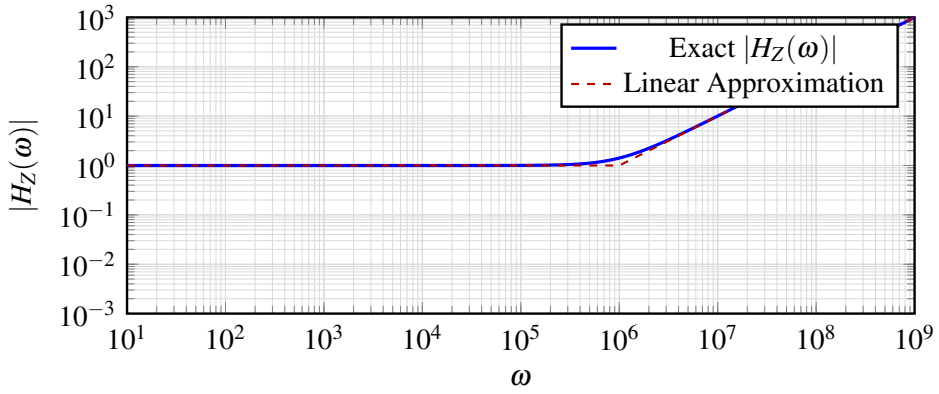
We can look at the phase plot as well:



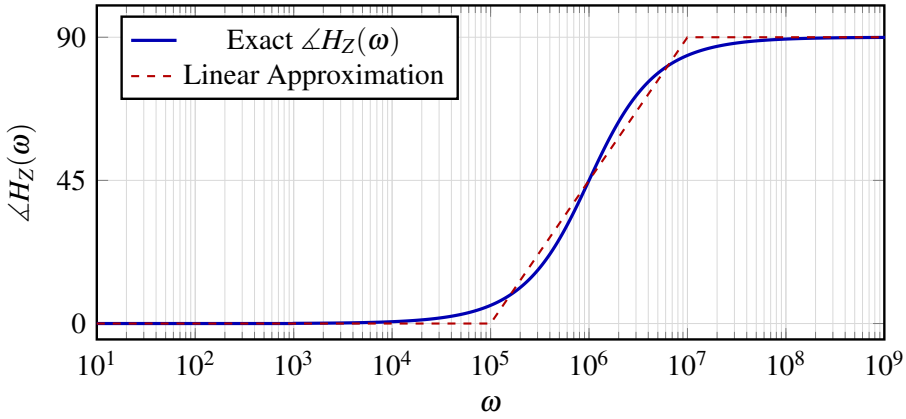
Now let's take a look at a single zero.

$$H_Z(\omega) = 1 + j\omega/\omega_z \tag{6}$$

We show that this Magnitude Bode plot rises with a slope of 1 after the zero at ω_z.

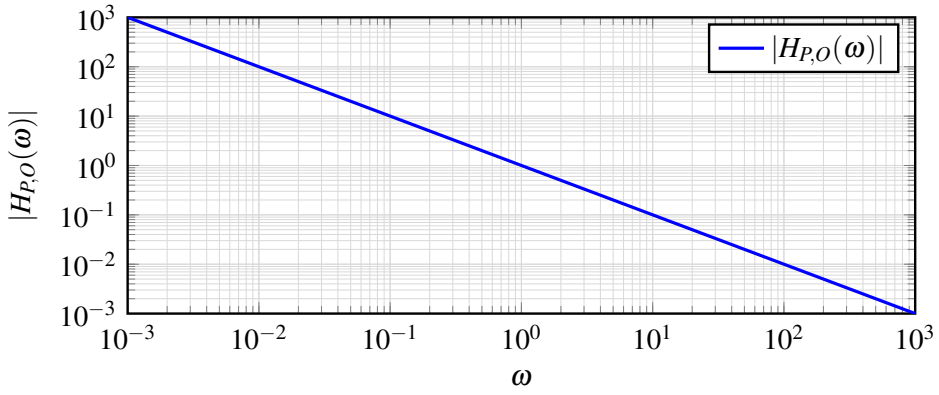


The phase plot is as follows:



Pole/Zero at the Origin

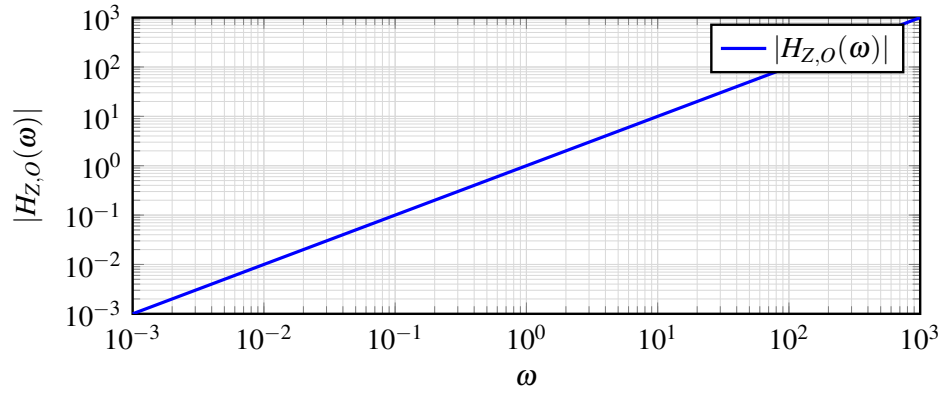
To plot a pole at the origin, recall that $H(\omega) = \frac{1}{j\omega}$ has magnitude $\frac{1}{\omega}$ and phase $-90°$.[6] If our transfer function has a pole at the origin, it will start off with a slope of -1. The phase of a pole at the origin is $-90°$ at all frequencies.



To plot a zero at the origin, recall that $H(\omega) = j\omega$ has magnitude ω and phase $90°$. If our transfer function

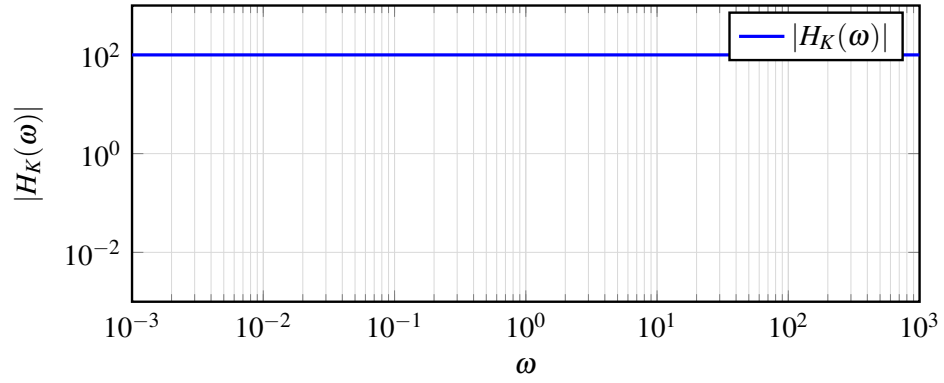
[6]For this subsection and the next, our linear approximations are actually exactly correct.

has a zero at the origin, it will start off with a slope of 1. The phase of a zero at the origin is 90° at all frequencies.



Constant Terms

Lastly, we show the plot of a constant $K = 100$. As expected, the plot remains constant. This implies that multiplication by K will shift up the entire bode plot up by K. Note that positive constants have a constant phase of 0° at all frequencies, while negative constants have a constant phase of $180° \equiv -180°$ at all frequencies.



3.5 Examples

We have previously seen examples of how to compute the bode plot of a bandpass filter and for an $n-$th order low-pass filter. At this point, see if you can go back and compose them yourself with the linear approximations presented above. See if you get the same results!

Transfer Function Example

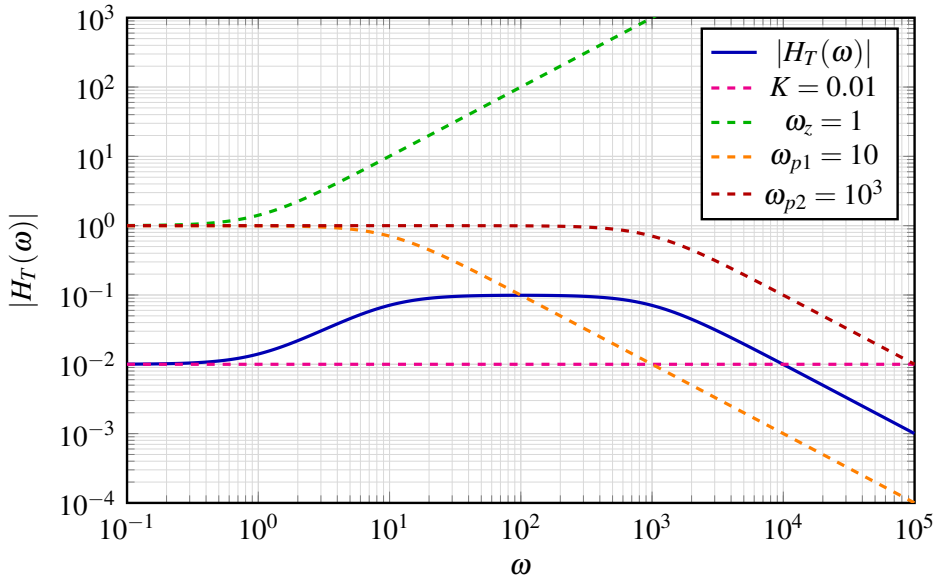
Now let's take a look at the Bode plot of a new transfer function.

$$H_T(\omega) = 100\frac{(1+j\omega)}{(j\omega)^2 + 1010(j\omega) + 10^4}$$

Our first step is to factor this into its rational transfer function form:

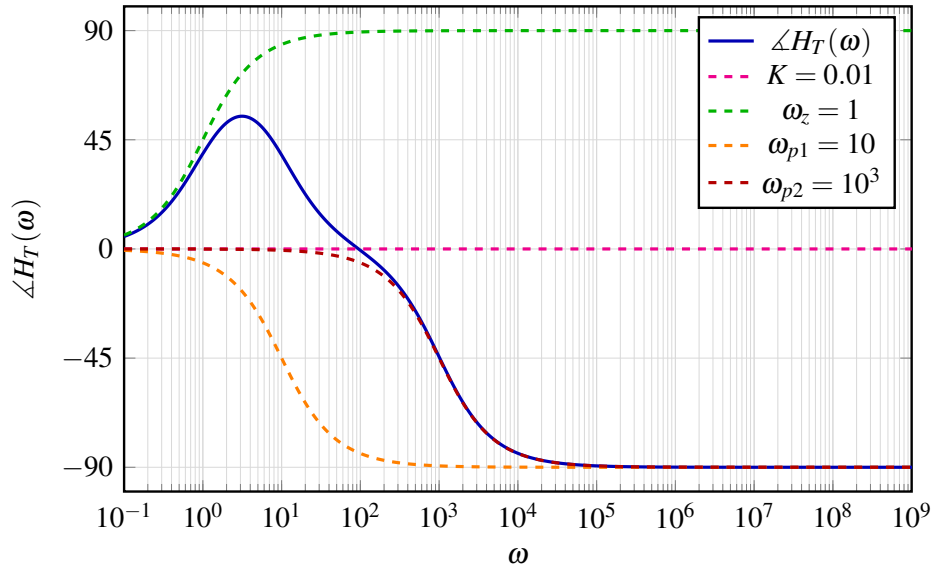
$$H_T(\omega) = 0.01\frac{(1+j\omega)}{(1+j\omega/10)(1+j\omega/10^3)}$$

With $H_T(\omega)$ in its rational form, we see that $K = 0.01, \omega_z = 1, \omega_{p1} = 10, \omega_{p2} = 10^3$. Below is a magnitude plot of each consituent component (following the building-block rules presented above), and the multiplication of all of these provides $|H_T(\omega)|$. The linear approximations are omitted to keep the plot legible, but the approximate result will very closely match the exact one.



To provide an analysis for this Bode plot, we see that the plot starts off at $K = 0.01$. Then at $\omega_z = 1$, it starts rising with slope 1. When it hits the pole at $\omega_{p1} = 10$, the slope of 1 is cancelled out by the -1 slope that the pole provides. Then the Bode plot stays constant until $\omega_{p2} = 10^3$ at which it drops off with a slope of 1. We've provided Bode plots of the individual terms to give you a sense of how we "add" Bode plots together.

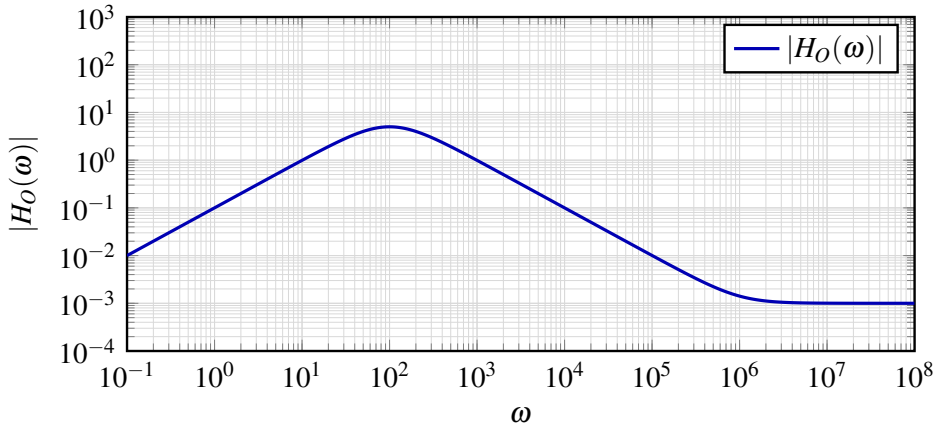
We can also plot the phase in a very similar way using our building blocks:



Zero at the Origin

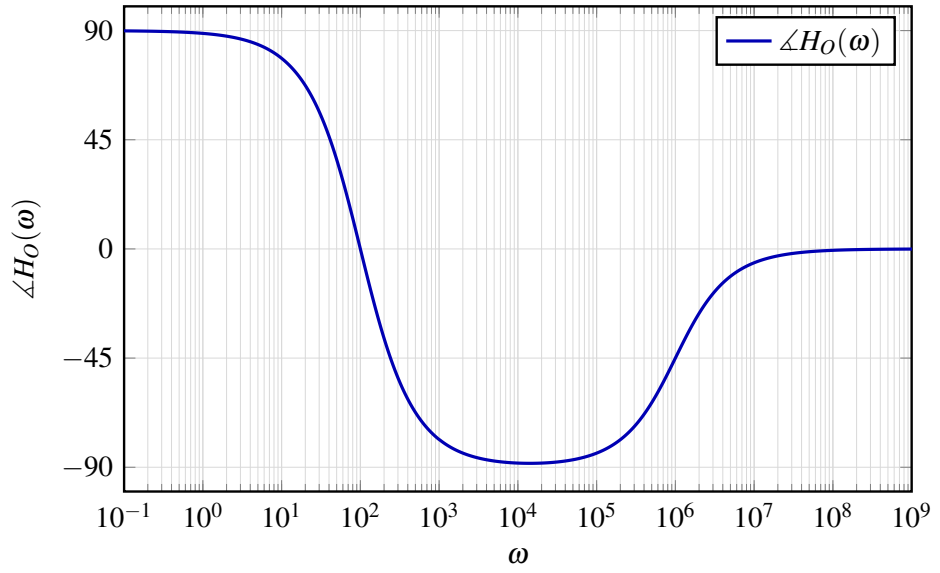
In our final example, we examine the effects of a zero at the origin. Only the final results are shown; the intermediate building blocks are left fo the reader to consider. We are given the following transfer function in rational form.

$$H_O(\omega) = 0.1\frac{(j\omega)(1+j\omega/10^6)}{(1+j\omega/10^2)^2} \tag{7}$$



Since there is a zero at the origin, the plot will initially start with a slope of 1. There are no additional zeros or poles before $\omega = 1$, so we can approximate $|H_O(1)| = K = 0.1$. Then the double pole at $\omega_p = 10^2$ provides a slope of -2 that will cancel out the slope of 1 making the overall slope after ω_p equal to -1. Lastly, there is a zero at $\omega_z = 10^6$ and we see that the addition of a slope of 1 makes $|H_O(\omega)|$ remains constant after ω_z.

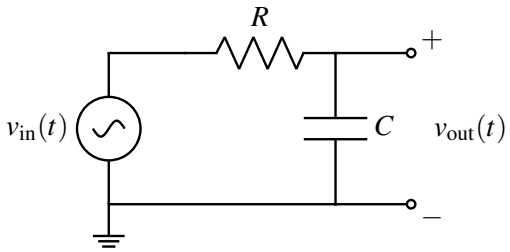
And for the phase, we have:



A Time Constant

When computing the cutoff frequency for a first order low-pass filter, we noticed that the $\omega_c = \frac{1}{RC} = \frac{1}{\tau}$. Here, we draw the connection between time constants and cutoff frequencies.

Recall from the note on differential equations that we defined the time constant of a first-order circuit to be the point at which the response $v_c(t)$ to a constant input was $1-e^{-1}$ away from its steady state value. With this in mind, let's try plugging in an exponential input $v_{in}(t) = V_0 e^{j\omega t}$ into an RC circuit and see what happens.[7]



The differential equation for this circuit is

$$\frac{d}{dt}v_{\text{out}}(t) = \lambda\left(v_{\text{out}}(t) - V_0 e^{j\omega t}\right) \tag{8}$$

for $\lambda = -\frac{1}{\tau}$. In Note 3 we showed that the steady state value of this differential equation is

$$v_{ss}(t) = \frac{-\lambda}{j\omega - \lambda} V_0 e^{j\omega t} \tag{9}$$

Therefore, plugging in for $\lambda = -\frac{1}{\tau}$, it follows that

$$v_{ss}(t) = \frac{1}{1 + j\omega\tau} V_0 e^{j\omega t} \tag{10}$$

Notice that $H(\omega) = \frac{1}{1+j\omega\tau}$ and the cutoff arises naturally as $\omega_c = \frac{1}{\tau}$. We can also realize that at steady state, $H(\omega)$ is in fact the eigenvalue for the differential equation with eigenfunction $e^{j\omega t}$. This is a crucial connection between differential equations and the frequency response of a linear system that you will see in later half of the course and in courses like EE120.

Contributors:

- Neelesh Ramachandran.
- Rahul Arya.
- Anant Sahai.
- Jaijeet Roychowdhury.
- Taejin Hwang.

[7] We should be inputting $v_{in}(t) = V_0\cos(\omega t)$ but we choose $e^{j\omega t}$ since it provides the same result while simplifying the math.

EECS 16B Designing Information Devices and Systems II

Note 7A: Controls Overview

1 Overview

We have seen how to model a continuous-time system (for example, defined by a system of differential equations $\frac{d}{dt}\vec{x}_c(t) = A_c\vec{x}_c(t) + B_c\vec{u}_c(t)$) by choosing a time interval Δ, deciding to apply piecewise constant inputs for durations of length $\Delta > 0$ seconds, and then sampling every Δ seconds to get a new discrete time system:

$$\vec{x}_d[i+1] = A_d\vec{x}_d[i] + B_d\vec{u}_d[i],$$

where $\vec{x}_d[i]$ is the value of $\vec{x}_c(i\Delta)$ and $\vec{u}_d[i]$ is the value of $\vec{u}_c(t)$ for the time interval $t \in [i, (i+1)\Delta)$.

A Side Note on Continuous vs. Discrete: Note that it is perfectly fine if what follows here doesn't make sense yet; we will cover it in more detail in discussion 7A. Note that the A_d matrix for the discrete time system is related to the original A_c matrix of the continuous-time system. It is easiest to see the exact relationship when there exists a coordinate system in which A_c is diagonal — $A_c = V\Lambda V^{-1}$. In the diagonal case, $\frac{d}{dt}\vec{\tilde{x}}_c(t) = \Lambda\vec{\tilde{x}}_c(t) + \widetilde{B}_c\vec{u}_c(t)$, the underlying system is basically a parallel set of scalar differential equations $\frac{d}{dt}\left(\vec{\tilde{x}}_c(t)\right)_k = \lambda_k\left(\vec{\tilde{x}}_c(t)\right)_k + \left(\widetilde{B}_c\vec{u}_c(t)\right)_k$. If $\lambda_k \neq 0$, this discretizes to $\left(\vec{\tilde{x}}_d(i+1)\right)_k = e^{\Delta\lambda_k}\left(\widetilde{x}_d[i]\right)_k + \frac{e^{\Delta\lambda_k}-1}{\lambda_k}\left(\widetilde{B}_c\vec{u}_d[i]\right)_k$. If $\lambda_j = 0$, this discretizes to $\left(\vec{\tilde{x}}_d[i+1]\right)_k = \left(\widetilde{x}_d[i]\right)_k + \left(\Delta\widetilde{B}_c\vec{u}_d[i]\right)_k$. The A and B matrices follow from this.

For the remainder of this module, we will almost entirely disregard the fact that our system may in fact really be continuous, focusing entirely on questions related to our discrete-time model. After all, in reality, most electronic control systems (like the MSP that we will use to develop a "robot car") have some intrinsic Δ in their ability to sample $\vec{x}_c(t)$ and vary their output $\vec{u}(t)$, so even if we knew that $\vec{x}_c(t)$ varied within the interval Δ, we would not be able to measure or react to this variation within the time interval.

Contributors:

- Neelesh Ramachandran.
- Rahul Arya.
- Anant Sahai.

EECS 16B Designing Information Devices and Systems II

Note 7B: System ID

1 Overview

Given a linear differential equation model for a system, we know how we can discretize it and obtain a discrete-time system model. However, this relies on knowing physical parameters to infinite precision and many times, it is not worth investing that much effort in trying to model the system exactly by hand. *System Identification* is about using data collected about the inputs $\vec{u}$ and states $\vec{x}$ to attempt to determine A and B.[1] This process is of great importance, because when we construct real-world mechanical systems, it is practically impossible to determine all their behaviors theoretically - we must do so empirically. In practice, data is also taken continuously to update our models, since in the real world, the model itself can change over time.

Modern system design leverages the ability to learn from data. Fortunately, EECS16A has already given you all the basic tools that you need to do this intelligently, as we see in the following section.

2 System Identification

Let our linear system be:

$$\vec{x}_d[t+1] = A\vec{x}_d[t] + B\vec{u}_d[t] \tag{1}$$

where we observe each of the $\vec{x}_d[t]$. That is, t refers to the current timestep. From here on, it is understood that $\vec{x}_d[t]$ is discrete, so we will use subscripts to denote indices of the discrete-time vector (such as, $\vec{x}_1[t]$ is the value of the first component of $\vec{x}_d[t]$ at time t). We may also refer to $\vec{x}_d[t]$ simply as $\vec{x}[t]$.

Let's express our unknown matrices A and B in terms of scalars a_{ij}, b_{ij}, as follows:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \qquad B = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1k} \\ b_{21} & b_{22} & \dots & b_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nk} \end{bmatrix} \tag{2}$$

Substituting into our relation for $\vec{x}[t]$, we break down our state, observation, and input vectors into their scalar components as well:

$$\begin{bmatrix} x_1[t+1] \\ x_2[t+1] \\ \\ x_n[t+1] \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1[t] \\ x_2[t] \\ \vdots \\ x_n[t] \end{bmatrix} + \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1k} \\ b_{21} & b_{22} & \dots & b_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \dots & b_{nk} \end{bmatrix} \begin{bmatrix} u_1[t] \\ u_2[t] \\ \vdots \\ u_k[t] \end{bmatrix} \tag{3}$$

Now comes the interesting part. Recall that our unknowns are in fact all the a_{ij} and b_{ij} scalars, and our knowns are all the components of $\vec{x}$ and $\vec{u}$ (x_1, x_2, u_1, u_2, etc.) When solving linear systems, we know that we

[1] Note that here, we are only considering discrete evolution, so $\vec{x}$, A, B are $\vec{x}_d$, A_d, B_d.

should put our unknowns into a vector. By considering each row of the above matrix separately (using r to index the row), we obtain scalar equations of the form:

$$x_r[t+1] = \begin{bmatrix} a_{r1} & a_{r2} & \dots & a_{rn} \end{bmatrix} \begin{bmatrix} x_1[t] \\ x_2[t] \\ \vdots \\ x_n[t] \end{bmatrix} + \begin{bmatrix} b_{r1} & b_{r2} & \dots & b_{rk} \end{bmatrix} \begin{bmatrix} u_1[t] \\ u_2[t] \\ \vdots \\ u_k[t] \end{bmatrix} \tag{4}$$

for all $1 \leq r \leq n$. Notice that the two terms on the right of the equation are really just inner products, producing a scalar. As the inner product is symmetric for real vectors ($\langle x, y \rangle = \langle y, x \rangle$), we can swap the rows and columns and rewrite the above equation as:

$$x_r[t+1] = \begin{bmatrix} x_1[t] & x_2[t] & \cdots & x_n[t] \end{bmatrix} \begin{bmatrix} a_{r1} \\ a_{r2} \\ \vdots \\ a_{rn} \end{bmatrix} + \begin{bmatrix} u_1[t] & u_2[t] & \cdots & u_k[t] \end{bmatrix} \begin{bmatrix} b_{r1} \\ b_{r2} \\ \vdots \\ b_{rk} \end{bmatrix} \tag{5}$$

again for all $1 \leq r \leq n$.

Combining the two terms on the right, we finally obtain the equation

$$x_r[t+1] = \begin{bmatrix} x_1[t] & x_2[t] & \cdots & x_n[t] & u_1[t] & u_2[t] & \cdots & u_k[t] \end{bmatrix} \begin{bmatrix} a_{r1} \\ a_{r2} \\ \vdots \\ a_{rn} \\ b_{r1} \\ b_{r2} \\ \vdots \\ b_{rk} \end{bmatrix} \tag{6}$$

again for all $1 \leq r \leq n$.

Notice that we have managed to place all the unknowns involving the rth element of the state at time t in a single vector. Since we now have a linear equation with our unknowns in a vector, are we done, since we can do Gaussian elimination to solve for the unknowns?

Unfortunately not. Notice that there are $n+k$ unknowns, but only a single equation. We could try to get more equations by considering all values of r simultaneously (all the rows), but that would also bring in new unknowns, since we'd have to consider the a_{rj} and b_{rj} for *all* r. So is all hope lost?

No! Recall that our equation holds for *all* values of t, since we are assuming that our system's transition equation does not vary over time! Notice that for all timesteps $0 < t \leq T$, where T is the current time, the vector of unknowns remains the same. Thus, by considering all these values of t and stacking them

vertically, we obtain the system

$$\begin{bmatrix} x_r[1] \\ x_r[2] \\ \vdots \\ x_r[T] \end{bmatrix} = \begin{bmatrix} x_1[0] & \cdots & x_n[0] & u_1[0] & \cdots & u_k[0] \\ x_1[1] & \cdots & x_n[1] & u_1[1] & \cdots & u_k[1] \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ x_1[T-1] & \cdots & x_n[T-1] & u_1[T-1] & \cdots & u_k[T-1] \end{bmatrix} \begin{bmatrix} a_{r1} \\ a_{r2} \\ \vdots \\ a_{rn} \\ b_{r1} \\ b_{r2} \\ \vdots \\ b_{rk} \end{bmatrix} \tag{7}$$

which consists of T equations, not just 1! For $T \geq n+k$, we can use least squares to solve for our unknowns, and repeat the process for each value of r in order to fully determine our system. Typically, we create a matrix P of our unknowns across all values of r by stacking them horizontally to obtain

$$P = \begin{bmatrix} a_{11} & a_{21} & \dots & a_{n1} \\ a_{12} & a_{22} & \dots & a_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} & a_{2n} & \dots & a_{nn} \\ b_{11} & b_{21} & \dots & b_{n1} \\ b_{12} & b_{22} & \dots & b_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ b_{1k} & b_{2k} & \dots & b_{nk} \end{bmatrix} = \begin{bmatrix} A^\top \\ B^\top \end{bmatrix}. \tag{8}$$

We can follow a similar stacking procedure to obtain

$$D = \begin{bmatrix} x_1[0] & \cdots & x_n[0] & u_1[0] & \cdots & u_k[0] \\ x_1[1] & \cdots & x_n[1] & u_1[1] & \cdots & u_k[1] \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ x_1[T-1] & \cdots & x_n[T-1] & u_1[T-1] & \cdots & u_k[T-1] \end{bmatrix} \tag{9}$$

and

$$S = \begin{bmatrix} x_1[1] & x_2[1] & \cdots & x_n[1] \\ x_1[2] & x_2[2] & \cdots & x_n[2] \\ \vdots & \ddots & \ddots & \vdots \\ x_1[T] & x_2[T] & \cdots & x_n[T] \end{bmatrix} \tag{10}$$

Thus, we can combine all our values of r into the single approximate equation

$$DP \approx S$$

and solve this using least squares to obtain

$$P = \left(D^\top D\right)^{-1} D^\top S \tag{11}$$

Why do we use least-squares here? Because we are going to assume that any measurement errors or unmodeled terms are all relatively small. So it makes sense to pick a solution that has a small residual, and least squares gives that to us. (We will see in the next note how we could make such solutions robust to a few points being hit with something much larger.)

Recall that $D^\top D$ is invertible exactly when D has linearly independent columns. We will discuss what happens when the columns made out of the x_r are dependent later - for now, we will simply comment that choosing our inputs randomly will ensure with high probability that the input columns are all linearly independent both of each other and of the earlier x_r columns.

Contributors:

- Neelesh Ramachandran.
- Rahul Arya.
- Anant Sahai.

EECS 16B Designing Information Devices and Systems II

Note 8: Stability

1 Overview

Given a discrete-time system, we know how to determine its behavior by supplying inputs. We will now discuss the problem of *disturbance*, which is unpredictable and can change the properties of our system. We will study here how it affects our ability to both control a system and to move it to where we want.

2 Stability

State stability:

Consider the system

$$\vec{x}_d[t+1] = A\vec{x}_d[t] + B\vec{u}_d[t]. \tag{1}$$

We say this system is stable in the sense of "state-stability" if, for every time t, we have that $\|\vec{x}_d[t]\| \leq K$ for some constant $K \in \mathbb{R}$. Note that K may not depend on the time t. This definition remains true even if we consider a disturbance added to the system.

However, the above definition of the "state-stability" seems a bit inadequate. In particular, let us say we add an unbounded input to our system — we choose an arbitrarily large $\vec{u}_d[t]$. In this case, it is not possible for the state to stay bounded, but for nothing to do with the "system" itself, per-se. The state is blowing up only because we chose an unbounded input. This motivates the idea of BIBO stability, which requires the input to be bounded.

Bounded Input Bounded Output (BIBO) stability:

Consider the systsem in eq. (1). This system is Bounded Input Bounded Output (BIBO) stable if for every bounded input (i.e. for every input such that $\|\vec{u}_d\| \leq \varepsilon$ for some $\varepsilon \in \mathbb{R}$,) the norm of the state remains bounded (i.e. for every time t we have that $\|\vec{x}_d[t]\| \leq K$ for some constant $K \in \mathbb{R}$). Note that K may not depend on the time t.

Note that this definition now removes the problem of unbounded inputs that we faced with the earlier definition. We can also extend this definition to the case with a disturbance, as long as the disturbance is also bounded.

It is often convenient to consider the case of zero initial condition when proving if something is BIBO stable or unstable. From there, one can extend to non-zero initial conditions.

3 Disturbance

Recall that we typically model the evolution of a discrete-time linear system as follows:

$$\vec{x}_d[t+1] = A\vec{x}_d[t] + B\vec{u}_d[t].$$

In reality, though, these models never capture the full behavior of a system - errors, due to nonlinearities in the system, factors not taken into account when building the model, or just random noise, will always be present. We model the total error in our model as an all-encompassing disturbance term, $\vec{w}$. We then obtain the full state equation:

$$\vec{x}_d[t+1] = A\vec{x}_d[t] + B\vec{u}_d[t] + \vec{w}[t].$$

where $\vec{w}$ is unobservable and varies arbitrarily at each time step.

At this point, a natural question to ask would be: if $\vec{w}$ is unknown and varies arbitrarily, how can we possibly do anything with our system? We can apply control inputs and observe the system extremely carefully, only for everything to be thrown off by an arbitrarily large $\vec{w}$ term! To address this, we will introduce the notion of *bounded disturbance*. Specifically, we assume that that $\|\vec{w}\| \leq \varepsilon$ for some small nonzero constant ε.

We think of both $\vec{w}$ and $\vec{u}$ as being inputs to the system. Hence, we can use the same definitions of stability as defined in the earlier section (without a disturbance).

4 Scalar Stability

First, we will consider the case of system stability in one dimension ($\vec{x}$ is a scalar quantity x). Thus, our system's behavior can be modelled as

$$x_d[t+1] = \lambda x_d[t] + w[t].$$

The use of λ may remind you of eigenvalues - we'll see why that's the case shortly. Let's say that our system is initially at rest: $x_d[0] = 0$. Therefore, we have:

$$\begin{aligned}
& x_d[0] = 0 \\
\implies \quad & x_d[1] = w[0] \\
\implies \quad & x_d[2] = \lambda w[0] + w[1] \\
\implies \quad & x_d[3] = \lambda^2 w[0] + \lambda w[1] + w[2] \\
& \vdots \\
\implies \quad & x_d[t] = \lambda^{t-1} w[0] + \lambda^{t-2} w[1] + \ldots + w[t-1].
\end{aligned}$$

For now, let's assume that $\lambda \geq 0$. Recall that we have no control over the noise. For our system to be stable, $x_d[t]$ should be bounded no matter how w is chosen. In this scenario, we can see that $x_d[t]$ is maximized when we maximize w. Our assumption is that w has an upper bound of ε, so we should set $w = \varepsilon$ at every timestep t. Therefore, we can write:

$$x_d[t] = \varepsilon\left(1 + \lambda + \lambda^2 + \cdots + \lambda^{t-1}\right).$$

Since the system should remain bounded after arbitrarily many timesteps, we can take the limit $t \to \infty$ to obtain

$$\lim_{t\to\infty} x_d[t] = x(\infty) = \varepsilon(1 + \lambda + \lambda^2 + \ldots).$$

The sum of this infinite geometric series[1] converges if and only if $|\lambda| < 1$. In other words, a scalar system where $\lambda \geq 0$ is stable exactly when $|\lambda| < 1$.

Unfortunately, the above analysis becomes somewhat more tricky when $\lambda < 0$, and breaks down entirely when λ is complex! However, it is still useful in that it has provided us with a conjecture – namely that a

[1] Recall that $\sum_{k=0}^{\infty} a\, r^k = \frac{a}{1-r}$.

scalar discrete-time system is stable exactly when $|\lambda| < 1$. To prove this conjecture, we need to: 1) show that our system is stable when $|\lambda| < 1$ and 2) show that it is unstable when $|\lambda| \geq 1$.

Let's prove the former claim first. We can write that:

$$x_d[T] = \sum_{t=0}^{T-1} \lambda^{T-t-1} w[t]$$

even in the complex case. We can upper-bound $|x_d[t]|$ to be

$$\begin{aligned} |x_d[T]| &= \left| \sum_{t=0}^{T-1} \lambda^{T-t-1} w[t] \right| \\ &\leq \sum_{t=0}^{T-1} |\lambda^{T-t-1} w[t]| \\ &= \sum_{t=0}^{T-1} |\lambda^{T-t-1}||w[t]| \\ &\leq \varepsilon \sum_{t=0}^{T-1} |\lambda|^{T-t-1}, \end{aligned}$$

since $|\lambda^{T-t-1}| \geq 0$ and $|w[t]| \leq \varepsilon$ by the definition of bounded noise. Now, since $|\lambda| < 1$, we can apply the geometric series formula, and we can take an upper-bound by summing to infinity, rather than stopping at $T-1$. We then obtain the final bound:

$$|x_d[T]| \leq \varepsilon \left(1 + |\lambda| + |\lambda|^2 + \cdots\right) = \varepsilon \frac{1}{1-|\lambda|}$$

This is a bounded quantity even as $T \to \infty$. Consequently, we have shown that our system is BIBO stable when $|\lambda| < 1$ (even when λ is a complex number.)

What if $|\lambda| \geq 1$? We'd like to show that our system is unstable, and to do so, we can construct any kind of error sequence we want that is *adversarial*; that is, it is the worst-case scenario for our particular system. More specifically, this adversarial error is a particular sequence of bounded $w[t]$ that takes $|x_d[T]|$ to infinity over time (indicating that the state is unbounded). Looking at the special case of real, positive λ for inspiration, we might conjecture that:

$$w[t] = \varepsilon$$

is one such sequence. It makes sense that if the error is bounded by a constant, and we want to consideer the worst-possible case, we just make the error as large as it can be at every timestep! This candidate sequence of $w[t]$ is indeed bounded (by ε).

For this candidate sequence, we see that (when $\lambda \neq 1$):[2]

$$\begin{aligned} x_d[T] &= \sum_{t=0}^{T-1} \lambda^{T-t-1} w[t] \\ &= \varepsilon(\lambda^0 + \lambda^1 + \cdots + \lambda^{T-1}) \\ &= \varepsilon \frac{\lambda^T - 1}{\lambda - 1}, \end{aligned}$$

so:

$$|x_d[T]| = \varepsilon \frac{|\lambda^t - 1|}{|\lambda - 1|}.$$

[2]The sum of a finite geometric series $\sum_{k=0}^{n} a\, r^k = a\frac{1-r^{n+1}}{1-r}$.

As $|\lambda^t| = |\lambda|^t$ goes to infinity when $|\lambda| > 1$, we have shown that our system is unstable when $|\lambda| > 1$.

Unfortunately, in the above derivation, we had to exclude the case when $|\lambda| = 1$. For this remaining case, we need to generate a new candidate sequence of noise. Some experimentation yields the candidate sequence:

$$w[t] = \varepsilon \lambda^t.$$

Note that since $|\lambda| = 1$, $|w[t]| = \varepsilon 1^t = \varepsilon$, so the noise is still bounded. Now, plugging into our formula for $x_d[T]$, we find that

$$\begin{aligned} x_d[T] &= \varepsilon \sum_{t=0}^{T-1} \lambda^{T-t-1} \lambda^t \\ &= \varepsilon \sum_{t=0}^{T-1} \lambda^{T-1} \\ &= \varepsilon T \lambda^{T-1}, \end{aligned}$$

so:

$$|x_d[T]| = T|\varepsilon|,$$

which goes to infinity over time. Thus, when $|\lambda| = 1$, we can supply bounded noise that still drives our system to infinity, so the system is unbounded.

Putting everything together, we have shown that whenever the parameter λ of our scalar discrete-time linear system has magnitude less than 1, our system is stablem, but that in all other cases, it is unstable.

5 Vector Stability

Now, we will look at the discrete-time vector case, with the state equation:

$$\vec{x}_d[t+1] = A\vec{x}_d[t] + B\vec{u}_d[t] + \vec{w}[t].$$

We will assume that we start at the target state $\vec{x}_d[0] = \vec{0}$, and that we supply no inputs thereafter, so all $\vec{u} = \vec{0}$. Note that this means that B cannot affect the stability of our system, so we will disregard it.

Now, recall that when we considered continuous-time systems with coupled differential equations, we were able to separate them into a set of decoupled differential equations by diagonalizing their state matrix.[3] We can do something similar here, by letting $A = V\Lambda V^{-1}$. Rearranging, we obtain

$$\begin{aligned} & \vec{x}_d[t+1] = V\Lambda V^{-1}\vec{x}_d[t] + \vec{w}[t] \\ \implies \quad & V^{-1}\vec{x}_d[t+1] = \Lambda V^{-1}\vec{x}_d[t] + V^{-1}\vec{w}[t] \\ \implies \quad & \vec{\tilde{x}}_d[t+1] = \Lambda \vec{\tilde{x}}_d[t] + \vec{\tilde{w}}[t], \end{aligned}$$

where $\vec{\tilde{x}}_d = V^{-1}\vec{x}_d$ and $\vec{\tilde{w}} = V^{-1}\vec{w}$. Clearly, if a single component of $\vec{\tilde{x}}$ is unbounded, then the system as a whole is unstable. So all we have to do is consider each of the components of $\vec{\tilde{x}}$, each of which gives us the scalar state equation

$$\tilde{x}_j[t+1] = \lambda_j \tilde{x}_j[t] + \tilde{w}_j.$$

It would seem that we can just apply the result from the previous section, to state that this component is stable exactly when $|\lambda_j| < 1$.

But recall, that result only applied when the input noise was bounded (that is, $|\tilde{w}_j| < \tilde{\varepsilon}$ for some finite constant $\tilde{\varepsilon}$). We were given that our original noise $\vec{w}$ was bounded, so there existed some ε such that each

[3] We will discuss the case when A is not diagonalizable later in the course.

component $\vec{w}_j < \varepsilon$. But after applying V^{-1}, can we be certain that the transformed noise is still bounded? We might suspect the answer to be yes, but for the sake of completeness, we will derive a proof here. Let

$$V^{-1} = \begin{bmatrix} v_{11} & v_{12} & \cdots & v_{1n} \\ v_{21} & v_{22} & \cdots & v_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ v_{n1} & v_{n2} & \cdots & v_{nn} \end{bmatrix},$$

and let m be the entry v_{ij} within V^{-1} with the largest magnitude. Thus,

$$\begin{aligned} V^{-1}\vec{w} &= \begin{bmatrix} v_{11} & v_{12} & \cdots & v_{1n} \\ v_{21} & v_{22} & \cdots & v_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ v_{n1} & v_{n2} & \cdots & v_{nn} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \\ &= \begin{bmatrix} v_{11}w_1 + v_{12}w_2 + \ldots + v_{1n}w_n \\ v_{21}w_1 + v_{22}w_2 + \ldots + v_{2n}w_n \\ \vdots \\ v_{n1}w_1 + v_{n2}w_2 + \ldots + v_{nn}w_n \end{bmatrix}. \end{aligned}$$

Any given row r of $\vec{\tilde{w}} = V^{-1}\vec{w}$ is bounded by:

$$v_{r1}w_1 + v_{r2}w_2 + \ldots + v_{rn}w_n \leq |m|(w_1 + w_2 + \ldots + w_n).$$

Since $|m|$ is a constant that does not depend on $\vec{w}$, and $w_1 + w_2 + \ldots + w_n$ is bounded (since $\|\vec{w}\|$ is bounded), each element of $\vec{\tilde{w}}$ is also bounded by some finite constant. Therefore, we can indeed apply our original result in the eigenbasis. Consequently, our system as a whole is stable exactly when $|\lambda_j| < 1$ for all the eigenvalues λ_j of A.

6 Continuous-Time Systems

At this point, we will take a brief aside from our discussion of discrete-time systems to consider the stability of continuous time systems (t is any valid time, not necessarily an integer/timestep). Consider a continuous-time scalar system whose behavior is described by the equation

$$\frac{d}{dt}x_c(t) = \lambda x_c(t) + u(t) + w(t),$$

where λ and $w(t)$ are possibly complex.

As before, we are interested to see, given bounded noise $|w(t)| \leq \varepsilon$ and input $u(t) = 0$, whether $|x_c(t)|$ remains bounded over time. Initially, let's assume that $\lambda \neq 0$. Then, from our knowledge of differential equations, we know that we can solve for the state:

$$x_c(t) = x_c(0)e^{\lambda t} + \int_0^t e^{\lambda(t-\theta)}w(\theta)\,d\theta.$$

Let's consider the leading term $x_c(0)e^{\lambda t}$ first. We know that $e^{\lambda t}$ behaves very differently for real and imaginary choices of λ. Thus, to study it, it makes sense to break λ up into real and imaginary components, as follows:

$$\lambda = \lambda_r + j\lambda_j.$$

Now, by Euler's identity, we can express

$$e^{\lambda t} = e^{\lambda_r t + j\lambda_j t} = e^{\lambda_r t}\left(\cos(\lambda_j t) + j\sin(\lambda_j t)\right).$$

Since the magnitude of $\cos(x) + j\sin(x)$ is bounded by 1 (it lies within the unit circle), we have that:

$$|e^{\lambda t}| = e^{\lambda_r t},$$

so it becomes unbounded as $t \to \infty$ if $\lambda_r > 0 \iff \text{Re}\{\lambda\} > 0$.

Going back to our expression for $x_c(t)$, we can now notice that the leading term $x_c(0)e^{\lambda t}$ will go to infinity when $\text{Re}\{\lambda\} > 0$, even if the noise $w = 0$ throughout, so long as $x_c(0) \neq 0$. What if $x_c(0) = 0$? Then we can adversarially apply some initial noise in order to perturb our state from 0, then set the noise to 0 and let the state blow up. So, our system is unstable.

Thus, a necessary condition for stability is $\text{Re}\{\lambda\} \leq 0$. Is this condition sufficient? To find out, we now need to look at the effects of input on our state over time by considering the second term in our expression for $x_c(t)$. Observe that, when $\text{Re}\{\lambda\} < 0$, we can present the upper bound:

$$\begin{aligned}
|x_c(t)| &= \left| x_c(0)e^{\lambda t} + \int_0^t e^{\lambda(t-\theta)} w(\theta)\, d\theta \right| \\
&\leq |x_c(0)e^{\lambda t}| + \left| \int_0^t e^{\lambda(t-\theta)} w(\theta)\, d\theta \right|
\end{aligned}$$

We've just seen how the first term in the above sum is bounded when $\text{Re}\{\lambda\} < 0$. Focusing on the second term, we can rearrange it as:

$$\begin{aligned}
\left| \int_0^t e^{\lambda(t-\theta)} w(\theta)\, d\theta \right| &\leq \int_0^t |e^{\lambda(t-\theta)} w(\theta)|\, d\theta \\
&= \int_0^t |e^{\lambda(t-\theta)}||w(\theta)|\, d\theta \\
&= \int_0^t e^{\lambda_r(t-\theta)} |w(\theta)|\, d\theta.
\end{aligned}$$

Now, since $e^{\lambda_r(t-\theta)}$ is always a positive real number, it is clear that we can maximize the above expression by setting $|w(\theta)|$ to its maximum value ε. We then obtain the bound:

$$\begin{aligned}
\left| \int_0^t e^{\lambda(t-\theta)} w(\theta)\, d\theta \right| &\leq \varepsilon \int_0^t e^{\lambda_r(t-\theta)}\, d\theta \\
&= \varepsilon e^{\lambda_r t} \int_0^t e^{-\lambda_r \theta}\, d\theta \\
&= \varepsilon e^{\lambda_r t} \left[-\frac{1}{\lambda_r} e^{-\lambda_r \theta} \right]_0^t \\
&= -\frac{\varepsilon}{\lambda_r} e^{\lambda_r t} \left[e^{-\lambda_r \theta} \right]_0^t \\
&= \frac{\varepsilon}{\lambda_r} e^{\lambda_r t} \left[e^{-\lambda_r \theta} \right]_t^0 \\
&= \frac{\varepsilon e^{\lambda_r t}}{\lambda_r} \left(1 - e^{-\lambda_r t} \right) \\
&= \frac{\varepsilon}{\lambda_r} \left(e^{\lambda_r t} - 1 \right).
\end{aligned}$$

Since $\lambda_r < 0$, $e^{\lambda_r t}$ is bounded for all t, and so is the above expression. Therefore, we have shown that both

terms in the closed-form expression of $x_c(t)$ are bounded when $\text{Re}\{\lambda\} < 0$.

Looking back at what we have shown, we know that our system is stable when $\text{Re}\{\lambda\} < 0$, and unstable when $\text{Re}\{\lambda\} > 0$. The only remaining case is when $\text{Re}\{\lambda\} = 0$, where $\lambda = j\lambda_j$. In this case, consider the bounded noise $w(t) = \varepsilon e^{j\lambda_j t}$. Observe that, even when we start with $x_c(0) = 0$, our state becomes

$$\begin{aligned} x_c(t) &= \int_0^t e^{\lambda(t-\theta)} w(\theta)\, d\theta \\ &= \int_0^t e^{\lambda(t-\theta)} \varepsilon e^{j\lambda_j \theta}\, d\theta \\ &= \varepsilon e^{j\lambda_j t} \int_0^t e^{-\lambda_j \theta + \lambda_j \theta}\, d\theta \\ &= \varepsilon e^{j\lambda_j t} \int_0^t 1\, d\theta \\ &= \varepsilon t e^{j\lambda_j t}. \end{aligned}$$

Thus, $|x_c(t)| = \varepsilon t$, so it goes to infinity over time despite the noise being bounded by $|w(t)| = \varepsilon$ throughout. Consequently, when $\text{Re}\{\lambda\} = 0$, our system is still unstable.

Putting everything together, we ultimately see that continuous-time scalar systems are stable exactly when $\text{Re}\{\lambda\} < 0$.

Now, what about continuous-time vector systems? By performing diagonalization in an analogous manner to what we did earlier for discrete-time systems, it can be seen that (diagonalizable) vector systems are stable exactly when $\text{Re}\{\lambda_i\} < 0$ for all the eigenvalues λ_i of the state matrix A. Thus, we now have derived a necessary and sufficient condition for the stability of diagonalizable continuous-time systems.

7 Closed-Loop Control

Now, we know (at least to some extent) when a system is stable in the absence of input. That is, we can determine when applying bounded perturbations over time to a system can never lead to unbounded deviations from the stable state. However, many real-world systems are not stable in this manner. They instead rely on continuous monitoring and intervention to keep them near a desired state. We will now begin to explore how we may choose inputs that achieve our goal of stability, returning to discrete-time systems.

In particular, imagine choosing inputs that linearly depend on the state vector. Specifically, let

$$\vec{u}_d[i] = K\vec{x}_d[i]$$

where K is a matrix that we can adjust. Then, our state equation becomes

$$\begin{aligned} \vec{x}_d[i+1] &= A\vec{x}_d[i] + B\vec{u}_d[i] + \vec{w}[i] \\ &= A\vec{x}_d[i] + BK\vec{x}_d[i] + \vec{w}[i] \\ &= (A + BK)\vec{x}_d[i] + \vec{w}[i]. \end{aligned}$$

In essence, it is as if we have replaced our original state matrix A with the new matrix $A + BK$, but continue to apply no other input. Ideally, we'd be able to choose a K that would move the eigenvalues of the state transition matrix $A + BK$ into the regime of stability. We will derive the general condition for the existence of such a K in future lectures. Right now, however, we will examine a few special cases. Consider the scalar case, with the system

$$x_d[t+1] = ax_d[t] + bu_d[t] + w[t]$$

If $|a| \geq 1$ (for instance, if $a = 3$), we know that this system will be unstable in the absence of input. But now,

imagine that we choose $u(t) = kx(t)$ for a constant k to be determined. Substituting into the state equation, we obtain:

$$x_d[t+1] = (a+bk)x_d[t] + w[t].$$

Thus, when $b \neq 0$, we can choose k to set our system's eigenvalues to whatever we want! In particular, by setting $k = -a/b$, we obtain

$$x_d[t+1] = w(t)$$

minimizing the system's eigenvalues and ensuring that noise cannot accumulate geometrically. Now, we will look at a simple 2D case, with the state equation

$$\vec{x}_d[t+1] = \begin{bmatrix} 0 & 1 \\ 3 & 2 \end{bmatrix} \vec{x}_d[t] + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \vec{u}_d[t] + \vec{w}[t].$$

Notice that with no input, the eigenvalues of our state matrix are 3 and -1, so this system is definitely unstable in the absence of input. Notice also that the input only affects the second state directly, so we have no direct way of manipulating the first state! This is in contrast to our scalar example, where our input could affect the entire (one-dimensional) state. With that in mind, let our unknown K be

$$K = \begin{bmatrix} k_1 & k_2 \end{bmatrix}$$

so we obtain the following state equation:

$$\vec{x}_d[t+1] = \left(\begin{bmatrix} 0 & 1 \\ 3 & 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} k_1 & k_2 \end{bmatrix} \right) \vec{x}_d[t] + \vec{w}[t].$$

We are interested in minimizing the magnitudes of both eigenvectors of our state transition matrix, which may be re-expressed as

$$\begin{aligned} A + BK &= \begin{bmatrix} 0 & 1 \\ 3 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ k_1 & k_2 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 \\ 3+k_1 & 2+k_2 \end{bmatrix} \end{aligned}$$

Computing the eigenvalues λ, we see that they must satisfy the characteristic polynomial:

$$\begin{aligned} (-\lambda)(2+k_2-\lambda) - (3+k_1) &= 0 \\ \lambda^2 - (k_2+2)\lambda - (k_1+3) &= 0 \end{aligned}$$

Notice that we have full control over both coefficients of the characteristic polynomial, even though we can't fully control the state matrix. Therefore, we can place the state matrix's eigenvalues wherever we like, so we can still choose a feedback control law to make our system stable! In the future, we will explore the condition that determines whether a K that stabilizes a system exists.

Contributors:

- Neelesh Ramachandran.
- Gireeja Ranade.
- Rahul Arya.
- Anant Sahai.

- Jaijeet Roychowdhury.

EECS 16B Designing Information Devices and Systems II

Note 9: Controllability

1 Overview

An overview of controllability, and the general motivation for what follows, was presented in Note 7A.

2 Special Cases of Controllability

Many of the systems that we study have the following form:

$$\vec{x}_d[t+1] = A_d\vec{x}_d[t] + \vec{u}_d[t],$$

Since we're only discussing discrete-time systems here, we will drop the subscript $_d$ (so $\vec{x}_d[t] \equiv \vec{x}[t], A \equiv A_d$, etc. And, t is the timestep). We want to use our observations of $\vec{x}[t]$ and apply some *control* to the system (via the input $\vec{u}[t]$) to make $\vec{x}[t]$ approach some target $\vec{x}^\star$ over time. Under what conditions can we do this? In this model, our input vector can influence the next timestep's state vector directly. Not only that, but comparing the dimensions of the quantities in our equation, the input vector has as many elements as the state vector itself.

So, if we have full control over $\vec{u}[t]$, this problem is easy! Given some $\vec{x}[0]$, we may choose $\vec{u}[0] = \vec{x}^\star - A\vec{x}[0]$, to drive $\vec{x}[1]$ to our desired state in a single time step. This is because we are setting the input to be the difference between where we start and where we want to end up.

However, in reality, we seldom have that many degrees of freedom in our input $\vec{u}[t]$. Imagine, for instance, that our state equation modeled the behavior of a circuit connected to an input voltage that we can vary. We have seen previously that the state vector of a circuit may include many components, like the voltages across capacitors or the currents through inductors, none of which we can directly change. Therefore, we use the following equation to obtain a more accurate model of our system:

$$\vec{x}[t+1] = A\vec{x}[t] + B\vec{u}[t].$$

This equation acknowledges that the number of inputs we can independently adjust at any single time might be different from the dimension of the state. Notice that we have added the matrix B, which constrains how our input $\vec{u}[t]$ can influence the evolution of our system over time. For instance, in a simple system with a two-dimensional state, the case of

$$B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

would mean that our input can only directly affect the second state. For now, we will treat the above discrete-time equation as an accurate model of our system (ignoring noise and other disturbances from Note 7B).

Before, we were always able to choose a $\vec{u}[0]$ to get to our target state in a single time step, no matter what

the initial state $\vec{x}[0]$ was. Let's try doing the same thing now for an arbitrary B:

$$\begin{aligned} & \vec{x}[1] = \vec{x}^\star \\ \implies \quad & A\vec{x}[0] + B\vec{u}[0] = \vec{x}^\star \\ \implies \quad & B\vec{u}[0] = \vec{x}^\star - A\vec{x}[0]. \end{aligned}$$

Observe here that we can only achieve our desired state if $\vec{x}^\star - A\vec{x}[0] \in \text{range}(B)$, which is not always the case. For example, B may be a "tall" matrix (more rows than columns). Let n be the dimension of $\vec{x}$ - in other words, let $\vec{x}$ be made up of n scalar components. For *any* desired state to be reachable, B would have to span all of $\mathbb{R}^n$, and so be of rank n.

So, is all hope lost? Not necessarily. Recall that we only wanted to achieve our target state at *some point*, not necessarily in a single time step. Consider the state transition matrix

$$A = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix},$$

with B defined as before (i.e. only allowing us to affect the second state). Since B has only one column, our control input $\vec{u}$ is one dimensional, and so can be treated as a scalar u. We know that, after 1 time step, we can only reach the states that can be written as

$$\vec{x}[1] = A\vec{x}[0] + Bu[0]$$

for a suitable choice of control $u[0]$. Since B is not of rank 2, the set of viable $\vec{x}[1]$ does not yet span all of $\mathbb{R}^2$. However, now consider the set of states that are reachable after 2 time steps. By our state transition equation, they can be written as

$$\begin{aligned} \vec{x}[2] &= A\vec{x}[1] + Bu[1] \\ &= A\left(A\vec{x}[0] + Bu[0]\right) + Bu[1] \\ &= A^2\vec{x}[0] + ABu[0] + Bu[1] \end{aligned}$$

Notice that if the vectors AB and B together span all of $\mathbb{R}^2$, then we would be able to choose coefficients $u[0]$ and $u[1]$ (that is, two different controls at timesteps 0 and 1) to reach *any desired* $\vec{x}[2]$, meaning that the system is controllable in 2 time steps.

3 General Controllability

Generalizing our equation for $\vec{x}[2]$ to the case of multidimensional control inputs (i.e. when B has k columns, not just 1, and our input has multiple components), in addition to arbitrarily large state dimensions n, we obtain

$$\vec{x}[2] = A^2\vec{x}[0] + AB\vec{u}[0] + B\vec{u}[1]$$

from an analogous calculation.

This can be reexpressed as

$$\vec{x}[2] = A^2\vec{x}[0] + \begin{bmatrix} AB & B \end{bmatrix} \begin{bmatrix} \vec{u}[0] \\ \vec{u}[1] \end{bmatrix},$$

using stacked matrix notation.

More generally, we can show that as time passes, and the effect of our control inputs propagate, the state

vector after n timesteps becomes:

$$\vec{x}[t] = A^n\vec{x}[0] + \begin{bmatrix} B & AB & \cdots & A^{t-1}B \end{bmatrix} \begin{bmatrix} \vec{u}[t-1] \\ \vdots \\ \vec{u}[0] \end{bmatrix}.$$

This pattern is familiar to us from our analysis of discretization, and the way we unrolled the recursion in an analogous way in dis 03A and dis 07A. We have chosen to order our inputs in "reverse" order, such that the most recently applied input comes on top. This structure allows us to present the stacked matrix in a more natural order (B is the effect of the most recent input, AB is the effect of the input before that, and so on). Therefore, we find that our system is controllable after t time steps if and only if the matrix

$$\mathscr{C} = \begin{bmatrix} B & AB & \cdots & A^{t-1}B \end{bmatrix}$$

has a column span that is n-dimensional.

The question now becomes - is *every* system controllable after sufficiently many time steps? That is to say, is it true that, for any $n \times n$ matrix A and $n \times k$ matrix B, there exists some constant t such that the matrix $\begin{bmatrix} B & AB & \cdots & A^{t-1}B \end{bmatrix}$ is of full rank?

Unfortunately, this is not the case - we can construct a trivial counterexample by considering the case when the input has no effect on the state transition, and $B = 0$! But even with nontrivial B matrices, some systems may not be controllable. Consider, for instance:

$$A = \begin{bmatrix} -2 & 2 \\ 2 & 1 \end{bmatrix}$$
$$B = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Observe that

$$AB = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$
$$A^2B = \begin{bmatrix} 4 \\ 8 \end{bmatrix},$$

and so on, so all of the columns of the stacked matrix will be linearly dependent. No matter how many timesteps we go, columns of this form will not add to our column span. Thus, even though this system has a nontrivial state transition equation, it is still not controllable even after infinitely many time steps.

4 Determining Controllability

We will now attempt to devise a general method of determining whether or not a system is controllable. In essence, we want to know whether the infinite sequence

$$\text{range}\left(\begin{bmatrix} B \end{bmatrix}\right), \text{range}\left(\begin{bmatrix} B & AB \end{bmatrix}\right), \text{range}\left(\begin{bmatrix} B & AB & A^2B \end{bmatrix}\right), \ldots$$

will ever reach $\mathbb{R}^n$. If, at some finite point, this sequence reaches $\mathbb{R}^n$, then that's great, and we know that our system is controllable! However, if our system turns out to be uncontrollable, then this sequence will keep going on forever and will never reach $\mathbb{R}^n$. Unfortunately, it is not possible to somehow evaluate an infinite

number of terms of our sequence. So then, with only a finite number of terms of the sequence, how can we ever confidently say: "No, this system is not controllable"?

Our approach will rely on the following key observation: that if all the columns of A^tB are linearly dependent on the previous A^kB for $k < t$, then all the columns of $A^{t+1}B$ will also be linearly dependent on the same set of A^kB. That is to say, if the ranges in the above sequence *ever stop growing* for even a single iteration, then they will *never start growing again*.

For simplicity, we will conduct our proof under the assumption that $B = \vec{b}$ is a column vector, so all the $A^tB = A^t\vec{b}$ have only one column. However, the exact same approach works in the most general case, except that the notation is more messy.

By the condition of the key observation, and the definition of linear dependence, there exist coefficients α_k such that:

$$A^t\vec{b} = \sum_{k=0}^{t-1} \alpha_k A^k\vec{b} \tag{1}$$

Thus, we may write the following sequence of equations:

$$\begin{aligned}
A^{t+1}\vec{b} &= A(A^t\vec{b}) \\
\text{[by eq. (1)]} \qquad &= A\sum_{k=0}^{t-1} \alpha_k A^k\vec{b} \\
\text{[bring } A \text{ inside sum]} \qquad &= \sum_{k=0}^{t-1} \alpha_k A^{k+1}\vec{b} \\
\text{[re-number indices]} \qquad &= \sum_{k=1}^{t} \alpha_{k-1} A^k\vec{b} \\
\text{[split sum; separate } k=t \text{ case]} \qquad &= \alpha_{t-1}A^t\vec{b} + \sum_{k=1}^{t-1} \alpha_{k-1} A^k\vec{b} \\
\text{[sub. eq. (1), bring in scalar]} \qquad &= \sum_{k=0}^{t-1} \alpha_{t-1}\alpha_k A^k\vec{b} + \sum_{k=1}^{t-1} \alpha_{k-1} A^k\vec{b} \\
\text{[group terms, separate } k=0] \qquad &= \alpha_{t-1}\alpha_0\vec{b} + \sum_{k=1}^{t-1} (\alpha_{t-1}\alpha_k + \alpha_{k-1}) A^k\vec{b}
\end{aligned}$$

So, we have shown that $A^{t+1}B$ can be expressed as a linear combination of the same preceding A^kB.

With this result, we are on the road to developing an algorithm to determine the controllability of a system. We iteratively construct the sequence of ranges as described above. If the dimension of the ranges ever stops growing, we know that the dimension will never start to grow again as the sequence continues, so we know what possible $\vec{x}^\star$ are reachable even as $t \to \infty$. The only issue is if the dimension of the ranges never stops growing. But this can never be the case, since the dimension of the ranges is bounded by n (since they are all subspaces of $\mathbb{R}^n$)!

Thus, in summary: After n iterations, the ranges will either have stopped growing during one of the intermediate iterations, or would have reached rank n and so will not be able to continue to grow thereafter.

The argument above was given for any single column $\vec{b}$ and so in spirit, it can apply to all the columns viewed one at a time. So, we might as well do them all together. Consequently, by considering the subspace

$$\text{range}\left(\begin{bmatrix} B & AB & \dots & A^{n-1}B \end{bmatrix}\right)$$

we will be able to determine whether our system is controllable. If this range is of dimension n (and so spans the entirety of $\mathbb{R}^n$), then our system is controllable. Otherwise, it is not. Typically, we refer to this span as range($\mathscr{C}$), where $\mathscr{C}$ is defined as

$$\mathscr{C} = \begin{bmatrix} B & AB & \cdots & A^{n-1}B \end{bmatrix}.$$

An immediate consequence of this result is that a system is controllable *if and only if* it is controllable in at most n timesteps.

The next natural question is as follows: even if our system is controllable, how do we calculate the necessary inputs needed to reach a desired state $\vec{x}^\star$? As it turns out, this is straightforward, if we consider the stacked matrix representation of $\vec{x}[n] = \vec{x}^\star$:

$$\vec{x}^\star = A^n\vec{x}[0] + \begin{bmatrix} B & AB & \cdots & A^{n-1}B \end{bmatrix} \begin{bmatrix} \vec{u}[n-1] \\ \vdots \\ \vec{u}[0] \end{bmatrix} = A^n\vec{x}[0] + \mathscr{C} \begin{bmatrix} \vec{u}[n-1] \\ \vdots \\ \vec{u}[0] \end{bmatrix}$$

Rearranging, we obtain

$$\mathscr{C} \begin{bmatrix} \vec{u}[n-1] \\ \vdots \\ \vec{u}[0] \end{bmatrix} = \vec{x}^\star - A^n\vec{x}[0].$$

Using Gaussian elimination, we can now determine a sequence of control inputs to use, if such a sequence exists. Thus, we have resolved the problem that was initially posed at the beginning of our discussion. Later, we will learn how to choose potentially better sequences of control inputs if the solution above is not unique.

Contributors:

- Neelesh Ramachandran.
- Rahul Arya.
- Anant Sahai.

EECS 16B Designing Information Devices and Systems II

Note 10A: Orthonormalization

1 Speeding up Least Squares

In 16A, you were introduced to least squares to approximate solutions to systems of equations. An example context you saw was radio transmissions from devices. We have a huge number n of devices that could potentially be transmitting its own unique signal. The m-long signal $\vec{y}$ we receive is the sum of the signals from each active device.

Specifically, let $A \in \mathbb{R}^{m \times n}$ be a matrix with $m > n$ (a tall matrix), where the kth column represents the signal that is sent by device k:

$$A = \begin{bmatrix} | & | & & | \\ \vec{S}_1 & \vec{S}_2 & \cdots & \vec{S}_n \\ | & | & & | \end{bmatrix}$$

Then, let $\vec{x} \in \mathbb{R}^n$ be a vector where the kth entry represents the strength of the kth device. Our received signal is

$$\vec{y} = A\vec{x} = \begin{bmatrix} | & | & & | \\ \vec{S}_1 & \vec{S}_2 & \cdots & \vec{S}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x[1] \\ x[2] \\ \vdots \\ x[n] \end{bmatrix} = \sum_{k=1}^{n} x[k]\vec{S}_k$$

This is the "noise-free" case. More realistically, $\vec{y}$ is only approximately given by the above, but there is another (presumably small) noise component as well. To find the $x[i]$ that best estimates which signal strengths were used, we need to use least squares to project $\vec{y}$ onto the columns of A.

$$\vec{x} = (A^\top A)^{-1} A^\top \vec{y}.$$

Now assume that often times, new devices and signals $\vec{S}_i$ are added to our database. To solve the least squares problem again, we will need to recalculate the entire equation above, including the inverse. However, matrix inversion is computationally expensive — for a $n \times n$ matrix, inversion takes O(n^3). We might have to do inversion at many time steps as more signals S_i are added to our database, so is there a way to reuse our prior computation? (Note that this may not be a very realistic issue, and that the methods we learn below are much more general and useful for other purposes.)

2 Orthogonal Vectors and Projection

Recall from 16A that two vectors $\vec{v}, \vec{w}$ are orthogonal if they are 90° apart. Remember that an equivalent definition is that they are orthogonal if and only if

$$\langle \vec{v}, \vec{w} \rangle = \vec{v}^\top \vec{w} = \vec{w}^\top \vec{v} = 0 \tag{1}$$

Recall that the orthogonal projection of a vector $\vec{y}$ on to any other nonzero vector $\vec{b}$ is

$$\vec{y}_{\vec{b}} = \frac{\vec{y}^\top \vec{b}}{\left\|\vec{b}\right\|^2}\vec{b} \tag{2}$$

Also recall that least squares is just an orthogonal projection of a vector onto an entire subspace of vectors, so

$$\vec{y}_A = A\hat{x} = A(A^\top A)^{-1}A^\top \vec{y} \tag{3}$$

In this section, we will show that if the columns of A are mutually orthogonal to each other, the projection of $\vec{y}$ onto span(A) is the sum of the projection of $\vec{y}$ onto each column of A individually. Let's take a look at the case where $j = 2$ and the songs are mutually orthogonal. Suppose the songs found so far are $\vec{S}_1$ and $\vec{S}_2$, i.e., $A_2 = \begin{bmatrix} | & | \\ \vec{S}_1 & \vec{S}_2 \\ | & | \end{bmatrix}$. Then, the projection of $\vec{y}$ onto A_2 is

$$\vec{y}_{A_2} = A_2\left(A_2^\top A_2\right)^{-1}A_2^\top \vec{y}. \tag{4}$$

Let's first compute the term $\left(A_2^\top A_2\right)^{-1}$:

$$A_2^\top A_2 = \begin{bmatrix} — & \vec{S}_1{}^\top & — \\ — & \vec{S}_2{}^\top & — \end{bmatrix}\begin{bmatrix} | & | \\ \vec{S}_1 & \vec{S}_2 \\ | & | \end{bmatrix} \tag{5}$$

$$= \begin{bmatrix} \vec{S}_1{}^\top \vec{S}_1 & \vec{S}_1{}^\top \vec{S}_2 \\ \vec{S}_2{}^\top \vec{S}_1 & \vec{S}_2{}^\top \vec{S}_2 \end{bmatrix} \tag{6}$$

$$= \begin{bmatrix} \|\vec{S}_1\|^2 & 0 \\ 0 & \|\vec{S}_2\|^2 \end{bmatrix}. \tag{7}$$

Thus, we have a diagonal matrix and so

$$\left(A_2^\top A_2\right)^{-1} = \begin{bmatrix} \frac{1}{\left\|\vec{S}_1\right\|^2} & 0 \\ 0 & \frac{1}{\left\|\vec{S}_2\right\|^2} \end{bmatrix}. \tag{8}$$

Then, substituting this matrix into the original expression, the projection of $\vec{y}$ onto span(A_2) is

$$\vec{y}_{A_2} = A_2 \left(A_2^\top A_2\right)^{-1} A_2^\top \vec{y} \tag{9}$$

$$= \begin{bmatrix} | & | \\ \vec{S}_1 & \vec{S}_2 \\ | & | \end{bmatrix} \begin{bmatrix} \frac{1}{\left\|\vec{S}_1\right\|^2} & 0 \\ 0 & \frac{1}{\left\|\vec{S}_2\right\|^2} \end{bmatrix} \begin{bmatrix} — & \vec{S}_1^{\,\top} & — \\ — & \vec{S}_2^{\,\top} & — \end{bmatrix} \vec{y} \tag{10}$$

$$= \begin{bmatrix} | & | \\ \vec{S}_1 & \vec{S}_2 \\ | & | \end{bmatrix} \begin{bmatrix} \frac{1}{\left\|\vec{S}_1\right\|^2} & 0 \\ 0 & \frac{1}{\left\|\vec{S}_2\right\|^2} \end{bmatrix} \begin{bmatrix} \vec{S}_1^{\,\top}\vec{y} \\ \vec{S}_2^{\,\top}\vec{y} \end{bmatrix} \tag{11}$$

$$= \begin{bmatrix} | & | \\ \vec{S}_1 & \vec{S}_2 \\ | & | \end{bmatrix} \begin{bmatrix} \frac{\vec{S}_1^{\,\top}\vec{y}}{\left\|\vec{S}_1\right\|^2} \\ \frac{\vec{S}_2^{\,\top}\vec{y}}{\left\|\vec{S}_2\right\|^2} \end{bmatrix} \tag{12}$$

$$= \left(\frac{\vec{S}_1^{\,\top}\vec{y}}{\left\|\vec{S}_1\right\|^2}\right)\vec{S}_1 + \left(\frac{\vec{S}_2^{\,\top}\vec{y}}{\left\|\vec{S}_2\right\|^2}\right)\vec{S}_2. \tag{13}$$

Observe that the first term in the sum above is the projection of $\vec{y}$ onto $\vec{S}_1$ and the second term is the projection of $\vec{y}$ onto $\vec{S}_2$. Generalizing this pattern, we can guess that the projection of $\vec{y}$ onto span(A_n) where A_n has mutually orthogonal columns is

$$\vec{y}_{A_n} = \left(\frac{\vec{S}_1^{\,\top}\vec{y}}{\left\|\vec{S}_1\right\|^2}\right)\vec{S}_1 + \left(\frac{\vec{S}_2^{\,\top}\vec{y}}{\left\|\vec{S}_2\right\|^2}\right)\vec{S}_2 + \cdots + \left(\frac{\vec{S}_n^{\,\top}\vec{y}}{\left\|\vec{S}_n\right\|^2}\right)\vec{S}_n. \tag{14}$$

Furthermore, observe that if $\vec{S}_1, \ldots, \vec{S}_n$ are unit vectors (i.e., they all have length 1), then the above would further reduce to

$$\vec{y}_{A_n} = \left(\vec{S}_1^{\,\top}\vec{y}\right)\vec{S}_1 + \left(\vec{S}_2^{\,\top}\vec{y}\right)\vec{S}_2 + \cdots + \left(\vec{S}_n^{\,\top}\vec{y}\right)\vec{S}_n \tag{15}$$

$$= \begin{bmatrix} | & & | \\ \vec{S}_1 & \cdots & \vec{S}_n \\ | & & | \end{bmatrix} \begin{bmatrix} — & \vec{S}_1^{\,\top} & — \\ & \vdots & \\ — & \vec{S}_n^{\,\top} & — \end{bmatrix} \vec{y} = A_n A_n^\top \vec{y} \tag{16}$$

Definition: A set of vectors $\{\vec{v}_1, \ldots, \vec{v}_n\}$ is **orthonormal** if all the vectors are mutually orthogonal to each other (i.e. $\vec{v}_i^\top \vec{v}_j = 0$ if $i \neq j$) and all are of unit length (i.e. $\|\vec{v}_i\| = 1 = \vec{v}_i^\top \vec{v}_i$). Thus above, A_n has orthonormal columns. We now will generalize what we did earlier by showing that for any matrix Q with n

orthonormal columns, $Q^\top Q = I_n$.

$$Q^\top Q = \begin{bmatrix} — & \vec{q}_1^\top & — \\ & \vdots & \\ — & \vec{q}_n^\top & — \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{q}_1 & \dots & \vec{q}_n \\ | & & | \end{bmatrix} \tag{17}$$

$$= \begin{bmatrix} \vec{q}_1^\top \vec{q}_1 & \vec{q}_1^\top \vec{q}_2 & \dots & \vec{q}_1^\top \vec{q}_n \\ \vec{q}_2^\top \vec{q}_1 & \ddots & & \vec{q}_2^\top \vec{q}_n \\ \vdots & & \ddots & \vdots \\ \vec{q}_n^\top q_1 & \vec{q}_n^\top \vec{q}_2 & \dots & \vec{q}_n^\top \vec{q}_n \end{bmatrix} \tag{18}$$

$$= \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix} = I_n \tag{19}$$

Remember that we are using the property of orthonormal vectors that $\vec{q}_i^\top \vec{q}_j = 0$ when $i \neq j$ and $\vec{q}_i^\top \vec{q}_i = ||\vec{q}_i||^2 = 1$. Using this, notice that the least-squares estimate with orthonormal vectors simplifies to $\vec{y}_{A_n} = A_n(A_n^\top A_n)^{-1} A_n^\top \vec{y} = A_n(I)^{-1} A_n^\top \vec{y} = A_n A_n^\top \vec{y}$. By direct algebraic manipulation, we formally validated our generalization in equation 16.

Note that the above proof is general and applies to non-square matrices. However, we will specially refer to square matrices whose columns are orthonormal as orthonormal [1] matrices. If a square matrix Q is orthonormal, we can additionally say that $Q^\top Q = QQ^\top = I \implies Q^\top = Q^{-1}$.

Thus, we can see that having orthonormal vectors $\vec{S}_i$ will make least squares must faster and only consist of 1 matrix multiplication. But now you may ask how can we even ensure that $\vec{S}_i$ are orthonormal?

3 Orthonormalization

We want to take a sequence of vectors $\vec{S}_1, \vec{S}_2, \dots \vec{S}_k$ and construct a new sequence of vectors $\vec{q}_1, \vec{q}_2, \dots, \vec{q}_k$ that are orthonormal (i.e. $\vec{q}_i^\top \vec{q}_j = \begin{cases} 0 & \text{if } i \neq j \\ 1 & \text{if } i = j \end{cases}$) and satisfy the property that the subspaces span$(\vec{S}_1, \vec{S}_2, \dots \vec{S}_\ell)$ spanned by the original set of vectors are always the same as the subspaces span$(\vec{q}_1, \vec{q}_2, \dots, \vec{q}_\ell)$ spanned by the new set of vectors.

This might seem hard but we will start at the beginning and proceed systematically. We first let $\vec{q}_1 = \frac{\vec{S}_1}{\|\vec{S}_1\|}$ to make it normal, and it will clearly have the same span as $\vec{S}_1$. We will then leverage what we know about projections and least-squares from 16A. We know that the residual vector after a projection is always orthogonal [2] to the subspace being projected upon. So to ensure that the new vector $\vec{q}_k$ is orthogonal, we can just remove all parts of $\vec{S}_k$ that lie in the span of our previous vectors. From the previous section and equation 16, since the $\vec{q}_i$ are orthonormal, this is equivalent to subtracting each individual projection onto

[1] In a bit of confusing notation, in math literature you will often see such matrices called orthogonal even when they want to explicitly require that each column is normalized to have unit norm. We will try to use "orthonormal" to avoid this confusion.

[2] Recall that this is how we actually derived the least-squares formula!

all of our previous $\vec{q}_i$ vectors. Consequently, we can recursively define

$$\vec{q}_k = \frac{\vec{S}_k - \sum_{\ell=1}^{k-1} \vec{q}_\ell (\vec{q}_\ell^\top \vec{S}_k)}{\|\vec{S}_k - \sum_{\ell=1}^{k-1} \vec{q}_\ell (\vec{q}_\ell^\top \vec{S}_k)\|}. \tag{20}$$

This has unit norm by construction and it is orthogonal to all the previous $\vec{q}_\ell$ because it removes all the projections of the new vector $\vec{S}_k$ onto the subspace spanned by them. This collection also preserves the same span because every new vector $\vec{q}_k$ is just a linear combination of the original vectors. It turns out that this very natural iterative process that we "discovered for ourselves" has a name: **Gram-Schmidt Orthonormalization**.

Said more slowly and using generic language, Gram Schmidt is a procedure that takes a list[3] of linearly independent vectors $\{\vec{S}_1, \ldots, \vec{S}_n\}$ and generates an orthonormal list of vectors $\{\vec{q}_1, \ldots, \vec{q}_n\}$ that span the same subspaces as the original list. Concretely, $\{\vec{q}_1, \ldots, \vec{q}_n\}$ satisfy the following:

$$\{\vec{q}_1, \ldots, \vec{q}_n\} \text{ is an orthonormal set of vectors} \tag{21}$$

$$\text{span}(\{\vec{S}_1, \ldots, \vec{S}_k\}) = \text{span}(\{\vec{q}_1, \ldots, \vec{q}_k\}) \quad \forall 1 \leq k \leq n \tag{22}$$

Proof of Orthonormality (21):

We first start with showing each vector is normal, or unit length. This is true by construction since we are dividing a vector by the norm of that vector, so the result must have norm 1. In other words, for all vectors $\vec{v}$,

$$\left|\left| \frac{\vec{v}}{\|\vec{v}\|} \right|\right| = \frac{1}{\|\vec{v}\|} \|\vec{v}\| = 1$$

Now onto orthogonality. We will use an inductive approach by first assuming that the first $k-1$ vectors are already orthonormal. We will then show that our new constructed vector $\vec{q}_k$ will be orthogonal to all previous ones, so $\vec{q}_p^T \vec{q}_k = 0$ for all $p = 1, \ldots, k-1$. Note that constant factors don't affect orthogonality, so for simplicity we will combine the norm division for both $\vec{q}_k$ and $\vec{q}_p$ below into one constant A.

$$\vec{q}_p^T \vec{q}_k = A \vec{q}_p^T (\vec{S}_k - \sum_{\ell=1}^{k-1} \vec{q}_\ell (\vec{q}_\ell^\top \vec{S}_k)) \tag{23}$$

$$= A(\vec{q}_p^T \vec{S}_k - \sum_{\ell=1}^{k-1} \vec{q}_p^T \vec{q}_\ell (\vec{q}_\ell^T \vec{S}_k)) \tag{24}$$

Since we assumed the first $k-1$ vectors are all orthonormal, the $\vec{q}_p^T \vec{q}_\ell$ will cause the only nonzero in the summation to occur when $\ell = p$. Then,

$$\vec{q}_p^T \vec{q}_k = A(\vec{q}_p^T \vec{S}_k - \vec{q}_p^T \vec{q}_p (\vec{q}_p^T \vec{S}_k)) \tag{25}$$

$$= A(\vec{q}_p^T \vec{S}_k - \vec{q}_p^T \vec{S}_k) = 0 \tag{26}$$

where we use the fact that $\vec{q}_p^T \vec{q}_p = 1$ since we assumed it is orthonormal. Thus, for any k we know that the next vector we construct will be orthogonal to all of our previous vectors. If this idea of induction is confusing, don't worry as CS 70 will cover it in much more detail.

[3] The fact that these are lists and not sets matters. The vectors are ordered. We don't just want the overall spans to be the same, we want the spans to be the same as we walk down the lists together.

Proof of Equivalent Span (22):

Again, we will use an inductive approach by assuming that the first $k-1$ vectors of S and Q both span the same space. This is true for our base case since $\vec{q}_1 = \vec{S}_1/\|\vec{S}_1\|$. Now all we need to show is that $\vec{S}_k$ can be written as a linear combination of $\{\vec{q}_1, \ldots, \vec{q}_k\}$. By construction of $\vec{q}_k$, that is exactly true with

$$\vec{S}_k = \left\| \vec{S}_k - \sum_{\ell=1}^{k-1} (\vec{q}_\ell^T \vec{S}_k)\vec{q}_\ell \right\| \vec{q}_k + \sum_{\ell=1}^{k-1} (\vec{q}_\ell^T \vec{S}_k)\vec{q}_\ell$$

Thus, for all k, the spans will be equivalent.

3.1 Example for three vectors

The above might have been a bit fast, so let's walk through the reasoning for the case of three vectors to make sure it is clear.

Consider three vectors $\{\vec{S}_1, \vec{S}_2, \vec{S}_3\}$ that are linearly independent of each other.

- **Step 1:** Find unit vector $\vec{q}_1$ such that span($\{\vec{q}_1\}$) = span($\{\vec{S}_1\}$).
  Since span($\{\vec{S}_1\}$) is a one dimensional vector space, we can simply scale $\{\vec{S}_1\}$ so that it is unit norm:

$$\vec{q}_1 = \frac{\vec{S}_1}{\left\|\vec{S}_1\right\|}. \tag{27}$$

- **Step 2:** Given $\vec{q}_1$ from the previous step, find $\vec{q}_2$ such that span($\{\vec{q}_1, \vec{q}_2\}$) = span($\{\vec{S}_1, \vec{S}_2\}$) and orthogonal to $\vec{q}_1$. We know that $\vec{S}_2-$ (the projection of $\vec{S}_2$ on $\vec{q}_1$) would be orthogonal to $\vec{q}_1$ from 16A. So first, we can find the error or residual

$$\vec{e}_2 = \vec{S}_2 - \left(\vec{q}_1^\top \vec{S}_2\right) \vec{q}_1, \tag{28}$$

  which is orthogonal to $\vec{q}_1$. Then, we can normalize to get $\vec{q}_2 = \frac{\vec{e}_2}{\|\vec{e}_2\|}$. Note that these operations preserve the span because $\vec{q}_1$ and $\vec{q}_2$ are just linear combinations of $\vec{S}_1$ and $\vec{S}_2$ and vice-versa.

- **Step 3:** Now given $\vec{q}_1$ and $\vec{q}_2$ in the previous steps, we would like to find $\vec{q}_3$ such that span($\{\vec{q}_1, \vec{q}_2, \vec{q}_3\}$) = span($\{\vec{S}_1, \vec{S}_2, \vec{S}_3\}$). We know that the projection of $\vec{S}_3$ onto the subspace spanned by $\vec{q}_1, \vec{q}_2$ is

$$\left(\vec{q}_2^\top \vec{S}_3\right) \vec{q}_2 + \left(\vec{q}_1^\top \vec{S}_3\right) \vec{q}_1. \tag{29}$$

  Consequently, we know that the error/residual

$$\vec{e}_3 = \vec{S}_3 - \left[\left(\vec{q}_2^\top \vec{S}_3\right) \vec{q}_2 + \left(\vec{q}_1^\top \vec{S}_3\right) \vec{q}_1\right]. \tag{30}$$

  is orthogonal to both $\vec{q}_1$ and $\vec{q}_2$. Normalizing, we have $\vec{q}_3 = \frac{\vec{e}_3}{\|\vec{e}_3\|}$.

The procedure given earlier is just a continuation of this pattern.

Inputs

- A list of linearly independent vectors $\{\vec{S}_1, \ldots, \vec{S}_n\}$.

Outputs

- An orthonormal list of vectors $\{\vec{q}_1, \ldots, \vec{q}_n\}$, where span($\{\vec{S}_1, \ldots, \vec{S}_k\}$) = span($\{\vec{q}_1, \ldots, \vec{q}_k\}$) for all $1 \leq k \leq n$.

Gram Schmidt Procedure

- compute $\vec{q}_1$: $\vec{q}_1 = \frac{\vec{S}_1}{\left\|\vec{S}_1\right\|}$
- for $(i = 2 \ldots n)$:
  (a) Compute the vector $\vec{e}_i$, such that span($\{\vec{q}_1, \ldots, \vec{q}_{i-1}, \vec{e}_i\}$) = span($\{\vec{S}_1, \ldots, \vec{S}_i\}$):

$$\vec{e}_i = \vec{S}_i - \sum_{j=1}^{i-1} \left(\vec{q}_j^{\top} \vec{S}_i\right) \vec{q}_j \tag{31}$$

  (b) Normalize to compute $\vec{q}_i$ itself: $\vec{q}_i = \frac{\vec{e}_i}{\|\vec{e}_i\|}$.

Contributors:

- Ashwin Vangipuram.
- Anant Sahai.
- Jennifer Shih.
- Rachel Hochman.
- Vasuki Narasimha Swamy.
- Steven Cao.

EECS 16B Designing Information Devices and Systems II

Note 10B: Speeding up OMP

1 Setup for OMP

In 16A, you were (usually) introduced to orthogonal matching pursuit (OMP), an algorithm that can find sparse approximate solutions to systems of equations.

In 16A, the context was radio transmissions from the Internet Of Things. We have a huge number n of devices that could potentially be transmitting a signal, but we know that at most k are actually on. The m-long signal $\vec{y}$ we receive is the sum of the signals from each active device. Each device has a particular song that it sings if it is transmitting, with the amplitude of that song depending on its distance from the receiver, etc.

Specifically, let $A \in \mathbb{R}^{m \times n}$ be a matrix with $n > m$ (a wide matrix), where the kth column represents the song that could be sent by device k if it was indeed transmitting:

$$A = \begin{bmatrix} | & | & & | \\ \vec{S}_1 & \vec{S}_2 & \cdots & \vec{S}_n \\ | & | & & | \end{bmatrix}$$

Then, let $\vec{x} \in \mathbb{R}^n$ be a vector where the kth entry represents the "volume" of the kth device. Our received signal is

$$\vec{y} = A\vec{x} = \begin{bmatrix} | & | & & | \\ \vec{S}_1 & \vec{S}_2 & \cdots & \vec{S}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x[1] \\ x[2] \\ \vdots \\ x[n] \end{bmatrix} = \sum_{k=1}^{n} x[k]\vec{S}_k$$

This is the "noise-free" case. More realistically, $\vec{y}$ is only approximately given by the above, but there is another (presumably small) noise component as well. This noise is not out to get us in any way and is unrelated to any of the specific songs.

From $\vec{y}$, we want to infer $\vec{x}$, given that at most $k < m < n$ entries of $\vec{x}$ are non-zero. OMP gives us a way to find which k entries are "on." Then, we can form a matrix A_k with just the columns for the "on" devices, and we can find an estimate for $\vec{x}$ by using standard least-squares.

At a high level, in each iteration of the algorithm, we track the residual $\vec{r}$: the part of $\vec{y}$ still unexplained by the best least-squares fit to our current list of hypothesized "on" devices. In the beginning, when we don't have any hypothesized "on" devices yet, the residual $\vec{r}$ is just $\vec{y}$ itself. We find one more hypothesized "on" device by correlating the residual $\vec{r}$ with each column of A and taking the one whose correlation $\vec{S}_\ell^\top \vec{r}$ has the maximum absolute value. Having added this new device ℓ to our list of hypothesized "on" devices, we recompute the least-square estimate and update the residual. Then we repeat until done. Eventually, we find all k "on" devices or decide that the residual is already small enough with fewer than k, allowing us to solve for $\vec{x}$.

Explicitly, let $A_k = [\vec{S}_{i[1]}, \vec{S}_{i[2]}, \cdots, \vec{S}_{i[k]}]$ denote the matrix slice whose columns correspond to the k hypothesized"on" devices at iteration k. Here $i[1]$ is the index of the device that was hypothesized in the first iteration, $i[2]$ for the winner at the second iteration, and so on through $i[k]$. To find the residual $\vec{r}$ after the k-th iteration, we need to project $\vec{y}$ onto the columns of A_k and then subtract this projection from $\vec{y}$ (recall the projection formula):

$$\vec{r} = \vec{y} - A_k(A_k^\top A_k)^{-1}A_k^\top \vec{y}.$$

However, matrix inversion is computationally expensive — for an $n \times n$ matrix, inversion is $O(n^3)$. We have to do inversion at every time step, making our algorithm slow when we are finding lots of columns.

Is there a way to avoid doing such computations every iteration?

The general secret to making an algorithm run faster is to find a way to avoid duplicating work. Is there some way we can instead better reuse the work we did in earlier iterations?

Yes! All we are doing is projecting $\vec{y}$ onto the subspace spanned by the k vectors $\{\vec{S}_{i[1]}, \vec{S}_{i[2]}, \ldots, \vec{S}_{i[k]}\}$. For general $\vec{S}_k$, we need to compute the projection matrix $A_k(A_k^\top A_k)^{-1}A_k^\top$. But if the $\vec{S}_i$ were all orthogonal, meaning that $\vec{S}_i^T \vec{S}_j = 0$ for $i \neq j$, then we have another faster way of computing the projection without having to invert a matrix.

2 Using Gram Schmidt to speed up OMP

Now, we would like to use Gram Schmidt to speed up OMP. Recall that in each iteration of OMP, we add one more device to our list of "on" devices, appending that song to a matrix of selected songs: $A_k = [A_{k-1} \mid \vec{S}_{i[k]}]$. Then, we use least squares to get the updated residual: $\vec{r}_k = \vec{y} - A_k(A_k^T A_k)^{-1}A_k^T \vec{y}$. This step is the one that most slows us down because we need to perform an expensive inversion each time. It also tells us that while the algorithm is running, the purpose of A_k is just to act as a representative for the subspace spanned by the selected columns of A.

The intuition is that instead of just appending the song $\vec{S}_{i[k]}$ to the matrix A_{k-1}, we can use Gram-Schmidt as we go to make sure that the relevant matrix Q_k is orthonormal. Let Q_k be the orthonormal matrix that represents the same subspace as A_k. How do we maintain Q_k?

First, we initialize $Q_0 = [\ ]$ just as we had initialized A_0. Then, every time we find a new song, we perform Gram-Schmidt orthonormalization before adding it to Q_k, so Q_k remains orthonormal at every iteration.

Recall that we want to solve for $\vec{x}$ in the following equation, where $\vec{x}$ has (at most) k nonzero entries:

$$A\vec{x} = \begin{bmatrix} | & | & & | \\ \vec{S}_1 & \vec{S}_2 & \cdots & \vec{S}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x[1] \\ x[2] \\ \vdots \\ x[n] \end{bmatrix} = \sum_{k=1}^{n} x[k]\vec{S}_k \approx \vec{y}.$$

Following through on our idea above, we get this procedure:

Procedure:

- At time $j = 0$, we have no selected columns, so our residual is all of $\vec{y}$. So we initialize our variables as follows: $\vec{r}_0 = \vec{y}$, $Q_0 = [\ ]$. We will also use a list (thought of as a vector, so that we can index into it naturally) $\vec{i}$ to hold the indices of the columns we have selected so far, initialized to an empty list [].
- Repeat for $j = 1, \ldots, k$: (or stop when the residual is too small, etc.)

(a) Correlate $\vec{r}_{j-1}$ with all of the columns of A — i.e. compute $A^\top \vec{r}_{j-1}$. Find the song with the highest absolute correlation $|\vec{S}_\ell^\top \vec{r}_{j-1}|$. Record the maximizer's index[1] at the end of $\vec{i}$. In mathematical language:

$$i[j] = \arg\max_\ell |\vec{S}_\ell^\top \vec{r}_{j-1}|. \tag{1}$$

(b) Orthonormalize $\vec{S}_{i[j]}$ relative to Q_{j-1} following the spirit of Gram-Schmidt:

i. Find $\vec{e}_j$, or the part of $\vec{S}_{i[j]}$ that is orthogonal to the subspace spanned by Q_{j-1}:

$$\begin{aligned}\vec{e}_k &= \vec{S}_{i[j]} - Q_{j-1}Q_{j-1}^\top \vec{S}_{i[j]} \\ &= \vec{S}_{i[j]} - \sum_{\ell=1}^{j-1} \left(\vec{q}_\ell^\top \vec{S}_{i[j]}\right) \vec{q}_\ell.\end{aligned} \tag{2}$$

This step does require a number of operations that is growing with the iteration count j, but the number of operations grows linearly in j instead of cubicly the way that inverting a generic $j \times j$ matrix from scratch by Gaussian elimination would cost.

ii. Normalize to find $\vec{q}_j$ itself: $\vec{q}_j = \frac{\vec{e}_j}{\|\vec{e}_j\|}$.

iii. Column concatenate[2] the matrix Q_{j-1} with $\vec{q}_j$ to update: $Q_j \leftarrow [Q_{j-1} \mid \vec{q}_j]$.

(c) To find the new residual, project $\vec{y}$ onto Q_j:

$$\begin{aligned}\vec{r}_k &= \vec{y} - Q_j Q_j^\top \vec{y} \\ &= \vec{y} - \sum_{\ell=1}^{j} \left(\vec{q}_\ell^\top \vec{y}\right) \vec{q}_\ell.\end{aligned}$$

We can speed this step up by noticing that in the previous iteration, we had:

$$\vec{r}_{j-1} = \vec{y} - \sum_{\ell=1}^{j-1} \left(\vec{q}_\ell^\top \vec{y}\right) \vec{q}_\ell.$$

Almost all the terms are the same in the sum, except the last one. This means we don't have to redo all that work. We can compute the new residual by updating the previous one. We just need to subtracting the projection of $\vec{y}$ onto $\vec{q}_j$:

$$\vec{r}_j \leftarrow \vec{r}_{j-1} - (\vec{q}_j^\top \vec{y})\vec{q}_j. \tag{3}$$

This is a lot faster than having to recompute everything.

- When the loop above terminates, the information that we are most interested in is in the list $\vec{i}$ — these are the indices of the selected columns of A. To get an actual sparse solution $\vec{x}$ to our original problem $A\vec{x} \approx \vec{y}$, we need to do one last thing: compute $\vec{x}$.

[1] This is why the notation $\arg\max_\ell f(\ell)$ is convenient. If $\ell = 5$ is where the function $f(\ell)$ takes on its maximum value, say $f(5) = 24$, then $\arg\max_\ell f(\ell)$ returns 5. Meanwhile $\max_\ell f(\ell)$ returns 24. In your calculus class, most likely the context was used to determine whether you were interested in 5 vs 24 when you were maximizing something. For writing an algorithm out explicitly, we need some notation to express our intentions.

[2] When trying to get an efficient implementation in a computer, it is important to avoid doing redundant copying of memory, especially for big arrays. After all, every actual copy would involve having to put charge onto a set of capacitors, and you will learn in 61C how this would all have to go through a set of wires in series, taking time because of RC time constants.

We can form the matrix slice $A_{\vec{i}}$ whose columns are the selected columns. (This can be built as we go above.)

$$A_{\vec{i}} = \begin{bmatrix} | & | & & | \\ \vec{S}_{i[1]} & \vec{S}_{i[2]} & \cdots & \vec{S}_{i[k]} \\ | & | & & | \end{bmatrix}$$

Let $\vec{x}_{\vec{i}}$ be the entries of $\vec{x}$ corresponding to the indices in $\vec{i}$. Then, we have the approximate equation to solve:

$$A_{\vec{i}}\vec{x}_{\vec{i}} = \begin{bmatrix} | & | & & | \\ \vec{S}_{i[1]} & \vec{S}_{i[2]} & \cdots & \vec{S}_{i[k]} \\ | & | & & | \end{bmatrix} \begin{bmatrix} x[i[1]] \\ x[i[2]] \\ \vdots \\ x[i[k]] \end{bmatrix} \approx \vec{y}.$$

The sliced matrix $A_{\vec{i}}$ has dimensions dimension $m \times k$ where $k < m$, so we can solve for the non-zero entries of $\vec{x}$ using standard least squares: $\vec{x}_{\vec{i}} = (A_{\vec{i}}^\top A_{\vec{i}})A_{\vec{i}}^\top \vec{y}$. All the other entries of our final solution $\vec{x}$ are zero.

The above algorithm is substantially faster than the naive implementation of OMP.

3 Going even faster

In the above algorithm, there are still a couple of things that feel like they are redoing work that we've already done before. First and foremost, it is the very end. We end up doing a least-squares computation to get the final $\vec{x}$ solution — despite us having done projections already as we were updating the residual. Do we really have to invert a matrix?

If we think about it, along the way, we have effectively already computed the coefficients of the $\vec{q}_k$ that we need to represent the projection of $\vec{y}$ on the subspace spanned by Q_k — this happened when we computed $\vec{q}_k^\top \vec{y}$ in (3). So we know the coefficients in one basis — we just need them in the original coordinates. How would we change coordinates back to get the coefficients for $\vec{x}_{\vec{i}}$?

In general, changing coordinates requires solving a system of equations. A general system of equations with k equations and k unknowns costs something $O(k^3)$ to solve by Gaussian Elimination[3]. But this isn't a general system of equations. Instead, it has a peculiar form. This is because $\vec{S}_{i[1]}$ is just a constant multiple times $\vec{q}_1$, $\vec{S}_{i[2]}$ is just a linear combination of $\vec{q}_1$ and $\vec{q}_2$, and so on with $\vec{S}_{i[j]} = \sum_{\ell=1}(\vec{q}_\ell^\top \vec{S}_{i[j]})\vec{q}_\ell$.

Writing this in matrix form $A_{\vec{i}} = Q_k R$, the matrix R has a shape that resembles the shape that we get at the end of the downward pass of Gaussian Elimination:

$$R = \begin{bmatrix} \vec{q}_1^\top \vec{S}_{i[1]} & \vec{q}_1^\top \vec{S}_{i[2]} & \cdots & \vec{q}_1^\top \vec{S}_{i[k]} \\ 0 & \vec{q}_2^\top \vec{S}_{i[2]} & \cdots & \vec{q}_2^\top \vec{S}_{i[k]} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & \vec{q}_k^\top \vec{S}_{i[k]} \end{bmatrix}. \tag{4}$$

Such matrices are called *upper-triangular* because all the potentially nonzero entries are in a triangle on top. Notice that all of these computations were essentially done during the computation of (2), with the last

[3]Recall that this is because each step downward in Gaussian elimination costs $O(k^2)$ operations since the matrix has size k^2, and there are k such steps.

one effectively being done during the normalization part. So we can reuse all that work if we saved those computations in such a matrix.

Solving a system of equations involving such a matrix is cheap since we can do it using back-substitution, the same way that we finished the upward pass of Gaussian Elimination. This only costs $O(k^2)$ computations since we only have to flow information upward through the matrix, essentially touching each entry only once. This buys us a speedup at the end.

3.1 Even more speed

Once we have noticed the above pattern, we can wonder if we could save even more duplicated work. Where can we start looking for something to speed up? OMP is making repeated passes through the potential columns as it hunts for ones to select. Each time it picks one, it has to execute (2) which has a cost that is growing linearly with the iteration count k. This is because we need to compute the many $\vec{q}_\ell^\top \vec{S}_{i[k]}$ and use them to compute the orthogonal direction that this new $i[k]$ column brings to the subspace we are projecting onto. Can we speed this up so that it doesn't take an increased amount of time each iteration?

At first glance, there doesn't seem like much that we can do. After all, we need to compute all these inner products to orthonormalize. The only question is whether we could somehow rearrange the computations so that these could have already been computed before.

This is when we realize that in every iteration of OMP, we have to do (1) where we compute all the inner-products between the columns of A and the current residual. But the residuals are related to each other! So what is it that we actually have to compute anew?

Notice that at the start of iteration $k+1$ by (3),

$$\begin{aligned}\vec{S}_\ell^\top \vec{r}_k &= \vec{S}_\ell^\top (\vec{r}_{k-1} - (\vec{q}_k^\top \vec{y})\vec{q}_k) \\ &= \vec{S}_\ell^\top \vec{r}_{k-1} - (\vec{q}_k^\top \vec{y})\vec{S}_\ell^\top \vec{q}_k. \end{aligned} \tag{5}$$

The term $\vec{S}_\ell^\top \vec{r}_{k-1}$ was computed last time, so we can just keep this. The term $\vec{q}_k^\top \vec{y}$ is common to this entire iteration, and so we can just keep this. The new thing we actually need to compute is $\vec{S}_\ell^\top \vec{q}_k = \vec{q}_k^\top \vec{S}_\ell$. Notice that these are exactly the computations that we need to do (2).

This means that we can refactor the computations to compute in the above manner and save some work[4]. It turns out that this perspective of carefully tracking progress also opens the door to efficient implementations of more nuanced algorithms. For example, once OMP decides that it is going to add a column to its list, it uses that column to the maximum extent reasonable. This is why once a column has been selected, it will never get selected again. In principle however, you can imagine that after using a little bit of that column, the residual could change so that the winner of (1) is no longer that particular column. Other related algorithms can be made that try to navigate this differently, but these are not in scope for 16B.

3.2 Being more aggressive to speed up even more

Up to this point, we have been getting speedups by noticing how we can reuse work. Notice that to do this, we have had to actually change the computations (leveraging our understanding of what the algorithm is

[4] In principle, from the point of view of complexity scaling with problem size, we can make each iteration cost the same amount by actually tracking running updates (2) for each of the columns of A. It costs m operations to compute an inner-product $\vec{q}_k^\top \vec{S}_\ell$ — we can use the same number of operations to update a running version of (2) as well. In practice, we can just compute (2) using the saved values for $\vec{q}_k^\top \vec{S}_\ell$ only for the chosen column at lower actual computational cost, even though the sum would superficially seem to have growing complexity with iteration j.

actually doing and the relevant math). However, the sped-up algorithms have been functionally identical to the original naive implementation of OMP.

To get further speedups, we have to be willing to potentially change the solutions that are found. The key insight is that in many problems, the various columns of A are approximately orthogonal to each other, of similar sizes, and there are many coefficients in $\vec{x}$ that are of the same relative size. This means that we can be more aggressive about selecting columns — we don't just have to take one maximizer in (1). We can add the column with the highest absolute correlation, as well as other columns whose absolute correlations are not that far away. When k is large, this lets us make many iterations worth of progress at the cost of a single iteration. Principled ways of deciding how many columns to add and to set thresholds require an understanding of probability and so are very much out of scope in 16B. However, you can play around with this to see what happens.

Contributors:

- Anant Sahai.
- Jennifer Shih.
- Rachel Hochman.
- Vasuki Narasimha Swamy.
- Steven Cao.

EECS 16B Designing Information Devices and Systems II

Note: Upper Triangulation

1 Motivation

When studying systems of linear differential equations, we have written them in the form

$$\frac{\mathrm{d}}{\mathrm{d}x}\vec{x} = A\vec{x},$$

where A is a matrix of scalar real coefficients, and $\vec{x}$ is the state vector. To solve such systems, we have developed a technique that involves diagonalizing A, solving for the state vector in the eigenbasis, and then making a change of basis back to the identity basis to obtain the full solution.

While the above technique is very effective, it relies on A being diagonalizable. Unfortunately, this assumption is not always true. For instance, consider

$$D = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Computing its characteristic polynomial, we find that

$$\det(D - \lambda I) = (\lambda - 1)^2,$$

so its only root is $\lambda = 1$. But

$$\text{Null}\,(D - \lambda I) = \text{Null}\left(\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}\right) = \text{span}\left\{\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right\},$$

which is only one dimensional. Thus, D only has one eigenvector despite being 2×2, and so is not diagonalizable. We call such matrices *defective*.

In this note, we will develop a new change of basis that has similar properties to diagonalization, but that works for *all* matrices, allowing us to solve arbitrary systems of differential equations!

2 Upper-Triangular Form

In particular, we will aim to show that any square matrix A can be transformed, by a change of basis, into the matrix

$$T = \begin{bmatrix} \lambda_1 & ? & ? & \cdots & ? \\ 0 & \lambda_2 & ? & \cdots & ? \\ 0 & 0 & \lambda_3 & \cdots & ? \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

where the λ_i are eigenvalues of T and T is in upper-triangular form. Notice that, since for defective matrices there are fewer than n distinct eigenvalues, here we will repeat each eigenvalue in the diagonal in accordance with its *multiplicity*. The multiplicity of an eigenvalue λ_i of a matrix A represents the number of times the linear factor $(\lambda - \lambda_i)$ appears in the characteristic polynomial of A. For instance, consider the defective matrix

$$D = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

whose characteristic polynomial was shown above to be

$$P_D(\lambda) = (\lambda - 1)^2.$$

Thus, we say that the eigenvalue $\lambda = 1$ of D has a multiplicity of 2 (even though the corresponding eigenspace of D is only one-dimensional).

But does this construction make sense? We need to fill n spots on the diagonal of T with eigenvalues repeated according to their multiplicity, so we need to ensure that the eigenvalues' multiplicities sum to n. The degree of the characteristic polynomial of an $n \times n$ matrix (such as A) is the highest power of λ in the expression $P_A(\lambda) = \det(A - \lambda I)$. This determinant will have one factor of λ for each entry in the diagonal of $A - \lambda I$, and since $A - \lambda I$ is $n \times n$, it will have n factors of λ. Thus the degree of $P_A(\lambda)$ is n. Therefore the sum of the multiplicities of all distinct eigenvalues of A, and in general any $n \times n$ matrix, will be n, so we can indeed produce the λ_i that we need for our desired form to make sense.

3 Computing Upper-Triangular Form

We wish to find a change of basis that converts an arbitrary $n \times n$ square matrix A into the form T. Let this change of basis be represented by the columns $\vec{v}_i$ of the matrix U, such that

$$A = UTU^{-1} = \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix} \begin{bmatrix} \lambda_1 & ? & ? & \cdots & ? \\ 0 & \lambda_2 & ? & \cdots & ? \\ 0 & 0 & \lambda_3 & \cdots & ? \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix}^{-1}.$$

Right-multiplying by U to get rid of the inverse, we obtain

$$A \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix} = \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix} \begin{bmatrix} \lambda_1 & ? & ? & \cdots & ? \\ 0 & \lambda_2 & ? & \cdots & ? \\ 0 & 0 & \lambda_3 & \cdots & ? \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda_n \end{bmatrix}.$$

Now, breaking this matrix down into a series of vector equations, we obtain the system

$$\begin{aligned}
A\vec{v}_1 &= \lambda_1\vec{v}_1 \\
A\vec{v}_2 &= (?)\vec{v}_1 + \lambda_2\vec{v}_2 \\
A\vec{v}_3 &= (?)\vec{v}_1 + (?)\vec{v}_2 + \lambda_3\vec{v}_3 \\
&\vdots \\
A\vec{v}_n &= (?)\vec{v}_1 + (?)\vec{v}_2 + \ldots + (?)\vec{v}_{n-1} + \lambda_n\vec{v}_n.
\end{aligned}$$

Note that we use the symbol ? to represent *different* unknown quantities each time it is written, not always the same value. Let's also temporarily forget that the λ_i are meant to be the eigenvalues of our system, and instead just treat them as arbitrary scalar coefficients.

Thus, we see that a change of basis that puts a matrix A into upper-triangular form is equivalent to constructing a basis $\{\vec{v}_i\}$ such that $A\vec{v}_i$ can be written as a linear combination of the vectors $\{\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_i\}$, for all i.

One of these equations should immediately stand out – specifically, $A\vec{v}_1 = \lambda_1\vec{v}_1$, since this is the equation defining $\vec{v}_1$ to be an eigenvector of A with eigenvalue λ_1. So a necessary condition to write any square matrix in upper-triangular form is that it must have at least one eigenvector, even if the matrix isn't diagonalizable. Since we are attempting to define our upper-triangularization procedure for every square matrix, we need to prove that every square matrix has at least one eigenvalue. Otherwise, we would not have the necessary condition for some matrices.

4 Existence of at least one eigenvector

Let's try to prove that any square matrix A has at least one eigenvector. Recall that we solve for eigenvalues and eigenvectors by considering the matrix $A - \lambda I$, and searching for eigenvalues λ that caused $A - \lambda I$ to have a nontrivial nullspace. To do so, we viewed the determinant $\det(A - \lambda I)$ as a polynomial $P_A(\lambda)$ in λ, and searched for its roots.

However, the Fundamental Theorem of Algebra tells us that every polynomial must have at least one distinct (possibly complex) root![1] Thus, we will obtain at least one eigenvalue λ such that $A - \lambda I$ has a nontrivial nullspace. By considering an element $\vec{v} \in \text{Null}(A - \lambda I)$, we obtain

$$\begin{aligned}
& (A - \lambda I)\vec{v} = \vec{0} \\
\implies \quad & A\vec{v} - \lambda I\vec{v} =, \vec{0} \\
\implies \quad & A\vec{v} - \lambda\vec{v} =, \vec{0} \\
\implies \quad & A\vec{v} = \lambda\vec{v},
\end{aligned}$$

so we have obtained an eigenvalue-eigenvector pair $(\lambda, \vec{v})$ for our matrix A, even if A were not diagonalizable.

[1] The Fundamental Theorem of Algebra actually tells us that the polynomial has n complex-valued roots, but crucially *some of them may be the same*. In particular they can *all* be the same, in which case we get one distinct eigenvalue.

5 Guessing a basis

So we know how to compute some $\vec{v}_1$ such that $A\vec{v}_1 = \lambda_1\vec{v}_1$. But what about the remaining $\vec{v}_i$ for $i \geq 2$? To determine these $\vec{v}_i$, we will make a guess. We will make a guess that the $\vec{v}_i$ not only form a basis, but in fact form an *orthonormal* basis - in other words, that $\vec{v}_i \perp \vec{v}_j$ for all $i \neq j$, and that $\|\vec{v}_i\| = 1$ for all i. Consider an arbitrary such orthonormal basis, starting with the known eigenvector $\vec{v}_1$ (normalized to be of magnitude 1) and constructing the remaining vectors using the Gram-Schmidt process. First, let's place the $n-1$ arbitrarily chosen vectors, which we will denote as $\vec{r}_1, \ldots, \vec{r}_{n-1}$ in a matrix R_{n-1} defined as

$$R_{n-1} = \begin{bmatrix} \vec{r}_1 & \vec{r}_2 & \cdots & \vec{r}_{n-1} \end{bmatrix},$$

so our full basis which turns A into an upper-triangular matrix will look like

$$U_n = \begin{bmatrix} \vec{v}_1 & R_{n-1} \end{bmatrix},$$

using block matrix notation. Notice that as the $\vec{r}_i$ are orthonormal, $R_{n-1}^\top R_{n-1} = I_{n-1}$, where I_k represents the k-dimensional identity matrix. And since $\vec{v}_1$ is orthogonal to each $\vec{r}_i$, the whole matrix U_n obeys $U_n^\top U_n = I_n$. This means that $U_n^{-1} = U_n^\top$, a fact we will use in just a bit.

Does this basis U turn A into an upper-triangular matrix? Let's find out, by computing the change of basis and using the fact we just derived:

$$\begin{aligned}
U_n^{-1}AU_n &= U_n^\top AU_n \\
&= \begin{bmatrix} \vec{v}_1 & R_{n-1} \end{bmatrix}^\top A \begin{bmatrix} \vec{v}_1 & R_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} \vec{v}_1 & R_{n-1} \end{bmatrix}^\top \begin{bmatrix} A\vec{v}_1 & AR_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} \vec{v}_1 & R_{n-1} \end{bmatrix}^\top \begin{bmatrix} \lambda_1\vec{v}_1 & AR_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} \vec{v}_1^\top \\ R_{n-1}^\top \end{bmatrix} \begin{bmatrix} \lambda_1\vec{v}_1 & AR_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} \vec{v}_1^\top(\lambda_1\vec{v}_1) & \vec{v}_1^\top(AR_{n-1}) \\ R_{n-1}^\top(\lambda_1\vec{v}_1) & R_{n-1}^\top(AR_{n-1}) \end{bmatrix} \\
&= \begin{bmatrix} \lambda_1\vec{v}_1^\top\vec{v}_1 & \vec{v}_1^\top AR_{n-1} \\ \lambda_1 R_{n-1}^\top\vec{v}_1 & R_{n-1}^\top AR_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} \lambda_1 & \vec{v}_1^\top AR_{n-1} \\ \lambda_1 R_{n-1}^\top\vec{v}_1 & R_{n-1}^\top AR_{n-1} \end{bmatrix}.
\end{aligned}$$

What does this matrix look like? Ideally, we'd like it to be upper-triangular, and so of the form

$$\begin{bmatrix} \lambda_1 & ? & ? & \cdots & ? \\ 0 & \lambda_2 & ? & \cdots & ? \\ 0 & 0 & \lambda_3 & \cdots & ? \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda_n \end{bmatrix}.$$

The top two blocks of $U_n^{-1}AU_n$ look alright, since the top row of an upper triangular matrix does not have to contain any zeros.

The bottom two blocks, however, might pose more of an issue. Specifically, comparing the two matrices above, for $U_n^{-1}AU_n$ to be upper triangular, $\lambda_1 R_{n-1}^\top \vec{v}_1 = \vec{0}$, and $R_{n-1}^\top A R_{n-1}$ must itself be an $n-1$-dimensional square upper triangular matrix.

Let's try to verify the first of our requirements. Recall that we chose the $\vec{r}_i$ to be orthonormal to $\vec{v}_1$, so $\vec{r}_i^\top \vec{v}_1 = 0$ for each i. So then

$$\lambda_1 R_{n-1}^\top \vec{v}_1 = \lambda_1 \begin{bmatrix} - & \vec{r}_1^\top & - \\ - & \vec{r}_2^\top & - \\ & \vdots & \\ - & \vec{r}_{n-1}^\top & - \end{bmatrix} \vec{v}_1 = \lambda_1 \begin{bmatrix} \vec{r}_1^\top \vec{v}_1 \\ \vec{r}_2^\top \vec{v}_1 \\ \vdots \\ \vec{r}_{n-1}^\top \vec{v}_1 \end{bmatrix} = \lambda_1 \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \vec{0},$$

as desired! Therefore, we can rewrite

$$U_n^{-1}AU_n = \begin{bmatrix} \lambda_1 & \vec{v}_1^\top A R_{n-1} \\ \vec{0} & R_{n-1}^\top A R_{n-1} \end{bmatrix}.$$

Unfortunately, recall that we chose the columns of R_{n-1} essentially arbitrarily, so long as together with $\vec{v}_1$ they formed an orthonormal basis for all of n-dimensional space. So there is no guarantee that $R_{n-1}^\top A R_{n-1}$ is upper-triangular, meaning that we aren't quite done yet.

Next, we'll learn how to make a *specific* choice of R_{n-1}, and thus U_n, so that we end up with an upper-triangular matrix. We'll study small-n cases, which will show us how the general case works.

6 Low-dimensional cases

The above approach doesn't always take us to an upper-triangular form, but it certainly takes us closer to one. When *does* it take us to upper-triangular form? Exactly when

$$R_{n-1}^\top A R_{n-1}$$

is itself upper-triangular. And in what case is that matrix upper-triangular? Well, if it's 1×1, then it is trivially upper-triangular, so we'd be done. In other words, we know that when $n = 2$, the above approach yields an upper-triangulation of the original matrix.

Let's take the time to work this out algebraically. Let M_2 be a 2×2 matrix. The above approach lets us construct an orthonormal basis

$$U_2 = \begin{bmatrix} \vec{v}_1 & R_1 \end{bmatrix}$$

such that

$$U_2^{-1} M_2 U_2 = \begin{bmatrix} \lambda_1 & \vec{v}_1^\top M_2 R_1 \\ \vec{0} & R_1^\top M_2 R_1 \end{bmatrix}.$$

But in the 2×2 case, R_1 is simply another unit vector, orthogonal to $\vec{v}_1$. Let this vector be $\vec{v}_2$, so

$$U_2 = \begin{bmatrix} \vec{v}_1 & \vec{v}_2 \end{bmatrix}.$$

Then we see that

$$U_2^{-1} M_2 U_2 = \begin{bmatrix} \lambda_1 & \vec{v}_1^\top M_2 \vec{v}_2 \\ \vec{0} & \vec{v}_2^\top M_2 \vec{v}_2 \end{bmatrix}.$$

Since all the components of the above matrix are in fact 1×1, we have expressed M_2 in upper-triangular form!

This step was fairly simple, on account of *all* 1×1 matrices being upper triangular. This stops being the case for 2×2 matrices. So, let's look at the next case: when $n = 3$. Consider some 3×3 matrix M_3, and construct an orthonormal basis

$$U_3 = \begin{bmatrix} \vec{v}_1 & R_2 \end{bmatrix}$$

such that

$$U_3^{-1} M_3 U_3 = \begin{bmatrix} \lambda_1 & \vec{v}_1^\top M_3 R_2 \\ \vec{0} & R_2^\top M_3 R_2 \end{bmatrix}.$$

As mentioned before, since $R_2^\top M_3 R_2$ is not necessarily upper-triangular, we run into a problem with our solution.

But wait! $R_2^\top M_3 R_2$ is a 2×2 matrix. And we just saw how to upper-triangularize arbitrary 2×2 matrices! Let

$$M_2 = R_2^\top M_3 R_2$$

and upper-triangularize it as $U_2^{-1} M_2 U_2$ for some orthogonal basis U_2.

Ideally, we'd be able to combine our "partial" upper-triangularization of M_3 with this complete upper-triangularization of M_2 in order to obtain an upper-triangularization of M_3 itself. Making substitutions, we might conjecture that an upper-triangularization might look something like

$$\begin{bmatrix} \lambda_1 & \vec{?}^\top \\ \vec{0} & U_2^{-1} M_2 U_2 \end{bmatrix} = \begin{bmatrix} \lambda_1 & \vec{?}^\top \\ \vec{0} & U_2^\top R_2^\top M_3 R_2 U_2 \end{bmatrix}.$$

Notice that we don't really care about the values of the elements above the diagonal, since they don't affect whether or not our result is upper-triangular, so we just denote them as $?$.

Can we construct a change of basis U_3, in terms of U_2, to write M_3 in the above form? Well, observe that the above form can be further rewritten as

$$\begin{bmatrix} \lambda_1 & \vec{?}^\top \\ \vec{0} & (R_2 U_2)^\top M_3 (R_2 U_2) \end{bmatrix}.$$

In other words, it looks very much like how U_3 acted on M_3, except with $R_2 U_2$ instead of just R_2. Thus, based on what the above U_3 looked like, we can conjecture that the alternative change of basis

$$U_3 = \begin{bmatrix} \vec{v}_1 & R_2 U_2 \end{bmatrix}$$

will rewrite A in upper-triangular form. Let's check this out and see if it works. Recall that we constructed

$$U_2 = \begin{bmatrix} \vec{v}_2 & \vec{v}_3 \end{bmatrix},$$

where $\vec{v}_2$ is an eigenvector of M_2 with eigenvalue λ_2 and $\vec{v}_3 \perp \vec{v}_2$. (Notice that we have incremented

subscript indices to avoid ambiguity.) Thus, our new U_3 looks like

$$U_3 = \begin{bmatrix} \vec{v}_1 & R_2 U_2 \end{bmatrix} = \begin{bmatrix} \vec{v}_1 & R_2\vec{v}_2 & R_2\vec{v}_3 \end{bmatrix}.$$

We wish to compute $U_3^{-1} M_3 U_3$ and verify that it is upper-triangular. To do so, we need to compute U_3^{-1}.

How did we do this when we considered that $U_3 = \begin{bmatrix} \vec{v}_1 & R_2 \end{bmatrix}$? Well, we looked at each column, and showed (albeit briefly) that the set of columns is orthonormal, by appealing to the Gram-Schmidt construction. Since any orthonormal matrix U obeys $U^\top U = I$, we obtained $U^\top = U^{-1}$. So we were able to say $U_3^{-1} = U_3^\top$.

Let's do the same thing here. Observe that the second and third columns of U_3, $R_2\vec{v}_2$ and $R_2\vec{v}_3$, both lie in the column space of R_2, which by construction is orthogonal to the first column $\vec{v}_1$. Furthermore, we see that the inner product of the second and third columns is

$$\begin{aligned} (R_2\vec{v}_2)^\top (R_2\vec{v}_3) &= \vec{v}_2^\top (R_2^\top R_2)\vec{v}_3 \\ &= \vec{v}_2^\top I_2 \vec{v}_2 \\ &= 0, \end{aligned}$$

relying on the fact that $R_2^\top R_2 = I_2$ as R_2 is orthonormal, and $\vec{v}_2^\top \vec{v}_3 = 0$ as the two vectors were constructed to be orthogonal. Thus, all the columns of U_3 are mutually orthogonal. To verify that they are of unit magnitude, we can simply compute their squared magnitude through inner products, where we see that

$$\begin{aligned} \vec{v}_1^\top \vec{v}_1 &= 1 \\ (R_2\vec{v}_2)^\top (R_2\vec{v}_2) = \vec{v}_2^\top R_2^\top R_2 \vec{v}_2 &= \vec{v}_2^\top \vec{v}_2 = 1 \\ (R_2\vec{v}_3)^\top (R_2\vec{v}_3) = \vec{v}_3^\top R_2^\top R_2 \vec{v}_3 &= \vec{v}_3^\top \vec{v}_3 = 1, \end{aligned}$$

since $\vec{v}_1$, $\vec{v}_2$, and $\vec{v}_3$ were all constructed to be of unit magnitude.

Therefore, we have shown that U_3 forms an orthogonal basis, so $U_3^{-1} = U_3^\top$. Thus, using similar techniques to what we did in the "partial" upper-triangularization, we can rewrite M_3 in this new basis as

$$\begin{aligned} U_3^{-1} M_3 U_3 &= U_3^\top M_3 U_3 \\ &= \begin{bmatrix} \vec{v}_1^\top \\ (R_2 U_2)^\top \end{bmatrix} M_3 \begin{bmatrix} \vec{v}_1 & R_2 U_2 \end{bmatrix} \\ &= \begin{bmatrix} \vec{v}_1^\top \\ (R_2 U_2)^\top \end{bmatrix} \begin{bmatrix} \lambda_1 \vec{v}_1 & M_3 R_2 U_2 \end{bmatrix} \\ &= \begin{bmatrix} \lambda_1 & \vec{?} \\ U_2^\top (R_2^\top \lambda_1 \vec{v}_1^\top) & U_2^\top R_2^\top M_3 R_2 U_2 \end{bmatrix} \\ &= \begin{bmatrix} \lambda_1 & \vec{?} \\ \vec{0} & U_2^\top M_2 U_2 \end{bmatrix} \\ &= \begin{bmatrix} \lambda_1 & ? & ? \\ 0 & \lambda_2 & ? \\ 0 & 0 & ? \end{bmatrix}, \end{aligned}$$

so we have successfully placed M_3 in upper-triangular form!

7 Induction

Let's take a quick step back. What have we done so far?

First, we developed a fairly intuitive change of basis that took an arbitrary $n \times n$ matrix to a "partial" upper-triangular form. Then, we saw that in the 2×2 case, this change of basis actually took our matrix to the final upper-triangular form. And now we've just seen that we can use our technique to upper-triangularize 2×2 matrices to upper-triangularize arbitrary 3×3 matrices as well.

What's next? Well, notice how our approach centers around resolving the case for $n \times n$ dimensions by solving the problem for $(n-1) \times (n-1)$ matrices. If we were writing a computer program to upper-triangularize matrices, we could have used recursion. To make sure our program is correct, we would like to make sure that, assuming we can upper-triangularize $(n-1) \times (n-1)$ matrices, we can upper-triangularize $n \times n$ matrices – that way, our recursive calls work as intended.

To formalize this, we will use a mathematical principle called *induction*, which will be covered far more extensively in other classes such as CS 70. Right now, we're just using it to make our recursion arguments mathematically precise.

The princiole of induction says in summary that if we have some statement $P(n)$, we only need to prove two things to prove $P(n)$ is true for all n:

- $P(1)$ is true, or more generally $P(n_0)$ is true for some n_0 (*base case*).
- *If* $P(n-1)$ is true (*inductive hypothesis*) *then* $P(n)$ is true (*inductive step*).

Then for every $n \geq 1$ (or $n \geq n_0$), $P(n)$ is true. Note that in the second bullet, we first *assume* $P(n-1)$ is true and then show that $P(n)$ is true; in particular, we don't say anything about when $P(n-1)$ is false.

This explanation was a bit abstract, so let's connect it to our problem. We consider $P(n)$ to be the statement "every $n \times n$ matrix can be upper-triangularized". Saying $P(n)$ is true for all n is like saying "every $n \times n$ matrix can be upper-triangularized for all n", or equivalently "every square matrix can be upper-triangularized". And we need to prove that $P(1)$ is true (which we did; we showed, *very* briefly, that all 1×1 matrices are upper-triangular). The last thing we need to show is that if $P(n-1)$ is true then $P(n)$ is true. More explicitly, given that we can upper-triangularize $(n-1) \times (n-1)$ dimensional matrices, we want to show that we can upper-triangularize $n \times n$ matrices.

Consider an arbitrary $n \times n$ matrix A. We know that we can produce an unit eigenvector of A $\vec{v}_1$ with eigenvalue λ_1 that, along with the columns of the matrix R_{n-1} (produced using Gram-Schmidt), form an orthogonal basis of n-dimensional space.

In analogy with our approach for the case of $n = 3$, we then consider the $(n-1) \times (n-1)$ matrix

$$M_{n-1} = R_{n-1}^\top A R_{n-1}$$

and, using our inductive hypothesis, produce an orthogonal matrix U_{n-1} such that

$$U_{n-1}^{-1} M_{n-1} U_{n-1}$$

is upper-triangular.

Then, we let

$$U_n = \begin{bmatrix} \vec{v}_1 & R_{n-1} U_{n-1} \end{bmatrix}.$$

Before, to compute U_3^{-1}, we showed that U_3 was orthogonal by considering each pair of its columns. This approach is not quite going to work, since we have n columns, not just 3. Instead, we show that $U_n^\top U_n = I_n$, which implies that $U_n^\top = U_n^{-1}$. This equation is what we originally wanted. This also implies that the columns of U_n are orthonormal, since the entries of $U_n^\top U_n$ (and in general any matrix) are the inner products of the respective columns of U_n.

This can be done as follows:

$$\begin{aligned} U_n^\top U_n &= \begin{bmatrix} \vec{v}_1^\top \\ (R_{n-1}U_{n-1})^\top \end{bmatrix} \begin{bmatrix} \vec{v}_1 & R_{n-1}U_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \vec{v}_1^\top \vec{v}_1 & \vec{v}_1^\top R_{n-1}U_{n-1} \\ U_{n-1}^\top R_{n-1}^\top \vec{v}_1 & U_{n-1}^\top R_{n-1}^\top R_{n-1}U_{n-1} \end{bmatrix}. \end{aligned}$$

Let's look at each of the components of the above matrix. Since it was chosen to be a unit vector, $\vec{v}_1^\top \vec{v}_1 = 1$. As R_{n-1} was constructed using Gram-Schmidt to have its columns be orthogonal to $\vec{v}_1$, we have that $\vec{v}_1^\top R_{n-1} = \vec{0}^\top$ and that $R_{n-1}^\top \vec{v}_1 = \vec{0}$.

Looking at the bottom-right term, we recall that R_{n-1} was constructed to be an orthogonal matrix, so $R_{n-1}^\top R_{n-1} = I_2$. Furthermore, U_{n-1} was constructed to be an orthogonal basis for M_{n-1}, so $U_{n-1}^\top U_{n-1} = I_{n-1}$. Thus,

$$U_{n-1}^\top (R_{n-1}^\top R_{n-1}) U_{n-1} = U_{n-1}^\top I_{n-1} U_{n-1} = U_{n-1}^\top U_{n-1} = I_{n-1},$$

canceling out the middle terms first. Putting all of this together, we see that

$$U_n^\top U_n = \begin{bmatrix} 1 & \vec{0}^\top \\ \vec{0} & I_{n-1} \end{bmatrix} = I_n,$$

so U_n is indeed orthogonal, as we expected.

Now that we have shown U_n is orthogonal, we can write $U_n^{-1} = U_n^\top$. Reexpressing A in this change of basis, we see that

$$\begin{aligned} U_n^{-1} A U_n &= U_n^\top A U_n \\ &= \begin{bmatrix} \vec{v}_1^\top \\ (R_{n-1}U_{n-1})^\top \end{bmatrix} A \begin{bmatrix} \vec{v}_1 & R_{n-1}U_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \vec{v}_1^\top \\ (R_{n-1}U_{n-1})^\top \end{bmatrix} \begin{bmatrix} A\vec{v}_1 & AR_{n-1}U_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \vec{v}_1^\top \\ (R_{n-1}U_{n-1})^\top \end{bmatrix} \begin{bmatrix} \lambda_1\vec{v}_1 & AR_{n-1}U_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \lambda_1 \vec{v}_1^\top \vec{v}_1 & \vec{?} \\ U_{n-1}^\top R_{n-1}^\top \lambda_1 \vec{v}_1 & U_{n-1}^\top R_{n-1}^\top A R_{n-1} U_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \lambda_1 & \vec{?} \\ \lambda_1 U_{n-1}^\top (R_{n-1}^\top \vec{v}_1) & U_{n-1}^\top R_{n-1}^\top A R_{n-1} U_{n-1} \end{bmatrix} \\ &= \begin{bmatrix} \lambda_1 & \vec{?} \\ \vec{0} & U_{n-1}^\top M_{n-1} U_{n-1} \end{bmatrix}. \end{aligned}$$

Since $U_{n-1}^\top M_{n-1} U_{n-1}$ is upper-triangular, so is $U_n^{-1} A U_n$, so we have successfully upper-triangularized an arbitrary $n \times n$ matrix A with an orthogonal change of basis by applying the inductive hypothesis. Thus we completed the inductive step.

By induction, we can therefore upper-triangularize *arbitrary* square matrices using orthogonal changes of basis, which is what we had aimed to prove! Awesome!

8 Schur Decomposition

There's are still a couple loose ends to clear up, however. Recall that we had initially hoped for the elements along the main diagonal of T, the upper-triangularization of A, to be the eigenvalues of A. But though our construction made λ_1 an eigenvalue, the remaining λ_i were eigenvalues of different matrices, and we have not yet seen whether they are also eigenvalues of A itself.

To see that that is in fact the case, recall that we have just shown that we can write

$$A = UTU^{-1} = \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix} \begin{bmatrix} \lambda_1 & ? & ? & \cdots & ? \\ 0 & \lambda_2 & ? & \cdots & ? \\ 0 & 0 & \lambda_3 & \cdots & ? \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix}^{-1},$$

where the λ_i are not necessarily the eigenvalues of A. Consider a particular λ_i. To show that it is an eigenvalue of A, we must show that $\det(A - \lambda_i I) = 0$. Recalling that $\det(AB) = \det(A)\det(B)$, we have that

$$\begin{aligned} \det(A - \lambda_i I) &= \det\Big(UTU^{-1} - \lambda_i UU^{-1}\Big) \\ &= \det\Big(U(T - \lambda_i I)U^{-1}\Big) \\ &= \det(U) \cdot \det(T - \lambda_i I) \cdot \det\Big(U^{-1}\Big). \end{aligned}$$

Let's look at each of the elements of this product individually. First, observe that

$$\det(U) \cdot \det\Big(U^{-1}\Big) = \det\Big(UU^{-1}\Big) = \det(I) = 1,$$

so we can cancel the determinant of U with the determinant of its inverse, to write

$$\det(A - \lambda_i I) = \det(T - \lambda_i I).$$

In other words, we have shown that the characteristic polynomial of A remained unchanged under a change of basis. Now, observe that

$$T - \lambda_i I = \begin{bmatrix} \lambda_1 - \lambda_i & ? & ? & \cdots & ? \\ 0 & \lambda_2 - \lambda_i & ? & \cdots & ? \\ 0 & 0 & \lambda_3 - \lambda_i & \cdots & ? \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda_n - \lambda_i \end{bmatrix}.$$

Thus, the ith pivot element of the above matrix must be 0, since it will equal $\lambda_i - \lambda_i = 0$. If we have an upper-triangular matrix with only $n-1$ nonzero pivots, then it has linearly dependent columns, so its determinant is zero. Thus,

$$\det(T - \lambda_i I) = 0 \implies \det(A - \lambda_i I) = 0,$$

so each λ_i is an eigenvalue of A, as expected! This way of representing A is known as the *Schur Decomposition* of A.

9 Complex Inner Products

There's one subtlety that we skipped over in the above proof. Specifically, we assumed throughout that the notions of orthogonality and inner products were defined on our vector spaces. But how do you take the inner product of two complex-valued vectors? If you try to reuse the definition of the dot product, you'll get some weird results that cause our understanding of these concepts to break down. For instance,

$$\left\| \begin{bmatrix} i \\ 1 \end{bmatrix} \right\|^2 = \begin{bmatrix} i \\ 1 \end{bmatrix} \cdot \begin{bmatrix} i \\ 1 \end{bmatrix} = -1 + 1 = 0,$$

which doesn't seem to make sense, since only the zero vector should have a norm of zero.

For now, we should assume that we are working in the vector space of reals $\mathbb{R}^n$ using the standard definition of the dot product. This, however, requires all the λ_i to be real, which may not always be the case. For now, we will make that assumption, though in future lectures we will see a small generalization of the dot product which will ensure our above result is true in all cases. In any case, the adjustment to our proof is minimal, given this more general notion of the inner product.

10 The Spectral Theorem

Finally, we will use our decomposition to obtain some interesting results about diagonalizing real, symmetric matrices.

First, we claim that the eigenvalues of a real, symmetric matrix are all themselves real. To prove it, let's consider a real, symmetric $n \times n$ matrix $A = A^\top$ and an eigenvalue λ of A. By the definition of eigenvectors, there exists some nonzero vector x such that

$$Ax = \lambda x.$$

To show λ is real, we'd like to get a result that looks like $\lambda = \overline{\lambda}$. So, striking out blindly, we can take the conjugate to get $\overline{\lambda}$ involved somehow, to obtain

$$A\bar{x} = \bar{\lambda}\bar{x}.$$

Notice that $A = \bar{A}$, since we assumed A was real. Now, let's try to take advantage of A's symmetric nature, by taking the transpose and using the fact that $A = A^\top$, to obtain

$$\bar{x}^\top A = \bar{x}^\top \bar{\lambda}.$$

At this point we can post-multiply both sides by x to obtain

$$\begin{aligned} & \bar{x}^\top Ax = \bar{\lambda}\bar{x}^\top x \\ \implies \quad & \lambda\bar{x}^\top x = \bar{\lambda}\bar{x}^\top x \\ \implies \quad & (\lambda - \bar{\lambda})\bar{x}^\top x = 0. \end{aligned}$$

So by basic arithmetic, either $\lambda = \bar{\lambda}$, or $\bar{x}^\top x = 0$. But since we chose x to be a nonzero vector, the former equality must be the one that is true, so λ is real, as desired.

Now, we will make a stronger claim. We assert that, not only are all the eigenvalues of A real, but that A can be *diagonalized*, meaning that it has n linearly independent eigenvectors. Furthermore, we claim that these eigenvectors can be chosen such that they are all orthogonal.

What is our goal? We wish to show that we can express

$$A = Q^\top \Lambda Q,$$

where Q is an orthogonal matrix whose columns are the eigenvectors of A, and Λ is a diagonal matrix containing the eigenvalues of A.

Notice that, since Λ is a diagonal matrix, it is also upper-triangular, with the elements along its diagonal clearly being the eigenvalues of A. Thus, our desired diagonalization of A is also a Schur decomposition of A.

So the first question to ask should be: can we construct a Schur decomposition of an arbitrary real, symmetric matrix A? The critical assumption needed when doing so was that all the λ_i were real produced during the induction, as otherwise our arguments related to orthogonality broke down. Let's look at our procedure and try to show that this is the case.

First, consider λ_1. λ_1 was chosen to be an eigenvalue of A. But since all the eigenvalues of A are real (since A is symmetric, using the result from above), we know that λ_1 is real. Great!

Next, let's look at λ_2. Looking at the induction, λ_2 was chosen to be an eigenvalue of $M_{n-1} = R_{n-1}^\top A R_{n-1}$, where R_{n-1} was an $n \times (n-1)$ orthogonal matrix constructed in a particular fashion. Right now, the exact construction of R_{n-1} isn't super important. What *is* important is that, as $A = A^\top$,

$$M_{n-1}^\top = (R_{n-1}^\top A R_{n-1})^\top = R_{n-1}^\top A^\top R_{n-1} = R_{n-1}^\top A R_{n-1} = M_{n-1},$$

so M_{n-1} is symmetric. And so, as λ_2 is an eigenvalue of the symmetric matrix M_{n-1}, it is itself real. This looks promising!

In a similar manner, λ_3 is an eigenvalue of $M_{n-2} = R_{n-2}^\top M_{n-1} R_{n-2}$, which is symmetric, so λ_3 is itself real. And this recursive argument can be continued in a similar fashion to show that *all* the λ_i are real! Awesome[2]!

Since all the λ_i are real, the induction involved in the Schur form proof works out, so we can write

$$A = UTU^\top,$$

where U is an orthogonal matrix and T is upper-triangular. Our goal is to show that $T = \Lambda$ - in other words, that T is in fact diagonal! Since we know that T is already upper-triangular, one way of doing this would be

[2] We could alternatively frame this argument using induction, like we did when deriving Schur form, if you're more comfortable with that. But both approaches are fundamentally equivalent and lead to the same result.

to show that

$$T = T^\top,$$

so the "upper" part of T consists entirely of zeros as well, so it is diagonal. How can we get $T^\top$ from our upper-triangular decomposition? We might as well just blindly take the transpose of the entire equation, just so we get the desired term *somewhere*. Doing so, we obtain

$$A^\top = UT^\top U^\top.$$

But since A is symmetric, $A = A^\top$, so we can write

$$A = UT^\top U^\top.$$

So we have shown that, when working in the basis of U, A becomes both T and $T^\top$? How can this be true? Clearly, the only way for this to be possible is if

$$T = T^\top,$$

as desired! Thus, we have shown that we can write

$$A = UTU^\top,$$

where T is a diagonal matrix made up of A's eigenvalues, and U is an orthogonal matrix. So we have diagonalized A! This completes the proof of what is known as the *real spectral theorem*.

Contributors:

- Druv Pai.
- Rahul Arya.
- Anant Sahai.

EECS 16B Designing Information Devices and Systems II

Note: Minimum Energy Control

1 Planning

Recall from our understanding of control that any n-dimensional discrete-time system with state equation

$$\vec{x}[i+1] = A\vec{x}[i] + B\vec{u}[i]$$

can be controlled to reach any desired state from any other in at most n time steps if and only if the controllability matrix

$$\mathscr{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}$$

is of full row rank. We will only consider the controllable case from now on.

For simplicity, let's consider the case of scalar control only, so B is in fact a column vector. Then, we know that, starting at an initial state $\vec{x}[0] = \vec{0}$, we can reach any target state $\vec{x}^\star$ in n time steps by applying the control

$$\vec{u} = \begin{bmatrix} u[n-1] \\ u[n-2] \\ \vdots \\ u[0] \end{bmatrix}$$

chosen such that

$$\vec{x}^\star = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix} \begin{bmatrix} u[n-1] \\ u[n-2] \\ \vdots \\ u[0] \end{bmatrix} = \mathscr{C}\vec{u}.$$

In the case of scalar control, $\mathscr{C}$ is an $n \times n$ matrix (of full rank, by our assumption of controllability) and so is invertible. Thus, we can uniquely choose our control inputs to be

$$\vec{u} = \mathscr{C}^{-1}\vec{x}^\star.$$

However, what if we didn't want to arrive at the state $\vec{x}^\star$ after n time steps, but only need to be there after some longer duration $t > n$? There should be many ways to do so, since we have so many more choices of input we can impose. In particular, notice that for the first $t - n$ steps, we can apply *any* controls we want, since the final n steps will always be sufficient to bring us to $\vec{x}^\star$ from wherever we might have ended up.

So are we done? Not quite. Imagine the linear system of our robot car from lab, and consider the problem of bringing it to a particular state (i.e. assigning particular values to the wheel velocities) at a certain time. One way (that is perhaps the most natural) would be to apply a steady input to gradually ramp up the wheel velocities, so we reach the target state at the desired time. Another way, however, could be to apply large random inputs, accelerating and decelerating each wheel, until just before the target time t, at which point

we would apply large controls to set the wheel velocities to their desired values. Though both approaches accomplish the same goal, the former should seem more "natural" than the latter.

More generally, in the case of arbitrary controllable linear systems, we will aim to choose a series of scalar inputs $\vec{u}$ that reach the target state while minimizing a cost function, which expresses how "unnatural" our inputs are.

Since in our robotic car context we are trying to minimize the "size" of the inputs, a plausible choice for this cost function would be the norm $\|\vec{u}\|$ of the inputs. In this context, the first case of applying steady inputs has a smaller norm than the large, randomly varying inputs of the second case, suggesting that this definition of the cost function is in accordance with our intuition. We will aim to compute the control input that reaches a target value while minimizing this quantity, known as the *minimum energy control*.

2 Minimum Energy Control

In the previous section, we focused on controllable systems with scalar inputs, where $\mathscr{C}$ was of full rank. Now, we will consider the more general problem of arbitrary n-dimensional systems with scalar inputs, with an initial state $\vec{x}[0] = \vec{0}$. We will denote

$$\mathscr{C}_t = \begin{bmatrix} B & AB & A^2B & \cdots & A^{t-1}B \end{bmatrix}$$

where A and B are the matrices in the state transition equation.

By expanding out the state transition matrix, we wish to choose the t-dimensional input vector $\vec{u}$ such that

$$\vec{x}[t] = \mathscr{C}_t \vec{u} = \vec{x}^\star.$$

We will assume that $\vec{x}^\star$ lies in the column space of $\mathscr{C}_t$, as otherwise the target state can never be reached, and so the problem is unsolvable. But since $\mathscr{C}_t$ may have a null space, it is possible for infinitely many solutions $\vec{u}$ to exist, and we wish to pick the solution with minimum norm.

Let's try to make sense of this problem by the geometric viewpoint. Let $\vec{u}_0$ be a particular solution to the above equation (that is not necessarily of minimum norm), and recall that the null space of $\mathscr{C}_t$ is a subspace of t-dimensional space. Furthermore, observe that for any vector $\vec{a} \in \text{Null}(\mathscr{C}_t)$, $\vec{u}_0 + \vec{a}$ is a solution to our equation, since

$$\mathscr{C}_t(\vec{u}_0 + \vec{a}) = \mathscr{C}_t\vec{u}_0 + \mathscr{C}_t\vec{a} = \mathscr{C}_t\vec{u}_0 = \vec{x}^\star.$$

Furthermore, from the same calculation, any possible solution $\vec{u}$ where $\mathscr{C}_t\vec{u} = \vec{x}^\star$ can be written in the form $\vec{u}_0 + \vec{a}$ for some $\vec{a} \in \text{Null}(\mathscr{C}_t)$. Thus, we can plot the space of possible $\vec{u}$ by first plotting all vectors in $\text{Null}(\mathscr{C}_t)$, and then translating them by $\vec{u}_0$.

Intuitively, we don't want to waste input "size" on parts of the control $\vec{u}$ which live in $\text{Null}(\mathscr{C}_t)$. Thus the control input $\vec{u}$ with minimum norm should be one that is entirely orthogonal to $\text{Null}(\mathscr{C}_t)$. This would seem to be analogous to what we saw when studying least squares, where again we were interested in choosing a point on a subspace such that the error vector was orthogonal to said subspace.

Let's try to prove this rigorously. Imagine that we had some orthonormal basis

$$\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_t$$

of our t-dimensional space of inputs, ordered such that the first n vectors $\vec{v}_1, \ldots, \vec{v}_n$ form a basis for those vectors orthogonal to $\text{Null}(\mathscr{C}_t)$, and the remaining $t - n$ vectors $\vec{v}_{n+1}, \ldots, \vec{v}_t$ form a basis for $\text{Null}(\mathscr{C}_t)$. We

will worry about constructing this basis explicitly later – for now, let's just assume that it exists. (It has to — we can get a basis for the nullspace using 16A techniques and then orthonormalize it using Gram-Schmidt. Then, we can extend it to a full basis for the space again using Gram-Schmidt.)

Consider our $\vec{u}_0$ expressed in this basis, as follows:

$$\vec{u}_0 = \alpha_1\vec{v}_1 + \alpha_2\vec{v}_2 + \cdots + \alpha_t\vec{v}_t.$$

Observe that the norm of $\vec{u}_0$ is

$$\begin{aligned}\|\vec{u}_0\| &= \sqrt{\vec{u}_0^\top \vec{u}_0} \\ &= \sqrt{\alpha_1^2 + \alpha_2^2 + \ldots + \alpha_t^2},\end{aligned}$$

since the $\vec{v}_i$ are orthonormal, so $\vec{v}_i^\top \vec{v}_j$ is 1 if $i = j$ and 0 otherwise.

Now, observe that in this basis, any $\vec{a} \in \text{Null}(\mathscr{C}_t)$ can be represented as

$$\vec{a} = \beta_{n+1}\vec{v}_{n+1} + \beta_{n+2}\vec{v}_{n+2} + \ldots + \beta_t\vec{v}_t,$$

since (by definition) it has components only in the the null space of $\mathscr{C}_t$. Thus, we can write any solution $\vec{u} = \vec{u}_0 + \vec{a}$ as

$$\vec{u} = \vec{u}_0 + \vec{a} = \alpha_1\vec{v}_1 + \alpha_2\vec{v}_2 + \ldots + \alpha_n\vec{v}_n + (\alpha_{n+1} + \beta_{n+1})\vec{v}_{n+1} + \ldots + (\alpha_t + \beta_t)\vec{v}_t.$$

In a similar manner to what we did before, we see immediately that the norm of this expression is

$$\|\vec{u}\| = \sqrt{\alpha_1^2 + \alpha_2^2 + \ldots + \alpha_n^2 + (\alpha_{n+1} + \beta_{n+1})^2 + \ldots + (\alpha_t + \beta_t)^2}.$$

To minimize the norm of $\vec{u}$, therefore, we should set $\beta_i = -\alpha_i$ for all valid $i > n$. Therefore, our minimum energy control must be

$$\begin{aligned}\vec{u} &= \vec{u}_0 + \vec{a} \\ &= \alpha_1\vec{v}_1 + \alpha_2\vec{v}_2 + \ldots + \alpha_n\vec{v}_n + (\alpha_{n+1} + \beta_{n+1})\vec{v}_{n+1} + \ldots + (\alpha_t + \beta_t)\vec{v}_t \\ &= \alpha_1\vec{v}_1 + \alpha_2\vec{v}_2 + \ldots + \alpha_n\vec{v}_n + (\alpha_{n+1} - \alpha_{n+1})\vec{v}_{n+1} + \ldots + (\alpha_t - \alpha_t)\vec{v}_t \\ &= \alpha_1\vec{v}_1 + \alpha_2\vec{v}_2 + \ldots + \alpha_n\vec{v}_n.\end{aligned}$$

Observe that this solution is entirely orthogonal to Null($\mathscr{C}_t$), as we had expected. All that remains now is to demonstrate the existence of the $\vec{v}_i$.

3 Constructing an Orthonormal Basis

In truth, this task is rather simple. From Gaussian elimination, we know how to compute a basis of Null($\mathscr{C}_t$), which we can orthonormalize using Gram-Schmidt to produce the $\vec{v}_{n+1}, \ldots, \vec{v}_t$. We can then extend this basis again using Gram-Schmidt to span all of t-dimensional space, to produce the $\vec{v}_1, \ldots, \vec{v}_n$, demonstrating the existence of our desired basis. Although this description is not fully precise, with enough effort we can make it rigorous and solve the problem of minimum energy control computationally. We don't get nice expressions, however, and so there is less insight to be had with this purely procedural approach.

Instead, we will choose to attack this problem from a different direction, which is not entirely unreasonable

and will prove useful in our later derivation of the SVD. By definition, the $\vec{v}_i$ are an orthonormal basis of t-dimensional space. But recall from earlier that, by the real spectral theorem, the eigenvectors of a real symmetric matrix can give such an orthonormal basis. Wouldn't it be interesting if some symmetric matrix Q had exactly these eigenvectors $\vec{v}_1, \ldots, \vec{v}_t$? To be extra useful, it would be nice if $\vec{v}_{n+1}, \ldots, \vec{v}_t$ are in fact all members of Null(Q) — in other words, it would be great if these were all the eigenvectors corresponding to $\lambda = 0$. That is, we want Q to have the same nullspace as $\mathscr{C}_t$.

How do we generate such a symmetric matrix Q? Since it has the same null space as $\mathscr{C}_t$, perhaps we could try writing it in the form

$$Q = D\mathscr{C}_t,$$

where D is some unknown matrix of full column rank. With this choice of D, we get that Q and $\mathscr{C}_t$ have the same null space. Now, as Q is symmetric, we can take transposes to obtain

$$Q = Q^\top = \mathscr{C}_t^\top D^\top \implies D\mathscr{C}_t = \mathscr{C}_t^\top D^\top.$$

Looking at the latter equality, a natural conjecture would be to try $D = \mathscr{C}_t^\top$, so $Q = \mathscr{C}_t^\top \mathscr{C}_t$. Let's see if this works out.

First, we show that Null$(\mathscr{C}_t) \subseteq$ Null(Q). Specifically, for any $\vec{v} \in$ Null$(\mathscr{C}_t)$, we'd like to show that $\vec{v} \in$ Null(Q). Indeed,

$$\mathscr{C}_t\vec{v} = \vec{0} \implies \mathscr{C}_t^\top\mathscr{C}_t\vec{v} = \mathscr{C}_t\vec{0} = \vec{0} \implies \vec{v} \in \text{Null}(Q).$$

Thus Null$(\mathscr{C}_t) \subseteq$ Null(Q).

Let's try to prove the opposite relation, that is Null$(\mathscr{C}_t) \supseteq$ Null(Q), in order to show equality. Specifically, for any $\vec{v} \in$ Null(Q), we'd like to show that $\vec{v} \in$ Null$(\mathscr{C}_t)$. Well, for any given such $\vec{v}$, by definition it is true that

$$Q\vec{v} = \vec{0} \implies \mathscr{C}_t^\top\mathscr{C}_t\vec{v} = \vec{0}.$$

We'd like to show that $\mathscr{C}_t\vec{v} = \vec{0}$, which is equivalent to writing $\|\mathscr{C}_t\vec{v}\| = 0$. By the definition of the norm of a real vector, we'd like to show that

$$(\mathscr{C}_t\vec{v})^\top(\mathscr{C}_t\vec{v}) = \vec{v}^\top\mathscr{C}_t^\top\mathscr{C}_t\vec{v} = 0.$$

But wait! This is basically what we had in the definition of $\vec{v}$, just pre-multiplied by $\vec{v}^\top$. Putting everything together into a proof, we have

$$\begin{aligned}
& \vec{v} \in \text{Null}(Q) \\
\implies \quad & Q\vec{v} = \vec{0} \\
\implies \quad & \mathscr{C}_t^\top\mathscr{C}_t\vec{v} = \vec{0} \\
\implies \quad & \vec{v}^\top\mathscr{C}_t^\top\mathscr{C}_t\vec{v} = 0 \\
\implies \quad & \|\mathscr{C}_t\vec{v}\| = 0 \\
\implies \quad & \mathscr{C}_t\vec{v} = \vec{0} \\
\implies \quad & \vec{v} \in \text{Null}(\mathscr{C}_t),
\end{aligned}$$

so Null$(Q) \subseteq$ Null$(\mathscr{C}_t)$, as we desired. Since we've proven this inequality in both directions, we have Null$(Q) =$ Null$(\mathscr{C}_t)$, as we had hoped.

Thus, we can produce an orthonormal basis of the eigenspace corresponding to $\lambda = 0$ of Q that provides us

with the $\vec{v}_{n+1}, \ldots, \vec{v}_t$ that we wanted.

Considering the remaining eigenvectors of Q for $\lambda \neq 0$, by the real spectral theorem they are all mutually orthogonal and will all be orthogonal to Null(Q). So we can choose n of them to form our $\vec{v}_1, \ldots, \vec{v}_n$, completing our construction of the $\{\vec{v}_i\}$.

4 Singular Values

Now that we know how to construct this Q, and have demonstrated that its eigenvectors are exactly the $\vec{v}_1, \ldots, \vec{v}_t$ that we had wanted, it is natural to speculate on the eigenvalues of each of these eigenvectors. We know that the eigenvalues of $\vec{v}_{n+1}, \ldots, \vec{v}_t$ are all 0, since they lie in the null space of Q. But what about the first n eigenvectors? Let's try to work this out algebraically.

By the definition of eigenvectors, we can perform manipulations very similar to what we did earlier involving nullspaces, to see that

$$\begin{aligned}
& & Q\vec{v}_i &= \lambda_i \vec{v}_i \\
\implies & & \mathscr{C}_t^\top \mathscr{C}_t \vec{v}_i &= \lambda_i \vec{v}_i \\
\implies & & \vec{v}_i^\top \mathscr{C}_t^\top \mathscr{C}_t &= \lambda_i \vec{v}_i^\top \\
\implies & & \vec{v}_i^\top \mathscr{C}_t^\top \mathscr{C}_t \vec{v}_i &= \lambda_i \vec{v}_i^\top \vec{v}_i \\
\implies & & \|\mathscr{C}_t \vec{v}_i\|^2 &= \lambda_i \|\vec{v}_i\|^2 \\
\implies & & \lambda_i &= \frac{\|\mathscr{C}_t \vec{v}_i\|^2}{\|\vec{v}_i\|^2} \\
& & &= \|\mathscr{C}_t \vec{v}_i\|^2.
\end{aligned}$$

In particular, notice that $\lambda_i \geq 0$. For $i > n$, we already knew that $\lambda_i = 0$. But for $i \leq n$, since $\vec{v}_i \notin$ Null$(\mathscr{C}_t)$, $\lambda_i \neq 0$, so $\lambda_i > 0$.

This sounds interesting. Without loss of generality, we can order our $\vec{v}_i$ such that

$$\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n > \lambda_{n+1} = \lambda_{n+2} = \ldots = \lambda_t = 0,$$

and place them as columns of the eigenvector matrix

$$V = \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_t \\ | & & | \end{bmatrix}.$$

We can then write the eigendecomposition of Q as

$$Q = V \Lambda V^{-1} = V \begin{bmatrix} \lambda_1 & 0 & \ldots & 0 \\ 0 & \lambda_2 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \lambda_t \end{bmatrix} V^\top.$$

Since all the $\lambda_i \geq 0$, we can choose

$$\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_n > \sigma_{n+1} = \sigma_{n+2} = \ldots = \sigma_t = 0$$

where $\lambda_i = \sigma_i^2$ for all i. Recall that $\lambda_i = \|\mathscr{C}_t\vec{v}_i\|^2$, so $\sigma_i = \|\mathscr{C}_t\vec{v}_i\|$. These σ_i are known as the *singular values* of $\mathscr{C}_t$, and will prove important in the next section.

5 Constructing Another Orthonormal Basis For the Output

Now, we can change coordinates of our input $\vec{u}$ to be in V-basis. If we do that, we get a new matrix for $\mathscr{C}_t$ that takes inputs in these new coordinates. Since $V^\top = V^{-1}$ by orthonormality, we know that $\mathscr{C}_t = \mathscr{C}_t V V^{-1} = (\mathscr{C}_t V)V^{-1}$. This means that if we consider $\widetilde{\vec{u}} = V^{-1}\vec{u}$ to be the input in V-basis, then $\widetilde{\mathscr{C}_t} = \mathscr{C}_t V$ would be the counterpart of $\mathscr{C}_t$. Since $\vec{v}_{n+1}, \ldots, \vec{v}_t$ are all in the nullspace of $\mathscr{C}_t$, we know that the last $t-n$ columns of $\widetilde{\mathscr{C}_t}$ are all $\vec{0}$. So what about the first n columns of $\widetilde{\mathscr{C}_t}$? These are $\mathscr{C}_t\vec{v}_i$ for $i = 1, \ldots, n$. What are these like? After all, this is the matrix that we would have to invert in order to find the minimum energy controls in this new basis.

For any valid choice of i,j, we see that

$$\begin{aligned}(\mathscr{C}_t\vec{v}_j)^\top(\mathscr{C}_t\vec{v}_i) &= \vec{v}_j^\top\mathscr{C}_t^\top\mathscr{C}_t\vec{v}_i \\ &= \sigma_i^2\vec{v}_j^\top\vec{v}_i = \begin{cases} 0 & \text{if } i \neq j \\ \sigma_i^2 & \text{if } i = j \end{cases}\end{aligned}$$

since $\vec{v}_i$ and $\vec{v}_j$ are orthonormal.

If the $\mathscr{C}_t\vec{v}_i$ are orthogonal, it'd be reasonable to try and use them to create an orthonormal basis of the column space of $\mathscr{C}_t$. We have n nonzero mutually orthogonal vectors corresponding to $i \leq n$, and the column space of $\mathscr{C}_t$ is n dimensional, so we're still OK! To make each of these vectors of unit length, we should scale them down by their length $\|\mathscr{C}_t\vec{v}_i\| = \sigma_i$, to obtain the orthonormal basis $\vec{w}_1, \vec{w}_2, \ldots, \vec{w}_n$, where we define

$$\vec{w}_i = \frac{\mathscr{C}_t\vec{v}_i}{\sigma_i}$$

for all valid $i \leq n$. Rearranging, we get

$$\mathscr{C}_t\vec{v}_i = \sigma_i\vec{w}_i.$$

Horizontally stacking this result for all $i \leq n$, we find that

$$\mathscr{C}_t\begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix} = \begin{bmatrix} | & & | \\ \vec{w}_1 & \cdots & \vec{w}_n \\ | & & | \end{bmatrix}\begin{bmatrix} \sigma_1 & 0 & \ldots & 0 \\ 0 & \sigma_2 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \sigma_n \end{bmatrix}.$$

Is anything missing? Well, recall that each of the $\vec{v}_i$ was t dimensional, but we only use n of them here, so the matrix stacking them horizontally is not square! To fix this, we can simply "pad" that matrix with the remaining $\vec{v}_i$ for $i > n$, and introduce additional zero columns to the diagonal matrix of the σ_i, to obtain

$$\mathscr{C}_t\begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_t \\ | & & | \end{bmatrix} = \begin{bmatrix} | & & | \\ \vec{w}_1 & \cdots & \vec{w}_n \\ | & & | \end{bmatrix}\begin{bmatrix} \sigma_1 & 0 & \ldots & 0 & 0 & \ldots & 0 \\ 0 & \sigma_2 & \ldots & 0 & 0 & \ldots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & \sigma_n & 0 & \ldots & 0 \end{bmatrix},$$

Now, since V is a square matrix with orthonormal columns, $V^{-1} = V^\top$, so we can rearrange to obtain

$$\mathscr{C}_t = \begin{bmatrix} | & & | \\ \vec{w}_1 & \cdots & \vec{w}_n \\ | & & | \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_n & 0 & \dots & 0 \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_t \\ | & & | \end{bmatrix}^\top = W\Sigma V^\top,$$

defining W and Σ to be the horizontally stacked $\vec{w}_i$ and the diagonal matrix of the σ_i respectively.

The above decomposition is called the Singular Value Decomposition (SVD) of $\mathscr{C}_t$. We will return to it in a moment.

6 Applications to Planning

The singular value decomposition of $\mathscr{C}_t$ can be interpreted as follows - the columns of V formed a t-dimensional orthonormal basis of the domain of $\mathscr{C}_t$, and the columns of W formed an n-dimensional orthonormal basis of its column space.

Moreover, they mapped between each other in a very clean manner, with $\mathscr{C}_t\vec{v}_i = \sigma_i\vec{w}_i$ for $i \leq n$, and $\mathscr{C}_t\vec{v}_i = \vec{0}$ for $i > n$. Let's see how we can use this property of the SVD can help us find the minimum cost control input.

Recall that we showed that our control vector $\vec{u}$ must have components only along $\vec{v}_1, \dots, \vec{v}_n$, in order to minimize its norm. Thus, for $\mathscr{C}_t\vec{u} = \vec{x}^\star$, we can use projections onto these two bases to see that

$$\begin{aligned} & \mathscr{C}_t\vec{u} = \vec{x}^\star \\ \implies \quad & \mathscr{C}_t\left(\sum_{i=1}^n \langle\vec{u}, \vec{v}_i\rangle\vec{v}_i\right) = \sum_{i=1}^n \langle\vec{x}^\star, \vec{w}_i\rangle\vec{w}_i \\ \implies \quad & \sum_{i=1}^n \langle\vec{u}, \vec{v}_i\rangle(\mathscr{C}_t\vec{v}_i) = \sum_{i=1}^n \langle\vec{x}^\star, \vec{w}_i\rangle\vec{w}_i \\ \implies \quad & \sum_{i=1}^n \sigma_i\langle\vec{u}, \vec{v}_i\rangle\vec{w}_i = \sum_{i=1}^n \langle\vec{x}^\star, \vec{w}_i\rangle\vec{w}_i, \end{aligned}$$

so

$$\sigma_i\langle\vec{u}, \vec{v}_i\rangle = \langle\vec{x}^\star, \vec{w}_i\rangle \implies \langle\vec{u}, \vec{v}_i\rangle = \frac{\langle\vec{x}^\star, \vec{w}_i\rangle}{\sigma_i}.$$

for all valid $i \neq n$. Substituting back into our expression for $\vec{u}$, recalling that it does not have any components in Null($\mathscr{C}_t$), we obtain

$$\vec{u} = \sum_{i=1}^n \frac{\langle\vec{x}^\star, \vec{w}_i\rangle}{\sigma_i}\vec{v}_i,$$

which is a clean expression for the minimum energy control to reach our desired state in terms of the SVD.

7 Outer Products

Now, we will look at a new interpretation of matrix multiplication, in order to construct an alternative way of writing the SVD in terms of what are known as *outer products*.

Recall that, for real vectors $\vec{x}$ and $\vec{y}$ expressed as columns with n components, their inner product is defined as $\vec{y}^\top \vec{x}$, which yields a 1×1 matrix typically treated as a scalar.

Similarly, we will define their *outer product* to be $\vec{x}\vec{y}^\top$. Let's see what this means. Let

$$\vec{x} = \begin{bmatrix} x_1 & x_2 & \cdots & x_m \end{bmatrix}^\top$$
$$\vec{y} = \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix}^\top,$$

where it is possible that $m \neq n$. Then, by the definition of matrix multiplication,

$$\vec{x}\vec{y}^\top = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix} = \begin{bmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_n \\ x_2y_1 & x_2y_2 & \cdots & x_2y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_my_1 & x_my_2 & \cdots & x_my_n \end{bmatrix}.$$

So while the inner product took two vectors of the same dimension and produced a scalar, the outer product takes two vectors of possibly different dimensions and yields a matrix!

Furthermore, notice that this matrix cannot be any arbitrary matrix - since each of its columns are a scalar multiple of $\vec{x}$, it cannot be of a rank greater than 1. It is straightforward to show that any matrix of rank 0 or 1 can be produced by an outer product of two vectors, but we will not discuss the details here.

Now, why are we interested in the outer product? Well, recall that we can express real matrix multiplication in terms of inner products. Specifically, we know that

$$\begin{bmatrix} - & \vec{x}_1^\top & - \\ & \cdots & \\ - & \vec{x}_m^\top & - \end{bmatrix} \begin{bmatrix} | & & | \\ \vec{y}_1 & \cdots & \vec{y}_n \\ | & & | \end{bmatrix} = \begin{bmatrix} \vec{x}_1^\top\vec{y}_1 & \vec{x}_1^\top\vec{y}_2 & \cdots & \vec{x}_1^\top\vec{y}_n \\ \vec{x}_2^\top\vec{y}_1 & \vec{x}_2^\top\vec{y}_2 & \cdots & \vec{x}_2^\top\vec{y}_n \\ \vdots & \vdots & \ddots & \vdots \\ \vec{x}_m^\top\vec{y}_1 & \vec{x}_m^\top\vec{y}_2 & \cdots & \vec{x}_m^\top\vec{y}_n \end{bmatrix},$$

where the element at the ith row and jth column of the product of two matrices X and Y is the dot product of the ith row of X and the jth column of Y.

However, what if we were interested in the columns of X and the rows of Y instead? As it turns out, it is the case that

$$\begin{bmatrix} | & & | \\ \vec{x}_1 & \cdots & \vec{x}_n \\ | & & | \end{bmatrix} \begin{bmatrix} - & \vec{y}_1^\top & - \\ & \cdots & \\ - & \vec{y}_n^\top & - \end{bmatrix} = \vec{x}_1\vec{y}_1^\top + \vec{x}_2\vec{y}_2^\top + \ldots + \vec{x}_n\vec{y}_n^\top.$$

This result can be proved by applying the definition of matrix multiplication, but it is tedious and so will be omitted.

Instead, we will look at an example that demonstrates the main ideas behind the proof. Consider the product

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

From our knowledge of the matrix product as the composition of dot products representing each element in

the result, we obtain

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix}.$$

We will not simplify this result, for reasons that will become clear in a moment. Now, calculating the product of these matrices using outer products, we obtain

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \begin{bmatrix} 5 & 6 \end{bmatrix} + \begin{bmatrix} 2 \\ 4 \end{bmatrix} \begin{bmatrix} 7 & 8 \end{bmatrix} = \begin{bmatrix} 1 \cdot 5 & 1 \cdot 6 \\ 3 \cdot 5 & 3 \cdot 6 \end{bmatrix} + \begin{bmatrix} 2 \cdot 7 & 2 \cdot 8 \\ 4 \cdot 7 & 4 \cdot 8 \end{bmatrix}.$$

Notice that the terms in our matrix multiplication evaluated using dot products correspond exactly to the terms in our sums of outer products, so our outer product definition of matrix multiplication is consistent with our previous definitions, at least in this example. A slightly more rigorous form of this argument can be used to prove this result in general, but it is best to understand this result at an intuitive level.

8 Outer Product Form of the SVD

Now, we will use this new interpretation of matrix multiplication as a sum of outer products in order to obtain an alternative way of expressing the SVD. Recall that the SVD of a matrix A represented it as the product

$$A = U\Sigma V^\top,$$

where Σ was a matrix with nonzero entries only along its main diagonal. Let A be an $m \times n$ matrix, so U is a square $m \times m$ matrix and $V^\top$ is a square $n \times n$ matrix. Additionally, without loss of generality, assume that $m \geq n$, so A is a "tall" matrix (the same results can be obtained if A is "wide" by considering $A^\top$, which would become "tall").

Let the columns of U be $\vec{u}_1$ through $\vec{u}_m$, the nonzero diagonal entries of Σ be σ_1 through σ_n (moving from the top-left to the bottom-right entry), and the rows of $V^\top$ be $\vec{v}_1^\top$ through $\vec{v}_n^\top$. From our understanding of outer products, we know how to express $UV^\top$ in terms of the $\vec{u}_i$ and $\vec{v}_i$. But how does Σ affect this interpretation?

Let's consider just the first two terms of the SVD - the product $U\Sigma$. By the outer product interpretation of

matrix multiplication, we have that

$$\begin{aligned} U\Sigma &= \begin{bmatrix} | & & | \\ \vec{u}_1 & \cdots & \vec{u}_m \\ | & & | \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & 0 \\ 0 & 0 & \cdots & \sigma_n \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix} \\ &= \vec{u}_1 \begin{bmatrix} \sigma_1 & 0 & \cdots 0 \end{bmatrix} + \vec{u}_2 \begin{bmatrix} 0 & \sigma_2 & \cdots & 0 \end{bmatrix} + \cdots + \vec{u}_n \begin{bmatrix} 0 & 0 & \cdots & \sigma_n \end{bmatrix} \\ &\quad + \vec{u}_{n+1} \begin{bmatrix} 0 & 0 & \cdots & 0 \end{bmatrix} + \ldots + \vec{u}_m \begin{bmatrix} 0 & 0 & \cdots & 0 \end{bmatrix} \\ &= \begin{bmatrix} | & | & & | \\ \sigma_1\vec{u}_1 & \sigma_2\vec{u}_2 & \cdots & \sigma_n\vec{u}_n \\ | & | & & | \end{bmatrix}, \end{aligned}$$

since the inner products involving the zero row vector yield the zero matrix, and so can be disregarded.

Notice what has happened here. The σ_i have acted as coefficients for the first n columns of $\vec{u}$, with the subsequent columns vanishing entirely. Now, we can multiply by V and apply the outer-product interpretation of matrix multiplication to obtain

$$\begin{aligned} A &= U\Sigma V^\top \\ &= \begin{bmatrix} | & | & & | \\ \sigma_1\vec{u}_1 & \sigma_2\vec{u}_2 & \cdots & \sigma_n\vec{u}_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} - & \vec{v}_1^\top & - \\ - & \vec{v}_2^\top & - \\ & \cdots & \\ - & \vec{v}_n^\top & - \end{bmatrix} \\ &= \sigma_1\vec{u}_1\vec{v}_1^\top + \sigma_2\vec{u}_2\vec{v}_2^\top + \cdots + \sigma_n\vec{u}_n\vec{v}_n^\top, \end{aligned}$$

so any $m \times n$ matrix A (where $m \geq n$) can be expressed as the weighted sum of n rank-1 matrices of the form $\vec{u}_i\vec{v}_i^\top$, where the set of $\vec{u}_i$ and $\vec{v}_i$ are each mutually orthonormal. This interpretation of the SVD is known as the *outer product form*.

Contributors:

- Druv Pai.
- Rahul Arya.
- Anant Sahai.

EECS 16B Designing Information Devices and Systems II
Note: SVD

In this note, we'd like to explore and collect the fundamental properties of the SVD, so that from now on we'll be able to use it in a variety of contexts.

The following exposition derives heavily from Prof. Arcak's EECS16B reader.

Throughout this note, let's suppose A is an $m \times n$ matrix, with $\text{rank}(A) = r$. Note that $r \leq \min\{m, n\}$.

1 SVD Form

The *full SVD* (or just *SVD*) of A is the following decomposition of A:

$$A = U\Sigma V^\top = \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} \begin{bmatrix} \Sigma_r & 0_{r\times(n-r)} \\ 0_{(m-r)\times r} & 0_{(m-r)\times(n-r)} \end{bmatrix} \begin{bmatrix} V_r^\top \\ V_{n-r}^\top \end{bmatrix} \tag{1}$$

$$= \underbrace{\left[\begin{array}{ccc|ccc} | & & | & | & & | \\ \vec{u}_1 & \cdots & \vec{u}_r & \vec{u}_{r+1} & \cdots & \vec{u}_m \\ | & & | & | & & | \end{array}\right]}_{m\times m} \underbrace{\left[\begin{array}{ccc|c} \sigma_1 & & & \\ & \ddots & & 0_{r\times(n-r)} \\ & & \sigma_r & \\ \hline 0_{(m-r)\times r} & & & 0_{(m-r)\times(n-r)} \end{array}\right]}_{m\times n} \underbrace{\left[\begin{array}{c} - \ \vec{v}_1^\top \ - \\ \vdots \\ - \ \vec{v}_r^\top \ - \\ \hline - \vec{v}_{r+1} - \\ \vdots \\ - \ \vec{v}_n^\top \ - \end{array}\right]}_{n\times n} \tag{2}$$

where U, V, and Σ are chosen such that

- U is an $m \times m$ matrix with orthonormal columns $\vec{u}_1, \ldots, \vec{u}_m$ that live in $\mathbb{R}^m$.
- V is an $n \times n$ matrix with orthonormal columns $\vec{v}_1, \ldots, \vec{v}_n$ that live in $\mathbb{R}^n$.
- Σ is an $m \times n$ matrix which has an $r \times r$ diagonal block Σ_r in the upper left, and 0 elsewhere.
- U_r is an $m \times r$ matrix with the first r orthonormal columns $\vec{u}_1, \ldots, \vec{u}_r$ of U.
- V_r is an $n \times r$ matrix with the first r orthonormal columns $\vec{v}_1, \ldots, \vec{v}_r$ of V.
- Σ_r is an $r \times r$ matrix with the largest r *singular values* $\sigma_1 \geq \cdots \geq \sigma_r > 0$ of A.[1]
- U_{m-r} is an $m \times (m-r)$ matrix with the last $m-r$ orthonormal columns $\vec{u}_{r+1}, \ldots, \vec{u}_m$ of U.
- V_{n-r} is an $n \times (n-r)$ matrix with the last $n-r$ orthonormal columns $\vec{v}_{r+1}, \ldots, \vec{v}_n$ of V.

[1] We also consider the singular values $\sigma_{r+1} = \cdots = \sigma_{\min\{m,n\}} = 0$, although they don't show up explicitly in the matrix. We can consider them as the singular values associated with $\vec{u}_{r+1}, \ldots, \vec{u}_{\min\{m,n\}}$ and $\vec{v}_{r+1}, \ldots, \vec{v}_{\min\{m,n\}}$, and thus $\sigma_{r+1}, \ldots, \sigma_{\min\{m,n\}}$ can be thought of as the $(r+1)^{\text{th}}, \ldots, \min\{m,n\}^{\text{th}}$ entries on the diagonal of Σ.

Our matrices U, V, Σ are carefully constructed to have particular linear-algebraic properties. Some of these are listed below.

- $\text{col}(U_r) = \text{span}\{\vec{u}_1, \ldots, \vec{u}_r\} = \text{col}(A)$.
- $\text{col}(U_{m-r}) = \text{span}\{\vec{u}_{r+1}, \ldots, \vec{u}_m\} \perp \text{col}(A)$[2].
- $\text{col}(V_r) = \text{span}\{\vec{v}_1, \ldots, \vec{v}_r\} \perp \text{null}(A)$.
- $\text{col}(V_{n-r}) = \text{span}\{\vec{v}_{r+1}, \ldots, \vec{v}_n\} = \text{null}(A)$.

We have already derived several of these properties using the construction in our previous note. For completeness, when we put together an algorithm for constructing the SVD, we will provide the proof that our algorithm supplies vectors with these properties.

2 Space Efficiency: Outer Product Form

This particular decomposition requires us to store m^2 numbers for U, mn numbers for Σ, and n^2 numbers for V. So we need to store $m^2 + mn + n^2$ numbers total. But a lot of these entries of Σ are 0s. This seems redundant. The fact that Σ is so structured means that there is probably a way to simplify this representation. Let's try to discover this way by trying to do the multiplication $A = U\Sigma V^\top$, and simplify if possible.

$$A = U\Sigma V^\top \tag{3}$$

$$= \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} \begin{bmatrix} \Sigma_r & 0_{r\times(n-r)} \\ 0_{(m-r)\times r} & 0_{(m-r)\times(n-r)} \end{bmatrix} \begin{bmatrix} V_r^\top \\ V_{n-r}^\top \end{bmatrix} \tag{4}$$

$$= \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} \left(\begin{bmatrix} \Sigma_r \\ 0_{(m-r)\times r} \end{bmatrix} V_r^\top + \begin{bmatrix} 0_{r\times(n-r)} \\ 0_{(m-r)\times(n-r)} \end{bmatrix} V_{n-r}^\top \right) \tag{5}$$

$$= \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} \begin{bmatrix} \Sigma_r \\ 0_{(m-r)\times r} \end{bmatrix} V_r^\top \tag{6}$$

$$= \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} \begin{bmatrix} \Sigma_r V_r^\top \\ 0_{(m-r)\times r} V_r^\top \end{bmatrix} \tag{7}$$

$$= \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} \begin{bmatrix} \Sigma_r V_r^\top \\ 0_{(m-r)\times n} \end{bmatrix} \tag{8}$$

$$= U_r \Sigma_r V_r^\top + U_{m-r} 0_{(m-r)\times n} \tag{9}$$

$$= U_r \Sigma_r V_r^\top. \tag{10}$$

This form of the SVD is the *compact* SVD, and while we are not going to work with it past this section (and it's out of scope for everything else in the class), it has its own useful properties (U_r and V_r have orthonormal columns[3], and Σ_r is square and invertible) and is less expensive to compute/store, especially when r is small.

[2] This notation may be unfamiliar. By saying that a subspace is $\perp$ (orthogonal to) another subspace, we mean every vector in the first subspace is orthogonal to every vector in the second subspace, or vice versa.

[3] Note that U_r is $m \times r$ and V_r is $n \times r$. They're generally not square matrices, so we can't say $U_r^{-1} = U_r^\top$, because U_r^{-1} and V_r^{-1} don't exist. But because they have orthonormal columns, we can say that $U_r^\top U_r = V_r^\top V_r = I_r$.

Motivated by the fact that Σ_r is diagonal and still has a lot of 0s and therefore a lot of redundancy that we would like to optimize out, we attempt to complete the multiplication. This time we look at the columns of each matrix.

$$A = U_r \Sigma_r V_r^\top \tag{11}$$

$$= \begin{bmatrix} \vec{u}_1 & \cdots & \vec{u}_r \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix} \begin{bmatrix} \vec{v}_1^\top \\ \vdots \\ \vec{v}_r^\top \end{bmatrix} \tag{12}$$

$$= \begin{bmatrix} \sigma_1\vec{u}_1 & \cdots & \sigma_r\vec{u}_r \end{bmatrix} \begin{bmatrix} \vec{v}_1^\top \\ \vdots \\ \vec{v}_r^\top \end{bmatrix} \tag{13}$$

$$= \sum_{i=1}^{r} \sigma_i \vec{u}_i \vec{v}_i^\top. \tag{14}$$

This is the *outer product form of the SVD*.

Let's look at how much storage this requires. For each $\vec{u}_i$, we require m numbers. For each $\vec{v}_i$, we require n numbers. And each σ_i is one number. We need r of each, so we need to store $r(m+n+1)$ numbers, a huge improvement from $m^2 + n^2 + mn$ numbers as before. This is especially true when r is small. In fact, by storing $\sigma_i\vec{u}_i$ and $\vec{v}_i$ separately as two sets of vectors, we could represent A using $r(m+n)$ entries, whereas before we needed mn entries! This is another huge improvement and benefit of the SVD.

3 An Algorithm for Computing the SVD

So how do we actually calculate the SVD? We will work with the $n \times n$ symmetric matrix $A^\top A$ or $m \times m$ symmetric matrix $AA^\top$[4]. The one that is smaller is preferable to work with, since it requires less computation and storage. We will give a justification for the case that we're working with $A^\top A$, but provide algorithms for both cases. The justification for the case of $AA^\top$ is left to discussion section.

By the spectral theorem for real symmetric matrices, both of these matrices have all real eigenvalues, r of which are positive and the remaining are 0. Thus we can do either of the following procedures:

Method 1 To Find The Full SVD:

(a) Find eigenvalues $\lambda_1, \ldots, \lambda_n$ of $A^\top A$ and order them such that $\lambda_1 \geq \cdots \geq \lambda_r > 0$ and $\lambda_{r+1} = \cdots = \lambda_n = 0$.

(b) Find orthonormal eigenvectors $\vec{v}_1, \ldots, \vec{v}_n$ such that

$$A^\top A \vec{v}_i = \lambda_i \vec{v}_i \qquad \text{for } i = 1, \ldots, n. \tag{15}$$

(c) Define $\sigma_i = \sqrt{\lambda_i}$ for $i = 1, \ldots, \min\{m, n\}$.

[4]To check that they're symmetric, take the transpose of each matrix, and observe that the matrix equals its transpose. For example, we can check that $A^\top A$ is symmetric:

$$(A^\top A)^\top = (A)^\top (A^\top)^\top = A^\top A.$$

You can try the case for $AA^\top$ yourself.

(d) Find orthonormal vectors $\vec{u}_1, \ldots, \vec{u}_m$, obtaining $\vec{u}_1, \ldots, \vec{u}_r$ by the equation

$$\vec{u}_i = \frac{A\vec{v}_i}{\sigma_i} \qquad \text{for } i = 1, \ldots, r \tag{16}$$

and finding $\vec{u}_{r+1}, \ldots, \vec{u}_m$ by Gram-Schmidt.

Method 2 To Find The Full SVD:

(a) Find eigenvalues $\lambda_1, \ldots, \lambda_m$ of $AA^\top$ and order them such that $\lambda_1 \geq \cdots \geq \lambda_r > 0$ and $\lambda_{r+1} = \cdots = \lambda_m = 0$.

(b) Find orthonormal eigenvectors $\vec{u}_1, \ldots, \vec{u}_m$ such that

$$AA^\top \vec{u}_i = \lambda_i \vec{u}_i \qquad \text{for } i = 1, \ldots, m. \tag{17}$$

(c) Define $\sigma_i = \sqrt{\lambda_i}$ for $i = 1, \ldots, \min\{m, n\}$.

(d) Find orthonormal vectors $\vec{v}_1, \ldots, \vec{v}_n$, obtaining $\vec{v}_1, \ldots, \vec{v}_r$ by the equation

$$\vec{v}_i = \frac{A^\top \vec{u}_i}{\sigma_i} \qquad \text{for } i = 1, \ldots, r \tag{18}$$

and finding $\vec{v}_{r+1}, \ldots, \vec{v}_n$ by Gram-Schmidt.

If we wanted to form the compact SVD, no problem! We could just skip finding the "extra" vectors $\vec{u}_{r+1}, \ldots, \vec{u}_m$ and $\vec{v}_{r+1}, \ldots, \vec{v}_n$, and it wouldn't affect the computation of $\vec{u}_1, \ldots, \vec{u}_r$ and $\vec{v}_1, \ldots, \vec{v}_r$ at all.

4 Proving That the Algorithm Correctly Calculates the Full SVD

We have some things to show about this construction (specifically Method 1). First, we need to show that if U, Σ, and V are defined as in eq. (1) and eq. (2) using the vectors provided by our algorithm, then A really does equal $U\Sigma V^\top$. This is necessary to prove because otherwise the decomposition is wrong from the start. After we show $A = U\Sigma V^\top$, we would like to prove the list of properties we listed on the first couple pages.

We begin by proving that $A = U\Sigma V^\top$. We start with the fact that V has orthogonal columns and is square, so $V^\top = V^{-1}$. Thus $V^\top V = VV^\top = I_n$, so

$$A = AVV^\top = A \begin{bmatrix} V_r & V_{n-r} \end{bmatrix} \begin{bmatrix} V_r^\top \\ V_{n-r}^\top \end{bmatrix} \tag{19}$$

$$= \begin{bmatrix} AV_r & AV_{n-r} \end{bmatrix} \begin{bmatrix} V_r^\top \\ V_{n-r}^\top \end{bmatrix}. \tag{20}$$

Notice that in the algorithm, we defined $\vec{u}_i$ and $\vec{v}_i$ in a very particular way. That is, for $i \leq r$, we defined $\vec{u}_i$ and $\vec{v}_i$ such that $A\vec{v}_i = \sigma_i \vec{u}_i$. Therefore

$$AV_r = A \begin{bmatrix} \vec{v}_1 & \cdots & \vec{v}_r \end{bmatrix} = \begin{bmatrix} A\vec{v}_1 & \cdots & A\vec{v}_r \end{bmatrix} = \begin{bmatrix} \sigma_1 \vec{u}_1 & \cdots & \sigma_r \vec{u}_r \end{bmatrix} = U_r \Sigma_r. \tag{21}$$

We sort of did this computation in reverse in eq. (13), so we are allowed to make this simplification.

Note also that for $i > r$, we know $\vec{v}_i$ is an eigenvector of $A^\top A$ with eigenvalue 0. Thus

$$A^\top A\vec{v}_i = 0\vec{v}_i = \vec{0}. \tag{22}$$

Left-multiplying by $\vec{v}_i^\top$, we get

$$\vec{v}_i^\top A^\top A\vec{v}_i = \vec{v}_i^\top \vec{0} = 0. \tag{23}$$

But this leftmost term can be simplified further! It is indeed the squared norm of $A\vec{v}_i$:

$$0 = \vec{v}_i^\top A^\top A\vec{v}_i = (A\vec{v}_i)^\top (A\vec{v}_i) = \|A\vec{v}_i\|^2. \tag{24}$$

Since the squared norm of $A\vec{v}_i$ is 0, the norm of $A\vec{v}_i$ is 0 (by taking square roots):

$$\|A\vec{v}_i\|^2 = 0 \implies \|A\vec{v}_i\| = 0. \tag{25}$$

But the norm of $A\vec{v}_i$ is the length of the vector $A\vec{v}_i$. And the only vector with length 0 is the zero vector $\vec{0}$, so we must have $A\vec{v}_i = \vec{0}$. Therefore

$$AV_{n-r} = A\begin{bmatrix}\vec{v}_{r+1} & \cdots & \vec{v}_n\end{bmatrix} = \begin{bmatrix}A\vec{v}_{r+1} & \cdots & A\vec{v}_n\end{bmatrix} = \begin{bmatrix}\vec{0} & \cdots & \vec{0}\end{bmatrix} = 0_{m\times(n-r)}. \tag{26}$$

Returning to our original computation in eq. (20), we use the results of the previous two calculations to get

$$A = \begin{bmatrix}AV_r & AV_{n-r}\end{bmatrix}\begin{bmatrix}V_r^\top \\ V_{n-r}^\top\end{bmatrix} \tag{27}$$

$$= \begin{bmatrix}U_r\Sigma_r & 0_{m\times(n-r)}\end{bmatrix}\begin{bmatrix}V_r^\top \\ V_{n-r}^\top\end{bmatrix} \tag{28}$$

$$= U_r\Sigma_r V_r^\top + 0_{m\times(n-r)}V_{n-r}^\top \tag{29}$$

$$= U_r\Sigma_r V_r^\top. \tag{30}$$

We have shown that A is equal to its *compact* SVD. To ensure that A is equal to its *full* SVD, we just need to show that the compact SVD is exactly equal to the full SVD. But, we already did this, in particular in eq. (3) through eq. (10)! There, we've shown that the algorithm given by Method 1 produces the correct SVD.

5 Proving Properties of the SVD

The only thing that's left to prove now is the laundry list of properties on the first and second pages, for the construction of the SVD defined by Method 1.

We will do the proofs in a different order than the facts were presented, but this is only to avoid getting any cyclic proofs.

- V is an $n \times n$ matrix with orthonormal columns $\vec{v}_1, \ldots, \vec{v}_n$ that live in $\mathbb{R}^n$.
  - We know that $A^\top A$ is an $n \times n$ real symmetric matrix. Thus by the spectral theorem for real symmetric matrices, which we discussed in the last note, $A^\top A$ has n real eigenvalues $\lambda_1, \ldots, \lambda_n$ and n orthonormal eigenvectors $\vec{v}_1, \ldots, \vec{v}_n$. Our construction sets those as the columns of V in eq. (15), so the columns of V are orthonormal, as we desired.
- U is an $m \times m$ matrix with orthonormal columns $\vec{u}_1, \ldots, \vec{u}_m$ that live in $\mathbb{R}^m$.

– We know from the construction of the algorithm that U is an $m \times m$ matrix and can be written as

$$U = \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} = \left[\begin{array}{ccc|ccc} \vec{u}_1 & \cdots & \vec{u}_r & \vec{u}_{r+1} & \cdots & \vec{u}_m \end{array}\right]. \tag{31}$$

To show that U has orthonormal columns, we must show that $U^\top U = I_m$. To do this, we can do the computation:

$$U^\top U = \begin{bmatrix} U_r^\top \\ U_{m-r}^\top \end{bmatrix} \begin{bmatrix} U_r & U_{m-r} \end{bmatrix} = \begin{bmatrix} U_r^\top U_r & U_r^\top U_{m-r} \\ U_{m-r}^\top U_r & U_{m-r}^\top U_{m-r} \end{bmatrix}. \tag{32}$$

Since U_{m-r} was created by Gram-Schmidt to have $m-r$ orthonormal columns $\vec{u}_{r+1}, \ldots, \vec{u}_m$, each of which is orthogonal with the columns $\vec{u}_1, \ldots, \vec{u}_r$ of U_r, we know that

$$U_r^\top U_{m-r} = \begin{bmatrix} \vec{u}_1^\top \\ \vdots \\ \vec{u}_r^\top \end{bmatrix} \begin{bmatrix} \vec{u}_{r+1} & \cdots & \vec{u}_m \end{bmatrix} = \begin{bmatrix} \vec{u}_1^\top \vec{u}_{r+1} & \cdots & \vec{u}_1^\top \vec{u}_m \\ \vdots & \ddots & \vdots \\ \vec{u}_r^\top \vec{u}_{r+1} & \cdots & \vec{u}_r^\top \vec{u}_m \end{bmatrix} = \begin{bmatrix} 0 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 0 \end{bmatrix} = 0_{r \times (m-r)}. \tag{33}$$

By adapting this calculation, we can also show that

$$U_{m-r}^\top U_r = 0_{(m-r)\times r} \qquad \text{and} \qquad U_{m-r}^\top U_{m-r} = I_{m-r}.$$

The last quantity to compute is $U_r^\top U_r$, and the calculation is slightly different. We can again go column-by-column:

$$U_r^\top U_r = \begin{bmatrix} \vec{u}_1^\top \\ \vdots \\ \vec{u}_r^\top \end{bmatrix} \begin{bmatrix} \vec{u}_1 & \cdots & \vec{u}_r \end{bmatrix} = \begin{bmatrix} \vec{u}_1^\top \vec{u}_1 & \cdots & \vec{u}_r^\top \vec{u}_1 \\ \vdots & \ddots & \vdots \\ \vec{u}_1^\top \vec{u}_r & \cdots & \vec{u}_r^\top \vec{u}_r \end{bmatrix}. \tag{34}$$

This time, we haven't computed any vectors by Gram-Schmidt, so we can't say that everything is immediately zero. Instead, what we can do is use our construction for $\vec{u}_i$, introduced in eq. (16), i.e., $\vec{u}_i = \frac{A\vec{v}_i}{\sigma_i}$ for $i \leq r$. Then we can take the inner product of any two (not necessarily different) $\vec{u}_i$ to get

$$\vec{u}_i^\top \vec{u}_j = \left(\frac{A\vec{v}_i}{\sigma_i}\right)^\top \left(\frac{A\vec{v}_j}{\sigma_j}\right) \tag{35}$$

$$= \frac{\vec{v}_i^\top A^\top A \vec{v}_j}{\sigma_i \sigma_j}. \tag{36}$$

At this point we recall our definition of $\vec{v}_j$ as an eigenvector of $A^\top A$ with eigenvalue λ_j, so we

can write

$$\vec{u}_i^\top \vec{u}_j = \frac{\vec{v}_i^\top A^\top A \vec{v}_j}{\sigma_i \sigma_j} \tag{37}$$

$$= \frac{\vec{v}_i^\top \lambda_j \vec{v}_j}{\sigma_i \sigma_j} \tag{38}$$

$$= \frac{\lambda_j}{\sigma_i \sigma_j} \vec{v}_i^\top \vec{v}_j. \tag{39}$$

Here we know $\vec{v}_i$ and $\vec{v}_j$ are columns of V. But we have just proven that the columns of V are orthonormal! So we already know

$$\vec{v}_i^\top \vec{v}_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}. \tag{40}$$

Putting it all together,

$$\vec{u}_i^\top \vec{u}_j = \frac{\lambda_j}{\sigma_i \sigma_j} \vec{v}_i^\top \vec{v}_j = \frac{\lambda_j}{\sigma_i \sigma_j} \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} = \frac{\lambda_i}{\sigma_i^2} \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \tag{41}$$

where in the last equality we use step (c) of Method 1 to show that $\sigma_i^2 = \lambda_i$. Thus

$$U_r^\top U_r = \begin{bmatrix} \vec{u}_1^\top \vec{u}_1 & \cdots & \vec{u}_r^\top \vec{u}_1 \\ \vdots & \ddots & \vdots \\ \vec{u}_1^\top \vec{u}_r & \cdots & \vec{u}_r^\top \vec{u}_r \end{bmatrix} = \begin{bmatrix} 1 & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & 1 \end{bmatrix} = I_r. \tag{42}$$

Given everything we have found out, we can start again from eq. (32) and get

$$U^\top U = \begin{bmatrix} U_r^\top U_r & U_r^\top U_{m-r} \\ U_{m-r}^\top U_r & U_{m-r}^\top U_{m-r} \end{bmatrix} \tag{43}$$

$$= \begin{bmatrix} I_r & 0_{r \times (m-r)} \\ 0_{(m-r) \times r} & I_{m-r} \end{bmatrix} \tag{44}$$

$$= I_m. \tag{45}$$

Thus the columns of U are orthonormal, as we wanted to show.

- Σ is an $m \times n$ matrix which has an $r \times r$ diagonal block Σ_r in the upper left, and 0 elsewhere. Σ_r is an $r \times r$ matrix with the largest r *singular values* $\sigma_1 \geq \cdots \geq \sigma_r > 0$ of A.

  – Since we get the singular values and lay it out in the prescribed form during our construction, we just need to show that we actually get exactly r positive singular values. We know from the spectral theorem for real symmetric matrices that $A^\top A$ has n real eigenvalues, and r nonzero eigenvalues.

  We now claim that these nonzero eigenvalues are all positive. In fact, let $\vec{v}$ be an eigenvector of $A^\top A$ with eigenvalue λ. Then

  $$A^\top A \vec{v} = \lambda \vec{v}. \tag{46}$$

Left-multiplying by $\vec{v}^\top$,we get

$$\vec{v}^\top A^\top A\vec{v} = \lambda \vec{v}^\top \vec{v}. \tag{47}$$

But the left-hand side can be simplified:

$$\vec{v}^\top A^\top A\vec{v} = (A\vec{v})^\top (A\vec{v}) = \|A\vec{v}\|^2. \tag{48}$$

So can the right-hand side:

$$\lambda \vec{v}^\top \vec{v} = \lambda \|\vec{v}\|^2. \tag{49}$$

Thus

$$\|A\vec{v}\|^2 = \lambda \|\vec{v}\|^2. \tag{50}$$

We know that both $\|A\vec{v}\|^2$ and $\|\vec{v}\|^2$ are squared terms, so they are both ≥ 0. Thus $\lambda \geq 0$ as well. This is true for any arbitrary eigenvalue of $A^\top A$, so every eigenvalue of $A^\top A$ is non-negative. Thus if an eigenvalue is nonzero then it must be positive. Finally, in step (c) of Method 1, we set $\sigma_i = \sqrt{\lambda_i}$ for all eigenvalues λ_i, so there are r positive σ_i as well.

- U_r, V_r, U_{m-r}, V_{n-r} are sub-matrices of U and V with orthonormal columns.
  - This sub-matrix breakdown is just how we construct U and V. The orthonormality comes from the fact that the columns of U and V are orthonormal, so any subset of them will also be orthonormal. This includes the subsets of columns that become the columns of U_r, etc.
- $\text{col}(V_{n-r}) = \text{null}(A)$.
  - We first want to show that $\text{col}(V_{n-r}) \subseteq \text{null}(A)$, then $\text{null}(A) \subseteq \text{col}(V_{n-r})$. This implies that $\text{col}(V_{n-r}) = \text{null}(A)$.
    First, take any $\vec{v} \in \text{col}(V_{n-r})$. We want to show that $\vec{v} \in \text{null}(A)$. Let $\vec{v} = V_{n-r}\vec{w}$; $\vec{w}$ exists by the definition of $\text{col}(V_{n-r})$. We want to show that $\vec{v} \in \text{null}(A)$, so the natural thing to do is to multiply by A:

$$\begin{aligned} A\vec{v} &= AV_{n-r}\vec{w} && (51)\\ &= A\begin{bmatrix}\vec{v}_{r+1} & \cdots & \vec{v}_n\end{bmatrix}\vec{w} && (52)\\ &= \begin{bmatrix}A\vec{v}_{r+1} & \cdots & A\vec{v}_n\end{bmatrix}\vec{w}. && (53)\end{aligned}$$

At this point we would like to stop and consider what each of these columns are. We know that since $\vec{v}_{r+1}, \ldots, \vec{v}_n$ are eigenvectors of $A^\top A$ with eigenvalue 0,

$$A^\top A\vec{v}_{r+1} = \cdots = A^\top A\vec{v}_n = \vec{0}. \tag{54}$$

Therefore $\vec{v}_{r+1}, \ldots, \vec{v}_n \in \text{null}(A^\top A)$. But from EECS16A Note 23, we know that for any matrix A, we have

$$\text{null}(A) = \text{null}(A^\top A) \tag{55}$$

Thus $\vec{v}_{r+1}, \ldots, \vec{v}_n \in \text{null}(A)$, and so

$$\begin{aligned} A\vec{v} &= \begin{bmatrix}A\vec{v}_{r+1} & \cdots & A\vec{v}_n\end{bmatrix}\vec{w} && (56)\\ &= \begin{bmatrix}\vec{0} & \cdots & \vec{0}\end{bmatrix}\vec{w} && (57)\\ &= \vec{0}. && (58)\end{aligned}$$

Thus $v \in \text{null}(A)$. Since v is an arbitrary vector in $\text{col}(V_{n-r})$,

$$\text{col}(V_{n-r}) \subseteq \text{null}(A). \tag{59}$$

Now we want to try the other direction. Take any $\vec{v} \in \text{null}(A)$. We want to show that $\vec{v} \in \text{col}(V_{n-r})$. Since $\vec{v} \in \text{null}(A)$,

$$A\vec{v} = \vec{0} \tag{60}$$

Left-multiplying by $A^\top$,

$$A^\top A\vec{v} = A^\top \vec{0} = \vec{0} = 0\vec{v}. \tag{61}$$

Thus $\vec{v}$ is an eigenvector of $A^\top A$ with eigenvalue 0, so it's contained in the span of the eigenvectors of $A^\top A$ associated with the eigenvalue 0. But a basis for these eigenvectors is exactly $\vec{v}_{r+1}, \ldots, \vec{v}_n$, and so this span is $\text{col}(V_{n-r})$. Thus $v \in \text{col}(V_{n-r})$. Since v is an arbitrary vector in $\text{null}(A)$,

$$\text{null}(A) \subseteq \text{col}(V_{n-r}). \tag{62}$$

Thus by eq. (59) and eq. (62),

$$\text{null}(A) = \text{col}(V_{n-r}), \tag{63}$$

which is what we wanted to show.

- $\text{col}(V_r) \perp \text{null}(A)$.
  - Since V has orthonormal columns, we know that every vector in $\vec{v}_1, \ldots, \vec{v}_r$ is orthogonal to every vector in $\vec{v}_{r+1}, \ldots, \vec{v}_n$. Thus each of $\vec{v}_1, \ldots, \vec{v}_r$ is orthogonal to $\text{span}(\vec{v}_{r+1}, \ldots, \vec{v}_n) = \text{col}(V_{n-r})$. Thus any linear combination of $\vec{v}_1, \ldots, \vec{v}_r$ is too, so

    $$\text{span}(\vec{v}_1, \ldots, \vec{v}_r) \perp \text{col}(V_{n-r}) \tag{64}$$

    But the first term is just $\text{col}(V_r)$ by definition, and we just proved that $\text{col}(V_{n-r}) = \text{null}(A)$. So

    $$\text{col}(V_r) \perp \text{null}(A). \tag{65}$$

- $\text{col}(U_r) = \text{col}(A)$.
  - Remember that the columns of U_r are $\vec{u}_1, \ldots, \vec{u}_r$. From eq. (32) we obtain $\vec{u}_i = \frac{A\vec{v}_i}{\sigma_i}$, which is proportional to $A\vec{v}_i$.

    We already showed that $\vec{u}_1, \ldots, \vec{u}_m$ are orthonormal and hence $\vec{u}_1, \ldots, \vec{u}_r$ are orthonormal. This takes care of the proportionality, so all we need to show is that $\text{span}(A\vec{v}_1, \ldots, A\vec{v}_r) = \text{col}(A)$. But $A\vec{v}_1, \ldots, A\vec{v}_r$ are exactly the columns of AV_r, so the left hand side is $\text{col}(AV_r)$.

    First, we want to show $\text{col}(A) \subseteq \text{col}(AV_r)$. Let $\vec{v} \in \text{col}(A)$; we want to show that $\vec{v} \in \text{col}(AV_r)$. Indeed, let $\vec{v} = A\vec{w}$; this exists by the definition of $\text{col}(A)$. Then since $\vec{v}_1, \ldots, \vec{v}_n$ is an orthonormal basis for $\mathbb{R}^n$ and also the columns of V, there is a $\vec{y}$ such that $\vec{w} = V\vec{y}$. Then

    $$\vec{v} = A\vec{w} = AV\vec{y}. \tag{66}$$

    Writing V in terms of V_r and V_{n-r},

    $$\vec{v} = AV\vec{y} = A\begin{bmatrix} V_r & V_{n-r} \end{bmatrix}\vec{y} = \begin{bmatrix} AV_r & AV_{n-r} \end{bmatrix}\vec{y}. \tag{67}$$

But we already proved that

$$\text{col}(V_{n-r}) = \text{null}(A) \tag{68}$$

so

$$AV_{n-r} = 0_{m\times(n-r)}. \tag{69}$$

Thus

$$\vec{v} = \begin{bmatrix} AV_r & AV_{n-r} \end{bmatrix} \vec{y} = \begin{bmatrix} AV_r & 0_{m\times(n-r)} \end{bmatrix} \vec{y}. \tag{70}$$

Thus $\vec{v}$ is a linear combination of the columns of AV_r, so $\vec{v} \in \text{col}(AV_r)$. Since this is true for arbitrary $\vec{v}$,

$$\text{col}(A) \subseteq \text{col}(AV_r). \tag{71}$$

The reverse direction is easier. Take $\vec{v} \in \text{col}(AV_r)$. Then there is $\vec{w}$ such that

$$\vec{v} = AV_r\vec{w} = A(V_r\vec{w}). \tag{72}$$

So in the end $\vec{v}$ is a linear combination of columns of A, hence $\vec{v} \in \text{col}(A)$. Since this is true for arbitrary $\vec{v}$,

$$\text{col}(AV_r) \subseteq \text{col}(A). \tag{73}$$

Hence by eq. (71) and eq. (73),

$$\text{col}(AV_r) = \text{col}(A). \tag{74}$$

To wrap up the proof, recall that we showed at the beginning that the columns of AV_r, i.e., the vectors $A\vec{v}_1, \ldots, A\vec{v}_r$ are scaled versions of $\vec{u}_1, \ldots, \vec{u}_r$. Thus they span the same set, which is $\text{col}(A)$, as we desired.

- $\text{col}(U_{m-r}) \perp \text{col}(A)$.
  - Since U has orthonormal columns, every vector in $\vec{u}_1, \ldots, \vec{u}_r$ is orthogonal to every vector in $\vec{u}_{r+1}, \ldots, \vec{u}_m$. Thus each of $\vec{u}_{r+1}, \ldots, \vec{u}_m$ is orthogonal to $\text{span}(\vec{u}_1, \ldots, \vec{u}_r) = \text{col}(U_r)$. Thus any linear combination of $\vec{u}_{r+1}, \ldots, \vec{u}_m$ is too, so

    $$\text{span}(\vec{u}_{r+1}, \ldots, \vec{u}_m) \perp \text{col}(U_r). \tag{75}$$

    But the first term is just $\text{col}(U_{m-r})$ by definition, and we just proved that $\text{col}(U_r) = \text{col}(A)$. So

    $$\text{col}(U_{m-r}) \perp \text{col}(A) \tag{76}$$

    as desired.

We proved all the properties we wanted to prove, for Method 1 of finding the SVD. The corresponding properties for Method 2 can be similarly verified, as we will see in discussion.

Contributors:

- Druv Pai.
- Gireeja Ranade.

EECS 16B Designing Information Devices and Systems II
Note: PCA

Overview

In this note, we will examine a key application of the singular value decomposition (SVD), known as *principal component analysis* (PCA). We will explore its role in understanding large datasets with many dimensions, where PCA lets us determine the most important k-dimensional subspace for this data. To verify that the PCA approach is reasonable, we prove results involving *low-rank approximations* to matrices.

The key new idea here is that of approximation. When we collect data, there are potentially lots of real things that are influencing the exact values that we see. Our goal is often to figure out the most important of these. This allows us to compress or distill our data down to its important essence. How can we do this using the linear-algebraic tools that we have built?

1 Analyzing Matrix-type Data

Imagine we are conducting some sort of statistical study — say, we are interested in how a group of n students enjoy a set of m movies. Each student gives a numerical rating of each movie, indicating how much they enjoyed it. We place these ratings into a matrix R, where R_{ij} is the rating the i^{th} student gives to the j^{th} movie. Our goal will be to draw conclusions from this dataset that enable us to make future predictions.

For instance, if we are told how much a new student enjoys a subset of our movies, we should be able to predict how much they will enjoy the other movies in our set. Alternatively, if we produce a new movie and are told how much a subset of our students enjoy it, we should be able to predict how much the other students in our study will enjoy it.

How on earth can we come up with these predictions? Aren't we all *unique and special*, so knowing how a student enjoys some movies provides no information on how they will enjoy the rest? In such a case, then yes, nothing could be done and our problem would be unsolvable.

However, in practice, it turns out that students are not as completely unique and special as all that! Typically, there is an underlying structure that lets us predict how much a student will enjoy each movie. For instance, imagine that every j^{th} film has a certain intrinsic "goodness" g_j, and every i^{th} student has a "sensitivity" s_i. Then the i^{th} student's rating of the j^{th} movie can be computed as the product of the student's sensitivity and the film's goodness, yielding

$$R_{ij} = s_i g_j.$$

In such an ideal scenario, our ratings matrix would simply be

$$R = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix} \begin{bmatrix} g_1 & g_2 & \cdots & g_m \end{bmatrix} = \vec{s}\,\vec{g}^{\top}.$$

To take a slightly more complex scenario, imagine for simplicity that movies can be described by a mix of three properties - their levels of "romance," "comedy," and "action", represented by the vectors $\vec{r}$, $\vec{c}$, and $\vec{a}$. And imagine that each student computes how much they enjoy each movie by taking a linear combination of the "romance", "comedy", and "action" of each movie, where the coefficients of this linear combination vary with each student. Let these coefficients be stored in the vectors $\vec{s}_r$, $\vec{s}_c$, and $\vec{s}_a$ respectively. We'll call these coefficients the student's "sensitivities" to each of the three genres.

In such a case, we have that

$$R_{ij} = s_{r,i} r_j + s_{c,i} c_j + s_{a,i} a_j,$$

so the overall ratings matrix R can be expressed as

$$R = \vec{s}_r \, \vec{r}^{\top} + \vec{s}_c \, \vec{c}^{\top} + \vec{s}_a \, \vec{a}^{\top}.$$

The point is that, despite being an $n \times m$ matrix, an idealized R according to this model can be represented as the sum of only a constant number of outer products of n- and m-dimensional vectors.

What could we do if we knew these vectors? Well, imagine that we were given a new student and their ratings on a subset of the movies, stored in the vector $\vec{p}$. What do we want to know about them? Well, we've said that their sensitivities to romance, comedy, and action fully describe their movie preferences. So if we can recover their sensitivities to these genres, then we can make predictions about their ratings of a future movie, so long as we know what genres this future movie is in.

And how can we recover these sensitivities? We know that we can write each provided rating

$$p_j = w_r r_j + w_c c_j + w_a a_j,$$

where w_r, w_c, and w_a are the sensitivities of our new student. Stacking, we obtain the vector equation

$$\begin{bmatrix} r_1 & c_1 & a_1 \\ r_2 & c_2 & a_2 \\ \vdots & \vdots & \vdots \\ r_m & c_m & a_m \end{bmatrix} \begin{bmatrix} w_r \\ w_c \\ w_a \end{bmatrix} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_m \end{bmatrix}.$$

Using least squares, we can therefore recover the sensitivities w_r, w_c, and w_a, and use them to make future predictions. In a similar manner, if we are presented with a new movie and know how a subset of our students rate it, we can set up a least squares problem to solve for its romance, comedy, and action components, and then use these inferred components to make predictions for the rest of the students.

2 Analyzing Data using the SVD

What did we rely on when making our predictions? Fundamentally, we relied on being able to write our large ratings matrix R as a low-rank sum of outer products. In reality, however, there will always be noise and variation in our data. We need some way to construct a "low-rank" approximation of a large matrix that preserves the fundamental structure of our data.

Let's see if the SVD helps. After all, given an $m \times n$ wide matrix A with singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_m \geq 0,$$

we saw earlier that the SVD lets us write it as the sum of outer products

$$A = \sum_{i=1}^{m} \sigma_i \vec{u}_i \vec{v}_i^\top .$$

In general, all m of our singular values will be nonzero. However, we claim that when we have some linear, low-rank essential structure to our data as described above, most of our singular values will be very small. For instance, imagine that σ_1 and σ_2 are significantly larger than the others. Then we can write A as

$$(\sigma_1 \vec{u}_1 \vec{v}_1^\top + \sigma_2 \vec{u}_2 \vec{v}_2^\top) + \sum_{i=3}^{m} \sigma_i \vec{u}_i \vec{v}_i^\top ,$$

where the contribution of the summation is very small in comparison to the first two terms. This motivates approximating our dataset as just

$$\widehat{A} = \sigma_1 \vec{u}_1 \vec{v}_1^\top + \sigma_2 \vec{u}_2 \vec{v}_2^\top ,$$

and using this model to make predictions in the manner described.

As it turns out, empirically, this approach works quite well, and is the heart of what is known as *principal component analysis*. Still, it would be nice to somehow quantifiably justify its efficacy. In particular, why is it that taking just the first few terms of the SVD provides us with the "best" low-rank approximation to our dataset?

If you already believe the result, you might want to jump to the end and read Section 5.

3 Low-rank Approximations

Let's make our intuition about the SVD more formal. Given a matrix A, we will set a goal of computing a "low-rank" approximation $\widehat{A}$ that is of rank $\leq k$, but that is still "approximately equal" to A.

What does "approximately equal" mean? Well, we wish to minimize the magnitude of the error $A - \widehat{A}$, where the squared magnitude of a matrix is defined to be the sum of the squares of all its elements. Why? Because we have no a-priori reason to care about some entries more than other entries of the matrix. It turns out that this quantity is called the *Frobenius norm*, and we define it as

$$\|A\|_F^2 = \sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij}^2 .$$

Why do we care about minimizing this quantity $\left\|A - \widehat{A}\right\|_F^2$? One way to think about it is that our low-rank approximation is essentially a model that we are trying to fit to our data. For instance, in the above "movies" example, we are essentially trying to fit a model of student movie preferences to observed data. So it makes sense that we should try to pick a model that minimizes the error between the predicted and observed data points.

Our conjecture is that the best such approximation of an $m \times n$ matrix A (with $m \leq n$) with SVD

$$A = U\Sigma V^\top = \begin{bmatrix} | & & | \\ \vec{u}_1 & \cdots & \vec{u}_m \\ | & & | \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_m & 0 & \cdots & 0 \end{bmatrix} \begin{bmatrix} - & \vec{v}_1^\top & - \\ - & \vec{v}_2^\top & - \\ & \vdots & \\ - & \vec{v}_n^\top & - \end{bmatrix} = \sum_{i=1}^{m} \sigma_i \vec{u}_i \vec{v}_i^\top.$$

is

$$\widehat{A} = \sum_{i=1}^{k} \sigma_i \vec{u}_i \vec{v}_i^\top,$$

where $\sigma_1 \geq \sigma_2 \geq \cdots$. In other words, we sum up only the largest k outer products, rather than all m of them.

How can we prove this result? First, we should take the time to establish some properties of the Frobenius norm.

3.1 The Frobenius Norm

Consider an $m \times n$ matrix M. As stated above, the Frobenius norm of this matrix is defined to be

$$\|M\|_F^2 = \sum_{i=1}^{m} \sum_{j=1}^{n} M_{ij}^2.$$

First, let's look at how the columns of M relate to this quantity. Let the columns of M be m-dimensional vectors $\vec{v}_i$ such that

$$M = \begin{bmatrix} | & & | \\ \vec{v}_1 & \cdots & \vec{v}_n \\ | & & | \end{bmatrix}.$$

Observe that the squared norm of a particular vector is

$$\left\|\vec{v}_j\right\|^2 = \sum_{i=1}^{m} M_{ij}^2.$$

In other words, it is the sum of squares of all the elements in the j^{th} column. Summing this quantity over all the columns, we obtain the sum of squares of all the elements in M, which is the squared Frobenius norm. Expressed algebraically, it is the case that

$$\|M\|_F^2 = \sum_{i=1}^{m} \sum_{j=1}^{n} M_{ij}^2 = \sum_{j=1}^{n} \left(\sum_{i=1}^{m} M_{ij}^2 \right) = \sum_{j=1}^{n} \left\|\vec{v}_j\right\|^2.$$

A similar result can be derived in an exactly analogous manner for the rows of M.

Now, we will look at how the Frobenius norm of a matrix varies under an orthonormal change of basis. Let

Q be a matrix with m orthonormal columns, such that $Q^\top Q = I_{m\times m}$. Observe that, using the above result,

$$\begin{aligned}
\|QM\|_F^2 &= \left\| \begin{bmatrix} | & & | \\ Q\vec{v}_1 & \cdots & Q\vec{v}_n \\ | & & | \end{bmatrix} \right\|_F^2 \\
&= \sum_{j=1}^{n} \left\|Q\vec{v}_j\right\|^2 \\
&= \sum_{j=1}^{n} \vec{v}_j^\top Q^\top Q \vec{v}_j \\
&= \sum_{j=1}^{n} \vec{v}_j^\top \vec{v}_j \\
&= \sum_{j=1}^{n} \left\|\vec{v}_j\right\|^2 \\
&= \|M\|_F^2 ,
\end{aligned}$$

so the Frobenius norm of a matrix remains unchanged under pre-multiplication by Q. An intuitive way to interpret the above algebraic result is to notice that pre-multiplying by Q doesn't change the norm of any of the columns of M, so its Frobenius norm should similarly remain unchanged. It is interesting to note that here, we don't actually need Q to be square — that never came up in the above derivation. We just need orthonormal columns.

In a very similar manner, it can be shown that post-multiplication by a matrix $Q^\top$ with orthonormal rows again does not change the Frobenius norm. An easy way of seeing this is to simply take transposes everywhere in the above calculation, recognizing that the Frobenius norm of a matrix does not change if we take the transpose.

Now, combining the above observations with our knowledge of the SVD, we find that, given a matrix A with SVD $A = U\Sigma V^\top$,

$$\begin{aligned}
\|A\|_F &= \left\|U\Sigma V^\top\right\|_F \\
&= \left\|U^\top U\Sigma V^\top V\right\|_F \\
&= \|\Sigma\|_F ,
\end{aligned}$$

since U and V are both square[1] orthonormal matrices, so pre-multiplying by $U^\top$ and post-multiplying by V both do not change the Frobenius norm, as in both cases their rows and columns are all orthonormal. Writing out the Frobenius norm of Σ in terms of the singular values explicitly, we find that

$$\|A\|_F = \sqrt{\sum_{i=1}^{m} \sigma_i^2}.$$

This seems like a useful result that we should hang on to.

[1] Here, we are using the full form of the SVD, not the compact form.

3.2 Reducing to an easier case via the SVD

Now, let's get back to the question of low-rank approximations. As stated before, we wish to choose a rank at most k matrix $\widehat{A}$ that minimizes $\left\|A - \widehat{A}\right\|_F$. Looking at the SVD of A and applying simplifications very similar to those derived in the previous section, pre-multiplying by $U^\top$ and post-multiplying by V, we see that

$$\begin{aligned}\left\|A - \widehat{A}\right\|_F &= \left\|U\Sigma V^\top - \widehat{A}\right\|_F \\ &= \left\|U^\top U\Sigma V^\top V - U^\top \widehat{A} V\right\|_F \\ &= \left\|\Sigma - U^\top \widehat{A} V\right\|_F.\end{aligned}$$

Let $X = U^\top \widehat{A} V$, for convenience. Observe that, since U and V are both invertible, given any X we can choose a corresponding $\widehat{A}$, and vice-versa. Furthermore, it is clear that the rank of $\widehat{A}$ and X are the same, so X has rank no more than k. Thus, the problem reduces to choosing an X of rank no more than k that minimizes

$$\|\Sigma - X\|_F.$$

Essentially, the SVD tells us that all we really need to understand is the diagonal case. How can we best approximate a diagonal matrix with sorted entries down the diagonal using a rank at most k matrix?

Let's write out Σ and X explicitly, to try and get a better idea of what remains for us to prove. We wish to choose columns $\vec{x}_i$ that minimize

$$\left\| \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_m & 0 & \cdots & 0 \end{bmatrix} - \begin{bmatrix} | & & | & | & & | \\ \vec{x}_1 & \cdots & \vec{x}_m & \vec{x}_{m+1} & \cdots & \vec{x}_n \\ | & & | & | & & | \end{bmatrix} \right\|_F^2$$

Expressing the squared Frobenius norm as the sum of the squared norms of the columns, we see that we are tasked with minimizing

$$\|\sigma_1\vec{e}_1 - \vec{x}_1\|^2 + \cdots + \|\sigma_m\vec{e}_m - \vec{x}_m\|^2 + \left\|\vec{0} - \vec{x}_{m+1}\right\|^2 + \cdots + \left\|\vec{0} - \vec{x}_n\right\|^2,$$

while keeping the rank of X less than or equal to some given constant k. Note that we use the notation $\vec{e}_i$ here to represent the i^{th} column of the identity matrix $I_{m\times m}$, working in $\mathbb{R}_m$ throughout. It should be clear here that

$$\vec{x}_{m+1} = \vec{x}_{m+2} = \cdots = \vec{x}_n = \vec{0},$$

since making them nonzero would only hurt us by increasing the quantity that we are trying to minimize.

But what about the $\vec{x}_1, \ldots, \vec{x}_m$? Because of the constraint on the rank of X, we can't make them all equal to the corresponding columns of Σ, as then the rank of X would be m, and not k.

There is a natural guess that seems somewhat obvious: just pick $\vec{x}_1 = \sigma_1\vec{e}_1, \vec{x}_2 = \sigma_2\vec{e}_2, \ldots, \vec{x}_k = \sigma_k\vec{e}_k$ and set the rest of the $\vec{x}_i = \vec{0}$ for $i > k$. This definitely makes X have rank k and it does turn out to be the right answer, but is it really obvious? How can we be sure that there isn't a better choice out there?

This actually requires proof.

4 The Projection Perspective

In the previous section, we used the SVD to distill our problem down to the diagonal case. How can we choose an X of rank no more than k that minimizes

$$\|\Sigma - X\|_F .$$

We saw that what matters here is choosing the m vectors $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_m$ that are matched up with the non-zero parts of Σ in the above subtraction, since the rest of X should clearly be all zeros. We have a guess as to what the optimal choice should be, but we need to prove that it is indeed optimal.

The projection perspective approaches this problem by stepping back a bit. Dealing with the constraint on the rank of X is annoying. What does this condition really mean from a geometric point of view? It means that all these m vectors $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_m$ must lie in a k-dimensional subspace. This is what rank means.

Let's reason about the problem in terms of a basis for this subspace. Imagine that we are already given this subspace as the span of some $m \times k$ matrix Q. What kind of matrix should we ask for? We know that orthonormal matrices are easy to work with, and any other matrix with linearly independent columns can be orthonormalized, so for convenience, let's assume that all the columns of Q have been orthonormalized, so $Q^\top Q = I_{k\times k}$. Then what should each of the $\vec{x}_i$ be, assuming they are constrained to lie in the column space of Q?

Well, once we're given Q, we simply need to choose each $\vec{x}_i$ to minimize the term it is involved in — specifically, each $\vec{x}_i$ should minimize

$$\|\sigma_i \vec{e}_i - \vec{x}_i\|^2 .$$

And how can we choose such an $\vec{x}_i$? We know this from 16A! It is simply the projection of $\sigma_i \vec{e}_i$ onto the column space of Q, which from least squares is just

$$\vec{x}_i = Q(Q^\top Q)^{-1} Q^\top (\sigma_i \vec{e}_i) = QQ^\top (\sigma_i \vec{e}_i) = \sigma_i QQ^\top \vec{e}_i,$$

simplifying by recalling that $Q^\top Q = I_{k\times k}$, as we chose the columns of Q to be an orthonormal basis.

This means that finding the right orthonormal Q is enough to solve our problem. In hindsight, this is not surprising since our initial motivation was to discover a subspace that best captures the data.

4.1 The finding Q problem

So our problem has been simplified further. Rather than trying to find an arbitrary $m \times n$ matrix X with rank-k, we can instead find an $m \times k$ orthonormal matrix Q, and then compute $\vec{x}_i$ by projecting each of the columns of Σ onto the column space of Q.

Since we're now working with Q, rather than X, it makes sense to write the quantity we're interested in minimizing in terms of Q. Substituting in our least-squares solution for each of the $\vec{x}_i$ into our earlier expression for the squared Frobenius norm, and setting all the $\vec{x}_i = \vec{0}$ for all $i > m$ (as we argued would be optimal), we find that the new quantity to minimize becomes

$$\begin{aligned}\|\sigma_1 \vec{e}_1 - \vec{x}_1\|^2 + \cdots + \|\sigma_m \vec{e}_m - \vec{x}_m\|^2 + \left\|\vec{0} - \vec{x}_{m+1}\right\|^2 + \cdots + \left\|\vec{0} - \vec{x}_n\right\|^2 &= \sum_{i=1}^{m} \left\|\sigma_i \vec{e}_i - \sigma_i QQ^\top \vec{e}_i\right\|^2 + \sum_{i=m+1}^{n} \left\|\vec{0} - \vec{0}\right\|^2 \\ &= \sum_{i=1}^{m} \sigma_i^2 \left\|\vec{e}_i - QQ^\top \vec{e}_i\right\|^2 .\end{aligned}$$

Can we simplify this expression somewhat? Notice that $\left(\vec{e}_i - QQ^\top \vec{e}_i\right) \perp QQ^\top \vec{e}_i$, since we know from 16A that the least squares projection is orthogonal to the residual error vector. Thus, by the Pythagorean theorem, we can write

$$\|\vec{e}_i\|^2 = \left\|\vec{e}_i - QQ^\top \vec{e}_i\right\|^2 + \left\|QQ^\top \vec{e}_i\right\|^2 \implies \left\|\vec{e}_i - QQ^\top \vec{e}_i\right\|^2 = \|\vec{e}_i\|^2 - \left\|QQ^\top \vec{e}_i\right\|^2.$$

Making this substitution in our earlier expression, noticing that $\|\vec{e}_i\| = 1$ and rearranging, our problem reduces to finding an orthonormal Q that minimizes the quantity

$$\sum_{i=1}^{m} \sigma_i^2 (\|\vec{e}_i\|^2 - \left\|QQ^\top \vec{e}_i\right\|^2) = \sum_{i=1}^{m} \sigma_i^2 - \sum_{i=1}^{m} \sigma_i^2 \left\|QQ^\top \vec{e}_i\right\|^2.$$

Notice that we have broken the quantity we are minimizing into two summations. The first summation is simply the squared Frobenius norm of Σ, which does not depend on Q. The second is the negation of the squared Frobenius norm of $QQ^\top \Sigma$. Thus, in order to minimize the overall quantity, we should aim to choose our Q to *maximize* $\left\|QQ^\top \Sigma\right\|_F^2$, since we are subtracting this quantity from another fixed value.

4.2 Working with this even simpler problem

Cool! We've managed to simplify our problem further, by taking advantage of properties of projections and orthonormal bases. Recall from earlier that $\|QM\|_F^2 = \|M\|_F^2$ when Q has orthonormal columns. Then we see immediately that the quantity we are trying to maximize can be slightly simplified as

$$\left\|QQ^\top \Sigma\right\|_F^2 = \left\|Q^\top \Sigma\right\|_F^2. \tag{1}$$

Be aware that, since Q is not square, $QQ^\top \neq I_{m \times m}$, as the rows are not necessarily mutually orthogonal. So we can't use this property to make any further simplifications. However, it is nice to see that (1) feels very reminiscent of the maximizing correlation goal we've seen in 16A.

How can we show that our guessed solution (just choose the first k columns of the identity for Q) is indeed the best choice for maximizing (1)? That choice would give $\left\|Q^\top \Sigma\right\|_F^2 = \sum_{j=1}^{k} \sigma_j^2$. One way to show that this is optimal is to show that we cannot possibly do any better than that.

To do this, let's just expand this product out directly. For notational convenience, let the columns of Q be $\vec{q}_i$, and so

$$Q^\top = \begin{bmatrix} - & \vec{q}_1^\top & - \\ - & \vec{q}_2^\top & - \\ & \vdots & \\ - & \vec{q}_k^\top & - \end{bmatrix}.$$

Remember that each of the $\vec{q}_i$ are m-dimensional orthonormal vectors.

Now, writing out our product, we obtain

$$\begin{aligned}
\left\|Q^\top \Sigma\right\|_F^2 &= \left\| \begin{bmatrix} - & \vec{q}_1^\top & - \\ - & \vec{q}_2^\top & - \\ & \vdots & \\ - & \vec{q}_k^\top & - \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_m & 0 & \cdots & 0 \end{bmatrix} \right\|_F^2 \\
&= \sum_{i=1}^{k} \left\|\vec{q}_i^\top \Sigma\right\|_F^2 = \sum_{i=1}^{k} \sum_{j=1}^{m} \left(\vec{q}_i^\top \vec{e}_j\right)^2 \sigma_j^2 \\
&= \sum_{i=1}^{k} \sum_{j=1}^{m} Q_{ji}^2 \sigma_j^2 \\
&= \sum_{j=1}^{m} \sigma_j^2 \sum_{i=1}^{k} Q_{ji}^2.
\end{aligned}$$

Thus, we have expressed our desired Frobenius norm as a linear combination of the squared singular values σ_j^2. For convenience and to focus our attention on what is going to matter, let the coefficients of our linear combination be denoted as

$$d_j = \sum_{i=1}^{k} Q_{ji}^2, \tag{2}$$

so we can express the term we want to maximize as

$$\left\|Q^\top \Sigma\right\|_F^2 = \sum_{j=1}^{m} \sigma_j^2 d_j. \tag{3}$$

Now, observe that the Frobenius norm of $Q^\top$ itself is $\|Q\|_F = k$, since Q has k unit-norm columns. Expressing this quantity in terms of the d_j, we see that

$$\sum_{j=1}^{m} d_j = \left\|Q^\top\right\|_F^2 = \|Q\|_F^2 = k. \tag{4}$$

Furthermore, notice that, since they are defined as a sum of squares in (2), each of the $d_j \geq 0$. As $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_m$, we could try to upper-bound the quantity we are trying to maximize by choosing $d_1 = k$ and setting the other $d_j = 0$.

That would give $k\sigma_1^2$ which is bigger than our guessed optimal solution. But is this extreme choice of d_j actually achievable by some Q? The challenge is that we know a lot about the columns of Q since they are orthonormal. But d_j is related to the j^{th} *row* of Q. How can we understand anything about the rows of Q?

If Q were square, then the fact that $Q^\top Q = I$ would tell us that $Q^\top$ is the inverse of Q and so $Q^\top$ would also be orthonormal. But Q is decidedly not square, so what can we do?

Observe that the k columns of Q can be extended using Gram-Schmidt to form a full orthonormal basis for

$\mathbb{R}_m$. Let this extended basis be

$$\widehat{Q} = \begin{bmatrix} | & & | & | & & | \\ \vec{q}_1 & \cdots & \vec{q}_k & \vec{q}_{k+1} & \cdots & \vec{q}_m \\ | & & | & | & & | \end{bmatrix}.$$

By the properties of orthonormal square matrices, we know that in addition to the columns, the *rows* of $\widehat{Q}$ also form an orthonormal basis, and so are all of unit norm. Thus, looking at the j^{th} row, we can write

$$\sum_{i=1}^{m} \widehat{Q}_{ji}^2 = 1$$

for all j.

This tells us that the rows of $\widehat{Q}$ can't be too large, and thus, truncating the sum by removing the terms corresponding to the $(k+1)^{\text{th}}$ column onwards, we see that

$$d_j = \sum_{i=1}^{k} Q_{ji}^2 \leq \sum_{i=1}^{m} \widehat{Q}_{ji}^2 = 1. \tag{5}$$

So in addition to requiring that they sum to k, we have established that each individual $0 \leq d_j \leq 1$ for all j.

Now, we have found a simpler problem inside our original problem.

4.3 Working with the inner problem

At this point, we have turned even the simplified problem into something[2] only involving numbers. From (3), we know we want to maximize $\sum_{j=1}^{m} \sigma_j^2 d_j$ over the choice of d_j that we know must satisfy certain constraints. From (4), we know their overall sum $\sum_{j=1}^{m} d_j = k$. We also from (5), that $0 \leq d_j \leq 1$ for all j.

A consequence of this is that our previous overly greedy assignment of values to d_j is not possible if $k > 1$, since we must have $d_1 \leq 1 < k$. Instead, if we wish to maximize our linear combination of the σ_j^2, we should set

$$d_1 = d_2 = \cdots = d_k = 1$$

and

$$d_{k+1} = d_{k+2} = \cdots = d_m = 0,$$

in order to place as much weight as possible on the largest squared singular values without violating the constraints on the d_j. This is very intuitive and gives us $\sum_{j=1}^{k} \sigma_j^2$ as the best we can do.

Why is this intuitive? We can think of each d_j as telling us how much money we want to spend to buy product j. The store only has up to 1 liter of each product in stock. Each item costs one dollar per liter. The product j contains σ_j^2 grams of gold per liter. If we want to get as much gold as possible, what should we do if we have k dollars to spend? The answer is clearly buy out all the stock of the first k products since they all cost the same, but the first k products have more gold per liter than any of the other products.

The rigorous proof of this claim is omitted here in this note, but only because you are walked through this exact proof on the homework. (It follows from a straightforward argument that doing anything else can't be

[2] You will see in later courses like 127 and 170 that such problems are actually called "linear programs" and are extremely useful. They arise naturally in lots of settings. In 127 and 170, you will learn ways of thinking about them that are generally useful. In our particularly simple case here, we can reason about the problem directly.

optimal since anything else could be improved by moving it closer to this solution.)

But is *this* assignment of d_j above actually achievable in our original problem of interest? It certainly satisfies all of our inequalities, but that is not sufficient to show it is actually possible — perhaps we could have derived another inequality that makes this assignment impossible. After all, the d_j are defined in (2) from the underlying Q matrix. Maybe no Q matrix exists that satisfies our desired asignments of d_j?

Consequently, the best way to show that it is in fact feasible is to present such a Q. We want the first k columns of $Q^\top$ to all be of unit norm, and the remaining columns to all be 0. Thus, one choice would be to write

$$Q^\top = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 & \cdots & 0 \end{bmatrix}.$$

It is straightfoward to verify that the rows of $Q^\top$ form an orthonormal basis of a k-dimensional subspace, so this choice of Q is valid, and so it is optimal.

So what low-rank approximation does this optimal choice of Q correspond to? Substituting back, we see that

$$\begin{aligned} \widehat{A} &= UXV^\top \\ &= UQQ^\top \Sigma V^\top. \end{aligned}$$

Now, observe that

$$QQ^\top = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 & \cdots & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix}.$$

In words, $QQ^\top$ is a diagonal matrix where the first k entries along the diagonal are 1, and the remainder are 0. Thus, we see that

$$\widehat{A} = U \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_k & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix} V^\top = \sum_{i=1}^{k} \sigma_i \vec{u}_i \vec{v}_i^\top,$$

as expected. This completes the proof.

The entire proof here is not something that we would expect you to be able to come up with on your own at

the level of 16B. However, it is elementary enough that you should be able to follow the steps, and the goal is to help you begin to feel how each part follows naturally.

5 Using the PCA idea

The above theorem tells us that if we want to find the best k-dimensional subspace to use to approximate a set of (real) data, we can take the data, arrange it into a matrix M by stacking the columns of data, and then taking the SVD. Looking at the SVD in outer product form $M = \sum_i \sigma_i \vec{u}_i \vec{v}_i^\top$, where the σ_i are in decreasing order, we can just choose to truncate the sum to the top k terms. We get an approximation of all the data $\widehat{M} = \sum_{i=1}^{k} \sigma_i \vec{u}_i \vec{v}_i^\top$, but more importantly, we also get a basis for our subspace of interest. Namely, the $\vec{u}_i$ give us such an orthonormal basis. The first such $\vec{u}_1$ is called the first principal component (for columns), the second such $\vec{u}_2$ is called the second principal component (for columns), and so on.

If we had arranged the data into rows instead, then we could do the same thing and we would use the first $\vec{v}_i$ instead as the principal components. (Or if we want to keep them in row form, $\vec{v}_i^\top$.) The choice depends on the problem context. What is the nature of the data and what are we trying to do?

Once we have learned such a subspace from training data, we can use it for dimensionality reduction when dealing with new raw data points. We just project the new raw data points onto the subspace. Although our goal is for the projections to be close to the original raw data points, what we work with are the coefficients of the projections in the subspace basis that we have found. For example, if the raw data points have $m = 100$ entries in them, and we decided to use $k = 3$ to choose a 3 dimensional subspace, then we have managed to distill the essential information into just 3 coefficients. These are also easy to calculate by the orthonormality of the $\vec{u}_i$. For the case of $k = 3$, we just keep $\begin{bmatrix} \vec{u}_1^\top \vec{x} \\ \vec{u}_2^\top \vec{x} \\ \vec{u}_3^\top \vec{x} \end{bmatrix} = \begin{bmatrix} \vec{u}_1^\top \\ \vec{u}_2^\top \\ \vec{u}_3^\top \end{bmatrix} \vec{x}$ instead of the whole m-dimensional raw data point $\vec{x}$.

Such dimensionality reduction is often very useful in reducing the computational burden[3] for doing classification, etc.

In some problem contexts, the data that we are given has DC offsets[4] that we don't want to consider as directions of variation. When that happens, those DC offsets are often estimated and removed before one does PCA analysis. Such offsets can be found by taking appropriate means/averages of the training data, and then subtracting them off. However, it is useful to consider that "de-meaning" step as being something separate from the core task of subspace discovery. Instead, it should be considered an application-specific data cleaning or preprocessing step. It should only be done if you actually believe that such irrelevant DC offsets might exist.

Contributors:

[3] For reasons that you will learn in future courses like 189, the advantage is not simply computational. Distilling the data down by dimensionality reduction can also be helpful in eliminating irrelevant noise that could confuse subsequent stages of learning. Even more subtly, it can help make problems tractable that otherwise might seem impossible. For example, if the data that is being recorded is a microphone trace that is hundreds of samples long, then many approaches to learning how to classify words might require thousands of examples of each word. By reducing the dimensionality of the incoming microphone trace, it can become possible to learn from fewer examples using those approaches. All of this is not in scope for 16B.

[4] What do we mean? Consider two-dimensional data. Our approach to PCA will find the best line through the origin that goes through the data. But what if the data was actually along a line that didn't go through the origin? How would we find this? In that case, we would want to "center" the data first. The average of all the data points that actually lie along a straight line will also lie on that same straight line. Consequently, the same will also be approximately true if the points are approximately on a line.

- Rahul Arya.
- Anant Sahai.
- Ayan Biswas.

EECS 16B Designing Information Devices and Systems II

Note 15: Linearization

Overview

So far, we have spent a great deal of time studying linear systems described by differential equations of the form:

$$\frac{\mathrm{d}}{\mathrm{d}t}\vec{x}(t) = A\vec{x}(t) + B\vec{u}(t) + \vec{w}(t)$$

or linear difference equations (aka linear recurrence relations) of the form:

$$\vec{x}(t+1) = A\vec{x}(t) + B\vec{u}(t) + \vec{w}(t).$$

Here, the linear difference equations came from either discretizing a continuous model or were learned from data.

In the real world, however, it is very rare for physical systems to perfectly obey such simple linear differential equations. The real world is generally continuous, but our systems will obey nonlinear differential equations of the form:

$$\frac{\mathrm{d}}{\mathrm{d}t}\vec{x} = \vec{f}(\vec{x}(t), \vec{u}(t)) + \vec{w}(t),$$

where $\vec{f}$ is some non-linear function. Through linearization, we will see how to approximate general systems as locally linear systems, which will in turn allow us to apply all the techniques we have developed so far.

1 Taylor Expansion

First, we must recall how to approximate the simplest nonlinear functions — scalar functions of one variable — as linear functions. To do so, we will use the Taylor expansion we learned in calculus. This expansion tells us that for analytic functions $f(x)$ (functions that can be infinitely differentiated everywhere), we can write

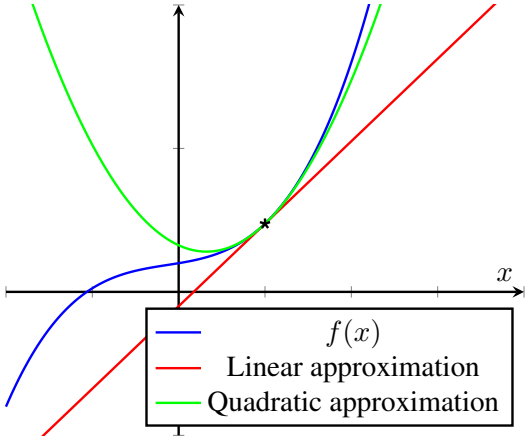
$$\begin{aligned} f(x) &= f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2 + \dots \\ &= f(x_0) + \frac{\mathrm{d}f}{\mathrm{d}x}\bigg|_{x=x_0}(x - x_0) + \frac{1}{2}\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\bigg|_{x=x_0}(x - x_0)^2 + \dots \end{aligned}$$

near any particular point $x = x_0$, known as the *expansion point*. By truncating this expansion after the first few terms, we can obtain a locally polynomial approximation for f of a desired degree d. For instance, we may take just the first two terms to linearly approximate $f(x)$ at x_0 as

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0).$$

It is further known that, when working in the "neighborhood" of $x = x_0$ (i.e. in a small interval around that point), the error of such a truncated approximation is bounded if the function is well behaved[1]. The exact magnitude of this bound can be determined[2] but is not very important for our purposes — the important part is that it is indeed bounded.

Before going further, let's see what these approximations look like.



We have plotted a nonlinear function $f(x)$ and its linear approximation constructed around $x_0 = 0.5$. Notice that though the approximation deviates greatly from $f(x)$ over the whole domain, in the close neighborhood of x_0, the error of the approximation is very small. Including one more term of our Taylor expansion, we obtain the quadratic approximation. Qualitatively, this approximation matches our function better in the neighborhood of our expansion point. In practice, we rarely go beyond the quadratic terms when trying to find a polynomial approximation of a function in a local region.

The ability to approximate is generally good because it lets us attack a problem that we don't know to solve by approximating it using another problem that we do know how to solve. But this only makes engineering sense if we have some way of dealing with the impact of the approximation error. Control provides a very useful example for this kind of thinking. If we approximate the f in a nonlinear differential equation by a linear function, why can we trust that everything will work out when we use this approximation to control? The answer is provided by the disturbance term $w(t)$ — our feedback control design for the linearized system should be able to reject bounded disturbances. In the neighborhood of the expansion point, our approximation differs from the true function by a small bounded amount. This approximation error can therefore be absorbed into the existing disturbance term — we just make the disturbance a little bigger. As long as the feedback controlled system stays within the region that the approximation is valid, we are fine. In effect, we pass the buck and let the feedback control deal with the approximation error. We'll see in a later note on classification how iteration provides another way to deal with the impact of approximation errors.

[1] For example, it has a bounded derivative beyond the level at which we truncate in the neighborhood of interest.

[2] Remember from your calculus courses — this is an application of the mean value theorem. You can take the maximum absolute value for the (presumably bounded) next derivative and use it in the expansion to get an explicit upper bound. The fact that the factorial $(k+1)!$ in the denominator is very rapidly growing is what makes this bound get very small as we add terms in the expansion. But for us, the more important fact is that the bound includes $(x - x_0)^{k+1}$. This is what makes the bound get very small in a tight enough neighborhood around x_0. This tells us that we can use appoximations that are not that high order (like linear) and still have a good local approximation. If the second derivative is bounded by 2, then in a neighborhood of distance 0.1 from x_0, the linear approximation will be good to a precision of 0.01.

2 Multivariate Scalar-Valued Functions

Unfortunately, we can't directly apply the Taylor expansions that we learned in our calculus courses to solve our problem of linearizing a nonlinear system. Why is that? Well, so far we only know how to linearize scalar functions of one variable. But any nontrivial control system has at least *two* variables: the state and the control input! Even with just a scalar state and one-dimensional control input, our system would look like something of the form

$$\frac{\mathrm{d}}{\mathrm{d}t}x(t) = f(x(t), u(t)).$$

So we need to figure out how to deal with $f(x, u)$. Ideally, we'd be able to pick an expansion point (x_0, u_0), around which we could linearize f as

$$f(x_0 + \delta x, u_0 + \delta u) \approx f(x_0, u_0) + a \times \delta x + b \times \delta u,$$

where a and b depend only on the function f and the x_0 and u_0. It makes sense that a should somehow represent the "sensitivity" of f to variations in x, and b the sensitivity of f with respect to variations in u.

To get a better understanding of how we can compute these two quantities, it will help to consider an example. Let $f(x, y) = x^3u^2$. Then we see that, near an expansion point (x_0, u_0), we can rewrite

$$\begin{aligned} f(x_0 + \delta x, u_0 + \delta u) &= (x_0 + \delta x)^3(u_0 + \delta u)^2 \\ &= (x_0^3 + 3x_0^2\delta x + 3x_0\delta x^2 + \delta x^3)(u_0^2 + 2u_0\delta_u + \delta u^2). \end{aligned}$$

Imagine that we are very near this expansion point, so $|\delta x|, |\delta u| \ll x_0, u_0$. In other words, we can treat δx and δu as two comparably tiny quantities, that are both much smaller than x_0 or u_0. Then which terms in the above product (when expanded out fully) are most significant?

Well, any term involving δx or δu will immediately be made small, so the most significant term is $x_0^3u_0^2$. This term is just a constant, and isn't very interesting since it doesn't vary at all as we move around the neighborhood of our expansion point. So what can we get next? Well, we want as few δx and δu's as possible in our terms. We've already got the only term with zero of them, so the next most significant terms will be those involving exactly one tiny quantity: $3x_0^2u_0^2\delta x$ and $2u_0x_0^3\delta u$. Every other term will have products of tiny terms and we know that products of tiny numbers are even tinier numbers. If you will recall, this is the same type of reasoning that you saw in your calculus course when you first encountered derivatives.

Thus, using only these most significant terms, we obtain the approximation

$$x^3u^2 \approx x_0^3u_0^2 + 3x_0^2u_0^2\delta x + 2u_0x_0^3\delta u.$$

The $x_0^3u_0^2$ term clearly corresponds to the $f(x_0, u_0)$ term in our desired linearization. In a similar manner, $3x_0^2u_0^2$ corresponds to the coefficient a and $2u_0x_0^3$ to the coefficient b.

How could we have obtained these coefficients directly from f?

2.1 Partial Derivatives

The key insight is the following. Imagine that u was in fact a constant, so our function was just the scalar valued function of a single scalar variable $f(x) = x^3u^2$. Then what is its derivative? Well, since u^2 is being treated as a constant, its derivative is $3x^2u^2$. Similarly, by treating x as a constant and writing our nonlinear function as $f(u) = x^3u^2$, we can differentiate to obtain $2ux^3$. These quantities are known as the *partial*

derivatives of f, and are denoted as $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial u}$.

Now when we evaluate the partial derivatives at our expansion point (x_0, u_0), we exactly get the terms from above. In other words, we obtain the desired coefficients by treating all the variables except the one of interest as a constant, and then differentiating f with respect to the remaining variable. Thus, we get

$$a = \left.\frac{\partial f}{\partial x}\right|_{x=x_0,u=u_0} \text{ and } b = \left.\frac{\partial f}{\partial u}\right|_{x=x_0,u=u_0}$$

Then we can linearize any well-behaved scalar-valued function of 2 variables about an expansion point (x_0, u_0) as

$$f(x,u) = f(x_0,u_0) + \left(\left.\frac{\partial f}{\partial x}\right|_{x=x_0,u=u_0}\right)(x-x_0) + \left(\left.\frac{\partial f}{\partial u}\right|_{x=x_0,u=u_0}\right)(u-u_0)$$

If we had more than 2 variables as the input, and instead a vector of variables, we can generalize this to get the linearization of $f(\vec{x})$ around an expansion point $\vec{x}^*$:

$$f(\vec{x}) = f(\vec{x}^*) + \sum_{i=1}^{n}(\left.\frac{\partial f}{\partial x_i}\right|_{\vec{x}=\vec{x}^*})(x_i - x_i^*)$$

3 Vector-Valued Functions

With partial derivatives in hand, we can return to our original problem of linearizing a vector-valued function $\vec{f}(\vec{x}, \vec{u})$.

First, we should ask ourselves what the desired form of the linearization could be. Recalling our experience with linear systems, we'd probably like our linearization near an expansion point $(\vec{x}^*, \vec{u}^*)$ to look something like

$$\vec{f}(\vec{x},\vec{u}) \approx \vec{f}(\vec{x}^*,\vec{u}^*) + A(\vec{x}-\vec{x}^*) + B(\vec{u}-\vec{u}^*),$$

where A and B are some matrices that presumably depend on $\vec{f}$ and $\vec{x}^*$ and $\vec{u}^*$.

Let our state $\vec{x}$ be n-dimensional and our control $\vec{u}$ be k-dimensional. Since $\frac{\mathrm{d}}{\mathrm{d}t}\vec{x} = \vec{f}(\vec{x},\vec{u})$, the output of $\vec{f}$ must also be n-dimensional. Thus, by the rules of matrix multiplication, we know that A must be an $n \times n$ matrix and B an $n \times k$ matrix, for the dimensions to all work out. But what are the contents of these two matrices? To determine this, we will look at our vectors elementwise, expressing the function we wish to linearize as

$$\vec{f}(\vec{x},\vec{u}) = \begin{bmatrix} f_1(\vec{x},\vec{u}) \\ f_2(\vec{x},\vec{u}) \\ \vdots \\ f_n(\vec{x},\vec{u}) \end{bmatrix},$$

where the f_i are each scalar-valued functions. Let's now consider a single one of these f_i, and try to linearize it about an expansion point. Note that though we write each f_i as functions of the vectors $\vec{x}$ and $\vec{u}$, they are really simply functions of the $n+k$ individual scalars $x_1, \ldots, x_n, u_1, \ldots, u_k$ that form the components of the two vector-valued arguments. Thus, we can use the linearization techniques from the earlier section to

approximate

$$f_i(\vec{x},\vec{u}) \approx f_i(\vec{x}^*,\vec{u}^*) + \sum_{j=1}^{n} \left(\frac{\partial f_i}{\partial x_j}\bigg|_{(\vec{x}^*,\vec{u}^*)} \right) (x_j - x_j^*) + \sum_{j=1}^{k} \left(\frac{\partial f_i}{\partial u_j}\bigg|_{(\vec{x}^*,\vec{u}^*)} \right) (u_j - u_j^*).$$

We can rearrange the above linearization as the matrix multiplication

$$\begin{aligned} f_i(\vec{x},\vec{u}) &\approx f_i(\vec{x}^*,\vec{u}^*) + \begin{bmatrix} \frac{\partial f_i}{\partial x_1} & \cdots & \frac{\partial f_i}{\partial x_n} \end{bmatrix}\bigg|_{(\vec{x}^*,\vec{u}^*)} (\vec{x} - \vec{x}^*) + \begin{bmatrix} \frac{\partial f_i}{\partial u_1} & \cdots & \frac{\partial f_i}{\partial u_k} \end{bmatrix}\bigg|_{(\vec{x}^*,\vec{u}^*)} (\vec{u} - \vec{u}^*) \\ &= f_i(\vec{x}^*,\vec{u}^*) + \frac{\partial f_i}{\partial \vec{x}}\bigg|_{(\vec{x}^*,\vec{u}^*)} (\vec{x} - \vec{x}^*) + \frac{\partial f_i}{\partial \vec{u}}\bigg|_{(\vec{x}^*,\vec{u}^*)} (\vec{u} - \vec{u}^*) \end{aligned}$$

The two row vectors above can be thought of as the derivatives[3] of $f_i(\vec{x},\vec{u})$ with respect to $\vec{x}$ and $\vec{u}$, so we denote these rows by $\frac{\partial f_i}{\partial \vec{x}}$ and $\frac{\partial f_i}{\partial \vec{u}}$.

This was the linearization for just one element of $\vec{f}$, so we can now stack the above linearization for all f_i to obtain a linearization for the original vector-valued $\vec{f}(\vec{x},\vec{u})$:

$$\vec{f}(\vec{x},\vec{u}) \approx \vec{f}(\vec{x}^*,\vec{u}^*) + \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}\bigg|_{(\vec{x}^*,\vec{u}^*)} (\vec{x} - \vec{x}^*) + \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \cdots & \frac{\partial f_1}{\partial u_k} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial u_1} & \cdots & \frac{\partial f_n}{\partial u_k} \end{bmatrix}\bigg|_{(\vec{x}^*,\vec{u}^*)} (\vec{u} - \vec{u}^*),$$

We call these 2 matrices the *Jacobian* of $\vec{f}$ with respect to $\vec{x}$ and $\vec{u}$, evaluated at $(\vec{x}^*,\vec{u}^*)$. We will denote them as J_x, J_u. This gives us

$$\vec{f}(\vec{x},\vec{u}) \approx \vec{f}(\vec{x}^*,\vec{u}^*) + J_x(\vec{x} - \vec{x}^*) + J_u(\vec{u} - \vec{u}^*)$$

so we have solved for $A = J_x$ and $B = J_u$ in our desired linear model.

Formally, the Jacobian is the partial derivative of a vector-valued function $\vec{f}$ with respect to a vector input $\vec{x}$ so we say the Jacobian is $\frac{\partial \vec{f}}{\partial \vec{x}}$, and J_x is the Jacobian evaluated at $\vec{x}^*$ so $J_x = \frac{\partial \vec{f}}{\partial \vec{x}}\Big|_{\vec{x}=\vec{x}^*}$. Once again it makes sense for these derivatives to be matrices since a matrix acting on a vector returns a vector.

One great development is that modern software frameworks like PyTorch and TensorFlow have automatic support for computing derivatives of functions that are expressed naturally using code. This means that all these ideas around linearization are now quite easy to implement — you don't have to compute all the derivatives by hand in practice.

4 Equilibrium Points

Now, let's see how this linearization plugs back into our original differential equation (the discrete-time case behaves analogously for the purposes of linearization, but we will focus on the continuous-time case to

[3] In general, the derivative of a scalar valued function of a vector has to be a row vector. This is because it needs to act on an change in a column vector and return a scalar that represents the change to the function. Rows multiply columns to give scalars. For those of you who might have taken a course in multivariable calculus, it is vital that you not confuse the gradient ∇f with the derivative. The gradient is a column vector and is the transpose of the derivative of the function f.

avoid duplicating calculations). We see that

$$\frac{\mathrm{d}}{\mathrm{d}t}\vec{x}(t) = f(\vec{x},\vec{u}) = \vec{f}(\vec{x}^*,\vec{u}^*) + J_x(\vec{x}(t)-\vec{x}^*) + J_u(\vec{u}(t)-\vec{u}^*) + \vec{w}(t).$$

Notice that even though we are using an approximation, we have used an equals sign $=$ above. This is because we have folded the approximation error into the disturbance term $\vec{w}(t)$.

So have we gotten this into a form that we know how to deal with? The first apparent problem is that we have $\vec{x}(t)-\vec{x}^*$ on the right, but $\vec{x}(t)$ on the left. To fix this, we will define new state variables $\Delta\vec{x}(t) = \vec{x}(t)-\vec{x}^*$ and new control inputs $\Delta\vec{u}(t) = \vec{u}(t)-\vec{u}^*$.

By rewriting $\vec{x}(t) = \Delta\vec{x}(t) + \vec{x}^*$ and expanding out the derivative, we obtain

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}t}\vec{x}(t) = \frac{\mathrm{d}}{\mathrm{d}t}\vec{x}^* + \frac{\mathrm{d}}{\mathrm{d}t}\Delta\vec{x}(t) &= \vec{f}(\vec{x}^*,\vec{u}^*) + J_x(\vec{x}(t)-\vec{x}^*) + J_u(\vec{u}(t)-\vec{u}^*) + \vec{w}(t)\\ &= \vec{f}(\vec{x}^*,\vec{u}^*) + J_x\Delta\vec{x}(t) + J_u\Delta\vec{u}(t) + \vec{w}(t)\end{aligned}$$

Since $\vec{x}^*$ is a constant for all time, we know $\frac{\mathrm{d}}{\mathrm{d}t}\vec{x}^* = 0$. Then the final term to remove to get a linear model is the $\vec{f}(\vec{x}^*,\vec{u}^*)$ on the right side of the equation. Notice that this term depend only on the choice of expansion point. Thus, when we linearize a system, it is our job to pick expansion points such that

$$\vec{f}(\vec{x}^*,\vec{u}^*) = \vec{0}. \tag{1}$$

Such points $(\vec{x}^*,\vec{u}^*)$ are called *equilibrium points* of $\vec{f}$. When we linearize a dynamical system around such an equilibrium, the approximation is now a linear system:

$$\frac{\mathrm{d}}{\mathrm{d}t}\Delta\vec{x}(t) = J_x\Delta\vec{x}(t) + J_u\Delta\vec{u}(t) + \vec{w}(t) \tag{2}$$

where now $A = J_x$ and $B = J_u$. The system's stability can be checked by looking at the eigenvalues of A and the local controllability can be checked by looking at A, B together. At this point, if the system is controllable, we can use feedback control to place the eigenvalues of $A + BK$ where we want and thereby get a feedback control law that will stabilize the linearized system about the chosen operating point. Note that this feedback control will drive $\Delta\vec{x} \to 0$ which means it drives $\vec{x} \to \vec{x}^*$. In this way, we can think of $\vec{x}^*$ as our desired value of $\vec{x}(t)$. It is also important to remember that the actual control applied will be $\vec{u}(t) = \vec{u}^* + K(\vec{x}(t)-\vec{x}^*)$ — because the K has been calculated for the linearized system and we need to apply a control in the original nonlinear system.

In general, there are are many potential solutions for an equilibrium point. Which one you choose depends on the context or design goal. For example, one might want to find a feedback control law that stabilizes a nonlinear system about a particular state $\vec{x}^*$. In that case, you will want to solve for the the corresponding $\vec{u}^*$ that satisfies (1) with that particular state $\vec{x}^*$. It can be thought of as the control that would keep the original nonlinear system held at that particular point in the absence of all disturbances. To find it, we can use numerical[4] or graphical[5] methods to solve the nonlinear system of equations (1).

The ability to linearize is what makes linear control practical since many (if not most) real-world systems

[4] So, how do you solve nonlinear systems of equations like (1)? This can be challenging at times but fortunately software packages exist for you to use. For example, SciPy has scipy.optimize.fsolve for you. Alternatively, you can use the Gauss-Newthon method.

[5] By graphical, we literally mean methods that involve drawing the graph of the function and seeing where it crosses zero or related intuitive approaches.

that we encounter are fundamentally nonlinear. The linearized system of differential equations can also be discretized to give rise to a discrete-time linear control system. Here, the critical choice is that we must choose a discretization time interval length that is small enough so that the approximation remains valid. The system must not move outside of the region of approximation validity (i.e. the neighborhood where the approximation error induced by linearization is within the desired bound) during the interval between taking samples. As long as we sample fast enough, we are free to design our control laws in discrete-time.

5 Tracking Trajectories

You've seen in the homework that we can do linear feedback control to keep a linear system stably following a desired chosen open-loop trajectory. Looking at the equations earlier, we can similarly consider what happens when $\vec{x}^*$ is no longer constant, and so we want $\vec{x}(t)$ to follow some desired trajectory $\vec{x}^*(t)$. Then to get the same linear system as (2), we will want operating points such that

$$\frac{\mathrm{d}}{\mathrm{d}t}\vec{x}^*(t) = \vec{f}(\vec{x}^*(t), \vec{u}^*(t)). \tag{3}$$

How do we find such nominal solutions? That is a bit out of the scope of this course but it turns out that the same kinds of iterative numerical approaches that work for finding equilibrium points can be leveraged here as well. We linearize locally, solve, and then relinearize and resolve repeatedly.

However, there is one more challenge that occurs when we use linearization around a nominal trajectory — the resulting A and B matrices are now generically functions of time. So does that mean we are doomed? Not at all! While one has to be careful, often we can succeed by taking the same modeling philosophy that we have taken so far. If we assume that we are sampling the system fast enough, we can hope that the A and B matrices don't change too much from one step to the next. As a result, we can approximate the A matrices as being locally constant over many samples and just fold the approximation error into the disturbances. Here, we need to be a bit more careful because any error in A (due to time-variation) multiplies the deviation of the state and we need the product to stay small. If the state is small and the Frobenius norm of the change in A is small, the product will also be small. Similarly for B — but here we must ensure that the control that we apply times the change in B stays small. This means that the feedback controls should be gentle enough. You will learn how to navigate the resulting tension in more advanced control courses.

Finally, what about planning a nominal trajectory? Here, it turns out that the minimum-norm approach that you have learned can be iterated to get plans. The reason is that there was nothing in the approach that required the A and B matrices to stay constant. So it is possible to guess a set of controls to get to a destination, linearize, find a minimum norm solution, and then iterate this process to get a plan. Not all starting guesses will work, and so sometimes, more brute force guessing or smarter heuristics have to be used to speed this process up. This is also a topic for more advanced robotics courses.

Contributors:

- Rahul Arya.
- Anant Sahai.
- Ashwin Vangipuram.

EECS 16B Designing Information Devices and Systems II

Note 16: Classification

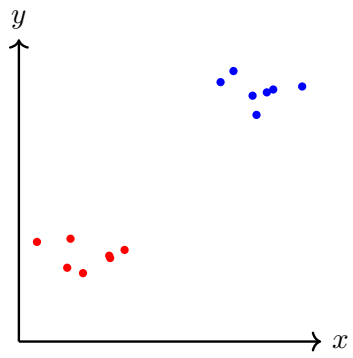
Overview

In this note, we will look at various related techniques for learning how to *classify* data. Classification is when we want to take an observation and return a discrete label corresponding to that observation. You have already seen this idea in 16A during the positioning module. Given recorded data, you had to decide whether a particular Gold Code was present or not — this is called a problem of binary classification since there are two labels: "present" or "not present." But you also had to decide which shift was present — this also involved choosing among discrete alternatives and so can be considered a classification problem. It is not binary however since there are more than two choices.

In this note, we will focus on the problem of *binary classification* — when there are two alternatives. We do this to focus on the issue of how to learn the classification rule from data. In 16A's positioning module, you already knew the codes and designed the rules to classify incoming data. Here, we want to learn the key parameters of the rules from training data: lots of labeled examples of each class. This training data plays the same role[1] that system input-output traces played in system identification.

1 Known Approaches

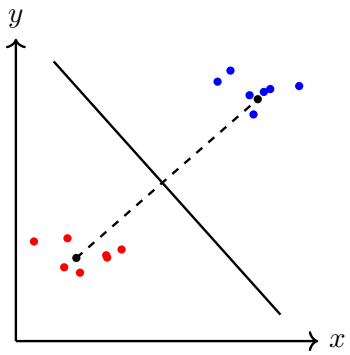
Before we do anything, let's see what this problem looks like visually. Imagine that our data is two-dimensional, and our points look something like the following:



The categories of our points are represented by their color. It is clear that our points can be separated by a line passing from the top-left to the bottom-right of our diagram.

[1] Notice the rough parallels here to what we have seen in control. We could get a discrete-time model and control law for a system by discretizing known continuous-time dynamics given by a differential equation and then doing control design for it. The continuous-time differential equation represents an underlying known model for ground-truth. But we also know what to do if we don't have that. We could use system identification to learn the discrete-time model from data obtained from traces of state input pairs. In 16A, we already knew the underlying model that generates the data that we would be classifying — the Gold Codes. Here, we are going to be seeing how to do the counterpart of system identification for classification.

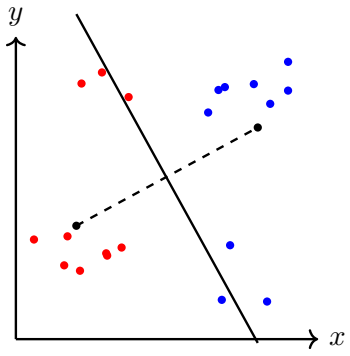
How could we construct such a line? One natural thing to do would be to take the average location of the points in each of the two categories, connect them with a segment, and then draw the perpendicular bisector of that segment, as shown below:



This strategy often works reasonably well. How can we understand what is going on here in a more systematic way? First, it is pretty clear that we are using the average location of the points in a category as a kind of paradigmatic representative for that category. This is easy to learn from training data. Then, what we are doing for classification is asking which paradigmatic representative we are closer to. We assign a point to the category which has the closest[2] paradigmatic representative. It is a basic fact of geometry that for two points, the points that are equidistant to both of them are the hyperplane that is the perpendicular bisector of the segment between the two points. Hence, such planes are useful for binary classification.

So are we done? To understand whether we need something better, we need to explore what can happen. We stick with the assumption that a hyperplane (or in the 2D case, a line) is the underlying right answer for how to separate two categories. However, the data that we learn from need not be tightly clustered around paradigmatic representatives. What else could happen?

For instance, consider the following sets of points:



It is clear visually that the best dividing line is a vertical line passing between the two categories. That cleanly splits the two categories. But because the observed averages (also called "means" in the literature to reflect language used in probability) of the two categories are not at the same height, their perpendicular bisector slopes downwards, meaning that it does not divide the data points correctly.

[2] If you recall, this was also the motivation used in 16A as a precursor to the maximum correlation approach. You noticed that $\|\vec{x}-\vec{t}\|^2 = \vec{x}^\top\vec{x} + \vec{t}^\top\vec{t} - 2\vec{t}^\top\vec{x}$. If we assumed that all of the templates $\vec{t}$ had the same length, then *minimizing* this distance to the templates was the same as *maximizing* the correlation with the templates. (After all, the $\vec{x}^\top\vec{x}$ is the same for each of the potential templates. So the only thing that is different for different templates (all of which have the same length) is the correlation.) If you think about it, this is actually a way that you could have discovered the Euclidean inner product as a concept for yourself.

Fundamentally, this "means-based" approach to learning does not work well in these kinds of cases there is some information in the distribution that can't be accurately captured by the mean of the data points of each category — it just calculates the averages. It also doesn't care about whether points are being classified correctly, since the separating line stays the same as long as the means are the same, even when it causes training data points to be misclassified.

2 Directly learning the rule

What this suggests is that we should try to learn the classification rule directly. For binary classification, a separating line is reasonable. How can we express this mathematically? If we think of the n-dimensional inputs that we want to classify as $\vec{x}$, this means that we want to compute $\sum_{i=1}^{n} w[i]x[i]$, where $w[i]$ and $x[i]$ are the i^{th} entry of $\vec{w}$ and $\vec{x}$ respectively, and compare it to some threshold. This is equivalent to computing $w[0]+\sum_{i=1}^{n} w[i]x[i]$ and comparing it to 0 — i.e. looking at the sign of the result. For notational convenience, we can augment our input with a 1 in the zeroth position and then what we want to learn is an $n+1$ dimensional vector $\vec{w}$.

How do we learn an $n+1$-dimensional vector $\vec{w}$ given a bunch of data? This sounds exactly like what we had to do in system identification, and so we might as well do what we know how to do — least squares.

But to do least-squares, we need to figure out how to get the "$\vec{y}$" that least squares is going to project onto an appropriate subspace to get the estimate for $\vec{w}$. In system-identification, the observed next states provided that information. After all, the goal of a system model is to predict the next state given the current state and the input. What should play that role here?

What we have are the binary labels. This is categorical information. What we need are target y values that are real numbers. Since we're going to use the sign of $\vec{x}^\top\vec{w}$ to determine the classification, it is natural to assign all the points in one category the target value -1, and all the points in the other the target value $+1$. Then we use least squares to construct an affine function that minimizes the squared error between its predictions and the actual values. We then draw our dividing line by looking at where our function equals 0.

To be explicit: let our raw observed training points be n-dimensional vectors $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_m$. We silently modify our data points to become:

$$\begin{bmatrix} 1 \\ \vec{x}_1 \end{bmatrix}, \begin{bmatrix} 1 \\ \vec{x}_2 \end{bmatrix}, \cdots, \begin{bmatrix} 1 \\ \vec{x}_m \end{bmatrix},$$

so that we can use the sign of $\vec{x}^\top\vec{w}$ to classify points.

Our target linear function is of the form $f(\vec{x}) = \vec{w}^\top\vec{x}$ for some unknown $\vec{w}$. Note that, if we want an affine function, to allow for a constant offset by varying the first entry of $\vec{w}$. Now, let the binary labels of our sample points be $\ell_1, \ldots, \ell_m$, where $\ell_i = +1$ if $\vec{x}_i$ is a point in the category we deem[3] "positive" and $\ell_i = -1$ if $\vec{x}_i$ is a point in the category we deem "negative."

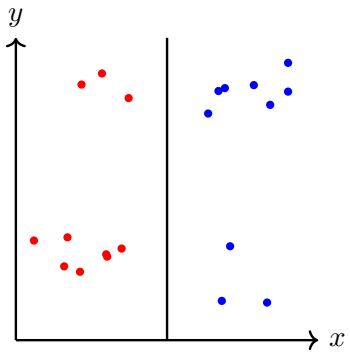
The standard least squares approach to classification is to set up the approximate equation:

$$\begin{bmatrix} \vec{x}_1^\top \\ \vec{x}_2^\top \\ \vdots \\ \vec{x}_m^\top \end{bmatrix} \vec{w} \approx \begin{bmatrix} \ell_1 \\ \ell_2 \\ \vdots \\ \ell_m \end{bmatrix} \tag{1}$$

[3] "+" and "-" are conventional names for these binary categories, but in actual situations, these could refer to any pair of mutually exclusive alternatives. "Cats" vs "Dogs" or "Red" vs "Blue." Anything.

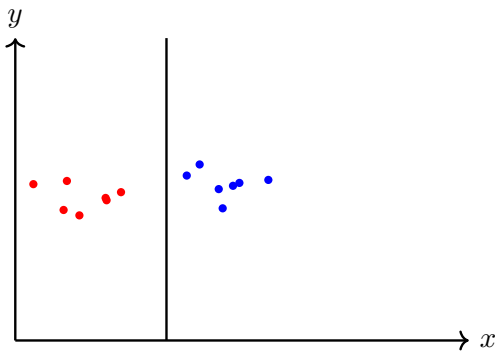
and solve it to get the solution $\vec{w}$ that minimizes $\sum_{i=1}^{m}(\vec{x}_i^\top \vec{w} - \ell_i)^2$.

We can try it for our example to see the resulting dividing line

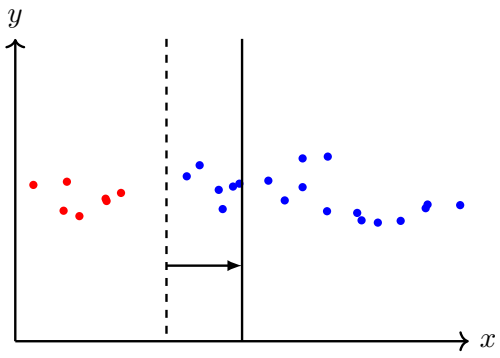


and this dividing line seems pretty good.

So are we done? Unfortunately not. Consider the following dataset:

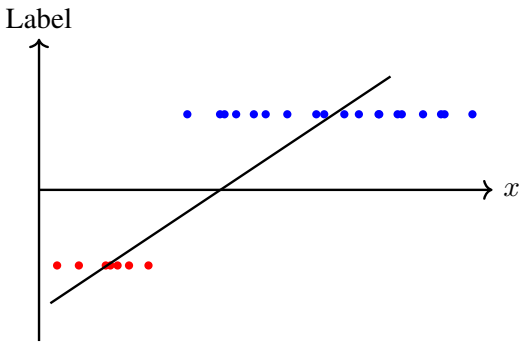


Least squares works correctly, as we expect, and constructs a dividing line. Now let's add some more points far to the right that are in the same $+1$ category. Our existing dividing line *already* classifies them correctly, as they are clearly already on the right side of the line. But what happens when we run least squares again?



As shown above, our new points "drag" our dividing line to the right, making our classifier *worse* than before!

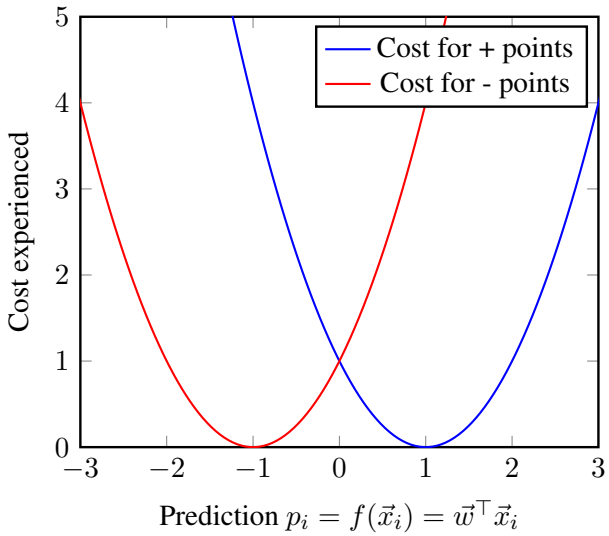
Why is least squares failing us? To see the problem, we will plot the x-coordinates of our points against their labels ($+1$ or -1), along with the line of best fit, omitting the y-coordinate since we clearly do not want it to be important for classification in this example:



Observe that the slope of the line of best fit is "dragged down" in order to avoid overestimating the labels of the points furthest to the right. However, this causes the dividing line to move to the right, making our linear classifier less accurate on the training data.

From the internal point of view of least-squares, the quantity $f(\vec{x}) = \vec{x}^\top \vec{w}$ is supposed to be a real number that it is trying to predict correctly by picking the right weights $\vec{w}$. Meanwhile, what we actually care about is getting the sign correct. Least-squares is minimizing the sum of the squared differences of the prediction and the true label (it is called "least squares" after all). So given a point with label $+1$, a prediction of -1 or $+3$ would both be penalized by a cost of $2^2 = 4$. But the former prediction is clearly much worse than the latter, since the first leads to a misclassification of the point while the second gives a prediction whose classification agrees with the training data.

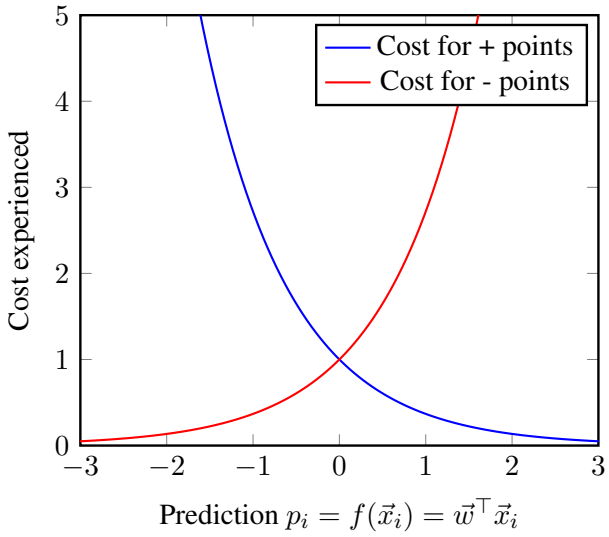
We can illustrate this behavior graphically. Let $p_i = \vec{x}_i^\top \vec{w}$ be the predicted value of point i, and let ℓ_i be its label. Plotting the cost function $(p_i - \ell_i)^2$ for points labeled -1 and $+1$, we obtain



The problem here comes from the cost curves bending back upwards where they really shouldn't. To achieve our goal of classification, we have no desire to penalize strong predictions that have the correct sign. But just using a quadratic cost makes us do this.

3 Exponential Cost Function

Ideally, we'd like a cost function that only severely penalizes errors that could lead to misclassifications. One such function is the following: for points in the $+1$ category, we use e^{-p_i}, and for points in the -1 category, we use e^{+p_i}. Plotting these functions, we obtain:



These look a lot better! For points in the $+1$ category, the predictions of -1 and $+3$ are no longer penalized equally: -1 has a high cost, while $+3$ is essentially not penalized at all, even though both correspond to an absolute error of 2 from the true label. So the issue we faced earlier about points far to one side of the dividing line has been resolved. This modification is known as using an *exponential cost function*.

Now, let's look at how we can compute the classifier that minimizes this new cost function. This is not least-squares anymore.

Let the cost function corresponding to the ith point be $c_i(p)$. In the case of quadratic-cost linear regression, this cost function looks like

$$c_i(p) = (p - \ell_i)^2.$$

Now, with our exponential cost function, our cost function becomes

$$c_i(p) = e^{-\ell_i p}.$$

Since in either case our goal is to pick the linear classifier with minimum cost, we can express our problem as trying to compute

$$\underset{\vec{w}}{\operatorname{argmin}} \left(\sum_{i=1}^{m} c_i(f(\vec{x}_i)) \right) = \underset{\vec{w}}{\operatorname{argmin}} \left(\sum_{i=1}^{m} c_i(\vec{w}^\top \vec{x}_i) \right).$$

The question is how can we solve such problems. In 16A, we were able to use geometry and the Pythagorean theorem to solve the least-squares case. Can we somehow leverage that work for more general cases?

4 Quadratic Approximations

How can we compute the desired $\vec{w}$? If our cost function were quadratic, we know how to use least squares to immediately identify the minimizing value of $\vec{w}$. But with our exponential cost function, or with some other arbitrary cost function, least squares doesn't give us the desired answer immediately anymore.

Rather than trying to solve the problem in one shot, we will instead try to develop an iterative way of computing the solution. Given a particular value of $\vec{w}$, we will attempt to make an improvement $\vec{\delta w}$ to $\vec{w}$ that lowers our cost. Then we will repeat this process, continuously updating $\vec{w}$ until we converge to a minimum.

How can we find such an improvement? Well, we know how to minimize quadratic costs using least squares. So if we approximate our cost function by a quadratic near a particular $\vec{w}$, we can hope to use least squares to give us an improvement. Then re-approximate the cost function near the improved $\vec{w}$, and repeat!

So the main remaining task is coming up with a quadratic approximation to our cost function. For convenience, let

$$c(\vec{w}) = \sum_{i=1}^{m} c_i(\vec{x}_i^\top \vec{w}),$$

so we are simply trying to compute

$$\underset{\vec{w}}{\operatorname{argmin}} \left(c(\vec{w})\right).$$

Let's review what we know that could be helpful. One thing we know is how to *linearize* $c(\vec{w})$ around a particular expansion point $\vec{w}_*$. We know that such a linear approximation looks like

$$c(\vec{w}_* + \vec{\delta w}) \approx c(\vec{w}_*) + \begin{bmatrix} \frac{\partial c}{\partial \vec{w}[1]} & \cdots & \frac{\partial c}{\partial \vec{w}[n]} \end{bmatrix} \vec{\delta w},$$

where all the partial derivatives are taken at $\vec{w} = \vec{w}_*$.

Another thing we know is how to compute a quadratic approximation for a *scalar* function $f(x)$ using the Taylor series about some point x_*, where

$$f(x_* + \delta x) \approx f(x_*) + \left(\frac{\mathrm{d}f}{\mathrm{d}x}\bigg|_{x=x_*}\right)(\delta x) + \frac{1}{2}\left(\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\bigg|_{x=x_*}\right)(\delta x)^2.$$

Our plan will be to try and understand where the quadratic term of the Taylor expansion comes from, and then apply similar reasoning to determine the quadratic term in our expansion for $c(\vec{w})$.

One argument is that we want our approximation of $f(x)$ to have the same value, first, and second derivatives at $x = x_*$, and it is clearly the only quadratic polynomial with that property. While this argument makes sense, it is not particularly useful since it does not obviously or immediately generalize to our multi-dimensional case.

Alternatively, we can ask ourselves what the purpose of the quadratic term is. With just the linear term, we model $f(x)$ as a linear function passing through $(x_*, f(x_*))$ with constant slope. The quadratic term essentially allows us to better approximate $f(x)$ by treating its *slope* as a linear function. We can see this by first approximating the derivative of f as a linear function

$$\frac{\mathrm{d}f}{\mathrm{d}x}\bigg|_{x=x_*+\delta x} \approx \left(\frac{\mathrm{d}f}{\mathrm{d}x}\bigg|_{x=x_*}\right) + \left(\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\bigg|_{x=x_*}\right)(\delta x).$$

We then integrate with respect to δx from x_* to $x_* + \delta x$ to see that

$$f(x_* + \delta x) - f(x_*) = \left(\frac{\mathrm{d}f}{\mathrm{d}x}\bigg|_{x=x_*}\right)(\delta x) + \frac{1}{2}\left(\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\bigg|_{x=x_*}\right)(\delta x)^2,$$

which when rearranged gives us our quadratic approximation. This "integration" is computing an area. The $\frac{1}{2}$ comes from the area of a triangle — the average slope over the duration δx isn't the initial slope $\frac{\mathrm{d}f}{\mathrm{d}x}\Big|_{x=x_*}$ nor is it the estimate for the final slope $\frac{\mathrm{d}f}{\mathrm{d}x}\Big|_{x=x_*} + \left(\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\Big|_{x=x_*}\right)\delta x$. It is halfway in between: $\frac{\mathrm{d}f}{\mathrm{d}x}\Big|_{x=x_*} + \frac{1}{2}\left(\frac{\mathrm{d}^2 f}{\mathrm{d}x^2}\Big|_{x=x_*}\right)\delta x$. Multiplying this average slope by the duration δx gives rise to the approximate update to the function value $f(x_*)$ to give us an estimate of $f(x_* + \delta x)$.

This line of reasoning is more amenable to generalization to functions of vectors. Consider the derivative of $c(\vec{w})$. We have that

$$\frac{\mathrm{d}c}{\mathrm{d}\vec{w}} = \begin{bmatrix} \frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]} \end{bmatrix}.$$

This is a function that takes in a column vector $\vec{w}$ (representing where we want to evaluate this derivative) and returns a row vector. We do not yet know how to linearize such a function. However, we *do* know how to linearize functions that take in column vectors and return *column* vectors. How can we get such a function? Simple — we take the transpose of the derivative! Doing so, we obtain

$$\left(\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\right)^\top = \begin{bmatrix} \frac{\partial c}{\partial w[1]} \\ \vdots \\ \frac{\partial c}{\partial w[n]} \end{bmatrix}.$$

Now we can linearize this about $\vec{w} = \vec{w}_*$, to obtain

$$\left(\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\bigg|_{\vec{w}=\vec{w}_*+\vec{\delta w}}\right)^\top \approx \begin{bmatrix} \frac{\partial c}{\partial w[1]} \\ \vdots \\ \frac{\partial c}{\partial w[n]} \end{bmatrix}\bigg|_{\vec{w}=\vec{w}_*} + \begin{bmatrix} \frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]\partial w[1]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[1]\partial w[n]} & \cdots & \frac{\partial c}{\partial w[n]\partial w[n]} \end{bmatrix}\bigg|_{\vec{w}=\vec{w}_*} \vec{\delta w}.$$

Finally, we can take transposes once more to obtain

$$\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\bigg|_{\vec{w}=\vec{w}_*+\vec{\delta w}} \approx \begin{bmatrix} \frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]} \end{bmatrix}\bigg|_{\vec{w}=\vec{w}_*} + (\vec{\delta w})^\top \begin{bmatrix} \frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]\partial w[n]} \end{bmatrix}\bigg|_{\vec{w}=\vec{w}_*}.$$

What next? Now that we've got a more accurate model of the derivative of our function, how can we use it to develop the desired quadratic approximation? In the purely scalar case, we could simply integrate, and the quadratic approximation popped out. However, we can't do the same thing here, since our functions all take in multiple arguments.

Let's think about what should happen at an intuitive level. Imagine trying to compute $c(\vec{w}_* + \vec{\delta w})$ using this new model of its derivative. We want to figure out how much $c(\vec{w})$ changes as we move $\vec{w}$ from $\vec{w}_*$

to $\vec{w}_* + \vec{\delta w}$. How a function changes under small perturbations, of course, is governed by its derivative. The first term in our above linear approximation is a constant, so it will contribute equally throughout this movement. The second term, however, grows linearly with $\vec{\delta w}$. So it will not contribute at all to begin with, but will contribute proportional to $\vec{\delta w}$ at the end of the movement. This suggests a "triangle-like" behavior similar to what we saw in the scalar case, where we should halve its impact to account for its linear growth over the movement. Basically, look at the average derivative for the actual range traveled in the input.

Thus, our intuition suggests that we can write

$$c(\vec{w}_* + \vec{\delta w}) - c(\vec{w}_*) = (\left[\begin{array}{ccc} \frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]} \end{array}\right]\Big|_{\vec{w}=\vec{w}_*} + \frac{1}{2}(\vec{\delta w})^\top \left[\begin{array}{ccc} \frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]} \end{array}\right]\Big|_{\vec{w}=\vec{w}_*})(\vec{\delta w}) \tag{2}$$

$$= \left[\begin{array}{ccc} \frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]} \end{array}\right]\Big|_{\vec{w}=\vec{w}_*} (\vec{\delta w}) + \frac{1}{2}(\vec{\delta w})^\top \left[\begin{array}{ccc} \frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]} \end{array}\right]\Big|_{\vec{w}=\vec{w}_*} (\vec{\delta w}) \tag{3}$$

which can be rearranged to give us a quadratic approximation for $c(\vec{w})$ near $\vec{w} = \vec{w}_*$.

The matrix $\left[\begin{array}{ccc} \frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]} \end{array}\right]\Big|_{\vec{w}=\vec{w}_*}$ in the quadratic term is known as the *Hessian*, often denoted by H. It is important to note that it is *not*, strictly speaking, the second derivative of c with respect to $\vec{w}$. *If* we were able to linearize

$$\left(\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\Big|_{\vec{w}=\vec{w}_*+\vec{\delta w}}\right) - \left(\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\Big|_{\vec{w}=\vec{w}_*}\right) \approx A(\vec{\delta w}),$$

using some matrix A, then A would be our desired second derivative, since we could post-multiply it by $\vec{\delta w}$ to obtain the change in the derivative over a small perturbation. But instead we can only write

$$\left(\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\Big|_{\vec{w}=\vec{w}_*+\vec{\delta w}}\right) - \left(\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\Big|_{\vec{w}=\vec{w}_*}\right) \approx (\vec{\delta w})^\top H,$$

so the Hessian matrix is not quite the second derivative, though it can be thought of as a *representation*[4] of it.

Now that we have developed the necessary intuition to define a quadtratic approximation of interest, we can "check our work." How can we "check our work" in this case? By seeing if what we got is consistent with what we know about scalar functions. Essentially, we know we can treat $c(\vec{w})$ as a scalar function of a

[4]This issue of representing derivatives is important and it turns out, at some point, it will no longer be possible to represent derivatives using ordinary vectors and matrices. But if you've internalized the spirit of 16AB, you will have no trouble coming up with the relevant definitions to deal with the multidimensional arrays that will arise as you consider higher-order derivatives. Although they are out of the scope of this course, if you follow this path forward, you will naturally discover higher-order *tensors* for yourself. The key is just to not confuse derivatives with their representatives.

scalar argument defined by only evaluating $c(\vec{w})$ along a particular direction starting at $\vec{w}_*$ and compute the quadratic approximation of this scalar function of a scalar argument using the Taylor series. We will then rewrite that approximation to show that it agrees with the above approximation.

To do so, let $\vec{r}$ be a unit vector pointing along our arbitrary direction, and let t be a (presumably) small scalar argument. Thus, $t\vec{r}$ is a small perturbation in the direction of $\vec{r}$. By our linearization of the derivative of $c(\cdot)$,

$$\begin{aligned}\left.\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\right|_{\vec{w}=\vec{w}_*+t\vec{r}} &\approx \left.\begin{bmatrix}\frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]}\end{bmatrix}\right|_{\vec{w}=\vec{w}_*} + (t\vec{r})^\top \left.\begin{bmatrix}\frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]}\end{bmatrix}\right|_{\vec{w}=\vec{w}_*} \\ &= \left.\begin{bmatrix}\frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]}\end{bmatrix}\right|_{\vec{w}=\vec{w}_*} + (\vec{r})^\top \left.\begin{bmatrix}\frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]}\end{bmatrix}\right|_{\vec{w}=\vec{w}_*} t.\end{aligned}$$

Now, we can define the scalar function of a scalar argument $c_r(t) = c(\vec{w}_* + t\vec{r})$. It is clear that

$$\frac{\mathrm{d}c_r}{\mathrm{d}t} = \left(\left.\frac{\mathrm{d}c}{\mathrm{d}\vec{w}}\right|_{\vec{w}=\vec{w}_*+t\vec{r}}\right)\vec{r},$$

so by our linearization

$$\frac{\mathrm{d}c_r}{\mathrm{d}t} \approx \begin{bmatrix}\frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]}\end{bmatrix}\vec{r} + (\vec{r})^\top \begin{bmatrix}\frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]}\end{bmatrix}(\vec{r})t.$$

Now, we can integrate with respect to t, to find that

$$\begin{aligned}c_r(t) - c_r(0) &\approx \begin{bmatrix}\frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]}\end{bmatrix} t\vec{r} + \frac{1}{2}(\vec{r})^\top \begin{bmatrix}\frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]}\end{bmatrix}(\vec{r})t^2 \\ &= \begin{bmatrix}\frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]}\end{bmatrix} t\vec{r} + \frac{1}{2}(t\vec{r})^\top \begin{bmatrix}\frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]}\end{bmatrix}(t\vec{r}).\end{aligned}$$

Substituting back our definition of $c_r(t)$, defining $\delta\vec{w} = t\vec{r}$, and rearranging, we find that

$$c(\vec{w}_* + \delta\vec{w}) \approx c(\vec{w}_*) + \begin{bmatrix}\frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]}\end{bmatrix}(\delta\vec{w}) + \frac{1}{2}(\delta\vec{w})^\top \begin{bmatrix}\frac{\partial^2 c}{\partial w[1]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[1]\partial w[n]} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial w[n]\partial w[1]} & \cdots & \frac{\partial^2 c}{\partial w[n]\partial w[n]}\end{bmatrix}(\delta\vec{w}),$$

which agrees. Since we can create any small $\delta\vec{w}$ by appropriately choosing t and $\vec{r}$, the above approximation

is reasonable for any small $\vec{\delta w}$. Thus, we have shown how the multivariate quadratic approximation is consistent with the Taylor series in the scalar case. Such consistency is always a good way to sanity check our definitions and approximations.

5 Iteratively Reweighted Least Squares

Now that we know how to come up with a quadratic approximation for a general function $c(\vec{w})$, let's see how to use it in our case. One approach is natural — we should iterate. We have seen this kind of iterative pattern before. The idea is to approximate the problem as something we know how to solve, solve the approximate problem, and then pass the solution on update the approximation and repeat the process. In effect, we are counting on the iterations to take care of the approximation error. Explicitly, we start with a candidate solution $\vec{w}_*$, and want to first come up with a quadratic approximation around it, and then find its minimizing $\vec{\delta w}$ in order to improve our solution. We defined

$$c(\vec{w}) = \sum_{i=1}^{m} c_i(\vec{w}^\top \vec{x}_i).$$

To approximate this function around a point, we clearly need to know its partial derivatives. Differentiating first with respect to the gth component of $\vec{w}$, we can use the standard univariate chain rule you know from calculus to obtain

$$\frac{\partial c}{\partial w[g]} = \sum_{i=1}^{m} c_i'(\vec{w}^\top \vec{x}_i) \left(\frac{\partial}{\partial w[g]} (\vec{w}^\top \vec{x}_i) \right).$$

Note that we use $c_i'(\vec{w}^\top \vec{x}_i)$ to represent the derivative of $c_i(p)$ with respect to p, evaluated at $\vec{w}^\top \vec{x}_i$. $c_i''(\cdot)$ is the analogous notation that we will use later to represent the second derivative. We use this notation for compactness later in the calculation.

Now, observe that we can expand out the inner product to obtain

$$\begin{aligned} \vec{w}^\top \vec{x}_i &= w[1]x_i[1] + w[2]x_i[2] + \cdots + w[g]x_i[g] + \cdots + w[n]x_i[n] \\ \implies \quad \frac{\partial}{\partial w[g]} (\vec{w}^\top \vec{x}_i) &= x_i[g], \end{aligned}$$

since all the other terms in the summation do not vary with $w[g]$. Thus,

$$\frac{\partial c}{\partial w[g]} = \sum_{i=1}^{m} c_i'(\vec{w}^\top \vec{x}_i) x_i[g].$$

Of course, to compute the Hessian, we require the second partial derivatives as well. Differentiating the

above quantity with respect to $w[h]$, we obtain

$$\begin{aligned}\frac{\partial^2 c}{\partial w[h]\partial w[g]} &= \frac{\partial}{\partial w[h]}\sum_{i=1}^{m} c_i'(\vec{w}^\top \vec{x}_i)x_i[g] \\ &= \sum_{i=1}^{m} \frac{\partial}{\partial w[h]}\left(c_i'(\vec{w}^\top \vec{x}_i)x_i[g]\right) \\ &= \sum_{i=1}^{m} x_i[g]\frac{\partial}{\partial w[h]}c_i'(\vec{w}^\top \vec{x}_i) \\ &= \sum_{i=1}^{m} x_i[g]c_i''(\vec{w}^\top \vec{x}_i)\frac{\partial}{\partial w[h]}(\vec{w}^\top \vec{x}_i),\end{aligned}$$

using the chain rule once again. Note that we pulled $x_i[g]$ out of the derivative as a constant, since it does not vary with $w[h]$. Now, we can use our above result on the partial derivatives of $\vec{w}^\top \vec{x}_i$, to find that

$$\frac{\partial^2 c}{\partial w[h]\partial w[g]} = \sum_{i=1}^{m} c_i''(\vec{w}^\top \vec{x}_i)x_i[g]x_i[h].$$

Next, we will use this result to compute the Hessian. Observe that the entry at the gth row and hth column of the Hessian is

$$\begin{aligned}H[g][h] &= \left.\frac{\partial^2 c}{\partial w[h]\partial w[g]}\right|_{\vec{w}=\vec{w}_*} \\ &= \sum_{i=1}^{m} c_i''(\vec{w}_*^\top \vec{x}_i)x_i[g]x_i[h] \\ &= \sum_{i=1}^{m} c_i''(\vec{w}_*^\top \vec{x}_i)(\vec{x}_i\vec{x}_i^\top)[g][h],\end{aligned}$$

since $(\vec{x}_i\vec{x}_i^\top)[g][h] = x_i[g]x_i[h]$. Thus, we can express the matrix

$$H = \sum_{i=1}^{m} c_i''(\vec{w}_*^\top \vec{x}_i)\vec{x}_i\vec{x}_i^\top.$$

Finally, we can plug back in to find the quadratic approximation of $c(\vec{w})$ near $\vec{w} = \vec{w}_*$. Using our result from the previous section,

$$\begin{aligned}c(\vec{w}_* + \vec{\delta w}) &= c(\vec{w}_*) + \begin{bmatrix}\frac{\partial c}{\partial w[1]} & \cdots & \frac{\partial c}{\partial w[n]}\end{bmatrix}(\vec{\delta w}) + \frac{1}{2}(\vec{\delta w})^\top H(\vec{\delta w}) \\ &= c(\vec{w}_*) + \sum_{g=1}^{n}\sum_{i=1}^{m} c_i'(\vec{w}_*^\top \vec{x}_i)x_i[g](\vec{\delta w})[g] + \frac{1}{2}(\vec{\delta w})^\top \sum_{i=1}^{m} c_i''(\vec{w}_*^\top \vec{x}_i)\vec{x}_i\vec{x}_i^\top(\vec{\delta w}) \\ &= c(\vec{w}_*) + \sum_{i=1}^{m} c_i'(\vec{w}_*^\top \vec{x}_i)\sum_{g=1}^{n} x_i[g](\vec{\delta w})[g] + \frac{1}{2}\sum_{i=1}^{m} c_i''(\vec{w}_*^\top \vec{x}_i)(\vec{\delta w})^\top \vec{x}_i\vec{x}_i^\top(\vec{\delta w}) \\ &= c(\vec{w}_*) + \sum_{i=1}^{m}\left[c_i'(\vec{w}_*^\top \vec{x}_i)\vec{x}_i^\top(\vec{\delta w}) + \frac{1}{2}c_i''(\vec{w}_*^\top \vec{x}_i)(\vec{\delta w})^\top \vec{x}_i\vec{x}_i^\top(\vec{\delta w})\right].\end{aligned}$$

This doesn't look particularly meaningful yet. Let's see what we can do to simplify it. First, notice that the leading term $c(\vec{w}_*)$ obviously does not depend on $\vec{\delta w}$. Since we are interested only in the value of $\vec{\delta w}$ that minimizes the overall cost, we can therefore ignore the leading term since it does not affect the minimizing value of $\vec{\delta w}$. Furthermore, observe that

$$(\vec{\delta w})^\top \vec{x}_i \vec{x}_i^\top (\vec{\delta w}) = ((\vec{\delta w})^\top \vec{x}_i) \times (\vec{x}_i^\top (\vec{\delta w})) = (\vec{x}_i^\top (\vec{\delta w}))^2,$$

since the two terms in the product are both simply the same scalar. Thus, we can rewrite the quantity we are trying to minimize as

$$\sum_{i=1}^{m} \left[c_i'(\vec{w}_*^\top \vec{x}_i)(\vec{x}_i^\top (\vec{\delta w})) + \frac{1}{2} c_i''(\vec{w}_*^\top \vec{x}_i)(\vec{x}_i^\top (\vec{\delta w}))^2 \right].$$

Observe that $\vec{x}_i^\top (\vec{\delta w})$ is just a scalar. Recall that when working with learning linear classifiers using least-squares, the expression $\vec{x}_i^\top \vec{w}$ came up a lot inside summations over i. Furthermore, our above expression to minimize now only has linear and quadratic terms in that same expression. Thus, it is natural to try and fit it into the form of the least squares cost function, using $\vec{\delta w}$ as the vector of weights that we want to learn.

To do so, we should first ask ourselves: what is our goal? What does the least squares cost function look like? Since we want to minimize the sum of the squared residuals, the least squares cost function looks like

$$\sum_{i=1}^{m} (b_i - \vec{a}_i^\top (\vec{\delta w}))^2,$$

where our points are the $\vec{a}_i$, the values we are trying to model are the b_i, and the parameters of our least squares model are in the vector $\vec{\delta w}$.

So we need to get a square of the form $(b_i - a_i^\top (\vec{\delta w}))^2$ inside our summation somehow, given linear and quadratic terms in $\vec{x}_i^\top (\vec{\delta w})$. You should recall that you know a technique that can do exactly this — completing the square! This was something that you saw when you leaned the quadratic formula to find the roots of a quadratic equation — the goal is to write a quadratic[5] as a perfect square plus a constant. When you learned the quadratic formula, the goal was to say that once an expression is that form, the roots are obvious. You just have to set the perfect square equal to the negative of the constant, and so you'll get two roots.

Completing the square of the summands, we see that

$$c_i'(\vec{w}_*^\top \vec{x}_i)(\vec{x}_i^\top (\vec{\delta w})) + \frac{1}{2} c_i''(\vec{w}_*^\top \vec{x}_i)(\vec{x}_i^\top (\vec{\delta w}))^2 = \frac{c_i''(\vec{w}_*^\top \vec{x}_i)}{2} \left(\vec{x}_i^\top (\vec{\delta w}) + \frac{c_i'(\vec{w}_*^\top \vec{x}_i)}{c_i''(\vec{w}_*^\top \vec{x}_i)} \right)^2 - \frac{(c_i'(\vec{w}_*^\top \vec{x}_i)^2)}{2 \times c_i''(\vec{w}_*^\top \vec{x}_i)}.$$

Notice again that the trailing additive constant does not depend on $\vec{\delta w}$, so it can be disregarded when trying to minimize. Similarly, we can drop the leading factor of $1/2$ in the squared term. Doing so, and then plugging this new summand back into the overall expression to be minimized, we are ultimately interested in computing

$$\operatorname*{argmin}_{\vec{\delta w}} \left[\sum_{i=1}^{m} c_i''(\vec{w}_*^\top \vec{x}_i) \left(\vec{x}_i^\top (\vec{\delta w}) + \frac{c_i'(\vec{w}_*^\top \vec{x}_i)}{c_i''(\vec{w}_*^\top \vec{x}_i)} \right)^2 \right].$$

We're very nearly there! The only issue is the weights in front of each summand. Their presence is why the technique we are developing is commonly known as *iteratively reweighted least squares*, because these

[5] Explicitly: $ax^2 + bx + c = a(x^2 + \frac{b}{a}x + \frac{c}{a}) = a((x + \frac{b}{2a})^2 + \frac{c}{a} - \frac{b^2}{4a^2}) = a((x + \frac{b}{2a})^2 - \frac{b^2 - 4ac}{4a^2})$. This clearly has roots whenever $(x + \frac{b}{2a})^2 = \frac{b^2 - 4ac}{4a^2}$. In other words, $x + \frac{b}{2a} = \pm \frac{\sqrt{b^2 - 4ac}}{2a}$, or put into traditional form: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

weights are updated at each iteration of the algorithm, while the points themselves remain the same $\vec{x}_i$.

However, we do not yet know how to solve a weighted least squares problem, we only know how to solve the regular unweighted case. So how can we get rid of these weights? The key is to recognize that, in both the quadratic and exponential cases, the second derivative of our cost function is always positive. Thus, we can simply take the square root of the weight inside our squared term, to obtain

$$\underset{\vec{\delta w}}{\operatorname{argmin}} \left[\sum_{i=1}^{m} \left(\sqrt{c_i''(\vec{w}_*^\top \vec{x}_i)} \vec{x}_i^\top (\vec{\delta w}) - \frac{-c_i'(\vec{w}_*^\top \vec{x}_i)}{\sqrt{c_i''(\vec{w}_*^\top \vec{x}_i)}} \right)^2 \right].$$

Notice the double minus sign in the scalar part of the squared term. This is simply to more clearly put our expression into the standard form of the least squares cost function.

Expressed in words, at each iteration, we set up a least squares problem with the m point, value pairs:

$$\left(\sqrt{c_i''(\vec{w}_*^\top \vec{x}_i)} \vec{x}_i^\top, -\frac{c_i'(\vec{w}_*^\top \vec{x}_i)}{\sqrt{c_i''(\vec{w}_*^\top \vec{x}_i)}} \right)$$

for $i \in \{1, 2, \ldots, m\}$.

We put the coefficients of the solution to this least squares problem in a vector $\vec{\delta w}$, and update our weights to be $\vec{w} = \vec{w}_* + \vec{\delta w}$. We make these weights our new $\vec{w}_*$ and repeat the procedure until we converge on a solution.

We have not proven that this process will always converge, but it can be shown in later classes that this is in fact the case for the examples given so far — the key is that the second derivative is always positive. Taking it as given that it converges on some $\vec{w}^*$, we can solve for $(\vec{w}^*)^\top \vec{x} = 0$ to see what our decision boundary is for classification.

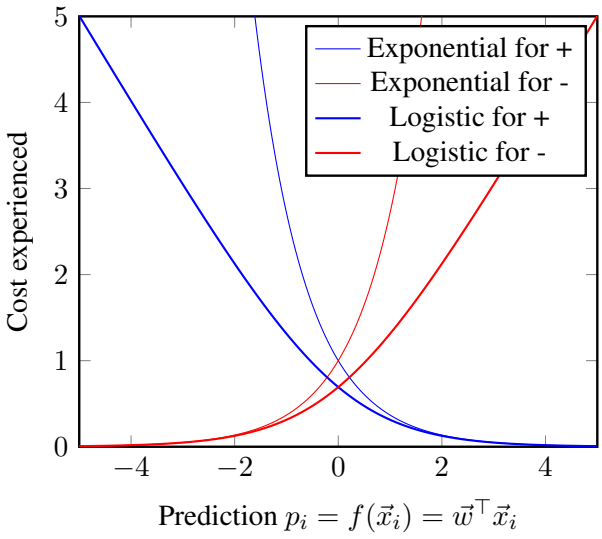
6 Choosing an even better cost function: deriving logistic regression

At this point, we actually have a very powerful generalization of least-squares at our disposal. We can use this iteratively reweighted procedure on pretty much any cost function whose second derivative is always positive. In the homework, you will explore this freedom and see that there is one more natural modification that we might want to make to our cost function. The issue is that the exponential cost function obsesses too much about points that are very largely misclassified. However, in many real contexts, we have some mislabeled points in our training data itself. It doesn't make sense to let those mislabeled points dominate the optimization. So the question is how can we mellow-out the exploding part of the exponential cost function without removing the very nice part. Explicitly, we like the part of e^{-p} that is decaying to zero, but don't like the exploding behavior on the negative p side.

The natural way to mellow out an exponential is to take a logarithm, but simply taking a logarithm of the exponential e^{-p} would give rise to $-p$ which is also not desirable because this cost can be made arbitrarily negative which doesn't make sense. A single correctly classified point shouldn't give unbounded happiness to the optimization. So what can be done? It turns out[6] that $\ln(1+x)$ has interesting behavior. For numbers x that are close to zero, $\ln(1+x) \approx x$. Meanwhile, if we consider a very big number x, then $\ln(1+x) \approx \ln(x)$ since $\ln(1+x) - \ln(x) = \ln\left(\frac{1+x}{x}\right) = \ln\left(1+\frac{1}{x}\right) \to 0$. So, this suggests using the cost function

[6] As the homework shows, this observation is natural once we consider bringing approximation thinking to Bode plots. But here, we will simply invoke this observation.

$c_i(p) = \ln\left(1 + e^{-\ell_i p}\right)$.



We can see the much mellower behavior of the logistic cost function in the plots above as compared to the exponential cost. It turns out that the almost linear behavior of the logistic cost functions in the misclassified region results in much better native resilience to outliers. A few mislabeled and crazily placed points just can't impact the chosen $\vec{w}$ that much. However the deeper reasons for that will have to wait for future courses like 127.

There are also deeper connections between probability and logistic regression, even though we have shown here that binary logistic regression can be naturally discovered for yourself on optimization-related grounds alone. Those connections too will have to wait for later courses.

7 Generalizing beyond binary classification (Bonus)

The ideas here can naturally generalize beyond the binary case to the maximum-correlation style perspective that you saw in 16A. The goal becomes to learn templates $\vec{w}_k$ corresponding to each of the K classes so that any given input $\vec{x}$ can be classified by computing the maximum correlation $\arg\max_k \vec{w}_k^\top \vec{x}$. Following the design pattern we have seen above, the key is choosing a loss function to optimize. The straightforward approach of doing binary least squares (or exponential or logistic) regression can be used to learn each of the $\vec{w}_k$ individually by treating the binary problem as "class k or not class k." But this ignores the fact that the final decision will involve taking a maximum.

Whereas logistic regression for the binary case emerges naturally, how to generalize it beyond the binary case is a bit out of the scope of 16B because many of the most natural ways to come up with a solution involve looking at the problem with the strategic perspective that probability provides, and probability is a topic for our successor course 70 (and realistically, for this intuition, you also need 126). However, it is possible to come up with a natural cost function that takes all the "predictions" $\vec{p}_i = \begin{bmatrix} \vec{w}_1^\top \vec{x}_i \\ \vec{w}_2^\top \vec{x}_i \\ \vdots \\ \vec{w}_K^\top \vec{x}_i \end{bmatrix} = \begin{bmatrix} \vec{w}_1^\top \\ \vec{w}_2^\top \\ \vdots \\ \vec{w}_K^\top \end{bmatrix} \vec{x}_i$

in at once[7]. Let us slightly refresh the notation to be more friendly to this case. Let the label ℓ_i for the i-th training data point $\vec{x}_i$ be a natural number from $1, 2, \ldots, K$. What we want is a cost function $c^\ell(\vec{p})$ that gives us a cost when the true label is ℓ and the vector of real predictions is $\vec{p}$. The desiderata are that the cost should be very low when $p[\ell]$ is actually the maximum with a clear advantage, and that the cost should be relatively higher when $p[\ell]$ is not the maximum. We also would like to essentially recover the case of logistic regression when $K = 2$.

Here, it is good to first wonder what the *two* predictions $p[1]$ and $p[2]$ should be for the case of binary logistic regression since earlier, we only had a single weight vector $\vec{w}$. If we consider the first position $p[1]$ to correspond to the score of the "-" class and the second position $p[2]$ to correspond to the score of the "+" class, then it is natural to split the difference and assign $\vec{w}_1 = -\frac{1}{2}\vec{w}$ and $\vec{w}_2 = +\frac{1}{2}\vec{w}$. This way, if $\vec{w}^\top \vec{x}$ is positive, then clearly $p[2]$ is going to be positive while $p[1]$ is negative — this means that $p[2]$ will win[8] the argmax and we will declare the second class, the "+" class, victorious in this case. Meanwhile, if $\vec{w}^\top \vec{x}$ is negative, then clearly $p[1]$ is going to be positive while $p[2]$ is negative — this means that $p[1]$ will win the argmax and we will declare the first class, the "-" class, victorious in this case.

So, what does the logistic loss look like? For something that was labeled "+" to begin with, the logistic loss was $\ln\left(1 + e^{-p}\right)$ which can be rewritten to involve $p[1]$ and $p[2]$ by observing:

$$\begin{aligned}
\ln\left(1 + e^{-p}\right) &= \ln\left(1 + e^{p[1]-p[2]}\right) \\
&= \ln\left(\frac{e^{p[2]} + e^{p[1]}}{e^{p[2]}}\right) \\
&= \ln\left(e^{p[2]} + e^{p[1]}\right) - \ln\left(e^{p[2]}\right) \\
&= \ln\left(e^{p[2]} + e^{p[1]}\right) - p[2]
\end{aligned}$$

where we were driven to try and make the expression as symmetric in the different $p[k]$ as possible. To verify the symmetry of the pattern, let us look at the case of something that was labeled "-."

$$\begin{aligned}
\ln(1 + e^{p}) &= \ln\left(1 + e^{p[2]-p[1]}\right) \\
&= \ln\left(\frac{e^{p[2]} + e^{p[1]}}{e^{p[1]}}\right) \\
&= \ln\left(e^{p[2]} + e^{p[1]}\right) - p[1].
\end{aligned}$$

This immediately suggests a natural extrapolation/generalization to the case of K classes:

$$c^\ell(\vec{p}) = \ln\left(\sum_k e^{p[k]}\right) - p[\ell]. \tag{4}$$

The intuition behind this is that the sum of exponentials is dominated by the largest term. So taking a log

[7] Notice how here, the "predictions" are just a bank of correlations of the data $\vec{x}_i$ with the various templates $\vec{w}_k$. This should be reminiscent of what you saw in 16A in Module 3.

[8] Notice that there is a slight difference between the maximum cross-correlation approach we are proposing here and what was done in 16A. In 16A, because of the nature of the problem, we chose to look at the maximum absolute value of the cross-correlation. Here, we are not taking the absolute value and are considering the correlation as a signed quantity.

of a sum of exponentials is tantamount to taking a softer version of the max. We would like the loss to be small when the max is indeed the desired one, and the loss function we have crafted certainly satisfies that. At the same time, when we are getting the answer wrong, we want the loss to be positive and since $\ln\left(\sum_k e^{p[k]}\right) \geq \max(p[1], \ldots, p[K]) \geq p[\ell]$ this is certainly also true.

The iteratively reweighted least-squares algorithm described above can be adapted[9] for this multi-class logistic loss function treating all the $\vec{w}_k$ together as one big parameter vector to be learned. Once again, we will get a sequence of least-squares-style problems to solve, and converge to a set of learned $\vec{w}_k$.

This is sometimes called multinomial logistic regression (as well as minimizing cross-entropy loss) and is one of the workhorse building blocks for modern neural-net based approaches to classification. You will learn more about this in later courses.

What is important to take away from this lesson is that understanding classification in machine learning draws on the same underlying family of ideas involving linearization and approximation thinking that we need to use for control, differential equations, or nonlinear transistor circuits[10]. The differences are only skin deep, while the commonalities in thinking and technical tools go down to the bone.

Contributors:

- Rahul Arya.
- Anant Sahai.
- Gaoyue Zhou.

[9]Remember, the heart of the iterative approach is to approximate the total loss in the neighborhood of existing set of weights by a quadratic, and then minimizing the quadratic to get an update. In general, such forms $\frac{1}{2}\vec{u}^\top H\vec{u} + \vec{q}^\top\vec{u}$ are minimized by $\vec{u} = H^{-1}\vec{q}$ — this can be seen by an appropriate change of coordinates and completing the square. The pseudo-inverse should be used here in place of the inverse in case H has a nullspace, as you've seen in the HW.

In our case, we stack all the $\vec{w}_k$ on top of each other to get a Kn-long parameter vector $\vec{w}$. Because the total loss is the sum of individual losses, you just have to do the quadratic approximation for each of those individual losses and then add them up. This gives rise to a quadratic approximation of the form $\frac{1}{2}\delta\vec{w}^\top(\sum_i H_i)\delta\vec{w} + (\sum_i \vec{q}_i^\top)\delta\vec{w} +$ stuff. In general, the standard scalar chain rule implies that this is going to involve computing $\frac{\partial}{\partial \vec{w}_k}c^{\ell_i}(\vec{p}_i) = \frac{\partial}{\partial p[k]}c^{\ell_i}(\vec{p}_i)\vec{x}_i^\top$. Basically, the different $\vec{w}_k$ experience different scalar derivatives $\frac{\partial}{\partial p[k]}c^{\ell_i}(\vec{p}_i)$ instead of a single c'. When the derivative is viewed as a Kn-long row vector, notice how there is a pattern to this: each of the K blocks is a multiple of $\vec{x}_i^\top$.

For the second derivative, we are going to get different terms that cross across the different weights $\vec{w}_k$. $\frac{\partial^2}{\partial w_j[h]\partial w_k[g]}c^{\ell_i}(\vec{p}_i) = \frac{\partial}{\partial w_j[h]}(\frac{\partial}{\partial p[k]}c^{\ell_i}(\vec{p}_i)x_i[g]) = (\frac{\partial^2}{\partial p[j]\partial p[k]}c^{\ell_i}(\vec{p}_i))x_i[h]x_i[g]$. This will in general result in a Hessian matrix that is made of $K \times K$ blocks, each of which are $n \times n$ in size and of the form $(\frac{\partial^2}{\partial p[j]\partial p[k]}c^{\ell_i}(\vec{p}_i))\vec{x}_i\vec{x}_i^\top$. Notice that each of these is a scalar $(\frac{\partial^2}{\partial p[j]\partial p[k]}c^{\ell_i}(\vec{p}_i))$ times the same matrix $\vec{x}_i\vec{x}_i^\top$.

This kind of scaled copy-paste for matrices happens often and so is usually denoted using what's called the Kronecker (or tensor) product $\otimes$ so that $H_i = \begin{bmatrix} \frac{\partial^2}{\partial p[1]\partial p[1]}c^{\ell_i}(\vec{p}_i) & \frac{\partial^2}{\partial p[2]\partial p[1]}c^{\ell_i}(\vec{p}_i) & \cdots & \frac{\partial^2}{\partial p[K]\partial p[1]}c^{\ell_i}(\vec{p}_i) \\ \frac{\partial^2}{\partial p[1]\partial p[2]}c^{\ell_i}(\vec{p}_i) & \frac{\partial^2}{\partial p[2]\partial p[2]}c^{\ell_i}(\vec{p}_i) & \cdots & \frac{\partial^2}{\partial p[K]\partial p[2]}c^{\ell_i}(\vec{p}_i) \\ \vdots & \ddots & \ddots & \vdots \\ \frac{\partial^2}{\partial p[1]\partial p[K]}c^{\ell_i}(\vec{p}_i) & \frac{\partial^2}{\partial p[2]\partial p[K]}c^{\ell_i}(\vec{p}_i) & \cdots & \frac{\partial^2}{\partial p[K]\partial p[K]}c^{\ell_i}(\vec{p}_i) \end{bmatrix} \otimes \vec{x}_i\vec{x}_i^\top$. Meanwhile, the derivative also has a representation using the Kronecker product: $\vec{q}_i^\top = \begin{bmatrix} \frac{\partial}{\partial p[1]}c^{\ell_i}(\vec{p}_i) & \frac{\partial}{\partial p[2]}c^{\ell_i}(\vec{p}_i) & \cdots & \frac{\partial}{\partial p[K]}c^{\ell_i}(\vec{p}_i) \end{bmatrix} \otimes \vec{x}_i^\top$.

For (4) in particular, the key distinction is that the partial derivative with respect to $p[k]$ is different depending on whether $\ell = k$ or whether $\ell \neq k$. When $\ell = k$, we pick up an additional -1 term in this derivative, which ends up resulting in another $-\vec{x}_i^\top$ term in the derivative with respect to $\vec{w}_k$. However, this additional term contributes nothing to the second derivative of (4) with respect to any of the $\vec{w}_k$ since it is effectively a constant. As a result, the second derivative doesn't care if $\ell = k$ or not. This is a computationally nice feature of this particular loss function.

[10]You'll see a HW problem where you use linearization to better understand the origins of gain in circuits.

EECS 16B Designing Information Devices and Systems II
Note: DFT

Overview

We have seen how polynomial interpolation can be used to fit an $(m-1)$-degree polynomial to any set of m points with distinct x-coordinates. Often, however, we are not given these x-coordinates explicitly. For instance, we may be given a sequence of observations $y_1, y_2, \ldots, y_m$, told that they are values sampled at a uniform rate from some sensor, and asked to interpolate between them. The meta-data from our sensor is always present in terms of when each sample was actually taken, but we have considerable flexibility in how we use it.

In this note, we will look at various natural ways of doing this interpolation and see how each one falls short. More important than the specific ways that each falls short is how confronting each of these obstacles helps us open our mind more truly to the possibilities and helps us design ways that overcome the obstacles that we faced. This will ultimately lead us to a family of ideas called the Discrete Fourier Transform via this interpolation context.

1 Problems with real-valued polynomials

One natural way of doing interpolation is to write our observations as

$$(0, y_1), (\Delta, y_2), (2\Delta, y_3), \ldots, ((m-1)\Delta, y_{m-1}),$$

where Δ is the time between observations, and then use polynomial interpolation to express y as a continuous function of time.

We know that polynomial interpolation uses a parametric function

$$g(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \ldots + \alpha_{m-1} x^{m-1},$$

that is a linear combination of the monomials $x^0, x^1, x^2, \ldots, x^{m-1}$. The α_i are the parameters that define the function. We proved last time that polynomial interpolation will always work, meaning that $g(x)$ will pass through all our sample points exactly.

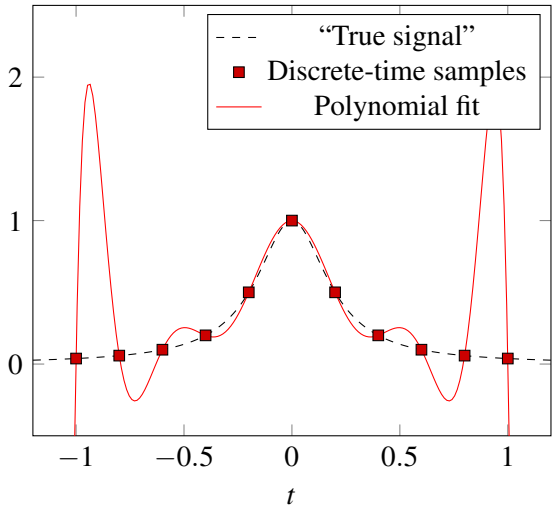
So what's more to do? As it turns out, although polynomial interpolation always produces *an* interpolation, it is not always a particularly desirable one. Let's try to see why. Imagine that we had a reasonably large number of samples - say, $m = 100$. Then our monomial basis will consist of the functions

$$x^0, x^1, x^2, \ldots, x^{99}.$$

The problem here is that x^{99} is really not a very nice function to work with over the reals. When x is just slightly less than 1, x^{99} quickly drops to zero. Try $(0.9)^{99} \approx 0.00003$ for example. Similarly, when x is just slightly greater than 1, the x^{99} starts blowing up rapidly. Try $(1.1)^{99} \approx 12500$. That is a huge dynamic range over a span of just 0.2. The consequence of this is that our coefficients α_i similarly become very small or very large in order to control these high-degree monomials, which is undesirable. The effect is that our

polynomial interpolations quickly become unreasonable - they may *technically* pass through all the sample points, but act "unreasonably" everywhere else.

Numbers don't have to get big before you start seeing this effect. For example, consider the following polynomial fit of $m = 11$ points:



Eleven is not that big of a number, and this is already starting to be pretty nasty in terms of how it is behaving. The fit turned out to be[1] $-220.94174208145742 * x^{10} + 494.90950226246065 * x^8 + -381.43382352942115 * x^6 + 123.35972850678912 * x^4 + -16.85520361990966 * x^2 + 1$ and the magnitude of those coefficients are getting big for the leading terms. In turn, these large coefficients are inducing oscillations present that did not exist in the original dataset.

Clearly, a better method of interpolation is needed for global approximation.

2 Escaping the desert of the reals: polynomials over the unit circle

Fundamentally, the problems we have faced with polynomial interpolation come down to the poor behavior of high-degree monomials away from $|x| = 1$, since their magnitudes vary enormously. We actually understand this very well since we have already seen the concept of stability for discrete-time systems. The reason that eigenvalues with magnitude less than 1 are stable is that raising them to high powers results in values close to zero, and hence the influence of disturbances a long time ago decays away. Similarly, eigenvalues with magnitudes greater than 1 cause violent instability because raising them to high powers blows up, and hence even small disturbances compound to overwhelm us. However, there are only two real numbers with absolute value equal to 1, so, with a large number of samples, we clearly cannot all place them such that $|x_i| = 1$.

Or can we? If we allow ourselves to use complex numbers, then we have infinitely many values z such that $|z| = 1$. We know that any complex number of the form $e^{j\theta}$, where θ is real, lies on the unit circle on the complex plane. We know from our studies of stability that the unit circle is special[2] — large powers neither

[1] Why are there only even terms here? Because all the odd terms are zero because the function we are interpolating is symmetric about the origin $f(x) = f(-x)$.

[2] It turns out that this property of not blowing up or shrinking to zero also occurs elsewhere — in finite fields. You will learn about finite fields in our successor course 70, and can learn even more about them in abstract algebra courses like Math 113 and Math 114. The fact that one can take large powers in finite fields without problems of instability turns out to be very useful

blow up nor go to zero. So, given a sequence $y_1, y_2, \ldots, y_m$, we may choose our x-coordinates to be such that our points become

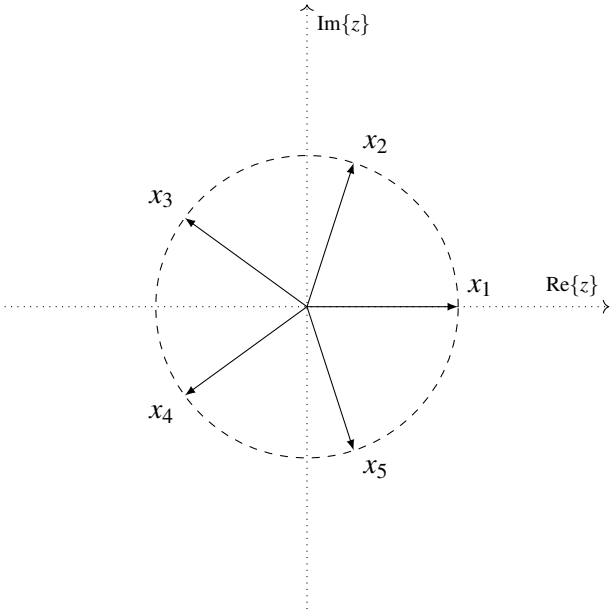
$$(1, y_1), (e^{2\pi j/m}, y_2), (e^{2\cdot 2\pi j/m}, y_3), \ldots, (e^{(m-1)\cdot 2\pi j/m}, y_{m-1}).$$

In other words, we let

$$x_i = (e^{j\frac{2\pi}{m}})^{i-1} = \omega_m^{i-1}, \ \forall i \in \{1, 2, 3, \ldots, m\}$$

where $\omega_m = e^{j\frac{2\pi}{m}}$ is the mth primitive root of unity.

We plot the x_i below on the complex plane, in the case of $m = 5$. Notice that they are all distinct, of magnitude 1, and equally spaced around the complex unit circle.



We can see clearly that the k-th power of $e^{j\theta}$ is $e^{jk\theta} = \cos(k\theta) + j\sin(k\theta)$ which always has a real part and imaginary part bounded by 1. Notice also the qualitative connection with what we traditionally associate with high degree polynomials — higher degrees are more "wiggly" than lower degrees. We get all these nice features without any of the pain of either blowing up or shrinking to zero. So this problem related to stability has been entirely exorcised.

3 Interpolating using complex numbers

Now, we can perform polynomial interpolation as usual, using these new, complex, values of x_i. Our previous results on the linear independence of the columns of a Vandermonde matrix still apply, since we never assumed there that our values were real. Thus, if our parametric function looks like

$$g(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \ldots + \alpha_{m-1} x^{m-1}$$

for communication-type applications like error-correcting codes and cryptography. This is also why Fourier-type analysis is also practiced in those areas.

with the coefficients/parameters stacked into a vector $\vec{\alpha}$, then we have

$$B\vec{\alpha} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^{m-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{m-1} \\ \vdots & \vdots & \ddots & \vdots & \\ 1 & x_m & x_m^2 & \cdots & x_m^{m-1} \end{bmatrix} \vec{\alpha} = \vec{y}.$$

Note that the matrix B is the same as the matrix B in the interpolation note, and the $\vec{y}$ are the samples.

Now, substituting in our chosen values for the x_i, our equation becomes

$$\begin{bmatrix} \omega_m^{0\cdot 0} & \omega_m^{0\cdot 1} & \omega_m^{0\cdot 2} & \cdots & \omega_m^{0\cdot(m-1)} \\ \omega_m^{1\cdot 0} & \omega_m^{1\cdot 1} & \omega_m^{1\cdot 2} & \cdots & \omega_m^{1\cdot(m-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \omega_m^{(m-1)\cdot 0} & \omega_m^{(m-1)\cdot 1} & \omega_m^{(m-1)\cdot 2} & \cdots & \omega_m^{(m-1)\cdot(m-1)} \end{bmatrix} \vec{\alpha} = \vec{y}.$$

When working with real numbers, the k-th column of our matrix B was composed of the stacked evaluations of the monomials x^k at $x_1, x_2, \ldots, x_m$. The same is true here, where now, because of our choice of x_i, they are the stacked evaluations of the monomials x^k at $\omega_m^0, \omega_m^1, \ldots, \omega_m^{m-1}$. As they are linearly independent, they continue to form a basis, so we can make any $\vec{y}$ as a linear combination of the columns of B.

It turns out that this particular $B = [\vec{b}_0, \vec{b}_1, \cdots, \vec{b}_{m-1}]$ matrix has a couple of nice properties that will be convenient for us later. First, notice that it is symmetric, by construction. Furthermore, we can compute the (squared) norm of each column (or row!) to be

$$\begin{aligned} \|\vec{b}_i\|^2 &= \langle \vec{b}_k, \vec{b}_k \rangle \\ &= \vec{b}_k^* \vec{b}_k \\ &= \sum_{i=0}^{m-1} \overline{\omega_m^{k\cdot i}} \cdot \omega_m^{k\cdot i} \\ &= \sum_{i=0}^{m-1} \omega_m^{-k\cdot i} \cdot \omega_m^{k\cdot i} \\ &= \sum_{i=0}^{m-1} 1 \\ &= m. \end{aligned}$$

Notice that, since $\vec{b}_k$ is a complex vector, we must take care to use the complex inner product when computing its norm. This involves a conjugate transpose instead of a transpose as in the real inner product. This result tells us that all the rows and columns of our B matrix have the same norm $\sqrt{m}$.

Furthermore, consider the inner product of any two *different* columns, $\vec{b}_a$ and $\vec{b}_b$. We see that

$$\begin{aligned}\langle \vec{b}_a, \vec{b}_b \rangle &= \vec{b}_b^* \vec{b}_a \\ &= \sum_{i=0}^{m-1} \overline{\omega_m^{b\cdot i}} \omega_m^{a\cdot i} \\ &= \sum_{i=0}^{m-1} \omega_m^{(a-b)\cdot i}.\end{aligned}$$

This is a finite geometric series with m terms and common ratio ω_m^{a-b}. Since this common ratio is not equal to 1, we can use the known formula for the sum of finite geometric series, to see that

$$\begin{aligned}\langle \vec{b}_a, \vec{b}_b \rangle &= \frac{\left(\omega_m^{a-b}\right)^m - 1}{\omega_m^{a-b} - 1} \\ &= \frac{\left(\omega_m^m\right)^{a-b} - 1}{\omega_m^{a-b} - 1}\end{aligned}$$

But since ω_m is an mth root of unity, $\omega_m^m = 1$. Thus,

$$\langle \vec{b}_a, \vec{b}_b \rangle = \frac{1^{a-b} - 1}{\omega_m^{a-b} - 1} = 0.$$

In other words, all the columns of B are mutually orthogonal. We can go a step further! Since all the columns have the same norm $\sqrt{m}$, we can normalize each column to obtain

$$U = \frac{1}{\sqrt{m}} B,$$

which has orthogonal columns of unit norm, and so is a complex orthonormal matrix!

Recall that we started with the equation $B\vec{\alpha} = \vec{y}$. We were searching for the coefficients α_i. Since B is known to be a square matrix of full rank, we can pre-multiply by its inverse to obtain

$$\vec{\alpha} = B^{-1}\vec{y}.$$

But now, since U is a square orthonormal matrix, we know that its inverse equals its conjugate transpose, $U^{-1} = U^*$. Thus, we see that the inverse of B is

$$B^{-1} = (\sqrt{m}U)^{-1} = (\sqrt{m})^{-1}U^{-1} = \frac{1}{\sqrt{m}}U^* = \frac{1}{\sqrt{m}}\left(\frac{1}{\sqrt{m}}B\right)^* = \frac{1}{m}B^*.$$

Substituting back, we find that the coefficients of our desired polynomial interpolation are

$$\vec{\alpha} = \frac{1}{m}B^*\vec{y}.$$

The $\vec{\alpha}$ are called the (polynomial-style) DFT coefficients that correspond to the vector $\vec{y}$. Often, an uppercase letter is used for them so $\vec{Y} = \vec{\alpha}$ and satisfies $\vec{y} = B\vec{Y}$. Equivalently, $\vec{Y} = \frac{1}{m}B^*\vec{y}$ and hence $Y[i] = \frac{1}{m}\vec{b}_i^*\vec{y}$.

Notice that B^* is a matrix with terms all of the same magnitude (1), and so (assuming that all the elements of $\vec{y}$ are about the same size) it is intuitive to expect that the DFT coefficients $(1/m)B^*\vec{y}$ is a vector whose

elements are within the same order of magnitude. Thus, by switching to samples taken on the unit circle over the complex plane, it seems that we have resolved the problem we had with our real polynomial interpolations, where the coefficients varied hugely in their magnitudes because the basis functions were poorly behaved over the points of interest.

4 Problems with our interpolation

Now that we have all this machinery available to us, let's try a simple example. Let our observations be

$$\vec{y} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}^\top,$$

so $m = 3$.

Substituting, our B matrix becomes

$$B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & e^{j2\pi/3} & e^{j4\pi/3} \\ 1 & e^{j4\pi/3} & e^{j2\pi/3} \end{bmatrix}.$$

Thus, we can solve for the coefficients of our polynomial interpolation, to obtain

$$\begin{aligned} \vec{\alpha} &= \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & e^{2j\pi/3} & e^{4j\pi/3} \\ 1 & e^{4j\pi/3} & e^{2j\pi/3} \end{bmatrix}^* \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \\ &= \frac{1}{3} \begin{bmatrix} 1 \\ e^{-2j\pi/3} \\ e^{-4j\pi/3} \end{bmatrix}. \end{aligned}$$

So our polynomial interpolation is

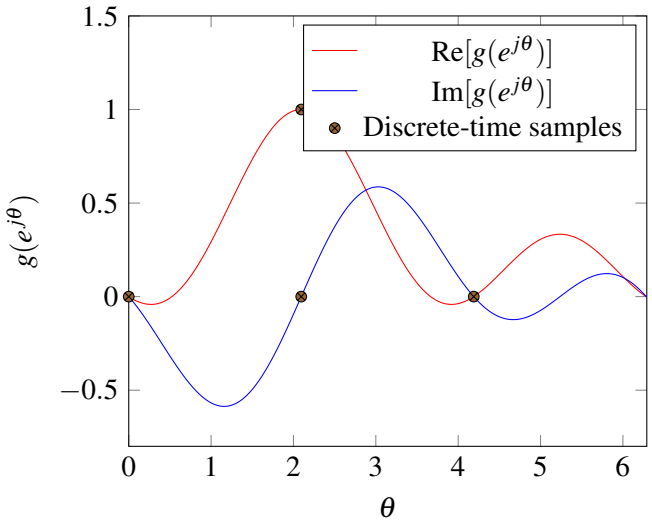
$$g(x) = \frac{1}{3} + \frac{e^{-2j\pi/3}}{3} x + \frac{e^{-4j\pi/3}}{3} x^2.$$

We can verify that $g(x)$ passes through our three observations. So are we done now?

Well, we really want to interpolate between our samples. Since our samples are located at $\omega_3^i = e^{j\frac{2\pi}{3}i}$ for $i \in \{0,1,2\}$, it makes sense to look at how $g(x)$ behaves at $e^{j\theta}$ for all $\theta \in [0, 2\pi)$, in order to obtain an equivalent to our previous real-valued polynomial interpolations. Here, our samples are interpreted as being at $\theta = 0, \frac{2\pi}{3}, \frac{4\pi}{3}$. Doing so, we obtain

$$\begin{aligned} g(e^{j\theta}) &= \frac{1}{3}\left(1 + e^{-2j\pi/3} e^{j\theta} + e^{-4j\pi/3} e^{j2\theta}\right) \\ &= \frac{1}{3}\left(1 + \cos(\theta - 2\pi/3) + \cos(2\theta - 4\pi/3)\right) + \frac{j}{3}\left(\sin(\theta - 2\pi/3) + \sin(2\theta - 4\pi/3)\right). \end{aligned}$$

Plotting the real and imaginary parts of this interpolation, we obtain:



At $\theta \in \{0, e^{j\frac{2\pi}{3}}, e^{j\frac{2\pi}{3}2}\}$, we can see that the imaginary component of our interpolation is 0 and the real component matches the sampled y_i, as expected. Unfortunately, everywhere in between, our interpolation has a nonzero imaginary component!

This is probably not what we wanted. Starting with only real sample points, we'd probably want our interpolated values between the samples to be real as well. So this is a problem. What can we do about it? Do we have any freedom?

5 Changing the basis functions without changing the matrix B

To resolve this, we will try to do as little work as possible. We have already got a pretty good method for finding parameter values — a unique solution always exists, the coefficients are all of similar orders of magnitude, and the B matrix has lots of nice properties. It is easy to invert. Because of the orthogonality properties inherent in B, it is also very easy to project onto subspaces that are defined by a subset of the columns — we can just keep those coefficients and zero out the others. This means that it is also very compatible with least-squares fitting. The only issue is that our interpolation has a nonzero imaginary component even when given purely real points to interpolate.

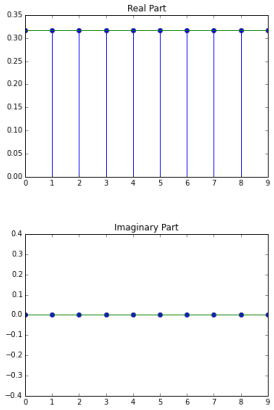
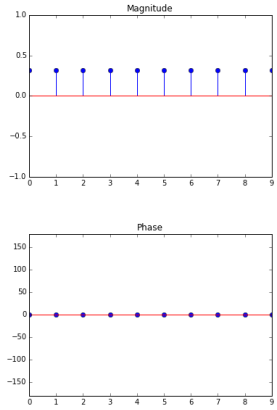
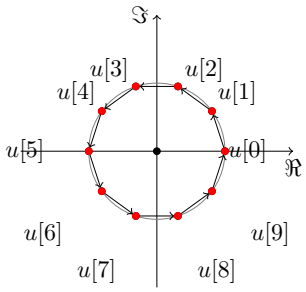
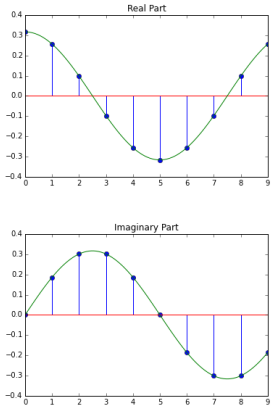
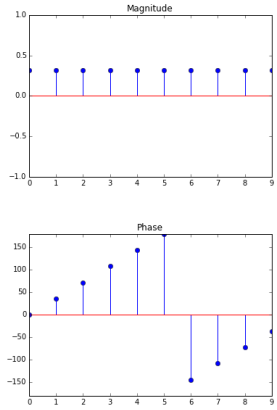
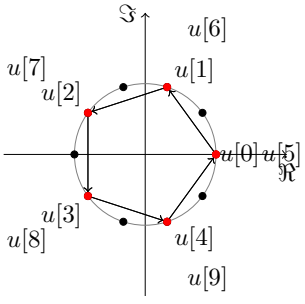
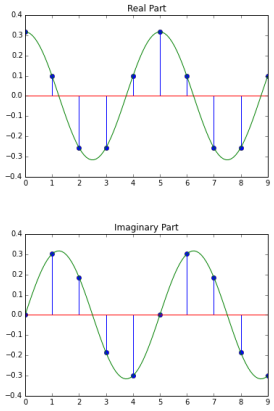
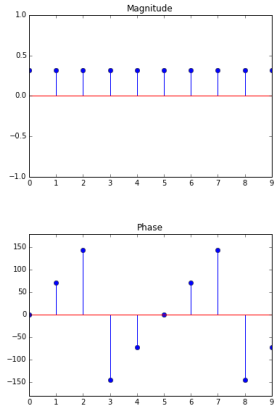
We want to make the smallest change possible to our approach such that, assuming we start with real samples y_i, we can be assured that our interpolation $h(e^{j\theta})$ is real for all real θ. In particular, we want to keep the same B matrix and α_is, but just change the parametric function we use to interpolate. In other words, we want to change the basis functions.

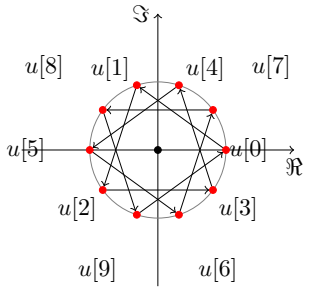
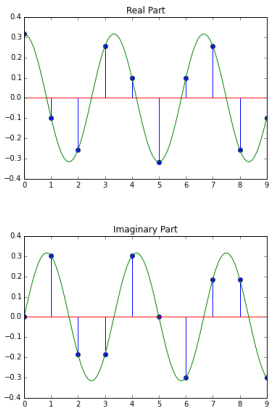
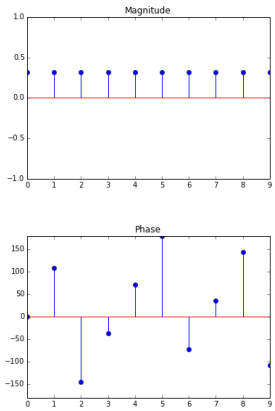
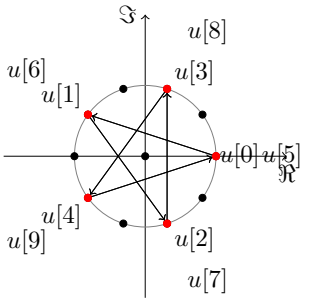
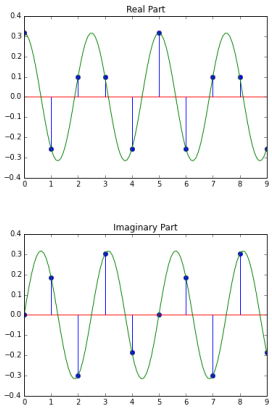
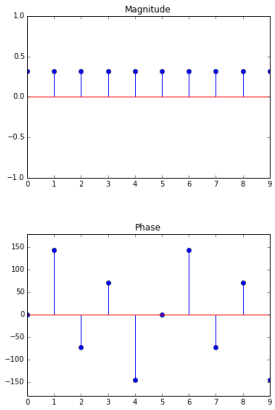
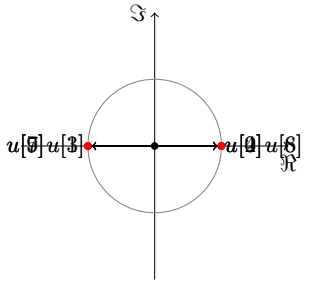
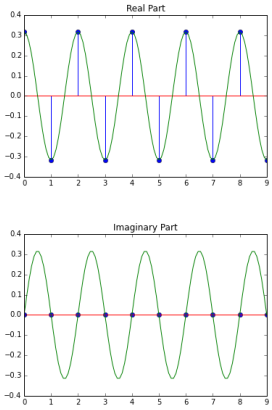
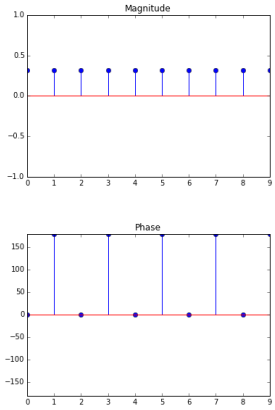
5.1 Taking a closer look at what is going on

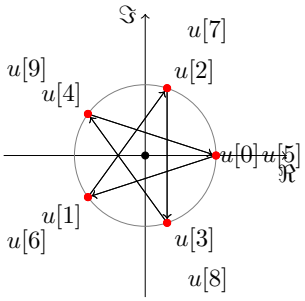
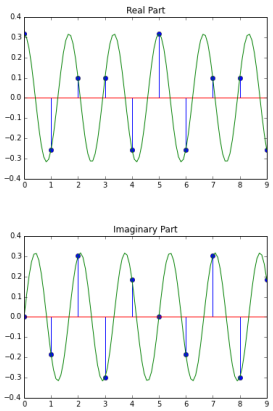
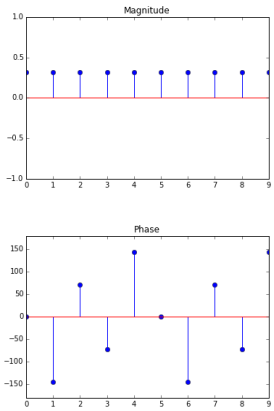
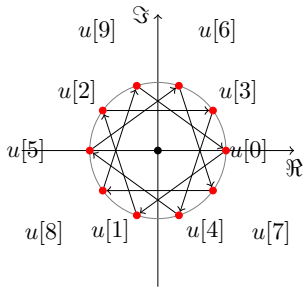
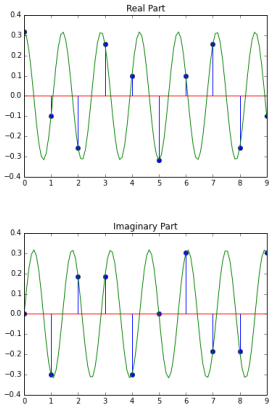
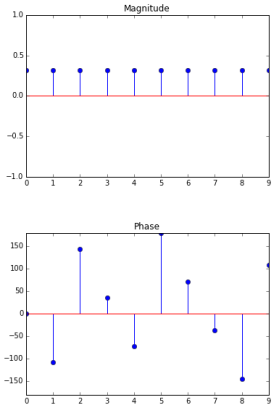
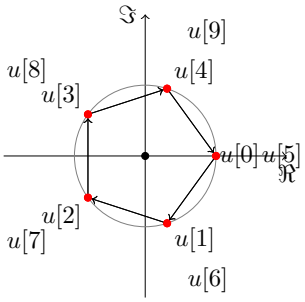
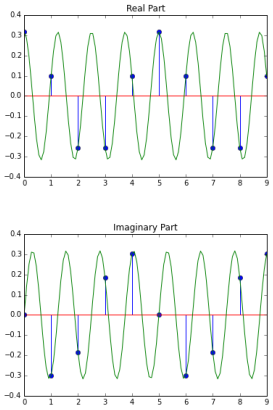
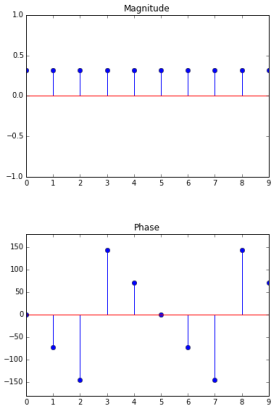
It is worth actually looking at an example of what the DFT basis vectors $\vec{b}_k$ or $\vec{u}_k$ look like. These are essentially complex exponentials, and as the k increases, they "wiggle" more and more. This is why k is often referred to as the frequency associated with that vector. They always have an integer number of periods between in the total duration m.

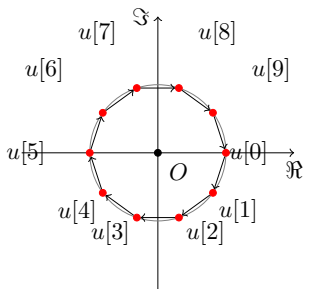
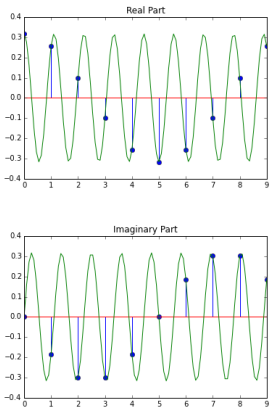
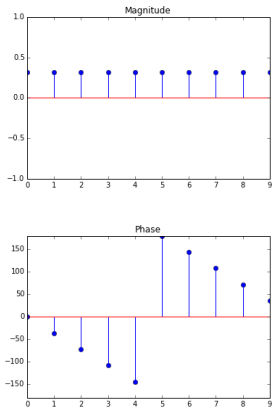
Here, we will illustrate $\vec{u}_k$ for the $m = 10$. The only reason we have chosen to illustrate $\vec{u}_k$ instead of $\vec{b}_k$ is so that the magnitude plot in polar coordinates of what the values inside the vector is will be slightly less boring. For $\vec{b}_k$, the magnitude of every entry in the vector is 1. Here, for the Cartesian Coordinates plots, we

have also drawn the underlying complex exponential that is being sampled to create the specific DFT basis vector being illustrated.

k	$u_k[t]$	Unit circle plots	Cartesian coordinates plots	Polar coordinates plots
0	1			
1	$e^{j\frac{2\pi t}{10}}$			
2	$e^{j\frac{2\pi 2t}{10}}$			

3	$e^{j\frac{2\pi 3t}{10}}$			
4	$e^{j\frac{2\pi 4t}{10}}$			
5	$e^{j\frac{2\pi 5t}{10}}$			

6	$e^{j\frac{2\pi 6t}{10}}$			
7	$e^{j\frac{2\pi 7t}{10}}$			
8	$e^{j\frac{2\pi 8t}{10}}$			

9	$e^{j\frac{2\pi 9t}{10}}$			

Looking at these vectors, we see that there are lots of symmetries in them. We also notice that there is a curious deja-vu feeling when looking at the circle diagrams — we go decagon to pentagon to lotus to pentagram to line and then walk back in the reverse order: back to pentagram (but reversed arrows), to lotus (also reversed arrows), to pentagon (reversed arrows again), to decagon (also reversed). The labels are also mirrored across the real axis, which tells us that there is a complex conjugation that is happening.

5.2 Returning to interpolation

Currently, our interpolation is of the form

$$g(e^{j\theta}) = \alpha_0 + \alpha_1 e^{j\theta} + \alpha_2 e^{j2\theta} + \cdots + \alpha_{m-1} e^{j(m-1)\theta}.$$

We would like to suppress the imaginary components of our interpolation. One way of doing so would be to add the complex conjugates of each term, so only the real terms remain. But there is no obvious way of getting complex conjugates to appear in our above equation without changing the coefficients α_i.

To do so, we need to take a closer look at our coefficients. Looking at one row of our equation for $\vec{\alpha}$, we see that

$$\begin{aligned}\alpha_i &= \frac{1}{m}\sum_{k=0}^{m-1} B^*[i][k]y_k \\ &= \frac{1}{m}\sum_{k=0}^{m-1} \omega_m^{-i\cdot k} y_k.\end{aligned}$$

Thus, since the y_j are real, its conjugate must equal

$$\overline{\alpha_i} = \frac{1}{m}\sum_{k=0}^{m-1} \omega_m^{i\cdot k} y_k.$$

But as ω_m is an mth root of unity,

$$\omega_m^{i\cdot k} = \omega_m^{i\cdot k - k\cdot m} = \omega_m^{k(i-m)}.$$

This is the mirroring across the real axis that we were seeing in the circle diagrams of the labels! This

means:

$$\overline{\alpha_i} = \frac{1}{m}\sum_{k=0}^{m-1} \omega^{k(i-m)} y_k = \frac{1}{m}\sum_{k=0}^{m-1} \omega^{-k(m-i)} y_k = \alpha_{m-i}.$$

In other words, the conjugates of the coefficients α_i are simply the coefficients of the "reversed" coefficient vector $\vec{\alpha}$.

So now we have some hope for leveraging complex conjugates! We remember from Phasor Analysis of circuits that by using a complex exponential with a negative sign together with every use with a positive sign, we will get real valued functions as long the coefficients are complex conjugates.

So can we do this? We originally generated the k-th column of the B matrix by viewing them as samples from the monomial "x^k" evaluated at the m-th roots of unity. How can we just change the function?

5.3 Aliases

This is where we get to a matter that is at the heart of all machine learning. Trying to learn a function from a finite amount of data is fundamentally an underspecified problem because there are always infinitely many functions that pass through any finite amount of data. If we view the actual data as a thing or object, we can view the abstract function that generated the data as its "name." The issue is that the same data has many possible names[3]. This issue is called "aliasing" in the literature. When we want to interpolate at some point where hadn't collected data, we need to summon[4] forth an evaluation by calling a specific name. This disambiguation is something that we are always doing, either implicitly or explicitly.

In this case, the symmetries inherent in the roots of unity let us see lots of the different names/functions that correspond to our columns of the B matrix. In particular, if we look at the k-th column $\vec{b}_k$, we see that it could have come from any formal monomial "x^{k+qm}" for any integer q. This is because[5] we can look at the i-th entry in $\vec{b}_k$ and notice that $(\omega_m^i)^{k+qm} = \omega_m^{ik}\omega_m^{qmi} = \omega_m^{ki}(\omega_m^m)^{qi} = \omega_m^{ki}(1)^{qi} = \omega_m^{ki}$.

This means we are free to summon forth any of these or even any convex combination (a linear combination where the weights sum to 1) of these "x^{k+qm}" and these are all valid choices. The resulting functions will interpolate all the data points.

6 "Natural" interpolation using the DFT coefficients

The reality of aliasing tells us that we need to make a choice for which functions to use to interpolate. We know what we want in this case. We want to be able to get real values for our interpolation if our original data was real, and this means that we need to leverage complex conjugates. This means that since we have $\alpha_1 = \overline{\alpha_{m-1}}$ we would prefer to summon forth x^{-1} instead of x^{m-1} associated with the parameter α_{m-1}. This is valid because $-1 = m-1+(-1)m$ and hence x^{-1} and x^{m-1} are both aliases of the last column of the B matrix. The same applies for $\vec{b}_{m-2}$ — we can view this as coming from x^{-2} and so on. This allows us to pair up the terms and cancel out all the imaginary parts.

[3] Notice how this classical terminology is somewhat curiously sample-centric. It views the sampled vectors as being the objects and the functions as being "names" — and so the same vector has many aliases. You might argue that reality is actually reversed and the phenomenon is actually one of Dopplegangers. The functions are the actual objects of interest. And many different functions can look the same if you only sample them at those points. The name of the game here is in choosing among these Dopplegangers.

[4] It is this deep cultural connection to the idea of "true name magic" that explains the classical terminology. When programming, we "invoke" and "call" functions — by saying their names. We do this to make them manifest. You saw this more deeply in 61A when you were doing your interpreter project. As a result, as engineers, names have power for us. Hence this curious terminology. Anyway, you'll get used to it.

[5] Because we are dealing with integer powers, we are allowed to do these manipulations.

6.1 The case of m odd

The story becomes easiest to understand if we stick to m being an odd number. Then, we have the 0-th coefficient for the constant term and the rest of the coefficients divide evenly into pairs. Imagine that we could replace the second half of the monomial terms with their conjugates, to obtain the new interpolation

$$h(e^{j\theta}) = \alpha_0 + \sum_{i=1}^{(m-1)/2} \left(\alpha_i e^{ji\theta} + \alpha_{m-i} e^{-ji\theta}\right) = \alpha_0 + \sum_{i=1}^{(m-1)/2} \left(\alpha_i e^{ji\theta} + \overline{\alpha_i e^{ji\theta}}\right).$$

The second equality is where we used the realness of the underlying $\vec{y}$ that we are interpolating.

Clearly, this interpolation is real, since each of the terms in the summation are real (by the property that adding a complex number to its conjugate gives twice the real part). But does it still pass through all of our sample points? It should by construction, but let's verify this. Since our sample points are of the form (ω_m^{i-1}, y_i), we can set up the system of equations to check as:

$$\begin{bmatrix} h(\omega_m^0) \\ h(\omega_m^1) \\ \vdots \\ h(\omega_m^{m-1}) \end{bmatrix} = \vec{y}.$$

Now, we can simplify the sum step by step. We start by splitting up the summation that defines our interpolation $h(\cdot)$:

$$\begin{aligned}
h(\omega_m^k) &= \alpha_0 + \sum_{i=1}^{(m-1)/2} \alpha_i (\omega_m^k)^i + \sum_{i=1}^{(m-1)/2} \overline{\alpha_i (\omega_m^k)^i} \\
&= \alpha_0 + \sum_{i=1}^{(m-1)/2} \alpha_i (\omega_m^k)^i + \sum_{i=m-1}^{(m+1)/2} \overline{\alpha_{m-i} (\omega_m^k)^{m-i}} \quad \text{(looking at the second sum backwards)} \\
&= \alpha_0 + \sum_{i=1}^{(m-1)/2} \alpha_i (\omega_m^k)^i + \sum_{i=(m+1)/2}^{m-1} \alpha_i (\omega_m^k)^{i-m} \quad \text{(conjugacy properties)} \\
&= \alpha_0 + \sum_{i=1}^{(m-1)/2} \alpha_i (\omega_m^k)^i + \sum_{i=(m+1)/2}^{m-1} \alpha_i (\omega_m^k)^i \quad \text{(root of unity property)} \\
&= \alpha_0 + \sum_{i=1}^{m-1} \alpha_i (\omega_m^k)^i.
\end{aligned}$$

So despite the different function being used to interpolate, on the sample points ω_m^k, it returns exactly the same thing as the traditional polynomial interpretation. Recall that we originally chose our $\vec{\alpha}$ to satisfy the equation $B\vec{\alpha} = \vec{y}$. Recall how we constructed the columns $\vec{b}_i$. Each of them were the evaluation of the ith basis function on the x-coordinates of our sample points. After changing our basis functions, the key observation is that although the functions are changed, their *evaluations* on these coordinates remain the same. Since the α_i were chosen to satisfy polynomial interpolation, this new function also interpolates perfectly.

So even after changing the basis functions used for the linearly parameterized function that we want to learn, the same B matrix remains valid, so the equation $B\vec{\alpha} = \vec{y}$ remains a necessary and sufficient condition in order for the interpolation to pass through all our sample points. Thus, what we can do is compute $\vec{\alpha}$ as

before, then plug into the new interpolation $h(\cdot)$, to get a purely real interpolation that passes through all of our sample points. So we have found exactly the interpolation that we want.

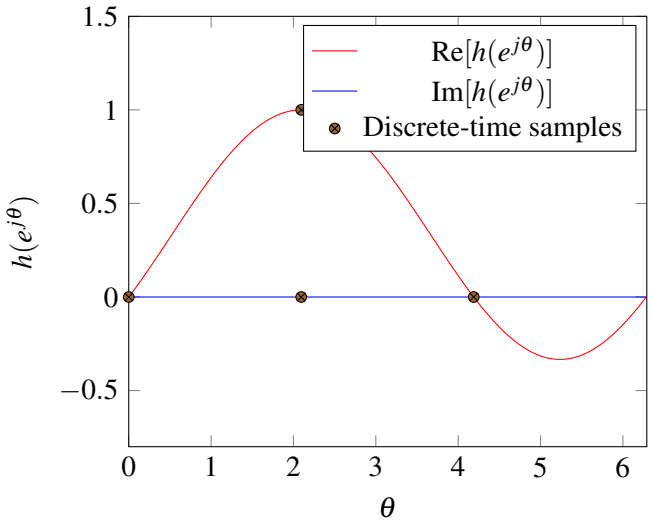
Let's verify that it works by returning to our example of interpolating $\vec{y} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$. Recall that we got $\vec{Y} = \frac{1}{3}\begin{bmatrix} 1 \\ e^{-2j\pi/3} \\ e^{-4j\pi/3} \end{bmatrix}$ for the DFT coefficients. Putting them into our interpolation we get

$$h(x) = \frac{1}{3} + \frac{e^{-2j\pi/3}}{3}x + \frac{e^{-4j\pi/3}}{3}x^{-1}$$

and expanding this out for arbitrary $x = e^{j\theta}$ on the unit circle, we get

$$h(e^{j\theta}) = \frac{1}{3} + \frac{e^{-j2\pi/3}}{3}e^{j\theta} + \frac{e^{+j2\pi/3}}{3}e^{-j\theta} = \frac{1}{3}(1 + 2\cos\left(\theta - \frac{2\pi}{3}\right))$$

Plotting the real and imaginary parts of this interpolation, we obtain:



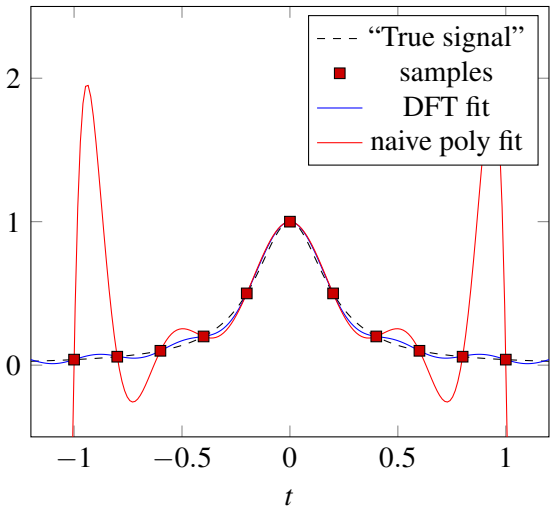
This clearly feels like a qualitatively "simpler" interpolation and thus satisfies our intuition for what Occam's razor demands.

Returning to our example of 11 points. The values were $\vec{y} = [\frac{1}{26}, \frac{1}{16}, \frac{1}{10}, \frac{1}{5}, \frac{1}{2}, 1, \frac{1}{2}, \frac{1}{5}, \frac{1}{10}, \frac{1}{16}, \frac{1}{26}]^\top$. The DFT coefficients $\vec{Y} = \frac{1}{11}B^*\vec{y}$ and these can be used to get an interpolation $Y[0] + \sum_{k=1}^{5} Y[k]e^{jk\theta} + \overline{Y[k]}e^{-ji\theta} = Y[0] + 2\sum_{k=1}^{5} |Y[k]|\cos\big(k\theta + \angle Y[k]\big)$. gives us a DFT-based interpolation of $0.255 + (-0.159 - j0.047)e^{j\theta} + (-0.159 + j0.047)e^{-j\theta} + (0.077 + j0.050)e^{j2\theta} + (0.077 - j0.050)e^{-j2\theta} + (-0.036 - j0.042)e^{j3\theta} + (-0.036 + j0.042)e^{-j3\theta} + (0.014 + j0.030)e^{j4\theta} + (0.014 - j0.030)e^{-j4\theta} + (-0.004 - j0.026)e^{j5\theta} + (-0.004 + j0.026)e^{-j5\theta}$ This interpolation has the sampled points being viewed as being regularly spaced starting at 0 and heading towards 2π.

However, the meta-data tells us that the samples were taken at $-1, -0.8, -0.6, -0.4, -0.2, 0, 0.2, 0.4, 0.6, 0.8, 1$. If we want to use the meta-data to line up the points and get an interpolation function in terms of the original points, we need to find the affine translation between the original x and θ. Here -1 for x translates to $\theta_0 = 0$. While $+1$ for x translates to $\theta_{10} = \frac{2\pi 10}{11}$. This means that $\theta(x) = \frac{2\pi 5}{11} + \frac{2\pi 5}{11}x$ is the affine map from

original x inputs to the corresponding θ. We see that $\theta(-1) = 0, \theta(-0.8) = \frac{2\pi}{11}, \theta(-0.6) = \frac{2\pi 2}{11}, \theta(-0.4) = \frac{2\pi 3}{11}, \theta(-0.2) = \frac{2\pi 4}{11}, \theta(0) = \frac{2\pi 5}{11}, \theta(0.2) = \frac{2\pi 6}{11}, \theta(0.4) = \frac{2\pi 7}{11}, \theta(0.6) = \frac{2\pi 8}{11}, \theta(0.8) = \frac{2\pi 9}{11}, \theta(1) = \frac{2\pi 10}{11}$.

Using this we can plot:



Notice how much better and more natural the DFT-based interpolation is for this data. This naturalness is why we use DFT-based interpolations or analogous DFT-based projections to get global approximations for data. Notice that this is indeed an approximation — the interpolation doesn't get our "true signal" perfectly right, but getting it perfectly right is in general too much to ask for in practice. The only way that would happen is if the true signal was indeed something that was perfectly representable in the basis that we had decided to use for fitting. What we are going for with our DFT-based interpolation is a good general-purpose approximation for signals that are smooth and not too wiggly over a finite domain of interest.

A further thing to notice is that by choosing to map the finite domain of interest to the unit circle, we are indeed making another implicit choice. After all, a circle has no natural beginning and no natural end — it is a loop. This means that the functions we are going to get are always going to be implicitly periodic. It turns out that there are perfectly natural ways to eliminate that assumption. For example, we can put all our sample points from 0 to π instead of from 0 to 2π, and just mirroring them on the other side of the unit circle. All sequences are naturally periodic if you walk through them up and then down — you are guaranteed to end up where you started. This goes by the name of the DCT and is something you'll see more about in 123. There are similar natural ways to adapt to nonuniformly spaced samples, etc. Orthogonality gets lost and the computations slow down, but it is possible to proceed.

6.2 The even m case

All that remains is the case of m even. Here, there is a slight twist because the DFT coefficients don't naturally come in complex conjugate pairs. As before, there is the special 0 coefficient α_0 which is real because the first column of the B matrix is real and we are assuming that the y_i values we are interpolating are real. But for the even m case, there is another special case of $\alpha_{\frac{m}{2}}$. This is also real because $\omega_m^{\frac{m}{2}} = -1$ and hence the middle column $\vec{b}_{\frac{m}{2}}$ of B consists entirely of alternating $-1, +1$s. Because of this, since the inverse of B has $\frac{1}{m}\vec{b}_k^*$ for its k-th row, we know that $\alpha_{\frac{m}{2}}$ is always real if the y_i values are all real.

For the rest of the coefficients, they come in complex conjugate pairs. So what we did for the odd m case will work for interpolation. But what about the $\frac{m}{2}$ coefficient? What function should we summon forth

here? $x^{\frac{m}{2}}$ isn't the right choice on its own — it will be complex. For the same reason, $x^{-\frac{m}{2}}$ also isn't the right choice on its own — it too will be complex. But the average of two aliases is also an alias, so we can use $\frac{1}{2}(x^{\frac{m}{2}}+x^{-\frac{m}{2}})$. Evaluated on $x=e^{j\theta}$ this is $\frac{1}{2}(e^{j\frac{m}{2}\theta}+e^{-j\frac{m}{2}\theta})=\cos\left(\frac{m}{2}\theta\right)$ which is definitely real.

Putting everything together, we get for our interpolation (when the original data was real):

$$h(e^{j\theta})=\alpha_0+\alpha_{\frac{m}{2}}\frac{1}{2}(e^{j\frac{m}{2}\theta}+e^{-j\frac{m}{2}\theta})+\sum_{i=1}^{\frac{m}{2}-1}\left(\alpha_i e^{ji\theta}+\overline{\alpha_i e^{ji\theta}}\right)$$

which is definitely real.

A final comment can be made on what should we do if the original samples $\vec{y}$ were complex to begin with. (This is not an even vs odd thing, but the even case is what brings it to mind more strongly.) The above expressions continue to be valid interpolations as long as we replace $\overline{\alpha_i}$ with what it actually comes from: α_{m-i}. It was the realness of $\vec{y}$ that gave the complex conjugacy relationship between the pairs of α_i, α_{m-i} coefficients. The fact that these are interpolations comes from the fact that $\vec{b}_k$ has many names, and we have chosen to invoke one of them. That didn't care about what $\vec{y}$ was like. That said, when we have $\vec{y}$ complex, we need to ask where they are coming from and how we want to decide on the best strategy for interpolating them. Sometimes the answer is as above, and sometimes the nature of the problem will want to have us use a different strategy. You'll learn more about that in 120 and 123. It turns out to be particularly relevant in the context of making communication systems.

7 Other DFTs

The DFT is an amazingly versatile tool. Interpolation and global approximations of functions is just one of its uses. The "Fourier style" of thinking turns out to be helpful in other ways as well.

As a result, there are actually multiple definitions that you will encounter in the literature and when using codebases or libraries that exist. It is helpful to always think about the DFT as a coordinate transformation. As we saw when we illustrated them, because of the interesting properties of exponentials and powers, the kth DFT basis vector represents a counterclockwise path through the m-th roots of unity starting at 1 and then moving by k roots at a time. So the 0th DFT basis vector just stays at 1. The 1st DFT basis vector goes through the n roots of unity one at a time clockwise starting with 1. The 2nd DFT basis vector takes two trips around the unit circle starting at 1 and moving two at a time. And so on.

There are several different ways to normalize the DFT basis vectors, each coming from different motivations. Below is a table of three such normalizations in widespread use. They don't have standardized names[6], and so we will give them names that we like.

Variant	Normalizing Factor	Notation	entries
Polynomial DFT basis	1	$\vec{b}_k$	$b_k[i]=e^{j\frac{2\pi ik}{m}}$
Orthonormal DFT basis	$\frac{1}{\sqrt{m}}$	$\vec{u}_k$	$u_k[i]=\frac{1}{\sqrt{n}}e^{j\frac{2\pi ik}{m}}$
Traditional/Classic DFT basis	$\frac{1}{m}$	$\vec{d}_k$	$d_k[i]=\frac{1}{n}e^{j\frac{2\pi ik}{m}}$

[6]To add to the confusion, inspired by certain discrete settings, some folks use the conjugate of the classic DFT with the basis vectors effectively going clockwise to start. For example, without any attempt to tell it to do otherwise, Wolfram Alpha and Mathematica default to this alternative. The moral is, you have to be careful and check to see if any software package is indeed giving you what you are asking for.

The table above defines the relevant matrices $B = [\vec{b}_0, \vec{b}_1, \cdots, \vec{b}_{m-1}]$ or $U = [\vec{u}_0, \vec{u}_1, \cdots, \vec{u}_{m-1}]$ or $D = [\vec{d}_0, \vec{d}_1, \cdots, \vec{d}_{m-1}]$ and correspondingly result in relationships (e.g. $B\vec{F} = \vec{f}$) between the learned parameters $\vec{F}$ and the original data $\vec{f}$. They clearly all differ only by scaling[7].

Their inverses are what we use to go from observations $\vec{y}$ to the corresponding DFT coefficients $\vec{Y}$. These are $B^{-1} = \frac{1}{m}B^*$ for the polynomial-style DFT, $U^{-1} = U^* = \frac{1}{\sqrt{m}}B^*$ for the orthonormal DFT, and $D^{-1} = B^*$ for the classic/traditional DFT.

You will learn more about the traditional/classic DFT in 120/123, and see why it is defined the way that it is. The reason has to do with eigenvalues and eigenvectors, and is intimately connected to Phasors and Transfer functions. Essentially, the coefficients obtained using the classic DFT represent the eigenvalues of a particular matrix, and they play the role of a Transfer function. You will see in 120/123 how you can take limits to explicitly connect these two.

Contributors:

- Rahul Arya.
- Anant Sahai.

[7] FYI: Numpy given normalization "none" will compute the Traditional/Classic map and solve the equation $D\vec{F} = \vec{f}$ to return $\vec{F}$. If you give Numpy the normalization "ortho", then it will compute the Orthonormal DFT basis version and solve $U\vec{F} = \vec{f}$ and return $\vec{F}$. Nobody bothered to define a separate normalization for the Polynomial version because it turns out that it can be computed by calling the inverse classic DFT on a "flipped" vector $\vec{f}$. Can you see why? This has to do with the complex conjugacy relationships between the columns of B. By appropriately "flipping" the vector, you can effectively multiply by the conjugate of B. Here, flipping involves reversing the order of almost all the entries, but keeping the 0-th entry the same.

Anyway, since this is an EECS course and the FFT (Fast Fourier Transform) is literally considered one of the most important algorithms of all time, we felt that it was important to connect to numpy here. The FFT itself is a topic for 170 and 120/123. It tells us that we can compute the DFT coefficients far faster than having to do a full matrix multiplication.